

5.2.3.2 Algorithm The algorithm is summarized as follows:

- Each node of the decision tree classifier requires us to efficiently examine all possible splits on each value of each attribute.
- After choosing a split attribute, it needs to partition all data into its subset.
- While evaluating splits on numeric attributes, it sorts on attribute value and incrementally computes the Gini index.
- While splitting on categorical attributes, it finds the Gini index for each subset and chooses the best; for large datasets, a greedy method is used.

Some of the drawbacks of SPRINT include [16] (i) an increase in the size of the training database while maintaining the attribute list at each node, (ii) a large cost to keep these attribute lists sorted, and (iii) involvement of a costly hash-join between vertically separated parts of a record through the record identifier.

5.2.4 RainForest

Studies have shown that no algorithm is uniformly most accurate over all datasets. Hence a unifying generic framework RainForest [16] has been developed for classification tree construction. It yields scalable versions of a wide range of classification algorithms and offers performance improvements above a factor of three over SPRINT, the fastest available scalable classification algorithm. Unlike SPRINT, RainForest requires a certain minimum amount of main memory proportional to the set of distinct values in a column of the input relation. However, given the current main memory costs, this requirement is readily met in most, if not all, workloads. This generic algorithm can be specialized to obtain scalable versions of most classification and regression tree construction algorithms available in the literature.

5.2.5 Overfitting

A tree T overfits if there is another tree T' that gives a higher error on the training data while giving a lower error on unseen test data. Such an overfitted tree does not generalize to unseen instances. This situation generally occurs when the data contains noise or irrelevant attributes, and the training set size is small. Overfitting can reduce the accuracy drastically, even around 10-25% as reported in [17]. Smaller consistent decision trees typically have higher generalization accuracy than larger consistent trees.

Let us explain the concept of overfitting (poor generalization) in the context of a two-class problem. A classifier with excellent performance over a training dataset may have poor generalization over an unseen test set, since its decision

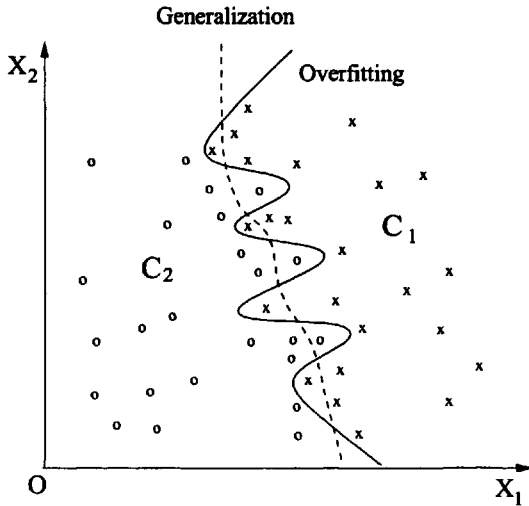


Fig. 5.2 Overfitting and generalization for decision boundaries in classification.

boundary overfits the training data as depicted in Fig. 5.2. On the other hand, a decision boundary with good generalization provides consistently good performance over both the training and test sets. This is also illustrated in the figure.

There are two popular approaches to prevent overfitting.

- Stop growing the tree beyond a certain point (pre-prune). For this purpose, one can use the statistical significance χ^2 or information gain, to assess the goodness of a split.
- First overfit and then post-prune. Here the tree building is divided into two phases, namely, growing and pruning.

Since it is hard to decide when to stop growing the tree, the second approach is more widely used. Although post-pruning requires more computations, it eventually generates a more reliable tree.

There exist three broad criteria for finding the correct final tree size. These are (i) cross-validation with separate test data, (ii) use of some criteria function, like Minimum Description Length (MDL), to choose the best size, and (iii) computation of statistical bounds, using all data for training but applying statistical tests to choose the right size.

Cross-validation consists of partitioning the dataset into two disjoint parts. The training set is used for building the tree, while a validation set is used for pruning the tree. The rule of the thumb is to keep two-thirds for training and to keep the remaining one-third for validation. The tree is evaluated on the validation set, keeping count of the correctly labeled data at each leaf as well as internal nodes. Starting bottom-up, the algorithm prunes nodes with

error less than its children. However, if the training dataset size is limited, k -fold cross-validation [18] is used.

Pruning using the MDL principle views a decision tree as a means for efficiently encoding the classes of records in the training set. The best tree is considered as the one that can encode records using the fewest number of bits. The cost of encoding a tree includes one bit for encoding the type of each node (e.g., leaf or internal), the cost C_{split} for encoding the attribute and value for each split, and the cost $N' \times E$ of encoding the N' records in each leaf (E being entropy). The problem is to compute the minimum cost subtree at the root of a built tree. Pruning proceeds in a bottom-up fashion. The basic steps are as follows:

- Suppose $\min C_\alpha$ is the cost of encoding the minimum cost subtree rooted at α .
- Compute $\min C_\alpha$ as $N'E + 1$ if α is a leaf, and $\min\{N'E + 1, C_{split} + 1 + \min C_{\alpha_1} + \min C_{\alpha_2}\}$ if α has children α_1 and α_2 .
- Prune the children of a node α if $\min C_\alpha = N'E + 1$.

5.2.6 Pruning and Building Integrated in Classification (PUBLIC)

This is an improved decision tree classifier [19] that integrates the pruning phase with the initial building phase. In PUBLIC, a node is not expanded in the building phase if it is determined that it will be pruned in the subsequent pruning phase. Before a node is expanded, the algorithm computes a lower bound on the minimum cost subtree rooted at the node. This estimate, based on the records contained in the leaf, is then used by PUBLIC to identify the nodes that are certain to be pruned later, such that no effort is initially expended on splitting them. Since building a subtree usually requires repeated scans over the data, a significant reduction in I/O and improvement in performance is realized by minimizing this wastage of effort. However, there exists a trade-off between the accuracy of the estimate and the actual cost involved.

5.2.7 Extracting classification rules from trees

The knowledge encoded by the decision tree can be extracted in the form of *IF-THEN* rules. One rule is created for each path from the root to a leaf, with each attribute-value pair along a path forming a conjunction. The leaf node holds the class prediction. Rules being easier for humans to understand, their usefulness in improving understandability and subsequent role in better human-machine interaction cannot be ignored. For example, sample rules can be of the form

```
IF age ≤ 30 AND student = yes THEN buys_computer = yes;  
IF 31 ≤ age ≤ 40 THEN buys_computer = yes;  
IF age ≤ 30 AND credit_rating = fair THEN buys_computer = no.
```

An example of a rule extracted from the decision tree of Fig. 5.1 is
 IF $30 > \text{age} \geq 20$ AND $\text{car type} = \text{sports}$ THEN $\text{driving risk} = \text{high}$.

Tree-based pruning limits the kind of pruning involved. When a node is pruned, all subtrees under it have to be pruned as well. One can also use rule-based pruning. Here, for each leaf of the tree, a rule is extracted using a conjunction of all tests upto the root. Then, based on the validation set, one independently prunes tests from each rule in order to generate the highest accuracy for that rule. The rules are maintained sorted, in decreasing order of their accuracy.

5.2.8 Fusion with neural networks

Decision trees and neural networks are the most commonly used non-parametric tools for pattern classification [20]. While in decision trees the number of tuples becomes smaller as the path between the root node and a new node increases, the decision boundaries of the neural net are formed by considering all the available input tuples as a whole. Hence a neural net can be expected to generate fewer rules, but with larger number of antecedent conditions [21]. In recent years, enormous work has been done in an attempt to combine the advantages of neural networks and decision trees [22]–[24].

Sethi [22] proposed a procedure for mapping a decision tree into a multilayered feed-forward neural network with two hidden layers. The mapping rules are as follows. (a) The number of neurons in the first hidden layer equals to the number of internal nodes in the tree. Each of these neurons implements one of the decision functions of the internal nodes. (b) The number of neurons in the second hidden layer equals the number of leaf nodes in the tree. (c) The number of neurons in the output layer equals to the number of distinct classes. Ivanova and Kubat [23] have directly mapped a decision tree into a three-layered network, such that each conjunction (in a rule) is modeled by a hidden node. The input domain is partitioned into a set of non-overlapping intervals of attributes, which are then mapped to input nodes of the network. Setiono and Liu [24] computed binary training patterns from the decision rules and attribute intervals. These are used to train a feed-forward network, which is then pruned to generate an optimal topology.

Determination of the optimal size of an artificial neural network (ANN) is a problem of considerable importance, as this has a significant impact on the effectiveness of its performance. In general, it is desirable to have small networks. This is because increasing the number of hidden nodes or links may improve the approximation quality of an ANN at the expense of deteriorating its generalization capability (due to the resulting redundancy). One way of improving the generalization behavior of an ANN is to use knowledge-based networks [25, 26]. Decision trees provide a way of extracting crude domain knowledge from a large dataset, which can be directly encoded into an ANN to formulate a knowledge-based network. Further details on knowledge-based networks are provided in Section 8.2.3.

5.3 BAYESIAN CLASSIFIERS

Statistical methods constitute one of the oldest learning paradigms in the literature [27], and work under the assumption that the underlying pattern generating mechanism is faithfully represented by a statistical model. Given that there are l possible hypotheses (pattern classes) $h_1, h_2, \dots, h_l$, and an arbitrary pattern belongs to hypothesis h_i with *a priori* probability $p_i \geq 0$, we have $\sum_{i=1}^l p_i = 1$. A decision rule assigns an unknown pattern $\mathbf{X}$ to a class h , such that the average loss incurred in the process is as small as possible.

5.3.1 Bayesian rule for minimum risk

Bayes' theorem provides a way of computing the probability of a particular event $\mathbf{X}$, given some set of observations. For training pattern or sample $\mathbf{X}$, the *posteriori* probability of a class or hypothesis h , $P(h|\mathbf{X})$, is expressed as

$$P(h|\mathbf{X}) = \frac{P(\mathbf{X}|h)P(h)}{P(\mathbf{X})}, \quad (5.6)$$

where $P(h) = \frac{|h|}{N}$ is the estimated *a priori* probability of h (given $|h|$ and N are the number of patterns in class h and the total number of patterns, respectively, and assuming that all hypotheses are equally likely), $P(\mathbf{X}|h)$ is the *posterior* probability of $\mathbf{X}$ conditioned on h , and $P(\mathbf{X})$ is the prior probability of $\mathbf{X}$ (and is constant).

The maximum posteriori (MAP) hypothesis is used to assign to the class h having maximum $P(h|\mathbf{X})$. It is expressed as

$$h_{MAP} \equiv \arg \max_{h \in H} P(h|\mathbf{X}) = \arg \max_{h \in H} P(\mathbf{X}|h)P(h),$$

where H is the set of hypotheses.

Given the probability distribution of the data, Bayesian classifiers are able to perform efficiently with the minimum error rate [8]. Here the expected loss (or conditional risk) of making a decision is the *minimum*. This statistically optimal classification rule is a generally accepted standard against which the performance of other classification algorithms is often compared. However, the practical difficulties involved include the requirement of initial knowledge about many probabilities and a significant computational cost.

5.3.2 Naive Bayesian classifier

The naive Bayesian classifier represents each pattern $\mathbf{X}$ as an n -dimensional vector of attribute values $[a_1, a_2, \dots, a_n]$. Given that there are l classes $C_1, C_2, \dots, C_l$, the classifier predicts an unknown sample $\mathbf{X}$ as belonging to the class having the highest posterior probability conditioned on $\mathbf{X}$. In other words, $\mathbf{X}$ is assigned to class C_i if and only if

$$P(C_i|\mathbf{X}) > P(C_j|\mathbf{X}) \quad (5.7)$$

for $1 \leq j \leq l$ and $j \neq i$. Using Eq. (5.6), we get

$$P(C_i|X) = \frac{P(X|C_i)P(C_i)}{P(X)}.$$

In order to reduce the computational expenses involved, the classifier makes the naive or simplified assumption that the n attributes are *conditionally independent* of one another. The class-conditional independence is expressed as

$$P(C_i|X) \propto P(C_i) \prod_{j=1}^n P(a_j|C_i).$$

As $P(X)$ is constant for all classes, and $P(C_i) = \frac{|C_i|}{N}$, one needs to maximize just $P(X|C_i)$. Therefore, we compute

$$P(X|C_i) = \prod_{j=1}^n P(a_j|C_i). \quad (5.8)$$

This greatly reduces the computation costs, since it only counts the class distribution.

If the i th attribute is categorical, $P(a_j|C_i)$ is estimated as the relative frequency of samples having value a_j as the j th attribute in class C_i . If, on the other hand, the j th attribute is continuous, then $P(a_j|C_i)$ is generally estimated through a Gaussian density function. Both cases are computationally easy.

When the Gaussian distribution is assumed, we have the normal density functions given by

$$P(X|C_i) = \frac{1}{(2\pi)^{\frac{n}{2}} |\Sigma_i|^{\frac{1}{2}}} \exp \left[-\frac{1}{2} (\mathbf{X} - \mathbf{m}_i)^T \Sigma_i^{-1} (\mathbf{X} - \mathbf{m}_i) \right], \quad (5.9)$$

for $i = 1, 2, \dots, l$, where $\mathbf{m}_i$ and Σ_i are the mean vector and the covariance matrix, respectively, for the i th class.

Bayesian classifiers are very simple, requiring only a single scan of the data, thereby providing high accuracy and speed for large databases. Their performance is comparable to that of decision trees and neural networks. However there arise inaccuracies due to (a) the simplified assumptions involved and (b) a lack of available probability data or knowledge about the underlying probability distribution. Bayesian classifiers have the minimum error rate, unless there are inaccuracies in their assumptions.

5.3.2.1 Example 3: Let Table 5.7 depict the attribute-class information related to a simple classification example, for which one needs to design the naive Bayesian classifier. Consider a numeric two-attribute two-class problem, to predict the classification of a new sample $\mathbf{X} = [a_1, a_2] = [1, 1]$. We compute

Table 5.7 Sample dataset for naive Bayesian classifier

Attributes		Class
a_1	a_2	
1	0	C_1
0	0	C_1
2	1	C_2
1	2	C_2
0	1	C_1

the *a priori* probabilities $P(C_1) = \frac{3}{5}$ and $P(C_2) = \frac{2}{5}$. The conditional probabilities $P(a_j|C_i)$ are estimated, for every attribute value given in the new sample $\mathbf{X}$, as $P(a_1 = 1|C_1) = \frac{1}{3}$, $P(a_1 = 1|C_2) = \frac{1}{2}$, $P(a_2 = 1|C_1) = \frac{1}{3}$, and $P(a_2 = 1|C_2) = \frac{1}{2}$, respectively. Assuming conditional independence of attributes, the conditional probabilities of Eq. (5.8) become

$$P(\mathbf{X}|C_1) = P(a_1 = 1|C_1) * P(a_2 = 1|C_1) = \frac{1}{3} * \frac{1}{3} = \frac{1}{9}$$

and

$$P(\mathbf{X}|C_2) = P(a_1 = 1|C_2) * P(a_2 = 1|C_2) = \frac{1}{2} * \frac{1}{2} = \frac{1}{4}.$$

This is used to evaluate

$$P(C_1|\mathbf{X}) = \frac{1}{9} * \frac{3}{5} = \frac{1}{15}$$

and

$$P(C_2|\mathbf{X}) = \frac{1}{4} * \frac{2}{5} = \frac{1}{10}.$$

Maximizing, we find that $P(C_2|\mathbf{X})$ is larger. Hence the sample $\mathbf{X}$ is predicted to be in class C_2 .

5.3.3 Bayesian belief network

A Bayesian network is a directed acyclic graph whose nodes represent random variables of interests, while the edges reflect the direct (causal) influence. It provides a graphical model of causal relationships, based on which learning can be performed. The conditional probabilities for the nodes are given all possible combinations of their parents. A Bayesian network is also termed a Belief or Probabilistic net [28]. It provides an efficient and effective representation of a probability distribution, in terms of the dependencies among the attributes. The nodes are statistically independent of their nondescendants, given the state of their parents. This implies that one can compute conditional probabilities of nodes, given the observed values of some nodes.

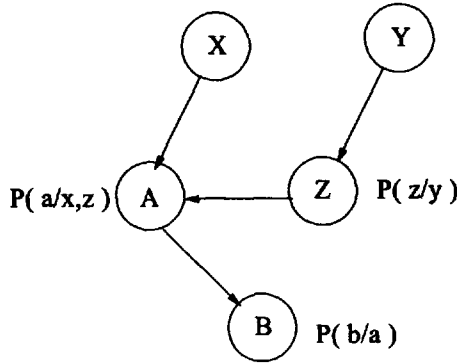


Fig. 5.3 A belief network.

5.3.3.1 Example 4: Figure 5.3 depicts a belief network consisting of five nodes X, Y, Z, A, B . Here X directly influences A , while Y also influences A through Z . Each node has a number of discrete states. For example, node X can take up states $\{x_1, x_2, \dots\}$, collectively denoted as x .

5.4 INSTANCE-BASED LEARNERS

These *nearest neighbor* rules are based on the concept of minimum distance classification from “instances” and can involve either a single prototype or multiple prototypes [8]. When the patterns of a class tend to cluster tightly about a typical representative pattern for that class, we use a single prototype from which to compute the minimum distance. Let $\mathbf{m}_i$ be the prototype for the $i = 1, \dots, l$ classes, such that the distance between an arbitrary pattern vector $\mathbf{X}$ and the i th prototype is given by D_i .

When a non-Euclidean metric like the *Mahalanobis distance* is used, the corresponding minimum distance classifier is termed the Mahalanobis classifier. Here

$$D_i^2 = (\mathbf{X} - \mathbf{m}_i)^T \Sigma_i^{-1} (\mathbf{X} - \mathbf{m}_i), \quad (5.10)$$

where Σ_i is the covariance matrix of the underlying Gaussian (or normal) distribution for class i . When the Euclidean distance is used, we have

$$D_i^2 = \|\mathbf{X} - \mathbf{m}_i\|^2 = \mathbf{X}'\mathbf{X} - 2(\mathbf{m}_i'\mathbf{X} - \frac{1}{2}\mathbf{m}_i'\mathbf{m}_i). \quad (5.11)$$

5.4.1 Minimum distance classifiers

In this category of decision rules, one does not make assumptions of an underlying statistical distribution. The minimum distance classifier computes the distance of an unlabeled pattern $\mathbf{X}$ from the prototype of each class, and

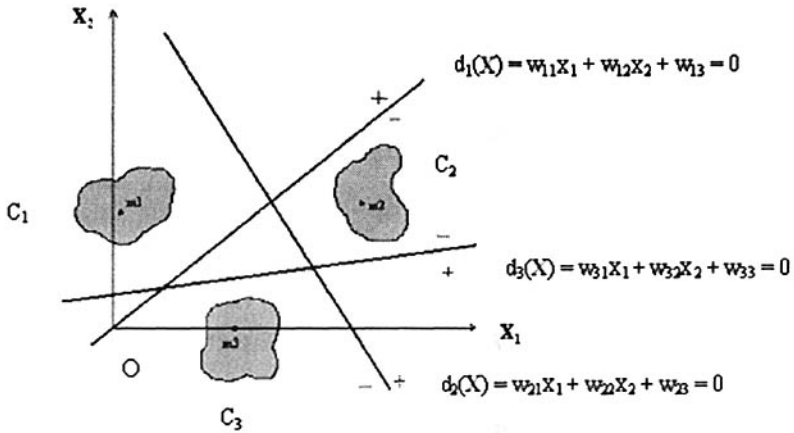


Fig. 5.4 Decision boundaries d_i for a three-class problem.

it assigns the pattern to that class to which it is closest. Ties are resolved arbitrarily. In other words, one generates a linear discriminant function

$$\min(D_i^2) = \max(d_i) = \max(\mathbf{m}_i' \mathbf{X} - \frac{1}{2} \mathbf{m}_i' \mathbf{m}_i),$$

from Eq. (5.11), where the decision boundary is $d_i(\mathbf{X}) = \mathbf{w}_i' \mathbf{X}$, for $i = 1, \dots, l$, such that $w_{ij} = m_{ij}$, $j = 1, \dots, n$, and $w_{i,n+1} = -\frac{1}{2} \mathbf{m}_i' \mathbf{m}_i$. Figure 5.4 illustrates the concept of the minimum distance based decision boundaries, generated for a three-class problem with single prototypes.

When each class is characterized by multiple prototypes, the minimum distance is computed from all of these as before. Let the i th class contain N_i prototypes $\mathbf{m}_i^1, \dots, \mathbf{m}_i^{N_i}$, such that the distance

$$D_i = \min_k \|\mathbf{X} - \mathbf{m}_i^k\| \quad (5.12)$$

for $k = 1, \dots, N_i$. The unlabeled pattern $\mathbf{X}$ is classified into the i th class if $D_i < D_j$ for all $j \neq i$. The piecewise linear discriminant function $d_i = \max_k \{(\mathbf{m}_i^k)' \mathbf{X} - \frac{1}{2} (\mathbf{m}_i^k)' \mathbf{m}_i^k\}$ for $k = 1, \dots, N_i$.

Instance-based learners are also called *lazy* learners because they store all the training samples. This may present difficulties when learning from very large datasets. On the other hand, decision trees and neural networks are termed *eager* learners since they are able to construct a generalization model, before receiving unknown or new samples to classify. Although lazy learners are faster at training, they are slower during application. They incur expensive computational costs when the number of stored samples or instances, with which to compare a given unlabeled sample, is large.

In this framework let us now present the k -nearest neighbor classifier, regression, radial basis function network, and case-based reasoning. The last two can, again, be categorized as soft computing approaches.

5.4.2 k -nearest neighbor (k -NN) classifier

Here all the instances correspond to pattern points in the n -dimensional space. The nearest neighbors are typically defined in terms of Euclidean distance. The target function could be either discrete- or real-valued. In case of discrete values, the algorithm assigns to a point $\mathbf{X}$ the label or class for the majority of its k -nearest neighbors from a sample pattern set $\mathbf{m}_i \in \{\mathbf{m}_1, \dots, \mathbf{m}_{l'}\}$ of known classification, where $l' \geq l$ and each of these instances belongs to one of the l classes. A nearest-neighbor (NN) classification rule assigns an unlabeled pattern $\mathbf{X}$ to the class of its nearest neighbor, where $\mathbf{m}_i \in \{\mathbf{m}_1, \dots, \mathbf{m}_{l'}\}$ is a nearest neighbor to $\mathbf{X}$ if the distance

$$D(\mathbf{m}_i, \mathbf{X}) = \min_j \{D(\mathbf{m}_j, \mathbf{X})\}, \quad (5.13)$$

for $j = 1, 2, \dots, l'$. A Voronoi diagram defines the decision surface induced by a 1-NN for a typical set of training examples. A k -NN ($k > 1$) rule consists of determining the k -nearest neighbors to $\mathbf{X}$ and using the *majority* of equal classifications in this group as the classification of $\mathbf{X}$.

For $k = 1$, the error rate is never worse than twice the Bayes' rate (which holds for unlimited number of samples). For large k , the algorithm is computationally more expensive, as compared to a decision tree or neural network, and requires efficient indexing. Moreover, it assigns equal weight to each attribute, thereby accommodating no feature selection. Different approaches to attain scalability for data mining incorporate (i) indexing to find k -nearest neighbors, (ii) an R-tree family, which works well up to 20 dimensions, (iii) a Pyramid tree for high-dimensional data, and (iv) clustering to reduce the dataset size.

5.4.2.1 Example 5: Figure 5.5 demonstrates the k -NN rule for $k = 5$. Considering the five nearest neighbors around $\mathbf{X}$, marked by the dotted line, we observe that there are three instances (patterns) from class C_2 and two from class C_1 . Taking a majority vote enables assignment of $\mathbf{X}$ to C_2 .

5.4.3 Locally weighted regression

The objective is to learn a new regression equation by weighting each training instance based on its distance from a new instance. The regression equation can be expressed as $\hat{f}(x) = w_0 + w_1 a_1(x) + \dots + w_n a_n(x)$. The algorithm tries to find the w_i s to minimize the error

$$E(X) \equiv \frac{1}{2} \sum_{x \in X} (f(x) - \hat{f}(x))^2.$$

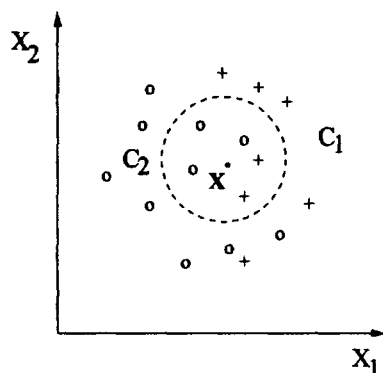


Fig. 5.5 A k -nearest neighbor rule for $k = 5$.

It constructs an explicit approximation of f over a local region surrounding query instance X_q . The target function f is approximated near X_q using the linear function

$$E(X_q) \equiv \frac{1}{2} \sum_{x \in k\text{-NN}_{of}\text{-}X_q} (f(x) - \hat{f}(x))^2 K(D(X_q, x)). \quad (5.14)$$

The squared error is minimized using a distance-decreasing weight K .

5.4.4 Radial basis functions (RBFs)

The RBFs are related to distance-weighted regression and artificial neural networks. In the framework of instance based learners, we can represent Eq. (2.25) as

$$y = f(x) \equiv w_0 + \sum_{j=1}^m w_j K(D(\mu_j, x)), \quad (5.15)$$

where each μ_j is an instance from the training data and $K(D(\mu_j, x))$ is the Gaussian kernel function of Eq. (2.24). Here the contribution of an instance is non-negligible only if the input region falls in the region of the instance. The training of an RBF can be summarized as follows:

- Choose the m kernel points μ_j s and the parameters of K – for example, the variance σ_j of the distribution.
- Choose the weights so as to minimize the error on the training data.

Every instance can be chosen as a kernel point, making all variances the same. Otherwise the instances can be clustered, excluding the target function, by using c -means or expectation maximization (EM)-like clustering algorithm [6] and choosing a representative within each cluster.

5.4.5 Case-based reasoning (CBR)

Cases are the symbolic representation of some typical situations, already experienced by the system. These are more complex than the simple training patterns of nearest-neighbor classifiers in the Euclidean space. The instances are conceptualized pieces of knowledge, representing an experience that teaches a lesson for achieving the goals of the system. CBR involves [9]

- adapting old solutions to meet new demands,
- using old cases to explain new situations or to justify new solutions, and
- reasoning from precedents to interpret new situations.

The system learns and becomes more efficient as a byproduct of its reasoning activity.

Examples of CBR encompass medical diagnosis and law interpretation, where the knowledge available is incomplete and/or evidence is sparse. Unlike traditional knowledge-based systems, a case-based system proceeds through a process of (i) remembering one or a small set of concrete instances or cases and (ii) basing decisions on comparisons between the new situation and the old ones. The basic operations include

- Case selection, where the cases belong to the set of examples encountered, and
- Case generation, where the constructed cases need not be any of the available examples.

Challenges in CBR include finding a good similarity metric, developing efficient techniques for indexing training cases, and methods for combining solutions. Some of the existing case selection methods [29] are k -NN based, using (i) condensed nearest neighbor (CNN), (ii) instance-based learning with case growing or pruning (e.g., IB3), and (iii) instance-based learning with feature weighting (e.g., IB4).

5.4.6 Granular computing and CBR

As mentioned in Section 2.3.2, granular computing [30] is an approach under soft computing. An information granule is a group of similar objects clubbed together by an indiscernibility relation. Granular computing is, therefore, performed using the information granules and not the data points (objects). This helps achieve information compression, leading to a computational gain. Here the cases are considered to be informative patterns or prototypes, characterizing the problem.

In the rough-fuzzy framework used in Ref. [31], cases are represented as information granules or clusters. These involve a reduced number of relevant features with variable size that (i) have less storage requirements, (ii) are

amenable to fast retrieval, and (iii) are suitable for mining data with large dimension and size. Fuzzy sets help in the linguistic representation of patterns, providing a fuzzy granulation of the feature space. Rough sets help in generating dependency rules to model the *informative* or *representative* regions in the granulated feature space. Fuzzy membership functions corresponding to these representative regions are stored as the cases.

5.5 SUPPORT VECTOR MACHINES

The Support Vector Machine (SVM) [32, 10] has recently become very popular as a high-performance classifier in several domains including pattern recognition, data mining and Bioinformatics. It has strong theoretical foundations and a good generalization capability. Another advantage of SVM is that, as a by-product of learning, it obtains a set of support vectors (SVs) which characterizes a given classification task or compresses a labeled dataset. Often the number of SVs is only a small fraction of the original dataset.

The basic idea is to construct a hyperplane as the decision surface such that the margin of separation between positive and negative examples is maximized. The *structural risk minimization* principle is used for the purpose. Here the error rate of a learning machine is considered to be bounded by the sum of the training error rate and a term depending on the Vapnik-Chervonenkis (VC)¹ dimension. Given a labeled set of N training samples (X_i, y_i) , where $X_i \in R^n$ and $y_i \in \{-1, 1\}$, the discriminant hyperplane is defined as

$$f(X_q) = \sum_{i=1}^N y_i \alpha_i K(X_q, X_i) + b. \quad (5.16)$$

Here $K(\cdot)$ is a kernel function and the sign of $f(X_q)$ determines the membership of query sample X_q . Constructing an optimal hyperplane is equivalent to determining all nonzero α_i s, which correspond to the support vectors, and the bias b .

A limitation of the SVM design algorithm, particularly for large datasets, is the need to solve a quadratic programming (QP) problem involving a dense $N \times N$ matrix, where N is the number of points in the dataset. Since most QP routines have quadratic complexity, SVM design requires huge memory and computational time for large data applications. However, approaches also exist for circumventing these shortcomings [33].

A simple method to solve the SVM QP problem has been described by Vapnik [10]. Here a 'chunking' algorithm uses the fact that the solution of the SVM problem remains the same if one removes the points that correspond to

¹The VC dimension of a system is defined as the largest set S of data samples for which the system can implement all possible $2^{|S|}$ dichotomies on S , where dichotomy implies a two-class categorization.

zero Lagrange multipliers of the QP problem (the non-SV points). The large QP problem can thus be decomposed into a series of smaller QP subproblems, whose ultimate goal is to identify all the nonzero Lagrange multipliers (SVs) while discarding the zero Lagrange multipliers (non-SVs). At every step, chunking solves a QP subproblem that consists of (a) the nonzero Lagrange multiplier points from the previous step and (b) a chunk of q other points. At the final step, the entire set of nonzero Lagrange multipliers are identified, thereby solving the large QP problem. Several variations of the algorithm exist depending upon the method of forming the chunks [32]. Chunking greatly reduces the training time as compared to batch learning of SVMs. However, it may not suitably handle large-scale training problems due to slow convergence of the chunking steps, when q new points are chosen randomly.

Active learning has recently become a popular paradigm for reducing the sample complexity of large-scale learning tasks [34]. Instead of learning from randomly selected samples, here the learner has the ability to select its own training data. This is done iteratively, and the output of a step is used to select examples for the next step. In the context of SVMs, active learning has been used to speed up chunking algorithms [35]. The points that split the current version space into two halves having equal volumes are selected at each step, as they are likely to be the active SVs. This active learning strategy queries for a single point at each step. However, a gain in computational time can be obtained by querying multiple instances at a time. A major limitation of this greedy method is that the selection of a new point is influenced only by the current hypothesis (separating hyperplane) available. Hence learning may be severely hampered if (i) a 'bad' example is queried, which drastically worsens the current hypothesis, and (ii) the current hypothesis itself is far from the optimal hypothesis (in the initial phase of learning), so that the examples queried are less likely to be the actual SVs.

5.6 FUZZY DECISION TREES

In this section we embark on a soft computing-based hybridization of decision trees with fuzzy sets. A decision tree that incorporates fuzzy set theoretic concepts at the input, output, and/or node levels can be termed a fuzzy decision tree. The data can be presented in fuzzy terms, the output decision may be provided as fuzzy membership values, and the measure determining the separability (or splitting) at the node level can be fuzzified.

The fusion of fuzzy sets with decision trees enables one to combine the uncertainty handling and approximate reasoning capabilities of the former with the comprehensibility and ease of application of the latter. This enhances the representative power of decision trees *naturally* with the knowledge component inherent in fuzzy logic, leading to better robustness, noise immunity, and applicability in uncertain or imprecise contexts.

Fuzzy decision trees [36] assume that all domain attributes or linguistic variables have predefined fuzzy terms, determined in a data-driven manner using fuzzy restrictions. The impurity factor, used for splitting a node, is modified for fuzzy representation, and a pattern can have nonzero match to one or more leaves. Techniques for the design of fuzzy decision trees have been reported in the literature [36]–[43].

Ichihashi et al. [37] extract fuzzy reasoning rules viewed as fuzzy partitions. An algebraic method to facilitate incremental learning is also employed. The intuitive factor used for splitting a node, which provides the maximal information gain, is defined as

$$Entropy = - \sum_{i=1}^l \frac{\sum_{j=1}^N \mu_{ij}}{N} \log_2 \frac{\sum_{j=1}^N \mu_{ij}}{N}, \quad (5.17)$$

where μ_{ij} is the membership of the j th pattern point to the i th decision class (leaf) and N is the total number of data objects. This uses a normalized version of fuzzy entropy, instead of the classical entropy component involved in the conventional expression of Eq. (5.4). If the relative frequency of a class i is greater than a threshold, then it is termed a leaf and labeled by the corresponding class with probability $\frac{\sum_{j=1}^N \mu_{ij}}{N}$.

Xizhao and Hong [38] discretize continuous attributes using fuzzy numbers and possibility theory. Pedrycz and Sosnowski [39], on the other hand, employ context-based fuzzy clustering for this purpose. Yuan and Shaw [40] induce a fuzzy decision tree by reducing classification ambiguity with fuzzy evidence. The input data are fuzzified using triangular membership functions around cluster centers obtained using Kohonen's feature map [44]. Wang et al. [41] present optimization principles of fuzzy decision trees based on minimizing the total number and average depth of leaves, proving that the algorithmic complexity of constructing a minimum tree is NP-hard. Fuzzy entropy and classification ambiguity are minimized at node level, and fuzzy clustering is used to merge branches.

Fuzzy knowledge-based networks [45, 46] typically incorporate fuzziness at the network level, using fuzzy neural networks. We now describe the formulation of a fuzzy knowledge-based network using fuzzy ID3 [11, 12]. Quantitative measures are defined to evaluate the effectiveness of the fuzzy decision tree and the linguistic rules. The novel concept of tree evaluation, in terms of its compactness and performance, enables extraction of only meaningful (less ambiguous) rules. A smaller or compact tree is more efficient in terms of both storage and time requirements, tends to generalize better to unknown test cases, and leads to the generation of more comprehensible linguistic rules. Quantitative evaluation of the linguistic rules not only minimizes human intervention, but also provides aids for knowledge discovery.

Discretization of continuous attributes, based on the distribution of pattern points in the feature space, is made in linguistic terms using quantiles. Unlike other fuzzy decision trees [36], this discretization to boolean form helps

in reducing the computational complexity while preserving the linguistic nature of the decision in rule form. New fuzziness measures, in terms of class memberships, are used at the node level of the tree to take care of overlapping classes. The extracted rules are mapped onto a fuzzy knowledge-based network. Unlike [22]–[24], the frequency of samples (representative of a rule) and the depth of the attributes in the decision tree are incorporated during the mapping. The effectiveness of the model is demonstrated on benchmark datasets [12].

5.6.1 Classification

Here we describe the incorporation of fuzziness at the input, output, and node levels of the fuzzy ID3 [12], to handle different forms of uncertainty. Input attributes are automatically discretized in linguistic terms, based on the distribution of pattern points in the feature space. Different forms of fuzzy entropy are computed at the node level, in terms of class membership, to take care of overlapping classes. Pruning is used to minimize noise, resulting in a smaller decision tree with more efficient classification. A new metric, called *T*-measure, is developed to evaluate the decision tree both in terms of performance and size.

5.6.1.1 Fuzziness at input Any input feature value is described in terms of some combination of overlapping membership values in the linguistic property sets *low* (L), *medium* (M), and *high* (H). An n -dimensional pattern $\mathbf{X}_i = [a_1, a_2, \dots, a_n]$ is represented as a $3n$ -dimensional vector [47]

$$\mathbf{X}_i = [\mu_{low(a_1)}(\mathbf{X}_i), \mu_{medium(a_1)}(\mathbf{X}_i), \mu_{high(a_1)}(\mathbf{X}_i), \dots, \mu_{high(a_n)}(\mathbf{X}_i)], \quad (5.18)$$

where the μ values indicate the membership functions of the corresponding linguistic functions *low*, *medium*, and *high* along each feature axis. Each μ value is then discretized, using a threshold (generally 0.5), to enable a convenient mapping in the ID3 framework. This discretization to boolean form speeds up computation by reducing the complexity of the search space. However, the linguistic flavor of the attributes is retained, thereby enabling the extraction of more user-friendly *natural* rules that are then mapped to the fuzzy knowledge-based network.

When the input feature is numerical, we divide it into three partitions (with range $[0, 1]$) using only two parameters P_{j1} and P_{j2} as depicted in Fig. 5.6. Features in linguistic and set forms can also be handled. *Quantiles* or *partition values*² [48] are used in order to minimize the influence of extreme values or noisy patterns.

²Quantiles or partition values are the values of a variate which divide the total frequency into a number of equal parts.

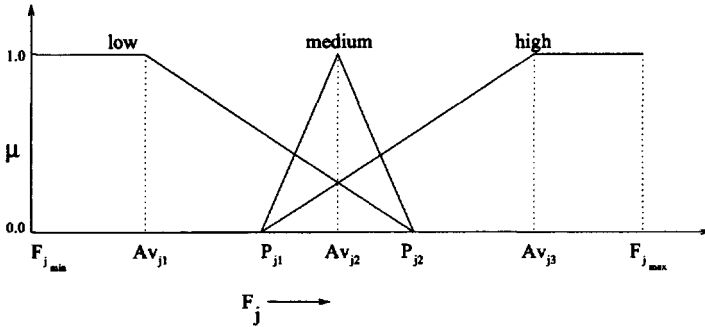


Fig. 5.6 Linguistic input membership functions.

Let $F_{j\max}$ and $F_{j\min}$ denote the maximum and minimum values encountered along feature F_j considering N training patterns $F_{1j}, F_{2j}, \dots, F_{Nj}$. Let these patterns be sorted in the ascending order of their values along the j th axis. The first quantile (P_{j1}) is the value of F_j that exceeds one-third of the measurements and is less than the remaining two-thirds. The second quantile (P_{j2}) is the value of F_j that exceeds two-thirds of the measurements and is less than the remaining one-third. In order to determine the two quantiles, we divide the measurements into a number of small class intervals of equal width δ and count the corresponding class frequencies f_i . The position of the k th partition value (here quantile, as $k = 1, 2$ for three partitions) is calculated as

$$P_{jk} = l_i + \frac{R_k - cf_{i-1}}{f_i} \cdot \delta, \quad (5.19)$$

where l_i is the lower limit of the i th class interval, $R_k = \frac{N \cdot k}{3}$ is the rank of the k th partition value, and cf_{i-1} is the cumulative frequency of the immediately preceding class interval, such that $cf_{i-1} \leq R_k \leq cf_i$. Then, from Fig. 5.6, we have

$$Av_{j1} = \frac{F_{j\min} + P_{j1}}{2}, \quad Av_{j2} = \frac{P_{j1} + P_{j2}}{2}, \quad \text{and} \quad Av_{j3} = \frac{P_{j2} + F_{j\max}}{2}.$$

The membership values of a pattern along the j th axis, in the corresponding three-dimensional linguistic space of Eq. (5.18), is computed as [12]

$$\mu_{low(a_j)}(\mathbf{X}_i) = \begin{cases} 1 & \text{for } F_{ij} < Av_{j1} \\ \frac{P_{j2} - F_{ij}}{P_{j2} - Av_{j1}} & \text{for } Av_{j1} \leq F_{ij} < P_{j2} \\ 0 & \text{otherwise,} \end{cases} \quad (5.20)$$

$$\mu_{medium(a_j)}(\mathbf{X}_i) = \begin{cases} 0 & \text{for } F_{ij} < P_{j1} \\ \frac{Av_{j2} - F_{ij}}{Av_{j2} - P_{j1}} & \text{for } P_{j1} \leq F_{ij} < Av_{j2} \\ \frac{P_{j2} - F_{ij}}{P_{j2} - Av_{j2}} & \text{for } Av_{j2} \leq F_{ij} < P_{j2} \\ 0 & \text{otherwise,} \end{cases} \quad (5.21)$$

$$\mu_{high(a_j)}(\mathbf{X}_i) = \begin{cases} 0 & \text{for } F_{ij} < P_{j1} \\ \frac{F_{ij} - P_{j1}}{Av_{j3} - P_{j1}} & \text{for } P_{j1} \leq F_{ij} < Av_{j3} \\ 1 & \text{otherwise.} \end{cases} \quad (5.22)$$

5.6.1.2 Output membership and fuzzy entropy Consider an l -class problem domain. The membership of the i th pattern in class k , lying in the range $[0, 1]$, is defined as [47]

$$\mu_{ik}(\mathbf{X}_i) = \frac{1}{1 + (\frac{z_{ik}}{f_d})^{f_e}}, \quad (5.23)$$

where z_{ik} is the weighted distance of the training pattern $\mathbf{X}_i$ from class C_k , and the positive constants f_d and f_e are the denominational and exponential fuzzy generators controlling the amount of fuzziness in the class membership set.

Fuzziness is incorporated into the ID3 algorithm at the node level by modifying the conventional decision function, with classical Shannon entropy, by the inclusion of different fuzzy measures. The fuzzy entropy considers the membership of a pattern to a class and helps enhance the discriminative power of an attribute. In order to reduce the effect of noise or exceptions, a node is pruned depending on the number of patterns reaching it. For this purpose, a threshold t is defined as a lower bound on the fraction of patterns allowed in an existing node.

Let us now provide the different fuzzy entropy or fuzziness measures, denoted as *cases a, b, d, and e*, respectively, investigated at the node level of the decision tree. Note that μ_{ij} , the membership of the j th pattern to the i th class, is calculated by Eq. (5.23) and p_k is the *a priori* probability of the k th class. Comparison is provided with *cases c* [37] and *f* [49].

Case a [12]:

$$Entropy = - \sum_{i=1}^l p_i \log_2 p_i - \frac{1}{N} \sum_{i=1}^l \sum_{j=1}^N [\mu_{ij} \log_2 \mu_{ij} + (1 - \mu_{ij}) \log_2 (1 - \mu_{ij})]. \quad (5.24)$$

The first term on the right is the classical entropy of Eq. (5.4), while the second term corresponds to fuzzy entropy [47].

Case b: Same as *Case a*, but without pruning.

Case c: Method of Ichihashi et al. using Eq. (5.17).

Case d [12]: *Entropy* =

$$- \sum_{i=1}^l \frac{\sum_{j=1}^N \mu_{ij}}{N} \log_2 \frac{\sum_{j=1}^N \mu_{ij}}{N} - \frac{1}{N} \sum_{i=1}^l \sum_{j=1}^N [\mu_{ij} \log_2 \mu_{ij} + (1 - \mu_{ij}) \log_2 (1 - \mu_{ij})]. \quad (5.25)$$

This is an amalgamation of the two forms of fuzzy entropy, where the first term on the right corresponds to Eq. (5.17) and the second term relates to the fuzzy entropy part of Eq. (5.24).

Case e [11]:

$$Entropy = - \sum_{i=1}^l p_i \log_2 p_i - \frac{1}{N} \sum_{i=1}^l \sum_{j=1}^N \min(\mu_{ij}, 1 - \mu_{ij}). \quad (5.26)$$

Here the first term on the right is the classical entropy of Eq. (5.4), while the second term corresponds to a fuzzy measure of the ambiguity present. When the class membership values are zero or one (crisp), the expression boils down to that of classical entropy. The fuzzy measure ensures that pattern points lying in overlapping regions are assigned lower weights during the construction of the decision tree; this is intuitively appealing. The reason for this lower weighting is that such ambiguous patterns (having μ values close to 0.5) lead to an increase in the entropy; thereby placing an impedance to its minimization.

Case f: Conventional ID3 algorithm, using Eq. (5.4).

5.6.1.3 Performance measure for decision tree Decision trees generated by different fuzzy entropy measures may vary in size and structure, and this influences the performance of both the tree and the rules extracted from it. In order to evaluate the efficiency of a decision tree the *T-measure* is introduced [12], keeping in view the following issues:

- The less the depths of the leaf nodes of the tree, the better it is since it takes less time to reach a decision.
- The existence of unresolved terminal nodes is undesirable.
- The distribution of labeled leaf nodes at different depths affects the performance of the tree; a tree whose frequently accessed leaf nodes are at lower depths is more efficient in terms of time.

DEFINITION 5.6.1 The **T-measure**, T , for a decision tree is defined as

$$T = \frac{2n - \sum_{i=1}^{N_{nodes}} w_i d_i}{2n - 1}, \quad (5.27)$$

where

$$w_i = \begin{cases} \frac{N_i}{N} & \text{for a resolved leaf node} \\ \frac{2N_i}{N} & \text{otherwise,} \end{cases} \quad (5.28)$$

n is the number of binary attributes of a pattern, d_i is the depth of a leaf node, N_{nodes} is the number of terminal (leaf or unresolved) nodes, N is the total number of patterns in the training set, and N_i is the total number of training set patterns that percolate down to the i th leaf node. The value of T lies in the interval $[0, 1)$. A value 0 for T is undesirable, and a value close to 1 signifies a good decision tree.

Let us demonstrate the evaluation of the T -measure with an example. Consider a two-class problem, with two-dimensional patterns. Let Fig. 5.7 depict

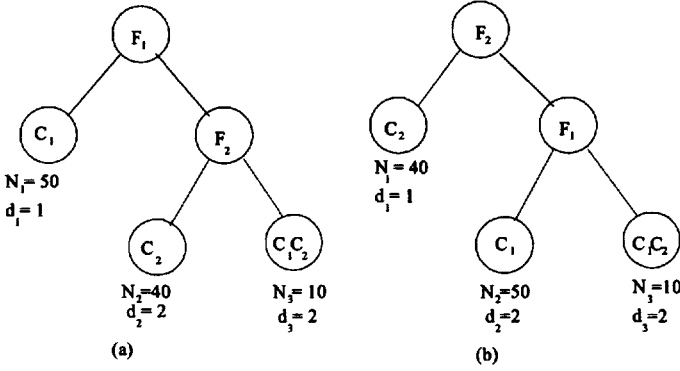


Fig. 5.7 Example demonstrating T -measure computation.

two decision trees generated by two different algorithms. For the decision tree in Fig. 5.7(a), $T = \frac{2 \times 2 - 0.5 \times 1 - 0.4 \times 2 - 0.2 \times 2}{2 \times 2 - 1} = 0.77$, while for the decision tree in Fig. 5.7(b) one obtains $T = \frac{2 \times 2 - 0.5 \times 2 - 0.4 \times 1 - 0.2 \times 2}{2 \times 2 - 1} = 0.73$. Hence we observe that the first decision tree is better than the second, since the fraction of patterns in the node at depth one is more in the first case.

Theorem. The value of T -measure lies within 0 and 1, that is, $0 \leq T < 1$.

Proof: Let us first establish the upper limit. By Eq. (5.28), we have

$$w_i \geq \frac{N_i}{N}, i = 1, 2, \dots, N_{\text{nodes}} \quad (5.29)$$

and

$$d_i \geq 1, i = 1, 2, \dots, N_{\text{nodes}}. \quad (5.30)$$

Hence

$$\sum_{i=1}^{N_{\text{nodes}}} w_i d_i \geq \sum_{i=1}^{N_{\text{nodes}}} \frac{N_i}{N}.$$

Since $\sum_{i=1}^{N_{\text{nodes}}} N_i = N$, one obtains

$$\sum_{i=1}^{N_{\text{nodes}}} w_i d_i \geq 1.$$

So

$$2n - \sum_{i=1}^{N_{\text{nodes}}} w_i d_i < 2n - 1,$$

that is,

$$T = \frac{2n - \sum_{i=1}^{N_{\text{nodes}}} w_i d_i}{2n - 1} < 1. \quad (5.31)$$

Now we check the lower bound for T . We have

$$w_i \leq \frac{2N_i}{N}, \quad i = 1, 2, \dots, N_{\text{nodes}} \quad (5.32)$$

and

$$d_i \leq n, \quad i = 1, 2, \dots, N_{\text{nodes}}. \quad (5.33)$$

Hence

$$\sum_{i=1}^{N_{\text{nodes}}} w_i d_i \leq \sum_{i=1}^{N_{\text{nodes}}} \frac{2N_i n}{N} = 2n,$$

that is,

$$0 \leq 2n - \sum_{i=1}^{N_{\text{nodes}}} w_i d_i,$$

that is,

$$0 \leq \frac{2n - \sum_{i=1}^{N_{\text{nodes}}} w_i d_i}{2n - 1} = T. \quad (5.34)$$

Thus one obtains

$$0 \leq T < 1. \quad (5.35)$$

5.6.2 Rule generation and evaluation

Here we explain the algorithm for extracting domain knowledge, in the form of rules, using the decision tree generated by the fuzzy ID3 [12]. As explained in Section 5.2.7, the path from the root to a leaf is traversed to generate the rule corresponding to a pattern from that class. In this manner, one obtains a set of rules for all the pattern classes, in the form of intersection of the features or attributes encountered along the traversal paths. The i th attribute is marked as A_i or $\bar{A}_i$ depending on whether the traversal is made along the right or left branch. Each rule is marked by its frequency – that is, the number of pattern points reaching this leaf node. Note that each leaf node that has pattern points corresponding to only one class is termed *resolved*.

5.6.2.1 Example 6: The scheme of extracting the rules from the decision tree is demonstrated with an example. Suppose the training set consists of 21 patterns, from three pattern classes, with three features F_1 , F_2 , and F_3 . After splitting each feature into the three linguistic variables *low*, *medium*, and *high* by Eq. (5.18), one obtains the nine-dimensional symbolic features $L_1, M_1, H_1, L_2, M_2, H_2, L_3, M_3, H_3$. Let the sample decision tree be shown in Fig. 5.8, and let the extracted rules be

1. $\bar{L}_1 \wedge \bar{H}_3 \rightarrow C_1; 2,$
2. $L_1 \wedge \bar{M}_3 \wedge \bar{M}_1 \rightarrow C_2; 6,$

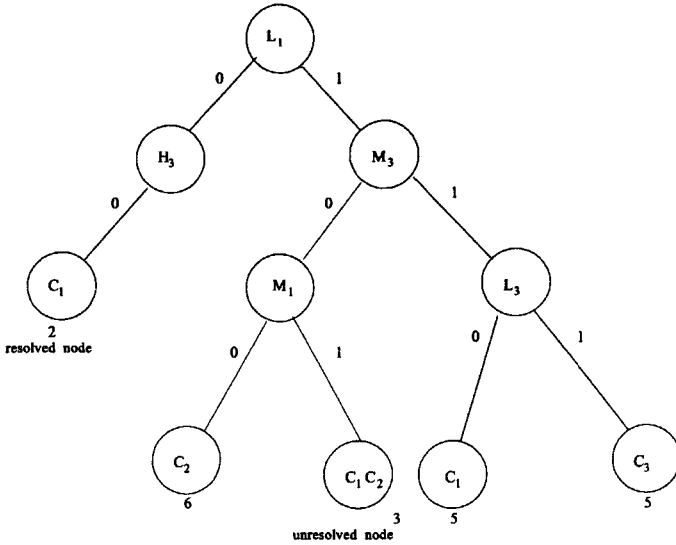


Fig. 5.8 Sample decision tree for rule generation.

$$3. L_1 \wedge \overline{M}_3 \wedge M_1 \rightarrow C_1, C_2; 3,$$

$$4. L_1 \wedge M_3 \wedge \overline{L}_3 \rightarrow C_1; 5,$$

$$5. L_1 \wedge M_3 \wedge L_3 \rightarrow C_3; 5.$$

5.6.2.2 Quantitative measures for rule evaluation The quantitative indices accuracy, user's accuracy, kappa, and confusion, as explained in Section 2.4.2 with reference to ANNs, are used for evaluating the extracted rules. A new measure to estimate the coverage of these rules is designed in the context of decision trees [12].

DEFINITION 5.6.2 Coverage is defined as the ratio between the total number of patterns associated with the rules corresponding to resolved leaf nodes, and the total number of patterns in all the rules and hence the terminal (resolved and/or unresolved) nodes.

When the rules can perfectly classify all the patterns, then coverage is 1, and when they cannot classify any pattern, then it is 0.

5.6.2.3 Example 7: For example, from Fig. 5.8 we have

$$\text{Coverage} = \frac{2 + 6 + 5 + 5}{2 + 6 + 3 + 5 + 5} = \frac{18}{21}.$$

5.6.3 Mapping of rules to fuzzy neural network

In this section we describe the mapping of the extracted rules to generate an optimal fuzzy knowledge-based neural network. The $3n$ -dimensional input vector of Eq. (5.18) is clamped at the input layer to the input nodes $[y_1^0, y_2^0, \dots, y_{3n}^0]$. Here $y_1^0, \dots, y_{3n}^0$ refer to the activation values of the $3n$ neurons in the input layer of the network depicted in Fig. 2.3. The l -dimensional output vector, in terms of class membership values (μ) of patterns [Eq. (5.23)], is clamped at the l nodes in the output layer of the MLP. During training, the weights are updated by backpropagating errors with respect to these membership values such that the contribution of uncertain or ambiguous pattern vectors is automatically reduced.

Let r_{ki} be the i th rule for class C_k with frequency f'_{ki} . Each rule is mapped using a single hidden node, modeling the conjunct, that connects the attributes corresponding to the appropriate pattern class. Therefore, one generates at least l hidden nodes in a single hidden layer for an l -class problem. For simplicity, rules involving only one class (pertaining to leaves) are selected and those corresponding to unresolved nodes of the decision tree are discarded. If there are two rules for a single class C_k , then that rule with the highest frequency is considered. Hence we use only l hidden nodes to model l classes. This constraint can of course be relaxed to incorporate other rules, albeit at the cost of increasing the size and computational complexity of the resultant network.

5.6.3.1 Example 8: The sample rules generated from Fig. 5.8 thus reduce to

1. $L_1 \wedge \overline{M}_3 \wedge \overline{M}_1 \rightarrow C_2; 6,$
2. $L_1 \wedge M_3 \wedge \overline{L}_3 \rightarrow C_1; 5,$
3. $L_1 \wedge M_3 \wedge L_3 \rightarrow C_3; 5.$

5.6.3.2 Network encoding These rules are used to initially encode an MLP, which then learns in the presence of training data. It is to be noted that these rules just serve as representatives, describing the major characteristics of the pattern classes, and as the starting point of the MLP for further learning. Therefore, the representative rulebase need not be too detailed or accurate; rather, a crude knowledge is sufficient to initiate the training procedure. This is the reason for sacrificing accuracy at the expense of simplicity at the decision tree level, by pruning the nodes and limiting the size of the extracted rulebase. The generalization aspect and other intricacies of the decision boundary are handled after the network mapping phase, during neural learning.

The weight w_{ki}^1 , between output node k (class C_k) and hidden node i (rule r_{ki}) is set at $W = \frac{f'_{ki}}{\sum_k f'_{ki}}$ where $f'_{ki} = 1$. This indicates the importance of this rule among all other rules determining the whole network. The scheme for mapping weight $w_{ia_j}^0$ between attribute a_j (L_j or M_j or H_j) and hidden

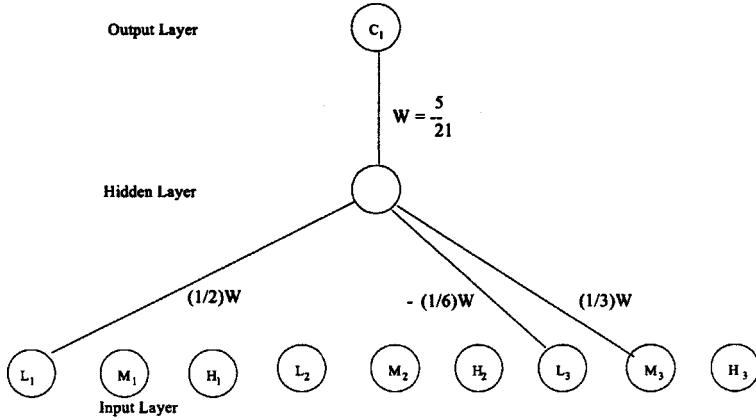


Fig. 5.9 Weight encoding scheme.

node i depends on the importance of feature a_j in the corresponding decision tree. Consider Fig. 5.8. We note from sample rule (2) that the attributes are selected in the order L_1, M_3, L_3 for class C_1 . So the weight $w_{ia_j}^0$ is assigned a value $\frac{2[Card(r_{ki})-j+1]}{Card(r_{ki})[Card(r_{ki})+1]} * W$, where $Card(r_{ki})$ indicates the number of features or attributes encountered along the traversal path from the root to the leaf containing the corresponding pattern. In other words, $Card(r_{ki})$ is the number of operands in the conjunct of rule r_{ki} for class C_k . It is to be noted that

$$\sum_{i=1}^{Card(r_{ki})} \left| \frac{2[Card(r_{ki})-j+1]}{Card(r_{ki})[Card(r_{ki})+1]} \right| * W = W. \quad (5.36)$$

5.6.3.3 Example 9: An example illustrating this scheme is provided in Fig. 5.9 for class C_1 . Sequentially, for $j = 1, 2, 3$, we obtain

$$w_{1L_1}^0 = \frac{2[3-1+1]}{3[3+1]} * W = \frac{1}{2}W,$$

$$w_{1M_3}^0 = \frac{2[3-2+1]}{3[3+1]} * W = \frac{1}{3}W$$

and

$$w_{1L_3}^0 = \frac{2[3-3+1]}{3[3+1]} * W = -\frac{1}{6}W.$$

Here the negative sign in the last expression takes care of the negation of the corresponding attribute L_3 .

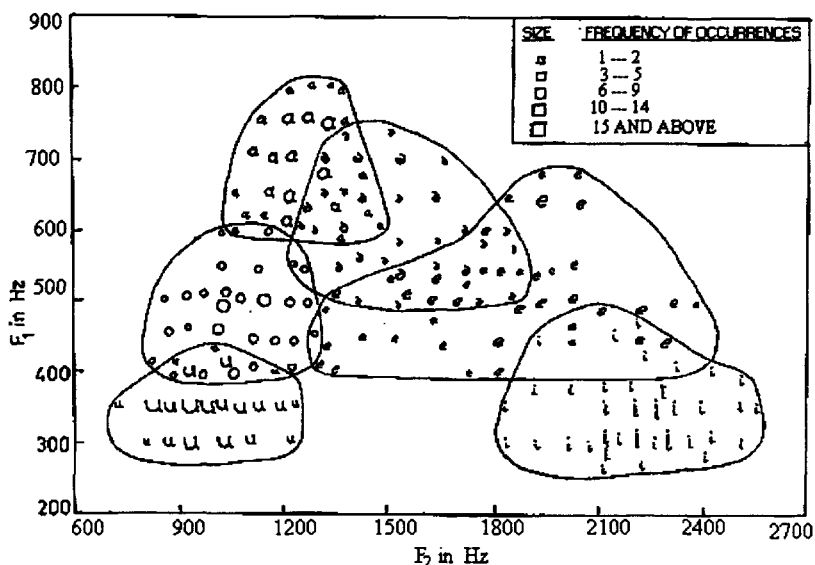


Fig. 5.10 Vowel diagram in F_1 - F_2 plane.

5.6.4 Results

The system was implemented on benchmark datasets, the detailed results of which are available in Ref. [12]. Different sizes of training sets were selected at random, and in each case the remaining data were kept aside as the test set. All results were averaged over 40 runs. The threshold t for pruning a node of the decision tree was set at 0.2 after several experiments.

In this section we provide sample results on two datasets, namely, *Vowel* and *Balance scale*. The *Vowel* data consists of a set of 871 Indian Telugu vowel sounds [47], uttered by three male speakers in the age group of 30 to 35 years, in a Consonant-Vowel-Consonant context. The three features F_1 , F_2 , and F_3 correspond to the first, second, and third vowel formant frequencies obtained through spectrum analysis of the speech data. Figure 5.10 shows a 2D projection of the 3D feature space of the six vowel classes ∂ , a , i , u , e , o in the F_1 - F_2 plane, for ease of depiction. The boundaries of the classes in the given data set are ill-defined (fuzzy).

The *Balance scale* data [50] consists of 625 instances generated to model psychological experimental results. There are four numeric attributes corresponding to the *left weight*, *left distance*, *right weight*, and *right distance*, and there are three output classes, namely, *tip right*, *tip left*, and *balanced*.

The fuzzy ID3 (Table 5.8) is used, for the different fuzzy measures involving cases a to f , to extract rules that are then quantitatively evaluated (Tables 5.9 and 5.10). These are then used for generating fuzzy knowledge-

Table 5.8 Performance of fuzzy ID3

Case	Train set (%)	Vowel			Balance scale		
		Recognition score (%)		T	Recognition score (%)		T
		Training	Testing		Training	Testing	
a	10	81.1	72.8	.70	89.8	79.4	.81
	20	77.6	71.9	.69	86.0	80.7	.78
	30	75.9	72.6	.68	83.2	80.9	.78
	40	74.7	71.0	.67	84.1	81.1	.78
	50	74.9	73.1	.67	81.9	79.8	.76
b	10	77.2	70.7	.53	92.4	80.2	.52
	20	73.5	68.8	.53	85.6	76.9	.52
	30	69.9	64.5	.53	79.7	76.6	.52
	40	65.6	62.8	.53	77.3	74.7	.52
	50	63.0	61.6	.53	74.3	70.9	.52
c	10	79.5	70.4	.70	90.1	75.2	.52
	20	76.1	70.0	.69	84.5	76.0	.52
	30	74.8	70.4	.70	80.6	75.9	.52
	40	74.1	70.6	.69	76.1	73.9	.52
	50	73.2	73.1	.67	75.1	71.7	.52
d	10	79.0	70.3	.71	89.8	77.6	.52
	20	76.3	71.5	.71	85.2	76.4	.52
	30	74.2	70.3	.69	78.9	75.9	.52
	40	74.5	71.1	.69	76.6	74.1	.52
	50	74.1	70.3	.69	73.0	72.4	.52
e	10	66.0	63.6	.64	71.1	65.4	.75
	20	61.3	64.5	.61	64.7	67.5	.71
	30	58.1	62.3	.60	69.8	68.7	.70
	40	58.9	60.4	.60	68.5	67.0	.68
	50	58.8	60.7	.60	69.2	68.5	.69
f	10	63.3	52.7	.64	71.0	63.8	.75
	20	57.7	51.4	.61	65.7	68.1	.72
	30	57.1	48.1	.60	69.6	68.8	.72
	40	57.2	48.4	.60	69.0	69.1	.70
	50	56.5	44.5	.59	72.7	68.2	.70

Table 5.9 Quantitative measures for evaluating rules in Vowel data

Case	Train set (%)	Accuracy (%)	User's accuracy (%)	Kappa	Confusion	Coverage
<i>a</i>	10	63.20	72.67	0.67	2.30	0.80
	20	62.70	73.15	0.67	2.37	0.78
	30	59.74	75.47	0.70	2.37	0.72
	40	59.62	77.14	0.72	2.20	0.73
	50	60.06	75.40	0.70	2.34	0.75
<i>b</i>	10	60.33	71.45	0.65	2.54	0.76
	20	60.14	73.58	0.67	2.24	0.79
	30	60.52	77.63	0.72	2.26	0.73
	40	59.46	79.36	0.75	2.38	0.69
	50	60.18	80.65	0.77	2.06	0.72
<i>c</i>	10	70.81	87.14	0.85	1.78	0.78
	20	66.90	84.66	0.81	2.04	0.75
	30	65.59	84.08	0.81	2.37	0.75
	40	62.62	84.20	0.81	2.32	0.72
	50	60.92	83.00	0.80	2.14	0.70
<i>d</i>	10	71.55	86.45	0.83	1.84	0.77
	20	65.26	84.80	0.82	2.07	0.74
	30	62.04	81.42	0.78	2.27	0.72
	40	62.22	82.82	0.80	2.26	0.71
	50	59.22	83.22	0.80	2.20	0.68
<i>e</i>	10	60.69	77.22	0.72	2.19	0.71
	20	59.70	71.80	0.67	1.89	0.60
	30	53.59	74.64	0.70	1.93	0.54
	40	53.39	73.95	0.69	1.99	0.52
	50	53.27	72.71	0.68	1.92	0.52
<i>f</i>	10	60.36	63.00	0.57	2.13	0.73
	20	57.15	66.00	0.60	2.29	0.60
	30	57.10	66.00	0.61	2.11	0.54
	40	55.72	67.00	0.61	1.97	0.50
	50	55.67	67.00	0.61	2.16	0.47

Table 5.10 Quantitative measures for evaluating rules in *Balance* data

Case	Train set (%)	Accuracy (%)	User's accuracy (%)	Kappa	Confusion	Coverage
<i>a</i>	10	84.76	91.72	0.87	1.73	0.92
	20	78.80	89.93	0.85	1.65	0.86
	30	75.48	88.93	0.83	1.80	0.83
	40	76.26	89.15	0.84	1.85	0.84
	50	73.44	88.62	0.84	1.77	0.80
<i>b</i>	10	86.13	97.41	0.96	2.38	0.88
	20	77.06	98.47	0.97	2.62	0.77
	30	70.13	99.75	1.00	2.93	0.70
	40	66.04	99.60	0.99	2.92	0.66
	50	62.34	100.00	1.00	3.00	0.62
<i>c</i>	10	81.78	89.37	0.83	1.78	0.90
	20	77.04	90.34	0.85	1.68	0.84
	30	73.96	88.70	0.83	1.75	0.81
	40	74.14	87.71	0.81	1.58	0.82
	50	73.37	85.98	0.79	1.68	0.83
<i>d</i>	10	73.94	87.01	0.81	1.52	0.82
	20	77.12	89.68	0.84	1.68	0.84
	30	74.52	87.98	0.81	1.60	0.82
	40	74.10	88.65	0.84	1.70	0.81
	50	73.94	87.01	0.81	1.52	0.82
<i>e</i>	10	67.51	71.78	0.53	1.36	0.91
	20	78.92	88.45	0.82	1.70	0.87
	30	75.59	89.20	0.84	1.75	0.83
	40	74.68	87.90	0.83	1.77	0.83
	50	74.36	87.30	0.82	1.80	0.83
<i>f</i>	10	78.00	89.08	0.83	1.72	0.85
	20	77.49	88.34	0.83	1.70	0.86
	30	75.86	88.42	0.83	1.70	0.84
	40	73.17	88.18	0.83	1.82	0.80
	50	67.51	71.78	0.53	1.36	0.91

based network (Table 5.11). The classification performance is provided for all three stages. Generally the knowledge-based networks result in better performance, in terms of network size and recognition scores. This is natural since the crude domain knowledge is encoded and further refined here, in the presence of training data.

Table 5.11 Performance of knowledge encoded MLP

Dataset	Model	Train set (%)	Recognition scores (%)		No. of links	No. of cycles
			Training	Testing		
Vowel	MLP	10	93.3	78.9	90.5	242
		20	87.5	79.6	91.2	203
		30	82.6	79.4	91.4	106
		40	76.8	77.6	91.8	27
	Knowledge-encoded MLP	10	88.7	78.5	63.6	102
		20	86.0	80.3	68.3	50
		30	82.9	80.2	66.5	35
		40	80.8	79.0	64.2	25
Balance scale	MLP	10	98.3	82.8	46.2	309
		20	91.8	87.5	47.8	331
		30	91.8	87.8	48.4	307
		40	91.3	88.6	47.1	312
	Knowledge-encoded MLP	10	94.8	82.7	39.4	277
		20	91.9	87.3	37.0	272
		30	91.3	89.1	35.7	252
		40	90.0	88.9	38.8	247

Knowledge encoding using linguistic rules extracted from the fuzzy decision tree generally enhances the performance of the knowledge-based system in terms of both network compactness and recognition scores. It is typically observed that the value of T decreases with an increase in size of the training data. This is because an increase in training set size leads to the consideration of a larger number of both noisy and good samples during the decision tree generation. The former influences the formation of a tree of greater depth, with an increased possibility of larger number of unresolved nodes, leading to a lower value of T .

5.7 CONCLUSIONS AND DISCUSSION

In this chapter we have described classification from the perspective of data mining. Different decision tree classifiers have been presented, followed by statistical models like the Bayesian classifier. Next we dealt with Instance-based learners – like the minimum distance classifier, nearest-neighbor classi-

fier, radial basis function, and case-based reasoning – and the Support Vector Machine. Finally, fuzzy decision trees are described.

Greedy splitting heuristics are efficient and adequate for most decision tree applications, but are essentially suboptimal. Crisp decisions that the trees usually output may not be adequate or useful in some settings. Therefore, sometimes results from multiple decision trees are combined to improve upon generalization accuracy. Current research issues involved with decision trees include (i) multiple splits on continuous attributes, using multi-interval discretization of continuous attributes [51], (ii) multi-attribute tests on nodes to handle correlated attributes, using multivariate linear splits as in oblique trees [1], (iii) methods of handling missing values, by either assuming the majority value, or taking the most probable path, (iv) allowing varying costs for different attributes, and (v) generation of *optimal* trees.

The design of a fuzzy knowledge-based network, based on linguistic rules extracted from a fuzzy decision tree [12], has also been described. Its major contributions include: (a) developing a new scheme for automatic linguistic discretization of continuous attributes using quantiles, (b) introducing the novel concept of a quantitative measure T to evaluate the goodness of the decision tree, in terms of its compactness and performance, (c) evaluating quantitatively the extracted linguistic rules with some new indices, (d) mapping the linguistic rules to a fuzzy knowledge-based network, incorporating frequency of samples and depth of attributes in the decision tree, and (e) using new fuzziness measures at the node level of the tree, to handle overlapping classes.

In this soft computing framework we integrate the generic merits of decision trees, like efficient exploration of large data, along with the uncertainty handling capability of fuzzy sets. It enhances the understandability of a decision by providing rules in linguistic form. Thereby, it improves the human-machine interaction in a system.

In the following chapter we move on to another major function of data mining, namely, unsupervised learning or clustering.

REFERENCES

1. S. K. Murthy, "Automatic construction of decision trees from data: A multi-disciplinary survey," *Data Mining and Knowledge Discovery*, vol. 2, pp. 345–389, 1998.
2. J. R. Quinlan, "Induction on decision trees," *Machine Learning*, vol. 1, pp. 81–106, 1986.
3. S. Rasoul Safavian and D. Landgrebe, "A survey of decision tree classifier methodology," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 21, pp. 660–674, 1991.

4. L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Monterey, CA: Wadsworth and Brooks/Cole, 1984.
5. J. R. Quinlan, *C4.5: Programs for Machine Learning*. San Mateo, CA: Morgan Kaufmann, 1993.
6. J. Han and M. Kamber, *Data Mining: Concepts and Techniques*. San Diego: Academic Press, 2001.
7. S. Haykin, *Neural Networks: A Comprehensive Foundation*. New York: Macmillan College Publishing Co., 1994.
8. J. T. Tou and R. C. Gonzalez, *Pattern Recognition Principles*. London: Addison-Wesley, 1974.
9. J. L. Kolodner, *Case-Based Reasoning*. San Mateo: Morgan Kaufmann, 1993.
10. V. Vapnik, *Statistical Learning Theory*. New York: John Wiley & Sons, 1998.
11. P. K. Singal, S. Mitra, and S. K. Pal, "Incorporation of fuzziness in ID3 and generation of network architecture," *Neural Computing and Applications*, vol. 10, pp. 155–164, 2001.
12. S. Mitra, K. M. Konwar, and S. K. Pal, "Fuzzy decision tree, linguistic rules and fuzzy knowledge-based network: Generation and evaluation," *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, vol. 32, pp. 328–339, 2002.
13. M. Mehta, R. Agrawal, and J. Rissanen, "SLIQ: A fast scalable classifier for data mining," in *Proceedings of 1996 International Conference on Extending Database Technology (EDBT '96)* (Avignon, France), March 1996.
14. J. Shafer, R. Agrawal, and M. Mehta, "SPRINT: A scalable parallel classifier for data mining," in *Proceedings of 1996 International Conference on Very Large Data Bases (VLDB '96)* (Bombay, India), pp. 544–555, September 1996.
15. T. Fukuda, Y. Morimoto, S. Morishita, and T. Tokuyama, "Data mining using two-dimensional optimized association rules: Scheme, algorithms, and visualization," in *Proceedings of 1996 ACM-SIGMOD International Conference on Management of Data (SIGMOD '96)* (Montreal, Canada), pp. 13–23, June 1996.
16. J. Gehrke, R. Ramakrishnan, and V. Ganti, "RainForest—A framework for fast decision tree construction of large datasets," *Data Mining and Knowledge Discovery*, vol. 4, pp. 127–162, 2000.

17. J. Mingers, "An empirical comparison of pruning methods for decision-tree induction," *Machine Learning*, vol. 4, pp. 227-243, 1989.
18. R. O. Duda and P. E. Hart, *Pattern Classification and Scene Analysis*. New York: John Wiley & Sons, 1973.
19. R. Rastogi and K. Shim, "PUBLIC: A decision tree classifier that integrates building and pruning," *Data Mining and Knowledge Discovery*, vol. 4, pp. 315-344, 2000.
20. B. D. Ripley, *Pattern Recognition and Neural Networks*. New York: Cambridge University Press, 1996.
21. H. J. Lu, R. Setiono, and H. Liu, "Effective data mining using neural networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 8, pp. 957-961, 1996.
22. I. K. Sethi, "Entropy nets: from decision trees to neural networks," *Proceedings of the IEEE*, vol. 78, pp. 1605-1613, 1990.
23. I. Ivanova and M. Kubat, "Initialization of neural networks by means of decision trees," *Knowledge-Based Systems*, vol. 8, pp. 333-344, 1995.
24. R. Setiono and H. Liu, "A connectionist approach to generating oblique decision trees," *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics*, vol. 29, pp. 440-444, 1999.
25. L. M. Fu, "Knowledge-based connectionism for revising domain theories," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 23, pp. 173-182, 1993.
26. G. G. Towell and J. W. Shavlik, "Knowledge-based artificial neural networks," *Artificial Intelligence*, vol. 70, pp. 119-165, 1994.
27. P. A. Devijver and J. Kittler, eds., *Pattern Recognition Theory and Applications*. Berlin: Springer-Verlag, 1987.
28. J. Pearl, "Fusion, propagation and structuring in belief networks," *Artificial Intelligence*, vol. 29, pp. 241-288, 1986.
29. D. W. Aha, D. Kibler, and M. K. Albert, "Instance-based learning algorithms," *Machine Learning*, vol. 6, pp. 37-66, 1991.
30. L. A. Zadeh, "Toward a theory of fuzzy information granulation and its centrality in human reasoning and fuzzy logic," *Fuzzy Sets and Systems*, vol. 19, pp. 111-127, 1997.
31. S. K. Pal and P. Mitra, "Case generation using rough sets with fuzzy representation," *IEEE Transactions on Knowledge and Data Engineering*.

32. C. J. C. Burges, "A tutorial on support vector machines for pattern recognition," *Data Mining and Knowledge Discovery*, vol. 2, pp. 1-47, 1998.
33. P. S. Bradley and O. L. Mangasarian, "Massive data discrimination via linear support vector machines," *Optimization Methods and Software*, vol. 13, pp. 1-10, 2000.
34. D. Cohn, L. Atlas, and R. Ladner, "Improving generalization with active learning," *Machine Learning*, vol. 15, pp. 201-221, 1994.
35. S. Tong and D. Koller, "Support vector machine active learning with application to text classification," *Journal of Machine Learning Research*, vol. 2, pp. 45-66, 2001.
36. C. Z. Janikow, "Fuzzy decision trees: Issues and methods," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 28, pp. 1-14, 1998.
37. H. Ichihashi, T. Shirai, K. Nagasaka, and T. Miyoshi, "Neuro fuzzy ID3: A method of inducing fuzzy decision trees with linear programming for maximizing entropy and algebraic methods," *Fuzzy Sets and Systems*, vol. 81, no. 1, pp. 157-167, 1996.
38. W. Xizhao and J. Hong, "On the handling of fuzziness for continuous-valued attributes in decision tree generation," *Fuzzy Sets and Systems*, vol. 99, pp. 283-290, 1998.
39. W. Pedrycz and A. Sosnowski, "Designing decision trees with the use of fuzzy granulation," *IEEE Transactions on Systems, Man, and Cybernetics - Part A*, vol. 30, pp. 151-159, 2000.
40. Y. Yuan and M. J. Shaw, "Induction of fuzzy decision trees," *Fuzzy Sets and Systems*, vol. 69, pp. 125-139, 1995.
41. X. Wang, B. Chen, G. Qian, and F. Ye, "On the optimization of fuzzy decision trees," *Fuzzy Sets and Systems*, vol. 112, pp. 117-125, 2000.
42. I. J. Chiang and J. Y. J. Hsu, "Integration of fuzzy classifiers with decision trees," in *Proceedings of Asian Fuzzy Systems Symposium*, pp. 266-271, 1996.
43. I. Hayashi, T. Maeda, A. Bastian, and L. C. Jain, "Generation of fuzzy decision trees by fuzzy ID3 with adjusting mechanism of AND/OR operators," in *Proceedings of International Conference on Fuzzy Systems (FUZZ-IEEE 98)*, pp. 681-685, 1998.
44. T. Kohonen, *Self-Organization and Associative Memory*. Berlin: Springer-Verlag, 1989.

45. S. Mitra, R. K. De, and S. K. Pal, "Knowledge-based fuzzy MLP for classification and rule generation," *IEEE Transactions on Neural Networks*, vol. 8, pp. 1338–1350, 1997.
46. M. Banerjee, S. Mitra, and S. K. Pal, "Rough fuzzy MLP: Knowledge encoding and classification," *IEEE Transactions on Neural Networks*, vol. 9, pp. 1203–1216, 1998.
47. S. K. Pal and S. Mitra, *Neuro-fuzzy Pattern Recognition: Methods in Soft Computing*. New York: John Wiley & Sons, 1999.
48. G. R. Davies and D. Yoder, *Business Statistics*. London: John Wiley & Sons, Inc., 1937.
49. Y. H. Pao, *Adaptive Pattern Recognition and Neural Networks*. Reading, MA: Addison-Wesley, 1989.
50. C. Blake and C. Merz, "UCI repository of machine learning databases," 1998. University of California, Irvine, Dept. of Information and Computer Sciences, <http://www.ics.uci.edu/~mlearn/MLRepository.html>.
51. U. Fayyad and K. Irani, "Multi-interval discretization of continuous-values attributes for classification learning," in *Proceedings of 13th International Joint Conference on Artificial Intelligence (IJCAI '93)* (Chambery, France), pp. 1022–1029, 1993.

This page intentionally left blank

6

Clustering in Data Mining

6.1 INTRODUCTION

Clustering is a useful technique for the discovery of some knowledge from a dataset. It maps a data item into one of several clusters, where clusters are natural groupings of data items based on similarity metrics or probability density models. Clustering pertains to unsupervised learning, when data with class labels are not available.

For example, when introducing a new product in the market, one groups or clusters the existing customers based on time series of payment history such that similar customers lie in the same cluster. The key requirement is the need for a good measure of similarity between the instances or patterns. This can help to identify micro-markets and develop separate policies for each. Analogously, one can group students of similar intelligence levels together so that efficient teaching strategies may be developed for each cluster.

The problem is to group N patterns into c desired clusters, such that the data points within clusters are more similar than across clusters. Some sample applications of clustering lie in customer segmentation, market basket customer analysis, attached mailing in direct marketing, and clustering of companies with similar growth.

Clustering is also very important in multimedia data mining, in order to cluster similar multimedia contents together for efficient indexing and storage in multimedia databases. For example, similar pictures can be clustered together for efficient indexing; thus, when a query is requested using a query

image or image description, then all the similar images can be effectively retrieved.

Clustering of data is broadly based on two approaches, namely, hierarchical and partitive [1, 2]. In partitional algorithms, the goal is to find a partition of c clusters that optimizes the chosen partitioning criterion. If a global optimality is desired, then one needs to exhaustively enumerate all partitions. Heuristic methods like the c -means and c -medoids have been designed to circumvent this problem. In the c -means algorithm [3], each cluster is represented by the center of gravity of the cluster. This need not essentially correspond to an object of the given pattern set. In the c -medoids algorithm [4], on the other hand, each cluster is represented by one of the representative objects in the cluster located near the center. A few variants of the c -means algorithm differ in (i) the selection of the initial c means, (ii) dissimilarity calculations, and (iii) strategies to calculate the cluster means.

Partitioning Around Medoids (PAM) [4] starts from an initial set of medoids, and it iteratively replaces one of the medoids by one of the nonmedoids if it improves the total distance of the resulting clustering. Although PAM works effectively for small data, it does not scale well for large datasets. The algorithms CLARA [4], CLARANS [5] (using randomized sampling), and the focusing technique for spatial data structures [6] are capable of dealing with the scalability issue.

Hierarchical methods can again be categorized as *agglomerative* and *divisive* algorithms, corresponding to bottom-up and top-down strategies, to build a hierarchical clustering tree (*dendrogram*). The optimal number of clusters is usually determined based on a validation index. There exist several clustering algorithms and validation indices in literature [1, 7], but they conventionally deal with numerical or quantitative data.

One of the major challenges to data mining [8, 9] is handling of *mixed media data*. This implies learning from data that is represented by a combination of various media, like (say) numeric, symbolic, images, text, etc. Symbolic or categorical clustering refers to the clustering of symbolic or categorical data. This is important from the point of view of data mining, where one has to mine for information from a set of symbolic objects. These objects are defined by attributes that can be quantitative (numeric or intervals) as well as qualitative. The similarity and dissimilarity measures between symbolic objects are often determined based on their position, span, and content features [10, 11].

Conceptual clustering [12, 13], from the Machine Learning community, is also applicable to a mixture of numeric, ordinal, and symbolic data. Here the focus is on interpretability or meaningfulness of the generated patterns, so that objects are clustered according to the concepts that they carry. The algorithm preserves cohesiveness within clusters while maintaining clear distinctness between clusters. Nonparametric probabilistic measures are used to determine the groupings.

This chapter is organized as follows. Section 6.2 elucidates the different distance measures and the concept of symbolic objects. The conventional

clustering categories are described in Section 6.3. The scalable versions of clustering algorithms, suitable for large data, are provided in Section 6.4. Soft computing-based approaches are dealt with in Section 6.5. Clustering with categorical attributes and symbolic clustering are described in Sections 6.6 and 6.7, respectively. Section 6.8 concludes the chapter.

6.2 DISTANCE MEASURES AND SYMBOLIC OBJECTS

Before moving on to clustering methodologies, let us explain what distance between a pair of objects refers to. In the context of mixed media data, the distance cannot be constrained to the numeric domain. Hence we need to compute distances between binary, nominal, and symbolic objects as well. The similarity measure and distance computation is also an integral part of multimedia data indexing, retrieval, and data mining techniques.

6.2.1 Numeric objects

Distances are normally used to measure the similarity or dissimilarity between two data objects X_i and X_j . The larger the similarity, the smaller the dissimilarity and hence the smaller the distance between the pair of objects. In the numeric domain, a popular measure is the Minkowski distance. It is defined as

$$d(X_i, X_j) = (|X_{i_1} - X_{j_1}|^q + |X_{i_2} - X_{j_2}|^q + \cdots + |X_{i_n} - X_{j_n}|^q)^{\frac{1}{q}}, \quad (6.1)$$

where q is a positive integer and n is the number of attributes involved. If $q = 1$, then d is termed *Manhattan* distance, while for $q = 2$ it is called the *Euclidean* distance. One can also use weighted distance or other dissimilarity measures [14].

6.2.2 Binary objects

In case of every pair of binary objects, a contingency table is designed. This is depicted in Table 6.1. Let there be two objects, such that location (1,1) of the 2×2 matrix denotes the number of features for which both take on values one. Analogously, location (2,2) corresponds to the number of features for which both X_i and X_j have values zero. Here b, c correspond to the occasions when the values of X_i and X_j mismatch.

The *simple matching coefficient* is used if the binary variable is symmetric. For example, in case of attribute gender, both states *female* and *male* are equally valuable and carry the same weight. Here the distance is invariant and is measured as

$$d(X_i, X_j) = \frac{b + c}{a + b + c + d}. \quad (6.2)$$

Table 6.1 Contingency table for binary variables

X_i	X_j		Sum
	1	0	
1	a	b	$a + b$
0	c	d	$c + d$
Sum	$a + c$	$b + d$	n

When the binary variable is asymmetric, the *Jaccard coefficient* is used. For example, the positive outcome of a disease test is considered more important than a negative outcome. Here the negative match of two zeros (corresponding to d in Table 6.1) is ignored. The measure is noninvariant and is defined as

$$d(X_i, X_j) = \frac{b + c}{a + b + c}. \quad (6.3)$$

6.2.2.1 Example 1: Let us consider an example to illustrate the process using Table 6.2. Let Sam, Mita, and Harry be three objects whose attributes correspond to their gender, symptoms (temperature, cold), and results of four tests. Here gender is a symmetric attribute, while the remaining are all asymmetric. Let the values Y and P be set to 1, and the value N be set to 0. (Here Y , P , and N correspond to symbolic outputs *yes*, *positive* and *negative*, respectively.) Considering only the asymmetric attributes, using Eq. (6.3), we have

$$d(\text{Sam}, \text{Mita}) = \frac{0 + 1}{2 + 0 + 1} = 0.33,$$

$$d(\text{Sam}, \text{Harry}) = \frac{1 + 1}{1 + 1 + 1} = 0.67,$$

and

$$d(\text{Harry}, \text{Mita}) = \frac{1 + 2}{1 + 1 + 2} = 0.75.$$

Table 6.2 Example to compute distance between binary objects

Name	Gender	Temperature	Cold	Test 1	Test 2	Test 3	Test 4
Sam	M	Y	N	P	N	N	N
Mita	F	Y	N	P	N	P	N
Harry	M	Y	P	N	N	N	N

6.2.3 Categorical objects

Nominal (or categorical) variables are a generalization of binary variables, in that each can take up more than two states. For example, color can be {red, yellow, blue, green}. One can use simple matching, considering p to be the total number of features and m to be the number of matches where X_i and X_j have the same state. Then using Eq. (6.2) we have

$$d(X_i, X_j) = \frac{p - m}{p}. \quad (6.4)$$

Another option would be to use a large number of binary variables, creating a new binary variable for each of the M nominal states.

There can be other types of variables as well. An ordinal variable can be discrete or continuous but, unlike nominal variables, here the order is important. An example of such a variable is rank. An interval variable specifies a lower and upper limit, between which its value lies. Variables can also be of mixed types, where a database may contain a mixture of numeric, symmetric binary, asymmetric binary, nominal, ordinal, and interval variables.

6.2.4 Symbolic objects

Let us now describe *symbolic* objects and the different dissimilarity measures, expressed as the distance between them. Symbolic objects are defined as the logical conjunction of events linking values and variables. The following are two examples of events: $e_1 = [\text{color} = \{\text{white}, \text{blue}\}]$, $e_2 = [\text{height} = [1.5 - 2.0]]$. Here e_1 indicates that the variable color takes a value either white or blue, while e_2 indicates that the variable height takes a value in the interval between 1.5 and 2.0, that is, $1.5 \leq \text{height} \leq 2.0$. For simplicity, we can drop the variable name and only take the value of that feature variable. Two symbolic objects A and B are written as the Cartesian product of features A_k and B_k , represented by $A = A_1 \times \cdots \times A_n$ and $B = B_1 \times \cdots \times B_n$. Let O_k denote the domain of the k th feature. Then the feature space can be expressed as a Cartesian product $O^{(n)} = O_1 \times \cdots \times O_n$.

The dissimilarity between two symbolic objects A and B is defined in terms of their distance involving position D_p , span D_s , and content D_c components [10, 11]. We have

$$D_p(A_i, B_i) = \cos \left[\left(1 - \frac{|\text{lower limit of } A_i - \text{lower limit of } B_i|}{\text{length of maximum interval along feature } i} \right) * 90 \right], \quad (6.5)$$

where the denominator indicates the difference between the highest and lowest values of the i th feature over all the objects. This measure holds for quantitative attributes only. The remaining two measures are defined for both

quantitative and qualitative attributes.

$$D_s(A_i, B_i) = \cos \left[\frac{|\text{length of } A_i| + |\text{length of } B_i|}{2 * (\text{span length of } A_i \text{ and } B_i)} * 90 \right], \quad (6.6)$$

where span length denotes the length of the minimum interval containing both A_i and B_i for quantitative values. The length of a qualitative feature value is the number of its elements, and the span length of two such feature values is the number of elements in their union.

$$D_c(A_i, B_i) = \cos \left[\frac{\text{length of intersection of } A_i \text{ and } B_i}{\text{span length of } A_i \text{ and } B_i} * 90 \right]. \quad (6.7)$$

Hence the distance $D(A, B)$ becomes

$$D(A, B) = \sum_{i=1}^n D(A_i, B_i), \quad (6.8)$$

where

$$D(A_i, B_i) = D_p(A_i, B_i) + D_s(A_i, B_i) + D_c(A_i, B_i)$$

with D_p , D_s , and D_c normalized to $[0,1]$ as per Eqs. (6.5)–(6.7).

6.3 CLUSTERING CATEGORIES

Clustering consists of partitioning data into *homogeneous* granules or groups, based on some objective function that maximizes the intercluster distances while simultaneously minimizing the intracluster distances. Traditional clustering algorithms can be broadly categorized into two main types [3]. Partitive algorithms like *c*-means divides the patterns into a set of spherical clusters, while minimizing the objective function. Here the number of clusters is pre-defined. Hierarchical methods, on the other hand, can again be grouped as agglomerative and divisive. Here no assumption is made about the shape or number of clusters, and a validity index is used to determine termination.

6.3.1 Partitional clustering

It basically involves enumerating c partitions, optimizing some criterion, over t iterations. The most commonly used numeric distance function is the square error, defined as $\sum_{i=1}^c \sum_{\mathbf{X} \in U_i} \|\mathbf{X} - \mathbf{m}_i\|^2$, where $\mathbf{m}_i$ is the mean of cluster U_i and $\mathbf{X}$ is the pattern vector (data object). The computational complexity of the algorithm is $O(N * c * t)$, where $c, t \ll N$. Generally, this works for well-separated, convex clusters. It is also scalable and efficient for large datasets. Even for very large databases, it is possible to work in a memory-efficient way with a small number of representatives and only periodic disk scans of the actual patterns.

Some of its drawbacks are as follows. It prefers splitting large clusters, and the gain from such splits can offset the merging of small clusters. A mean is not defined for categorical attributes and/or symbolic objects. Hence the algorithm cannot be used in applications involving categorical data, since it typically minimizes a cost function by changing the means of clusters. The algorithm is sensitive to noise and outliers, and it often terminates at a local optimum. The global optimum may, however, be found using techniques such as deterministic annealing and genetic algorithms. Finally, there is an inherent need to specify the value of c .

6.3.1.1 c -means algorithm The algorithm proceeds by partitioning the N objects into c nonempty subsets. During each partition, the centroids or means of the clusters are computed. The main steps of the c -means algorithm [3] are as follows:

- Assign initial means $\mathbf{m}_i$ (also called centroids).
- Assign each data object (pattern point) $\mathbf{X}_k$ to the cluster U_i for the closest mean.
- Compute new mean for each cluster using

$$\mathbf{m}_i = \frac{\sum_{\mathbf{X}_k \in U_i} \mathbf{X}_k}{|c_i|}, \quad (6.9)$$

where $|c_i|$ is the number of objects in cluster U_i .

- Iterate until the criterion function converges, that is, there are no more new assignments.

6.3.1.2 Example 2: Let us consider four two-dimensional samples $\mathbf{X}_1 = (1, 1)$, $\mathbf{X}_2 = (2, 2)$, $\mathbf{X}_3 = (3, 8)$, $\mathbf{X}_4 = (4, 8)$, to be clustered using $c = 2$. Let the first two patterns be chosen for initializing the means, as $\mathbf{m}_1 = (1, 1)$ and $\mathbf{m}_2 = (2, 2)$. The remaining two objects are assigned to U_2 based on their proximity to the closest mean. The cluster mean $\mathbf{m}_2$ then gets updated as

$$\mathbf{m}_2 = \left(\frac{2+3+4}{3}, \frac{2+8+8}{3} \right) = (3, 6).$$

The second iteration causes the assignment of $\mathbf{X}_1$, $\mathbf{X}_2$ to cluster U_1 and $\mathbf{X}_3$, $\mathbf{X}_4$ to cluster U_2 , with convergence to cluster means

$$\mathbf{m}_1 = \left(\frac{1+2}{2}, \frac{1+2}{2} \right) = (1.5, 1.5)$$

and

$$\mathbf{m}_2 = \left(\frac{3+4}{2}, \frac{8+8}{2} \right) = (3.5, 8).$$

6.3.1.3 Some variants Some of the variants of the conventional *c*-means algorithm are along the following lines.

1. Initial hierarchical clustering, followed by iterative relocation of patterns to the clusters.
2. ISODATA [3] employs splitting and merging operations on clusters, splitting if the sample variance in a cluster is more than a threshold while merging when the distance between the centroids of a pair of clusters is less than a threshold.
3. Instead of means, one could use *c*-modes (Section 6.6.3) to accommodate symbolic objects. This involves using new dissimilarity measures, and a frequency-based method to update modes of clusters.
4. Scalability of the algorithm is enhanced by retaining essential objects in main memory, while compressing or summarizing elements belonging to tight subclusters (using a clustering feature) and discarding redundant data. Hence large data are modeled only by the clustering feature and the retained objects in main memory.

6.3.1.4 Partitioning Around Medoids (PAM) The algorithm uses the most centrally located object in a cluster, the medoid, instead of the mean. Note that a medoid, unlike a mean, is essentially an existing data object from the cluster. It is closest to the corresponding mean. The basic steps are outlined as follows:

- Arbitrarily choose *c* objects as the initial medoids or seed points.
- Assign each remaining data object (pattern) to the cluster for the closest medoid.
- Replace each of the medoids by one of all the nonmedoids (causing the greatest reduction in square error), as long as the quality of clustering improves.
- Iterate until the criterion function converges.

For large *N* and *c*, the *c*-medoids [4] is computationally more costly than the conventional *c*-means. Here the computational complexity is $O((1 + \beta)c(N - c)^2)$, where β is the number of successful swaps (replacements). However, in the presence of noise and outliers, *c*-medoids is found to be more robust. This is because of the inherent robustness of medoids, as compared to means, with respect to noise. It is invariant to translations and orthogonal transformations of objects and is generally not influenced either by the order of presentation of objects or by the initial choice of seed points.

6.3.1.5 Example 3: Let us compare the working of PAM to that of c -means on the sample two-dimensional clustering problem. At the end of the first iteration, the cluster medoids are computed as $\mathbf{m}_1 = (1, 1)$ and $\mathbf{m}_2 = (3, 8)$ [closest to the mean $(3, 6)$].

6.3.1.6 Model-based clustering These methods aim to optimize the match between the given data and some mathematical model. Here the data are often assumed to be generated from c probability distributions, typically Gaussian or *Normal* around the cluster centers. This can also be termed as the probabilistic version of c -means clustering. One needs, however, to find the distribution parameters.

Expectation Maximization (EM) [15] is a popular iterative refinement algorithm that belongs to the category of model-based clustering. It differs from the conventional c -means algorithm in that each pattern point belongs to a cluster according to some weight (probability of membership). In other words, there are no strict boundaries between clusters. New means are computed based on weighted measures. It provides a statistical model of the data and is capable of handling the associated uncertainties. The algorithm can be characterized as follows:

- Initialize c cluster centers.
- Iterate between the two steps.
 - Expectation step: Assign each data point $\mathbf{X}_i$ to the cluster U_k with probability

$$P(X_i \in U_k) = p(U_k|X_i) = \frac{p(U_k)p(X_i|U_k)}{p(X_i)}, \quad (6.10)$$

where $p(X_i|U_k) = N(\mathbf{m}_k, E_k(X_i))$ follows the Normal distribution around mean $\mathbf{m}_k$ with expectation E_k .

- Maximization step: Estimate model parameters

$$\mathbf{m}_k = \frac{1}{N} \sum_{i=1}^N \frac{X_i P(X_i \in U_k)}{\sum_j P(X_i \in U_j)}. \quad (6.11)$$

In practice the algorithm converges fast, but may not reach the global optima. Convergence is guaranteed for certain forms of optimization functions. The computational complexity is $O(c * N * n * t)$, where n is the number of input features.

6.3.2 Hierarchical clustering

This method creates hierarchical nested partitions of the dataset, using a tree-structured dendrogram and some termination criterion. A matrix of similarity

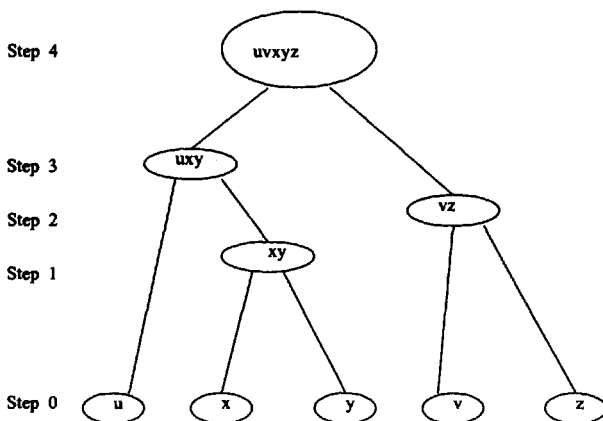


Fig. 6.1 A dendrogram for hierarchical clustering.

or dissimilarity is provided between every pattern pair. The decomposition of the dendrogram can be bottom-up, starting from individual data objects at the leaves as separate clusters and *agglomeratively* progressing upwards by merging the closest objects (or groups) into clusters. The top-down strategy, on the other hand, proceeds from a single cluster at the root and involves *divisive* splits while progressing down to the leaves. The merge or split decisions, if not properly made, may lead to low-quality clusters. Typically, the agglomerative strategy is more commonly used. However, in case of large databases, this method is not very practical since it scales at least quadratically with the number of data objects, that is, $O(N^2)$.

In agglomerative hierarchical clustering, the dendrogram shows how the clusters are merged hierarchically. A clustering of the data objects at any stage is obtained by cutting the dendrogram at the desired level, whereby each connected component in the tree corresponds to a cluster.

6.3.2.1 Example 4: Let there be five data objects u, v, x, y, z . The dendrogram of a hierarchical clustering, involving these objects, is depicted in Fig. 6.1. The merging of clusters proceeds in a sequence $xy, vz, uxy, uvxyz$ of four steps. The splitting for the divisive method would follow the reverse sequence. Although this method does not require the number of clusters c to be prespecified, it needs a termination condition.

6.3.2.2 Distance measure Generally, the distance matrix is used as the clustering criterion. The algorithm repeatedly merges closest clusters until the number of clusters becomes c . The distance involved can be either of the following:

- $d_{mean}(U_i, U_j) = \|\mathbf{m}_i - \mathbf{m}_j\|$ in the centroid approach, preferring break of large, nonhyperspherical clusters.

- $d_{\min}(U_i, U_j) = \min_{\mathbf{X} \in U_i, \mathbf{X}' \in U_j} \|\mathbf{X} - \mathbf{X}'\|$ in the minimum spanning tree approach, which is sensitive to outliers and slight change in position.

Analogically, there can be $d_{\text{average}}(U_i, U_j)$ and $d_{\max}(U_i, U_j)$.

The merging can follow the *single linkage* strategy, which combines two clusters such that the minimum distance between two points $\mathbf{X}$, $\mathbf{X}'$ from two different clusters U_i , U_j is the least. The disadvantage includes confusion created by overlapping objects. *Complete linkage*, on the other hand, merges two clusters U_i , U_j when all points in one cluster are “close” to all points in the other. This method is, however, sensitive to outliers. Several other hierarchical merging strategies are reported in the literature [1].

6.3.3 Leader clustering

This is another approach that is found to be useful in data mining. The leader clustering algorithm [2] chooses a *leader* as a representative (center) of a cluster, such that it is also its member. The algorithm depends on a threshold value to determine whether an object is similar (close) enough to the leader in order to lie in the same partition. If the threshold (or tolerance) is large, then more objects are grouped into less numbers of clusters, and vice versa. The main steps of the leader clustering algorithm are outlined below.

1. Initialize threshold θ ; initialize the first cluster center $\mathbf{m}_1$ by the first pattern. This is a leader.
2. **for** each of the remaining patterns $\mathbf{X}$ **repeat** steps 3–6.
3. Find nearest cluster U_i .
4. **if** distance $\|\mathbf{X} - \mathbf{m}_i\| < \theta$ **then** go to step 5 **else** go to step 6.
5. Update cluster center $\mathbf{m}_i$.
6. Generate new cluster center (leader).

The algorithm involves a single database scan, and it is required to store only the leaders (or cluster centers). It is incremental and robust to outliers, but is dependent on the order of pattern presentation. Moreover, one does not need to prespecify the number of clusters c . The algorithm is found to be suitable for use in large data.

6.4 SCALABLE CLUSTERING ALGORITHMS

Scalable algorithms were designed mainly by the Database community for working on very large datasets. Here the time and space complexities of the algorithms are of utmost concern. The number of database scans required is a good indicator of the algorithm’s feasibility. Often feature extraction becomes

useful for visualization. One could proceed by using an incremental (on-line) clustering that generates cluster representatives in a single database scan. A divide-and-conquer strategy may be applied by clustering a part of the data at a time in the main memory, generating their representatives, and finally merging these for all blocks of the data.

Some of the popular scalable algorithms include CLARA and CLARANS (Partitional), DBSCAN (Density-based), BIRCH (Hierarchical or Leader), CLIQUE (Grid-based), and CURE (Hierarchical). They are described below in this section.

6.4.1 Clustering large applications

This encompasses partitional algorithms like CLARA [4] and CLARANS [5].

6.4.1.1 Clustering LARGE Applications (CLARA) Here PAM is used to choose medoids from multiple random samples of data, returning the best clustering as the output. The computational complexity is $O(cs^2 + c(N - c))$ for a sample size s . Note that CLARA [4] considers *selected* sample datasets for the medoids, while conventionally PAM searches for the best c medoids from the entire data. However, CLARA cannot find the best clustering if any sampled medoid is not among the best c medoids. This is the trade-off for efficiency. Moreover, if s is not large enough, the effectiveness is lower; and if s is too large, the efficiency is not good.

6.4.1.2 Clustering Large Applications based on RANDOMized Search (CLARANS) Unlike CLARA that is confined to a fixed sample initially chosen, CLARANS [5] draws a sample with some randomness at each stage of the search. Each cluster is represented by a medoid. Multiple scans of the database are required by the algorithm. Here the clustering process searches through a graph, where each node is represented by a set of c medoids. Two nodes are termed as neighbors if they only differ by one medoid. Hence each node has $c * (N - c)$ neighbors.

The main steps are as follows:

- Initially, a node of c medoids is chosen randomly.
- Replace one of the c medoids at random, by selecting a neighbor node randomly.
- Assign data objects (pattern points) to the cluster with the closest medoid, by calculating average distance for this node; this requires one scan of the database.
- if the criterion function does not improve **then** revert back to the old medoid (node); **else** set the current node to be the neighbor node.
- **repeat** for a fixed number of times.

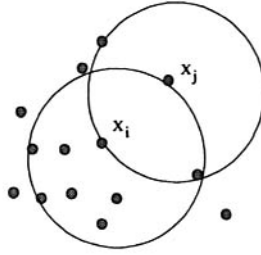


Fig. 6.2 Concept of density-reachable points.

CLARANS has been experimentally shown to be more effective than PAM and CLARA. It enables the detection of outliers. The computational complexity is $O(N^2)$. However the clustering quality is dependent on the sampling method used, and for large N the quality cannot be guaranteed.

6.4.2 Density-based clustering

This method clusters based on a local criterion such as density-connected points. The major features of the clustering include the abilities to (i) discover clusters of arbitrary shape and (ii) handle noise. The algorithm requires just one scan through the database. However, density parameters are needed for the termination condition. Some of the interesting studies in this direction are DBSCAN [16], OPTICS [17], DENCLUE [18], and CLIQUE [19].

Density-Based Spatial Clustering of Applications with Noise (DBSCAN) [16] discovers clusters of arbitrary shape in spatial databases in the presence of noise. The clusters are regarded as dense regions of objects in the data space, separated by regions of low density or noise. The objective is to determine the maximal set of *density-connected* points. The user is required to specify two parameters to define the minimum density. These are the ϵ -neighborhood (maximum radius) of an object and the minimum number of objects *MinPts* within this. Let *MinPts* = 5 and ϵ = 1 centimeter. Consider Fig. 6.2. A point X_i is directly *density-reachable* from a point X_j with respect to ϵ and *MinPts*, if

1. X_i belongs to $N_\epsilon(X_j)$, that is, $d(X_i, X_j) \leq \epsilon$, where N_ϵ is called a neighborhood function and $N_\epsilon(X_j)$ contains the points belonging to the neighborhood of X_j , and
2. core point condition $|N_\epsilon(X_j)| \geq \text{MinPts}$ holds.

A point X_i is density-reachable from a point X_j with respect to ϵ and *MinPts* if there is a chain of points $X_1, \dots, X_{N'}$, $X_1 = X_j$, $X_{N'} = X_i$ such that X_{k+1} is directly density-reachable from X_k . A point X_i is density-connected to a point X_j with respect to ϵ and *MinPts* if there is a point X_k

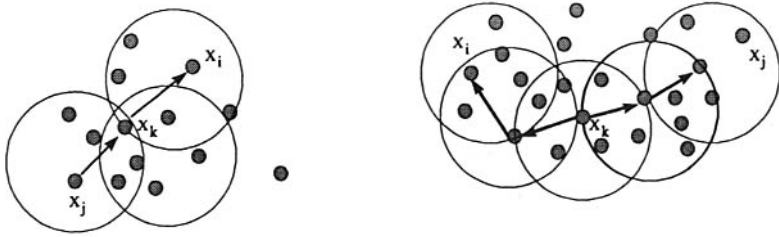


Fig. 6.3 Concept of density-connected points.

such that both X_i and X_j are density-reachable from X_k with respect to ϵ and $MinPts$. This is depicted in Fig. 6.3.

The algorithm is outlined as follows:

- Arbitrarily select a point X_i .
- Retrieve all points density-reachable from X_i with respect to ϵ and $MinPts$.
- if X_i is a core point then a cluster is formed.
- if X_i is a border point then no points are density-reachable from X_i and DBSCAN visits the next data point of the database.
- repeat the process until all the data objects (pattern points) have been processed.

The computational complexity of the algorithm is $O(N \log N)$. Using spatial access methods, DBSCAN is found to be efficient even for very large spatial databases. A generalized version of the algorithm has been recently developed [20] to cluster point objects as well as spatially extended objects.

A parallel version of DBSCAN, called PDBSCAN, has also been reported [21]. The main program of PDBSCAN, the master, starts a clustering slave on each available computer in the network and distributes the whole dataset into partitions onto the slaves. Every slave concurrently clusters only its local data. The basic steps of the algorithm are as follows:

1. Divide the database DB into c partitions $S_1, S_2, \dots, S_c$, such that $DB = \bigcup_{i=1}^c S_i$ and $S_i \cap S_j = \Phi$, for $i \neq j$. The partition S_i is distributed on slave computer C_i , where $i = 1, 2, \dots, c$.
2. Process the c partitions concurrently using DBSCAN on the available computers $C_1, C_2, \dots, C_c$.
3. Merge the clustering results obtained from the partitions $S_i, i = 1, 2, \dots, c$, into a clustering result for DB .

Since the run-time of DBSCAN only depends on the size of the input data, the partitions should be almost of equal size if we assume all computers to have

the same processing performance. Otherwise, the data may be distributed on computers according to their efficiency. In order to minimize communication costs, the nearby objects need to be organized on the same computer.

6.4.3 Hierarchical clustering

A major weakness of agglomerative clustering methods is that they do not scale well, the time complexity being at least $O(N^2)$ for N data objects. Moreover, they can never undo what was done in a previous step. Hence there have been integration of hierarchical with distance-based clustering, for suitably handling large datasets. These include (i) BIRCH [22], which uses a tree structure to store clustering features, and incrementally adjusts the quality of subclusters; (ii) CURE [23], which selects well-scattered points from the cluster and then shrinks them towards the center of the cluster by a specified fraction; and (iii) CHAMELEON [24], which performs hierarchical clustering using dynamic modeling.

6.4.3.1 *Balanced Iterative Reducing and Clustering Using Hierarchies (BIRCH)*

The algorithm [22] pre-clusters data points using a clustering feature tree (CF-tree). A CF-tree is height-balanced, storing the clustering features (CF) (summarized or condensed representations) for the application of a hierarchical clustering. The clustering feature is defined as

$$CF = (N, ls, ss),$$

where ls is the linear sum and ss is the square sum of N data points in a cluster or leaf.

The basic steps of the algorithm are as follows:

for each point

1. The CF-tree is traversed to find the closest cluster.
2. **if** the cluster is within a *threshold* **then** the point is absorbed into the cluster. The corresponding CF representation is updated.
3. Otherwise, the point starts a new cluster with its own CF representation (involving insertion and/or node split).

BIRCH requires only a single scan of the data, with the computational complexity being $O(N)$. The rebuilding process of the CF-tree is similar to insertion and node split of B+ trees. The algorithm is linearly scalable for large data and is capable of handling noise. The cluster summaries stored in the CF-tree are given to the hierarchical clustering algorithm for further processing. BIRCH can also be viewed as an extension of the leader algorithm (Section 6.3.3), since each point either is assigned to the cluster of the closest CF or is used to form a new cluster.

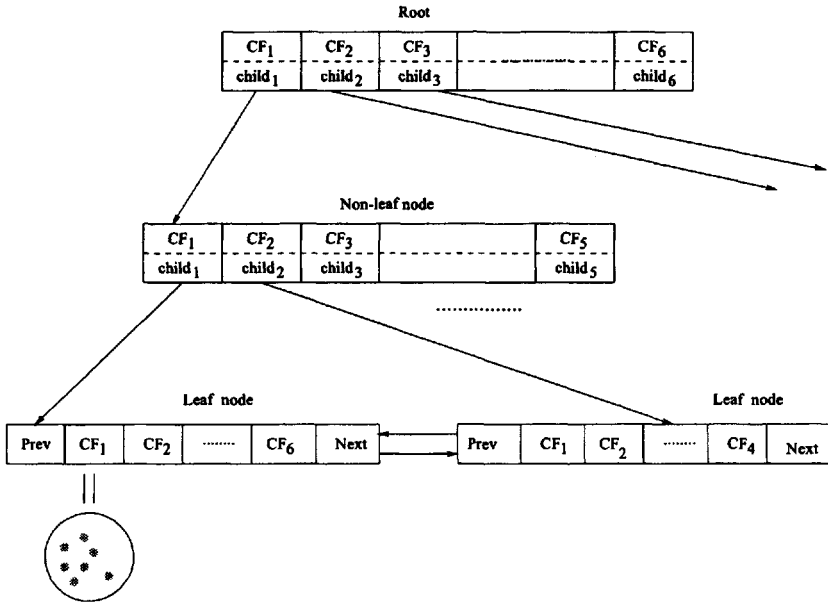


Fig. 6.4 A CF-tree.

However, it works for spherical clusters of uniform size and considers only limited available memory. The weakness of the algorithm is that it handles only numeric data and is sensitive to the order of the pattern point insertions.

6.4.3.2 Example 5: For example, let there be three points (3, 4), (2, 6), (4, 5) in a cluster. Then we have the clustering feature of the corresponding node to be

$$CF = (3, (3+2+4, 4+6+5), (3^2+2^2+4^2, 4^2+6^2+5^2)) = (3, (9, 15), (29, 77)).$$

The non-leaf nodes store sums of the CF s of their children. The parameters involved are the branching factor (maximum number of children per non-leaf node) and the threshold (maximum diameter of cluster stored at leaf node). Figure 6.4 depicts a CF-tree.

6.4.3.3 Clustering Using Representatives (CURE) Hierarchical clustering of large data typically uses a small number of representatives to represent a cluster. If it is centroid-based, then only one point is used. This involves too little information and can result in hyperspherical clusters. In a representative-object-based (minimum spanning tree) approach, on the other hand, one uses every point to represent a cluster. This involves too much information and can often be misleading.

CURE [23] uses a random sampling of original objects. A partitioning of the samples is made, followed by correct labeling. A small number of scattered representative points k are chosen. These are distributed over the cluster, such that each point in the cluster is close to one representative and the distance between clusters is the smallest distance between the representatives. This enables the capturing of the physical shape and geometry of the cluster. Farthest point heuristic is used to scatter the points over the cluster, and this is followed by shrinking it uniformly around its mean to dampen the effects of outliers. Availability of more than one representative point per cluster enables handling of nonspherical shapes. The algorithm is scalable for large databases, with a computational complexity of $O(N)$.

6.4.4 Grid-based methods

Grid-based clustering quantizes the pattern space into a finite number of cells, which form a multiresolution grid structure on which the operations are performed. These algorithms typically involve fast processing time since they are dependent only on the number of cells in each dimension of the quantized space and are typically independent of the number of actual data objects. This category of clustering algorithms encompasses STING [25] and CLIQUE [19].

6.4.4.1 Statistical Information Grid-based method (STING) The algorithm STING [25] divides the spatial area into rectangular cells using hierarchical structure, storing the statistical parameters of each numeric feature of the objects (like maximum, minimum, mean, etc.) within the cells. There are usually several levels of rectangular cells corresponding to different levels of resolution, each cell at a higher level being partitioned to form a number of cells at the next lower level in a top-down approach. The complexity of cluster generation is $O(N)$, while the complexity of response time of a query is $O(g)$, where $g \ll N$ is the number of grid cells at the lowest level.

The quality of clustering depends on the granularity at the lowest level of the grid structure: A finer granularity leads to an increased processing cost. The spatial relationship between neighboring cells at a particular level is not, however, considered in the hierarchy. This results in an isothetic shape of resulting clusters, with only horizontal and vertical cluster boundaries, such that no diagonal boundary can be detected.

6.4.4.2 Clustering In QUES (CLIQUE) The algorithm CLIQUE [19] integrates density-based and grid-based clustering methods. It is scalable to high-dimensional data in large databases. The algorithm is insensitive to the order of input tuples and does not presume any data distribution. However, accuracy may be degraded at the expense of its simplicity.

The algorithm finds clusters in all (nonoverlapping, rectangular) subspaces of the original pattern space. A unit in n -dimensions is defined as an inter-

section of one interval from each dimension. A cluster is expressed as a set of connected dense units in n -dimensions. It is assumed that if an n -dimensional unit is dense, then so are its projections in the $(n-1)$ -dimensional space. An *A priori*-like algorithm (described in Section 7.2.1) is used to generate candidate n -dimensional dense units. This enables generation of a minimal description for each of the clusters. In other words, it determines the maximal region that covers the clusters of connected dense units, followed by a minimal cover for each cluster.

6.4.5 Other variants

Recent research indicates that the concepts of proximity or clustering may not be meaningful in high dimensional data, mainly due to their inherent sparsity problem. A generalized projected clustering method [26] is designed to circumvent this problem by constructing clusters in arbitrarily aligned subspaces of lower dimensionality. The arbitrarily ORiented projected CLUSter generation (ORCLUS) technique searches for hidden subspaces with clusters which are created by inter-attribute correlations. The discovery of such correlations lead to projections which are not parallel to the original axis system. Extended cluster feature vectors are used to make the algorithm scalable for very large databases.

6.5 SOFT COMPUTING-BASED APPROACHES

In this section we concentrate on clustering, using different soft computing tools. These encompass fuzzy sets, neural networks, wavelets, rough sets, and evolutionary algorithms, the basics of which have been described in Section 2.2.

6.5.1 Fuzzy sets

Information granules can be viewed as collections of objects (patterns) drawn together by the criteria of indistinguishability, similarity or functionality, as in discretization or fuzzy partitioning. Fuzzy sets help in the semantics attachment to these granules (or clusters) of data. They also enable the determination and description of meaningful associations between the granules. In this section we describe some fuzzy clustering algorithms used for data mining.

6.5.1.1 Fuzzy c -means (FCM) This is a fuzzification of the c -means algorithm, proposed by Bezdek [27]. It partitions a set of N patterns $\{\mathbf{X}_k\}$ into

c clusters by minimizing the objective function

$$J = \sum_{k=1}^N \sum_{i=1}^c (\mu_{ik})^{m'} \|\mathbf{X}_k - \mathbf{m}_i\|^2, \quad (6.12)$$

where $1 \leq m' < \infty$ is the fuzzifier, $\mathbf{m}_i$ is the i th cluster center, $\mu_{ik} \in [0, 1]$ is the membership of the k th pattern to it, and $\|\cdot\|$ is the distance norm, such that

$$\mathbf{m}_i = \frac{\sum_{k=1}^N (\mu_{ik})^{m'} \mathbf{X}_k}{\sum_{k=1}^N (\mu_{ik})^{m'}} \quad (6.13)$$

and

$$\mu_{ik} = \frac{1}{\sum_{j=1}^c \left(\frac{d_{ik}}{d_{jk}} \right)^{\frac{2}{m'-1}}}, \quad (6.14)$$

$\forall i$, with $d_{ik} = \|\mathbf{X}_k - \mathbf{m}_i\|^2$, subject to $\sum_{i=1}^c \mu_{ik} = 1, \forall k$, and $\sum_{k=1}^N \mu_{ik} > 0, \forall i$. The algorithm proceeds as follows.

1. Pick the initial means $\mathbf{m}_i, i = 1, \dots, c$. Choose values for fuzzifier m' and threshold ϵ . Set the iteration counter $t = 1$.
2. **repeat** steps 3 and 4, by incrementing t , **until** $|\mu_{ik}(t) - \mu_{ik}(t-1)| > \epsilon$.
3. Compute μ_{ik} by Eq. (6.14) for c clusters and N data objects.
4. Update means $\mathbf{m}_i$ by Eq. (6.13).

Note that for $\mu_{ik} \in [0, 1]$ the objective function of Eq. (6.12) boils down to the hard c -means case, whereby a *winner-take-all* strategy is applied in place of membership values in Eq. (6.9).

6.5.1.2 Context-based clustering The conditional fuzzy c -means algorithm [28] uses a context, while performing fuzzy clustering. The aim is to determine the structure in a data set $\{\mathbf{X}_k\}$ for a given context or domain knowledge which is represented by a fuzzy set B defined over y_k , that is, $B(y_k)$. This introduces some focus and granularity in the search. It consists of a constraint-based optimization of the objective function of Eq. (6.12), such that $\mathbf{m}_i$ is given by Eq. (6.13) but Eq. (6.14) is modified to

$$\mu_{ik} = \frac{B(y_k)}{\sum_{j=1}^c \left(\frac{d_{ik}}{d_{jk}} \right)^{\frac{2}{m'-1}}}. \quad (6.15)$$

6.5.1.3 Fuzzy c -medoids This is a fuzzification of the c -medoids algorithm and is outlined as follows [29]:

1. Pick the initial medoids $\mathbf{m}_i, i = 1, \dots, c$.

2. **repeat** steps 3 and 4 **until** convergence.
3. Compute μ_{ik} for $i = 1, \dots, c$ and $k = 1, \dots, N$.
4. Compute new medoids

$$\mathbf{m}_i = \mathbf{X}_q,$$

where

$$q = \arg \min_{1 \leq j \leq N} \sum_{k=1}^N (\mu_{ik})^{m'} \|\mathbf{X}_j - \mathbf{X}_k\|^2 \quad (6.16)$$

refers to that j for which the minimum value of the expression is obtained.

Note that this boils down to the hard c -medoids with $\mu_{ik} = 1$, if $i = q$, and $\mu_{ik} = 0$ otherwise.

6.5.1.4 Granular clustering The granular clustering algorithm [30] organizes findings about data in the form of a collection of information granules, called hyperboxes. A compatibility measure guides the construction (growth) of the clusters. Indices like volume, compatibility measure, inclusion, overlap, normalized length, and sparsity are developed to describe the hyperboxes and express the relationships between such information granules. The family of the information granules result in a granular signature of the data. Abstraction is achieved through condensation of original data elements into granules, whose location and granularity reflects the essence of the structure of data. The basic steps of the algorithm consist of

- finding the two closest information granules to build a new granule embracing them, thereby condensing the data while reducing the dataset;
- repeating the first step until enough data condensation has been accomplished, as determined by a terminating criterion or a validation mechanism.

The indices *sparsity* and *overlap* are found to be useful in understanding the relevance of the variables, particularly their discriminatory abilities.

6.5.2 Neural networks

Neural network models based on variations of the self-organizing map (SOM) [31] are mainly used for clustering large datasets. The SOM is depicted in Fig. 2.4. It also results in good visualization of the results. The Adaptive Resonance Theory (ART) network [32] can also be viewed as a neural implementation of leader clustering (described in Section 6.3.3).

6.5.2.1 Construction of large SOM This was designed by Kohonen et al. [33]. The models of the larger map are initialized using the smaller ones, while pointers are initialized from the data to the models. The main steps of the algorithm are as follows:

- Initialize models of a very small SOM.
- Teach this small SOM.
- **repeat** until the desired map size is obtained.

The larger map can be fine-tuned with a parallel batch-map algorithm, which is typically used for faster computation. Let us consider Eq. (2.23). Assuming that the convergence to some ordered state is true, the expectation values of $\mathbf{m}_i(t+1)$ and $\mathbf{m}_i(t)$ for $t \rightarrow \infty$ become equal. Hence, in the stationary state, one has

$$E_t\{\alpha(t)[\mathbf{X}(t) - \mathbf{m}_i(t)]\} = 0, \quad \forall i,$$

where $E_t\{\cdot\}$ is the expectation value over the index of regression t . For a finite batch of $\mathbf{X}(t)$, we have

$$\mathbf{m}_i = \frac{\sum_t \alpha(t) \mathbf{X}(t)}{\sum_t \alpha(t)}. \quad (6.17)$$

Consider the Voronoi set $V_i = \{\mathbf{X} \mid \|\mathbf{X} - \mathbf{m}_i\| \text{ is minimum}\}$ such that $n_i = |V_i|$ is the number of samples of $\mathbf{X}(t)$ falling into V_i . Then the batch-map algorithm is outlined as follows:

1. Initialize the centers $\mathbf{m}_i, \forall i$.
2. **for** a finite set of samples $\mathbf{X}(t)$, compute a step of vector quantization

$$\bar{\mathbf{X}}_i = \frac{\sum_{\mathbf{X}(t) \in V_i} \mathbf{X}(t)}{n_i}, \quad \forall i. \quad (6.18)$$

3. Carry out one smoothing step

$$\mathbf{m}_i = \frac{\sum_j n_j \alpha \bar{\mathbf{X}}_j}{\sum_j n_j \alpha}, \quad \forall i, \quad (6.19)$$

where α is regarded as time invariant for simplicity.

4. **repeat** steps 2 and 3 until $\mathbf{m}_i$ becomes stationary.

The parallelization of the batch-map algorithm is possible due to the localized nature of the search for the winner. Here the data are distributed into several processors, and each executes the batch-map algorithm in parallel.

The algorithm has been applied to a large document database of patent abstracts [33] containing classes of 21 subsections, with 68,40,568 documents, and an initial dimension of 7,33,179 (number of different words) that is reduced to 500 using feature selection methods. The initial size of the SOM is 435. This later grows to a final size involving 10,02,240 nodes. The classification accuracy is found to be 64%.

6.5.2.2 Clustering of SOM This is a two-level approach described in Ref. [34]. The SOM is used to produce prototypes, which are then clustered using agglomerative and partitive methods. It is reported to have better performance than direct clustering, by resulting in a reduction in the computation time.

6.5.2.3 Dynamic self-organizing maps The dynamic self-organizing map (GSOM) [35] grows the nodes of the network according to the need of resolution. A spread factor is used to control the generation of maps with different dimensionality for different regions of interest in the feature space. Initially, one starts with a small map having a low spread factor. Hierarchical clustering is then done to achieve finer resolution for *interesting* clusters.

6.5.3 Wavelets

WaveCluster [36] uses a hybridization of grid and density-based approaches to cluster using wavelet transformation. The low-pass filter, inherent in the transform, helps to remove the outliers. It is possible to identify clusters at different levels of accuracy (i.e., fine or coarse) by using multiresolution. This implies that there are less clusters at a coarser resolution.

Multidimensional spatial data are considered as a multidimensional signal, and wavelet transform is applied to convert it to a number of frequency subbands. Convolution, with an appropriate kernel function as shown in Eqs. (3.11) and (3.12) as an example, results in a transformed space where the natural clusters become more distinguishable. A down sampling of the signal by two is made by skipping alternate signal samples. This results in the scales becoming coarser. However, at each scale the spatial relationships are preserved because of the inherent space-time localization principles of Discrete Wavelet Transform.

Consider Fig. 3.9. The high-frequency parts of the signal correspond to the boundaries of the clusters. Here the subbands LHi , HLi , and HHi refer to the horizontal, vertical, and diagonal features, respectively, constituting the high-frequency edge or discontinuity information at level i ($i = 1, 2, 3$). The low-frequency parts, with high amplitude, correspond to regions where the patterns (objects) are concentrated. This involves the actual clusters, falling in the average subband LLi . Details on the hierarchical decomposition of the subbands has been provided in Section 3.8.3. The main steps of the algorithm are as follows:

1. Quantize the feature space, and then assign objects to the units.
2. Apply wavelet transform on the feature space.
3. Find the connected components (clusters) in the (average) subband of the transformed feature space, at different levels.
4. Assign labels to the units.

5. Make the lookup table (map units in the transformed space to the original feature space).
6. Map the objects to the clusters.

WaveCluster is fast, with a linear time complexity $O(N)$, because of the finite length of filters. It efficiently handles large, high-dimensional spatial databases. Since the average subbands give approximations of the original feature space at different scales, this helps the algorithm to find clusters at different levels of details. It is possible to discover clusters with arbitrary shapes and complex structures. Outliers can be successfully handled using the inherent low-pass filter. No input parameter specifications are required, and the algorithm is insensitive to the order of input pattern presentation. WaveCluster is claimed to outperform the algorithms BIRCH, CLARANS, and DBSCAN.

6.5.4 Rough sets

Here the c -means algorithm is extended by viewing each cluster as an interval or rough set [37]. As mentioned in Section 2.2.6, a rough set Y is characterized by its lower and upper approximations $\underline{B}Y$ and $\overline{B}Y$, respectively. This permits overlaps between clusters. Adapting Eq. (6.9), the centroid $\mathbf{m}_i$ of cluster U_i is computed as

$$\mathbf{m}_i = \begin{cases} w_{low} \frac{\sum_{\mathbf{x}_k \in \underline{B}U_i} \mathbf{x}_k}{|\underline{B}U_i|} + w_{up} \frac{\sum_{\mathbf{x}_k \in (\overline{B}U_i - \underline{B}U_i)} \mathbf{x}_k}{|\overline{B}U_i - \underline{B}U_i|} & \text{if } \overline{B}U_i - \underline{B}U_i \neq \emptyset \\ w_{low} \frac{\sum_{\mathbf{x}_k \in \underline{B}U_i} \mathbf{x}_k}{|\underline{B}U_i|} & \text{otherwise,} \end{cases} \quad (6.20)$$

where the parameters w_{low} and w_{up} correspond to the relative importance of the lower and upper approximations, respectively. Here $|\underline{B}U_i|$ indicates the number of pattern points in the lower approximation of cluster U_i , and $|\overline{B}U_i - \underline{B}U_i|$ is the number of elements in the rough boundary lying between the two approximations.

Note that the expression boils down to Eq. (6.9) when the lower approximation is equal to the upper approximation, implying an empty boundary region. In order to determine whether an object $\mathbf{X}_k$ belongs to the upper or lower approximation of clusters, one computes the difference in its distance $d(\mathbf{X}_k, \mathbf{m}_i) - d(\mathbf{X}_k, \mathbf{m}_j)$ from cluster centroid pairs $\mathbf{m}_i$ and $\mathbf{m}_j$. If this is less than some threshold, then $\mathbf{X}_k \in \overline{B}U_i$ and $\mathbf{X}_k \in \overline{B}U_j$ and $\mathbf{X}_k$ cannot be a member of any lower approximation. Otherwise, $\mathbf{X}_k \in \underline{B}U_i$ such that distance $d(\mathbf{X}_k, \mathbf{m}_i)$ is minimum over the c clusters. A major disadvantage of this algorithm is the involvement of too many user-defined parameters.

6.5.5 Evolutionary algorithms

Evolutionary algorithms have been applied for optimally selecting the initial seed points (centroids) in c -means clustering [2]. Each solution string (chromosome) represents a valid c -partition of the N patterns. Here the length of the string is $c * n$, where n is the dimensionality of the patterns. A partition may also be encoded as a string of length N where the i th element denotes the cluster number of pattern X_i . The squared error value associated with a solution is translated into its fitness value, with a smaller error implying a larger fitness. The evolutionary operators selection, crossover, and mutation are applied to generate successive populations of solutions, until some termination criterion is satisfied. Selection ensures that highly fit individuals participate in the evolution with a higher probability. Crossover and mutation help in exploring the search space randomly.

Simulated Annealing [38], which updates a solution stochastically to permit its escape from local minimum, and Tabu Search [39], which searches in the vicinity of the current solution by storing possible solution vectors in a list while exploring the best solution found so far, have also been employed for determining optimal partitions.

Genetic algorithms (GA) have also been used for medoids-based clustering [2]. Here each chromosome consists of c parameters corresponding to the c medoids, with the parameter values corresponding to their record IDs in the database. The strings are not binary, as in conventional GAs (Section 2.2.5), but consist of integer values lying between 1 and N where N is the total number of objects in the dataset. The fitness of each individual is inversely proportional to the dissimilarity of the clusters. The selection procedure is the same as that of the conventional GAs.

Random Respectful Recombination (RRR) crossover operator is applied. Here the parameter values, which are common in both the parent individuals, are transported to the offsprings. The remaining parameter values of the offspring are chosen randomly from the rest of the parameter values of the parents.

6.5.5.1 Example 6: For example, with $c = 5$ and $N = 100$ and parent chromosomes 7 19 43 67 89 and 8 19 39 67 92, we find that the common values are 19, 67. These constitute the first two values in case of both offsprings. The remaining slots are randomly filled up with the values 8, 43, 89 and 89, 7, 39 from the parents, resulting in the children 19 67 | 8 43 89 and 19 67 | 89 7 39.

Mutation tries to replace the parameter value of a chromosome by another object (record number) not already present in it. First a random (integer) number in the range $[1, \dots, N]$ is generated. If this is different from the existing numbers in the current chromosomes, it replaces the old number by this new one.

6.5.5.2 Example 7: For example, a mutation on the 2nd gene of the parent 7 |19| 43 67 89 with the generation of a random number 34 produces an offspring chromosome given by 7 |34| 43 67 89.

6.5.5.3 Procedure The *c*-medoids algorithm, using GAs, involves the following steps.

1. Choose an initial population of *c* medoids.
2. Compute the dissimilarity (distance) of the clusters from the chromosomes.
3. Assign fitness values to the individuals, with more dissimilarity implying less fitness.
4. Generate new population of medoids by performing selection, crossover and mutation.
5. if termination criterion is satisfied **then** go to step 6 **else** go to step 2.
6. Return the best fit chromosome.

Clustering Large Applications based on Simulated Annealing (CLASA) applies simulated annealing to select better medoids [40]. Simulated annealing, unlike GAs, stochastically anneals a single chromosome with the help of a temperature parameter. A *cooling* schedule of the temperature is undertaken, as the search space is explored for determining the optimum set of medoids.

6.6 CLUSTERING WITH CATEGORICAL ATTRIBUTES

Traditional algorithms, suited for numeric or quantitative features, do not work well for categorical or nominal attributes. Viewing the feature space as points with (0/1) values of attributes does not solve the problem. The Jaccard coefficient of Eq. (6.3) is often used for handling binary values. However, it becomes hard to reflect the properties of the neighborhood of the points and the algorithms fail to capture the natural clustering of datasets. When using a traditional algorithm, involving centroids, as the cluster size grows, the number of attributes appearing in the mean goes up. Their values in the mean also decrease in a *ripple* effect. Thus it becomes very difficult to distinguish between two points.

Often one builds a weighted hyper-graph, with frequent itemsets involving categorical attributes. A hyper-edge in this graph represents a frequent item. The weight of the edge is the average of confidences of all association rules generated from its itemset. This hyper-graph partitioning algorithm is used to cluster items by minimizing the sum of weights of hyper-edges.

Some of the clustering algorithms, working on categorical attributes, include STIRR [41], ROCK [42], CACTUS [43], and *c*-modes [44].

6.6.1 Sieving Through Iterated Relational Reinforcements (STIRR)

STIRR [41] is based on nonlinear dynamic systems. It seeks a similarity based on co-occurrences of items in the same column. Each distinct value of each column becomes a node. The algorithm assigns weight to each node (either uniformly or randomly), such that the sum of all weights is one. An iterative approach is followed for assigning and propagating weights on the categorical values. However, it is difficult to analyze the stability of this method.

Table 6.3 Sample transactions set to demonstrate STIRR

Item purchased	Intermediate node	Transaction ID
A	α	1
A	α	2
B	β	3
B	α	4
C	α	1
C	β	3

6.6.1.1 Example 8: Let us consider an example in Table 6.3, demonstrating the relation between transactions of items purchased in a store, to illustrate how STIRR works. Figure 6.5 depicts the resulting undirected graph linking the items with the corresponding transactions. The intermediate nodes refer to the association or relationship between the items and the transactions, thereby defining possible clusters on the basis of similarity. The weights of the items, clusters, and transactions are represented by w_A, w_B, w_C ; w_α, w_β ; and w_1, w_2, w_3, w_4 ; respectively. These are initialized randomly. The subsequent weight update expression is given as $w_B = w_\alpha + w_4 + w_\beta + w_3$ or $w_B = w_\alpha \cdot w_4 + w_\beta \cdot w_3$. Upon convergence, clusters of transactions get reinforced by the larger weight values at the intermediate nodes.

6.6.2 Robust Hierarchical Clustering with Links (ROCK)

ROCK [42] uses the number of *common neighbors* between two data points in order to measure their similarity. Here common neighbors refer to nodes that differ from each other in only one item. As it uses a global knowledge of the similarity of data points in order to measure distances, the decision on which points to merge in a single cluster becomes very robust. It is basically a hierarchical clustering for categorical attributes, suitable for analyzing (say) market basket customers. A novel concept of cross-links is used for merging clusters. A similarity function $sim(T_i, T_j)$ captures the closeness or interconnectivity between transactions T_i and T_j . A pair of patterns T_i and T_j are

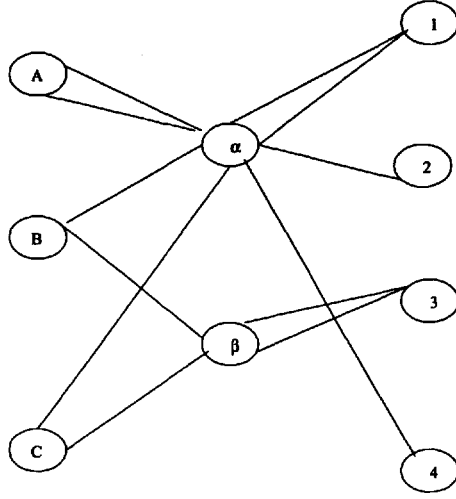


Fig. 6.5 Example demonstrating STIRR.

termed neighbors if $\text{sim}(T_i, T_j) \geq \theta$, where θ is a user-specified threshold, and $\text{link}(T_i, T_j)$ determines their number of common neighbors. A goodness measure is used to evaluate the clustering. Random sampling is employed for scaling up to large datasets.

6.6.2.1 Example 9: Let us consider an example with two clusters of transactions, at any point of time of the hierarchical clustering, involving items $\langle 1, 2, 3, 4, 5 \rangle$ and $\langle 1, 2, 6, 7 \rangle$, respectively, for $\theta = \frac{2}{3}$. Let the individual transactions be expressed as two natural partitions consisting of $\{1, 2, 3\}$ $\{1, 2, 4\}$ $\{1, 2, 5\}$ $\{1, 3, 4\}$ $\{1, 3, 5\}$ $\{1, 4, 5\}$ $\{2, 3, 4\}$ $\{2, 3, 5\}$ $\{2, 4, 5\}$ $\{3, 4, 5\}$ and $\{1, 2, 6\}$ $\{1, 2, 7\}$ $\{1, 6, 7\}$ $\{2, 6, 7\}$, respectively. We define

$$\text{sim}(T_i, T_j) = \frac{|T_i \cap T_j|}{|T_i \cup T_j|} \geq 0.5.$$

Here the transactions $\{1, 2, 6\}$ and $\{1, 2, 7\}$ are found to have five links (common neighbors, namely, $\{1, 2, 3\}$, $\{1, 2, 4\}$, $\{1, 2, 5\}$, $\{1, 6, 7\}$, $\{2, 6, 7\}$), validating their existence in the same cluster. On the other hand, transactions $\{1, 2, 3\}$ and $\{1, 2, 6\}$ are seen to have three links only (namely, $\{1, 2, 4\}$, $\{1, 2, 5\}$, $\{1, 2, 7\}$), implying less similarity and hence their existence in different clusters.

6.6.3 c-modes algorithm

An approach in Ref. [45] has used the *c*-means algorithm to cluster categorical data. Here multiple category attributes are converted into binary form,

with a 0/1 indicating the absence (or presence) of a category, and then the binary attributes are treated as numeric in the *c*-means framework. However, as mentioned earlier, this approach needs to handle a large number of binary attributes because the datasets often involve categorical attributes with hundreds or thousands of categories. This inevitably increases both computational and space costs of the *c*-means algorithm when used for data mining. The other drawback is that the cluster means, expressed as real values between 0 and 1, do not essentially indicate the characteristics of any of the clusters.

The *c*-modes algorithm [44] is capable of handling categorical or nominal attributes. It uses a partitional approach and proceeds along the lines of the *c*-means. However, the cluster means are replaced with their modes. Note that a mode is another measure of central tendency in a dataset, referring to that pattern or object having the maximum frequency of occurrence in a cluster. For example, the two modes of the set $\{(x, y), (x, z), (z, y), (y, z)\}$ are (x, y) and (x, z) . New dissimilarity measures are used to deal with categorical objects. A frequency-based method is used to update the modes of the clusters. A mixture of categorical and numerical data can be handled by the *c*-prototypes method [44]. This has applications in real-world databases involving mixed-type objects. As in the *c*-means, both these algorithms produce locally optimal solutions that are dependent on the initial modes and the order of presentation of objects in the dataset.

The basic steps of the *c*-modes algorithm are outlined as follows:

1. Select *c* initial modes.
2. Allocate an object to the cluster whose mode is the nearest to it. Update the mode of the cluster after each allocation.
3. After all objects have been allocated to clusters, retest the dissimilarity of objects against the current modes. If an object is found such that its nearest mode belongs to another cluster, reallocate the object to that cluster and update the modes of both clusters.
4. **repeat** step 3 **until** no object has changed clusters after a full cycle test of the whole dataset.

The major differences between CLARA (since PAM can use any dissimilarity measures, it can cluster objects with categorical attributes) and the *c*-prototypes algorithm include the following. (i) CLARA clusters based on samples, whereas *c*-prototypes directly works on the whole large dataset. (ii) CLARA optimizes its clustering result at the sample level, and hence may not produce a good clustering if the sample is biased; whereas the *c*-prototypes algorithm guarantees at least a locally optimal clustering. (iii) The efficiency of CLARA depends on the sample size, while the *c*-prototypes algorithm has no such restrictions.

6.7 HIERARCHICAL SYMBOLIC CLUSTERING

Symbolic objects include linguistic, nominal, boolean, and interval-type of features, along with quantitative attributes. Clustering in this domain involves the use of symbolic dissimilarity between the objects. In this section we outline conceptual clustering [12, 13] and a hierarchical symbolic clustering algorithm [46, 47] with different validity indices for determining the optimal number of meaningful clusters. The novelty of the method lies in transforming the different clustering validity indices, like Normalized Modified Hubert's statistic, Davies-Bouldin index, and Dunn's index, from the quantitative domain to the symbolic framework. The effectiveness of the algorithm is quantitatively evaluated on real-life benchmark datasets.

6.7.1 Conceptual clustering

Conceptual clustering [12, 13] is based on *Category Utility* CU of a cluster U_k , defined as the average over the l clusters. It is expressed as

$$\frac{\sum_{k=1}^l CU_k}{l}, \quad (6.21)$$

where

$$CU_k = P(U_k) * \left(\sum_i \sum_j P(a_i = V_{ij}|U_k)^2 - \sum_i \sum_j P(a_i = V_{ij})^2 \right). \quad (6.22)$$

Here $P(U_k)$ is the *a priori* probability of cluster U_k , $P(a_i = V_{ij})$ is the probability of feature a_i taking on value V_{ij} , and $P(a_i = V_{ij}|U_k)$ is the conditional probability of $a_i = V_{ij}$ in cluster U_k . This represents an increase in the number of feature values that can be correctly guessed for cluster U_k ($P(a_i = V_{ij}|U_k)^2$), over the expected number of correct guesses, given that no class information is available [$P(a_i = V_{ij})^2$].

The objective is to generate maximally cohesive clusters (high intraclass similarity) while achieving maximum separability (high interclass dissimilarity) among the clusters in a partition. Probabilistic measures, to evaluate the goodness of the partitioning, are expressed as [13]

$$M_{dk} = \sum_{i,j \in \{a_i\}_d} (P(a_i = V_{ij}|U_k)^2 - P(a_i = V_{ij})^2) \quad (6.23)$$

and

$$Var(U(k), U(l)) = \frac{1}{n} \sum_i \sum_j (P(a_i = V_{ij}|U_k) - P(a_i = V_{ij}|U_l))^2. \quad (6.24)$$

Here M_{dk} is the increase in predictability of an object X_d to cluster U_k . *Cohesion* of a partition structure is measured as the sum of the M_{dk} values of

all objects in the dataset, while $Var(U(k), U(l))$ is the variance of distribution match between clusters U_k and U_l in a given partition. *Distinctness* of a partition is taken as the average variance between clusters in that partition. Another popular conceptual clustering algorithm is COBWEB [48], which builds an incremental classification tree.

6.7.2 Agglomerative symbolic clustering

The agglomerative algorithm [46, 47] typically involves a repetition of the steps (i) assignment of pattern vector $\mathbf{X}$ to a cluster, (ii) all intercluster distance computation, and (iii) merging pairs of clusters which are closest to each other, until there is only one cluster left. Like typical hierarchical methods, the partitioning at any stage depends on the previously found clusters.

Let us now define two different measures for within-cluster and between-cluster distances in the symbolic framework. Let $\{\mathbf{X}_1, \dots, \mathbf{X}_{c_k}\}$ be a set of symbolic objects lying in a cluster U_k . Then the *average distance* within the cluster U_k is expressed as

$$S_a(U_k) = \frac{\sum_{i,i'} D(\mathbf{X}_i, \mathbf{X}_{i'})}{|c_k|(|c_k| - 1)}, \quad (6.25)$$

where $\mathbf{X}_i, \mathbf{X}_{i'} \in U_k, i \neq i', |c_k|$ is the number of samples in cluster U_k , and D indicates the symbolic dissimilarity of Eq. (6.8). The *between-cluster* distance is defined as

$$d_a(U_k, U_l) = \frac{\sum_{i,j} D(\mathbf{X}_i, \mathbf{X}_j)}{|c_k||c_l|}, \quad (6.26)$$

where $\mathbf{X}_i \in U_k, \mathbf{X}_j \in U_l$, such that $k \neq l$. We have used S_a and d_a in our computations, in terms of the symbolic dissimilarity D of Eq. (6.8). The algorithm is as follows.

1. Let the initial number of clusters be N , with each cluster having a weight (number of objects) of 1. Therefore $\mathbf{X}_i \in U_i, i = 1, \dots, N$.
2. Compute the weighted dissimilarities between all pairs of clusters in the dataset as

$$d_{aw}(U_i, U_j) = d_a(U_i, U_j) \left(\frac{|c_i| \cdot |c_j|}{|c_i| + |c_j|} \right)^{0.5}. \quad (6.27)$$

Note that the weighting term on the r.h.s. of Eq. (6.27) yields a value $\sqrt{50}$ for $|c_i| = |c_j| = 100$, while it results in $\sqrt{0.5}$ for $|c_i| = |c_j| = 1$ (singleton clusters). Hence the dissimilarity is highlighted for larger clusters. However, for $|c_i| = 100, |c_j| = 1$, we have $100/101 \simeq 1$, naturally implying a higher dissimilarity than that for $|c_i| = |c_j| = 1$. So there is a greater probability of merging a pair of smaller clusters as compared to larger clusters.

3. Determine the mutual pair (clusters) having the lowest average weighted dissimilarity $d_{a_{w\min}}$ by Eq. (6.27). Form a composite cluster U_k by merging the individuals of this pair, such that $|c_k| = |c_i| + |c_j|$. Reduce the number of clusters by 1.
4. repeat steps 2 and 3 until the number of clusters equals 1.
5. Compute cluster validity index V_t by Eqs. (6.29)–(6.32). Determine the stage t_0 , with clusters $c = c_0$, for $c = 2, \dots, \sqrt{N}$, at which V_t is optimum. This indicates the optimal number of clusters.

6.7.3 Cluster validity indices

To select the best among different partitioning, each of these can be evaluated using some validity index. Several validation methods have been proposed in the literature [7] for quantitative data. These include Normalized Modified Hubert's statistic, Davies–Bouldin index, and Dunn's index. Since the conventional centroid-based representation is not feasible in the symbolic framework, we modify these expressions using the average scatter S_a [Eq. (6.25)] within a cluster and d_a [Eq. (6.26)] between clusters [46, 47]. The performance is compared with that of the Cluster indicator [10].

6.7.3.1 Hubert's statistic Let $\mathbf{X}_i$ be the i th object and $L(i) = k$ if $\mathbf{X}_i \in U_k$. The Modified Hubert's statistic is a measure used for determining the optimal number of clusters in the quantitative domain [7]. For a cluster structure in the symbolic framework, it can be expressed in terms of symbolic dissimilarity as

$$\Gamma = \sum_{i=1}^{N-1} \sum_{j=i+1}^N D(\mathbf{X}_i, \mathbf{X}_j) d_a(U_{L(i)}, U_{L(j)}). \quad (6.28)$$

If $\mathbf{X}_i$ and $\mathbf{X}_j$ lie in two different clusters, d_a is computed using Eq. (6.26). However, when they belong to the same cluster, $d_a = 0$. From this, we get Normalized Modified Hubert's statistic $\hat{\Gamma}$ as

$$\hat{\Gamma} = \frac{1}{M} \sum_{i=1}^{N-1} \sum_{j=i+1}^N \frac{(D(\mathbf{X}_i, \mathbf{X}_j) - \bar{D})(d_a(U_{L(i)}, U_{L(j)}) - \bar{d}_a)}{s_D s_{d_a}}. \quad (6.29)$$

Here

$$\begin{aligned} \bar{D} &= \frac{1}{M} \sum_{i=1}^{N-1} \sum_{j=i+1}^N D(\mathbf{X}_i, \mathbf{X}_j), \\ \bar{d}_a &= \frac{1}{M} \sum_{i=1}^{N-1} \sum_{j=i+1}^N d_a(U_{L(i)}, U_{L(j)}), \\ s_D^2 &= \frac{1}{M} \sum_{i=1}^{N-1} \sum_{j=i+1}^N D^2(\mathbf{X}_i, \mathbf{X}_j) - \bar{D}^2, \end{aligned}$$

and

$$s_{d_a}^2 = \frac{1}{M} \sum_{i=1}^{N-1} \sum_{j=i+1}^N d_a^2(U_{L(i)}, U_{L(j)}) - \bar{d}_a^2,$$

where $M = \frac{N(N-1)}{2}$ is the total number of terms under the double summation. The terms s_D and s_{d_a} are the standard deviations of the entries of the matrices D and d_a respectively, while s_D^2 and $s_{d_a}^2$ are the corresponding variances [7]. The optimal partitioning occurs at $c = c_0$ for which $\Delta(\Delta\hat{\Gamma})$ is minimum. This corresponds to a sharp change in slope (also called *knee*) of the piecewise linear graph for Normalized modified Hubert's statistic, in case of well-separated clusters.

6.7.3.2 Davies–Bouldin index The Davies–Bouldin index is a function of the ratio of the sum of within-cluster distance to between-cluster separation. The best clustering, for $c = c_0$, minimizes

$$\frac{1}{c} \sum_{k=1}^c \max_{l \neq k} \left\{ \frac{S_a(U_k) + S_a(U_l)}{d_a(U_k, U_l)} \right\}, \quad (6.30)$$

for $1 \leq k, l \leq c$. Here the within-cluster distance is minimized and the between-cluster separation is maximized. The index is expressed in the symbolic framework, using Eqs. (6.25) and (6.26).

6.7.3.3 Dunn's index Like Davies–Bouldin index, Dunn's index is designed to identify sets of clusters that are compact and well separated. We maximize

$$\min_k \left\{ \min_{l \neq k} \left\{ \frac{d_a(U_k, U_l)}{\max_j S_a(U_j)} \right\} \right\}, \quad (6.31)$$

for $1 \leq j, k, l \leq c$. Here also the symbolic intercluster separation is maximized, while minimizing symbolic intracluster distances.

6.7.3.4 Cluster indicator The cluster indicator value at the t th iteration is defined as

$$CI_t = \frac{R_t}{R_{t+1}}, \quad (6.32)$$

where

$$R_t = \frac{\min_{k \neq l} d_a^t(U_k, U_l)}{\min_{k' \neq l'} d_a^{t+1}(U_{k'}, U_{l'}) + \min_{k'' \neq l''} d_a^{t-1}(U_{k''}, U_{l''})}. \quad (6.33)$$

This is maximized over different iterations for $t = 2, \dots, N-2$. Note that initially, at $t = 0$, there are N singletons. At $t = 1$, the pair of closest clusters are merged, resulting in $N-1$ clusters. Therefore, at the t th iteration we have $N-t$ clusters.

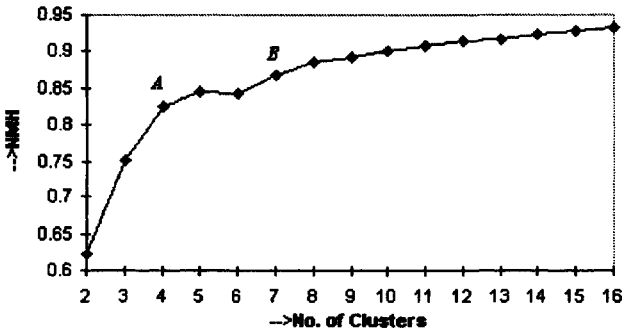


Fig. 6.6 Plot of Hubert's statistic for *Zoo* data.

6.7.4 Results

The symbolic clustering algorithm was implemented [46, 47] on benchmark symbolic data, namely, *Zoo*, *Auto Import*, and *Mushroom* [49]. The results were compared with that of conceptual clustering, and the measures *Cohesiveness* and *Distinctness* of Eqs. (6.23) and (6.24) were used for this purpose.

In this section we provide sample results for the *Zoo* data consisting of 100 instances of animals with 17 features and 7 output classes. The name of the animal constitutes the first attribute. There are 15 boolean features corresponding to the presence of hair, feathers, eggs, milk, backbone, fins, or tail; and whether airborne, aquatic, predator, toothed, breathes, venomous, domestic, catsize. The character attribute corresponds to the number of legs lying in the set $\{0, 2, 4, 5, 6, 8\}$.

The validity indices used, Cohesion and Distinctness values, the number of elements in each cluster (in parentheses), and the individual elements according to their sequential order of entry in the corresponding cluster are provided in Table 6.4. The symbolic clustering algorithm provided four clusters for the *Zoo* data with validity indices *CI*, Normalized Modified Hubert's statistic, Davies-Bouldin, and Dunn's, while Conceptual clustering generated two clusters (merging clusters 2, 3, and 4 into cluster 2 here). It is observed that the resulting partitions in the symbolic clustering are semantically meaningful and are very similar to those obtained by Kohonen's self-organizing feature map [35]. A plot of Hubert's statistic for the data, in Fig. 6.6, shows a sharp *knee* corresponding to four clusters. The Cohesion and Distinctness measures indicate better partitioning for four clusters.

Table 6.4 Symbolic clustering and evaluation of Zoo data

Index	Cohesion, distinctness	Cluster	Animals
CI, Normalized Modified Hubert's, Davies- Bouldin, and Dunn's	458.43, 0.58	1 (41)	aardvark, bear, girl, boar, cheetah, leopard, lion, raccoon, wolf, lynx, mongoose, polecat, puma, mink, platypus, dolphin, porpoise, seal, sealion, antelope, buffalo, deer, elephant, giraffe, oryx, gorilla, wallaby, calf, goat, pony, reindeer, pussycat, cavy, hamster, fruitbat, vampire, squirrel, hare, vole, mole, opossum.
		2 (21)	bass, catfish, piranha, chub, herring, carp, haddock, seahorse, sole, dogfish, pike, tuna, stingray, frog, toad, newt, tuatara, pitviper, slowworm, scorpion, seasnake.
		3 (21)	chicken, dove, parakeet, lark, pheasant, sparrow, wren, flamingo, ostrich, tortoise, crow, hawk, vulture, kiwi, rhea, penguin, duck, swan, gull, skimmer, skua.
		4 (17)	clam, seawasp, crab, starfish, crayfish, lobster, octopus, flea, termite, slug, worm, gnat, ladybird, housefly, moth, honeybee, wasp.
Conceptual	132.49, 0.50	1 (41)	aardvark, bear, girl, boar, cheetah, leopard, lion, raccoon, wolf, lynx, mongoose, polecat, puma, mink, platypus, dolphin, porpoise, seal, sealion, antelope, buffalo, deer, elephant, giraffe, oryx, gorilla, wallaby, calf, goat, pony, reindeer, pussycat, cavy, hamster, fruitbat, vampire, squirrel, hare, vole, mole, opossum.
		2 (59)	bass, catfish, piranha, chub, herring, carp, haddock, seahorse, sole, dogfish, pike, tuna, stingray, frog, toad, newt, tuatara, pitviper, slowworm, scorpion, seasnake. chicken, dove, parakeet, lark, pheasant, sparrow, wren, flamingo, ostrich, tortoise, crow, hawk, vulture, kiwi, rhea, penguin, duck, swan, gull, skimmer, skua. clam, seawasp, crab, starfish, crayfish, lobster, octopus, flea, termite, slug, worm, gnat, ladybird, housefly, moth, honeybee, wasp.

6.8 CONCLUSIONS AND DISCUSSION

In this chapter we have described clustering from the viewpoint of mining both homogeneous and heterogeneous data. Different distance measures have been defined for symbolic objects involving quantitative, binary, nominal (categorical) and ordinal attributes. Partitional and hierarchical methods, along with some scalable versions of clustering algorithms have been discussed. Use of soft computing-based approaches, involving fuzzy sets, neural networks, rough sets, genetic algorithms, and wavelets, are highlighted. Finally, clustering of symbolic objects has been outlined. Clustering has useful applications in data mining, pattern recognition, image segmentation, and Web mining.

Segmentation is a process of partitioning an image space into nonoverlapping meaningful homogeneous regions. The success of an image analysis system depends on the quality of segmentation. Remote sensing image data, containing observations from the Indian Remote Sensing (IRS) satellite for the city of Kolkata, India, has been segmented using fuzzy clustering based on thresholding [50]. The data contain images of four spectral bands and consist of six categories, namely, clear water (ponds, fisheries), turbid water (the river Ganga flowing through the city), concrete (buildings, roads, airport tarmac), habitation (concrete structures of lower density), vegetation (crop, forest areas), and open spaces (barren land, playgrounds). A rough-fuzzy hybridization of the EM algorithm [51], employing pixel classification, has been used to segment the multispectral satellite images into different landcover types.

The importance of clustering to Web mining, specifically in the domains of Web Content and Web Usage mining, make Web clustering an interesting topic of research. This includes clustering of Web documents, snippets and access logs. Usually the Web involves overlapping clusters. So a crisp usage of metrics is better replaced by fuzzy sets that can reflect, in a more natural manner, the degree of belongingness/membership to a cluster. A review of robust methods used for fuzzy Web clustering is presented in Ref. [52]. A robust algorithm is one whose performance is minimally affected in the presence of outliers. The algorithms considered include fuzzy *c*-means (FCM), fuzzy trimmed *c*-prototype, fuzzy *c*-least medians, and relational fuzzy *c*-means clustering, along with some techniques used to make them robust. These issues are further elaborated in Section 9.5.

The importance of symbolic clustering in real-world data is all the more evident, considering the availability of large volumes of mixed-media data that are distributed over the Internet. It enables efficient handling of heterogeneous data such as linguistic, nominal, boolean, interval, shape, color, etc. Partitioning of such data demands the use of symbolic measures for determining the similarity and dissimilarity between objects. The validity indices, expressed here in the symbolic framework, are generally found to provide better partitioning as compared to Conceptual clustering.

The determination of the optimal number of clusters is an open problem. The number of meaningful clusters selected, however, depends on the application domain. For example, if one desires plain clustering or segmentation, then one should go for coarser granules. On the other hand, if the goal is data condensation for data mining, then one should concentrate on finer granules as representative points.

REFERENCES

1. A. K. Jain and R. C. Dubes, *Algorithms for Clustering Data*. Englewood Cliffs, NJ: Prentice-Hall, 1988.
2. A. K. Jain, M. N. Murty, and P. J. Flynn, "Data clustering: A review," *ACM Computing Surveys*, vol. 31, pp. 264–323, 1999.
3. J. T. Tou and R. C. Gonzalez, *Pattern Recognition Principles*. London: Addison-Wesley, 1974.
4. L. Kaufman and P. J. Rousseeuw, *Finding Groups in Data: An Introduction to Cluster Analysis*. New York: John Wiley & Sons, 1990.
5. R. Ng and J. Han, "Efficient and effective clustering method for spatial data mining," in *Proceedings of 1994 International Conference on Very Large Data Bases (VLDB '94)* (Santiago, Chile), pp. 144–155, September 1994.
6. M. Ester, H. P. Kriegel, and X. Xu, "Knowledge discovery in large spatial databases: Focusing techniques for efficient class identification," in *Proceedings of 4th International Symposium on Large Spatial Databases (SSD '95)* (Portland, OR), pp. 67–82, August 1995.
7. J. C. Bezdek and N. R. Pal, "Some new indexes for cluster validity," *IEEE Transactions on Systems, Man, and Cybernetics, Part-B*, vol. 28, pp. 301–315, 1998.
8. J. Han and M. Kamber, *Data Mining: Concepts and Techniques*. San Diego: Academic Press, 2001.
9. S. Mitra, S. K. Pal, and P. Mitra, "Data mining in soft computing framework: A survey," *IEEE Transactions on Neural Networks*, vol. 13, pp. 3–14, 2002.
10. K. Chidananda Gowda and E. Diday, "Unsupervised learning through symbolic clustering," *Pattern Recognition Letters*, vol. 12, pp. 259–264, 1991.

11. K. Chidananda Gowda and T. V. Ravi, "Divisive clustering of symbolic objects using the concepts of both similarity and dissimilarity," *Pattern Recognition*, vol. 28, pp. 1277–1282, 1995.
12. R. Wilson and T. R. Martinez, "Improved heterogeneous distance functions," *Journal of Artificial Intelligence Research*, vol. 6, pp. 1–34, 1997.
13. G. Biswas, J. B. Weinberg, and D. H. Fisher, "ITERATE: A conceptual clustering algorithm for data mining," *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, vol. 28, pp. 219–230, 1998.
14. P. A. Devijver and J. Kittler, eds., *Pattern Recognition Theory and Applications*. Berlin: Springer-Verlag, 1987.
15. P. Bradley, U. M. Fayyad, and C. Reina, "Scaling clustering algorithms to large databases," in *Proceedings of 4th International Conference on Knowledge Discovery and Data Mining* (California), pp. 9–15, AAAI, 1998.
16. M. Ester, H. P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases," in *Proceedings of 1996 International Conference on Knowledge Discovery and Data Mining (KDD '96)* (Portland, OR), pp. 226–231, August 1996.
17. M. Ankerst, M. Breunig, H. P. Kriegel, and J. Sander, "OPTICS: Ordering points to identify the clustering structure," in *Proceedings of 1999 ACM-SIGMOD International Conference on Management of Data (SIGMOD '99)* (Philadelphia, PA), pp. 49–60, June 1999.
18. A. Hinneburg and D. A. Keim, "An efficient approach to clustering in large multimedia databases with noise," in *Proceedings of 1998 International Conference on Knowledge Discovery and Data Mining (KDD '98)* (New York), pp. 58–65, August 1998.
19. R. Agrawal, J. Gehrke, D. Gunopulos, and P. Raghavan, "Automatic subspace clustering of high dimensional data for data mining applications," in *Proceedings of 1998 ACM-SIGMOD International Conference on Management of Data (SIGMOD '98)* (Seattle, WA), pp. 94–105, June 1998.
20. J. Sander, M. Ester, H. P. Kriegel, and X. Xu, "Density-based clustering in spatial databases: The algorithm GDBSCAN and its applications," *Data Mining and Knowledge Discovery*, vol. 2, pp. 1–27, 1998.
21. X. Xu, J. Jaeger, and H. P. Kriegel, "A fast parallel clustering algorithm for large spatial databases," *Data Mining and Knowledge Discovery*, vol. 3, pp. 263–290, 1999.

22. T. Zhang, R. Ramakrishnan, and M. Livny, "BIRCH: An efficient data clustering method for very large databases," in *Proceedings of 1996 ACM-SIGMOD International Conference on Management of Data (SIGMOD '96)* (Montreal, Canada), pp. 103–114, June 1996.
23. S. Guha, R. Rastogi, and K. Shim, "An efficient clustering algorithm for large databases," in *Proceedings of 1998 ACM-SIGMOD International Conference on Management of Data (SIGMOD '98)* (Seattle, WA), pp. 73–84, June 1998.
24. G. Karypis, E. H. Han, and V. Kumar, "CHAMELEON: A hierarchical clustering algorithm using dynamic modelling," *Computer*, vol. 32, pp. 68–75, 1999.
25. W. Wang, J. Yang, and R. Muntz, "STING: A statistical information grid approach to spatial data mining," in *Proceedings of 1997 International Conference on Very Large Data Bases (VLDB '97)* (Athens, Greece), pp. 186–195, August 1997.
26. C. C. Aggarwal and P. S. Yu, "Redefining clustering for high-dimensional applications," *IEEE Transactions on Knowledge and Data Engineering*, vol. 14, pp. 210–225, 2002.
27. J. C. Bezdek, *Pattern Recognition with Fuzzy Objective Function Algorithms*. New York: Plenum Press, 1981.
28. W. Pedrycz, "Conditional fuzzy c-means," *Pattern Recognition Letters*, vol. 17, pp. 625–632, 1996.
29. R. Krishnapuram, A. Joshi, O. Nasraoui, and L. Yi, "Low complexity fuzzy relational clustering algorithms for web mining," *IEEE Transactions on Fuzzy Systems*, vol. 9, pp. 595–607, 2001.
30. W. Pedrycz and A. Bargiela, "Granular clustering: A granular signature of data," *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics*, vol. 32, pp. 212–224, 2002.
31. T. Kohonen, *Self-Organization and Associative Memory*. Berlin: Springer-Verlag, 1989.
32. G. A. Carpenter and S. Grossberg, "The ART of adaptive pattern recognition by a self organizing neural network," *IEEE Computer*, pp. 77–88, March 1988.
33. T. Kohonen, S. Kaski, K. Lagus, J. Salojärvi, J. Honkela, V. Paatero, and A. Saarela, "Self organization of a massive document collection," *IEEE Transactions on Neural Networks*, vol. 11, pp. 574–585, 2000.
34. J. Vesanto and E. Alhoniemi, "Clustering of the self-organizing map," *IEEE Transactions on Neural Networks*, vol. 11, pp. 586–600, 2000.

35. D. Alahakoon, S. K. Halgamuge, and B. Srinivasan, "Dynamic self-organizing maps with controlled growth for knowledge discovery," *IEEE Transactions on Neural Networks*, vol. 11, pp. 601–614, 2000.
36. G. Sheikholeslami, S. Chatterjee, and A. Zhang, "WaveCluster: A multi-resolution clustering approach for very large spatial databases," in *Proceedings of 1998 International Conference on Very Large Data Bases (VLDB '98)* (New York), pp. 428–439, August 1998.
37. P. Lingras and C. West, "Interval set clustering of Web users with rough k -means," Technical Report No. 2002-002, Dept. of Mathematics and Computer Science, St. Mary's University, Halifax, Canada, 2002.
38. S. Kirkpatrick, C. D. Gelatt Jr., and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, pp. 671–680, 1983.
39. F. Glover and M. Laguna, *Tabu Search*. Boston: Kluwer Academic Publishers, 1997.
40. S. C. Chu, J. F. Roddick, and J. S. Pan, "A comparative study and extensions to k -medoids algorithms," in *Proceedings of Fifth International Conference on Optimization: Techniques and Applications* (Hong Kong, China), pp. 1708–1717, 2001.
41. D. Gibson, J. M. Kleinberg, and P. Raghavan, "Clustering categorical data: An approach based on dynamical systems," in *Proceedings of 1998 International Conference on Very Large Data Bases (VLDB '98)* (New York), pp. 311–323, August 1998.
42. S. Guha, R. Rastogi, and K. Shim, "ROCK: A robust clustering algorithm for categorical attributes," in *Proceedings of 1999 International Conference on Data Engineering (ICDE '99)* (Sydney, Australia), pp. 512–521, March 1999.
43. V. Ganti, J. E. Gehrke, and R. Ramakrishnan, "CACTUS: Clustering categorical data using summaries," in *Proceedings of 1999 International Conference on Knowledge Discovery and Data Mining (KDD '99)* (San Diego, CA), 1999.
44. Z. Huang, "Extensions to the k -means algorithm for clustering large data sets with categorical values," *Data Mining and Knowledge Discovery*, vol. 2, pp. 283–304, 1998.
45. H. Ralambondrainy, "A conceptual version of the k -means algorithm," *Pattern Recognition Letters*, vol. 16, pp. 1147–1157, 1995.
46. K. Mali and S. Mitra, "Clustering of symbolic data and its validation," in *Advances in Soft Computing-AFSS 2002* (N. R. Pal and M. Sugeno,

- eds.), *Lecture Notes in Artificial Intelligence*, pp. 339–344, Heidelberg: Springer-Verlag, 2002.
47. K. Mali and S. Mitra, "Clustering and its validation in a symbolic framework," *Pattern Recognition Letters*, 2003 (to appear).
 48. D. Fisher, "Improving inference through conceptual clustering," in *Proceedings of 1987 AAAI Conference* (Seattle, WA), pp. 461–465, July 1987.
 49. C. Blake and C. Merz, "UCI repository of machine learning databases," 1998. University of California, Irvine, Dept. of Information and Computer Sciences, <http://www.ics.uci.edu/~mllearn/MLRepository.html>.
 50. S. K. Pal, A. Ghosh, and B. UmaShankar, "Segmentation of remotely sensed images with fuzzy thresholding, and quantitative evaluation," *International Journal of Remote Sensing*, vol. 21, pp. 2269–2300, 2000.
 51. S. K. Pal and P. Mitra, "Multispectral image segmentation using rough set initialized EM algorithm," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 40, pp. 2495–2501, 2002.
 52. A. Joshi and R. Krishnapuram, "Robust fuzzy clustering methods to support web mining," in *Proceedings of ACM-SIGMOD Workshop on Data Mining and Knowledge Discovery*, August 1998.

Association Rules

7.1 INTRODUCTION

Rule mining constitutes another major function of data mining. Here the relationship between attributes is expressed in the form of rules, thereby enhancing the understandability of the underlying information to users. As explained in Section 1.10, rules can broadly be of three types, namely, association rules, dependency rules and classification rules. In this chapter we dwell on association rules.

Let us consider a database of customer transactions T , where each transaction is a set of items. The objective is to find all rules of the form $X \Rightarrow Y$, which correlate the presence of one set of items X with another set of items Y . An example of such a rule is

98% of people who purchase diapers and baby food also buy baby soap.

For this purpose, one needs to ensure that (i) support of X and Y are greater than a user threshold s , and (ii) conditional probability (confidence) of Y given X is greater than a user threshold c .

A rule must have some minimum user-specified *Confidence*. A rule

$$1 \ \& \ 2 \Rightarrow 3$$

is defined to have 90% confidence if when a customer bought items 1 and 2, in 90% of those cases, the customer also bought item 3. A rule must also have some minimum user-specified *Support*. This implies that the rule $1 \ \& \ 2 \Rightarrow 3$

should hold in some minimum percentage of transactions, in order to have business value.

There can be any number of items in the consequent and antecedent parts of a rule. It is also possible to specify constraints on rules, such as finding only those rules involving expensive imported products.

Sample applications of association rules include

- Market basket analysis.
- Attached mailing in direct marketing.
- Fraud detection for medical insurance and credit cards.
- Department store floor or shelf planning.

An association rule can generally be viewed as being defined over attributes of a relation and has the form $C_1 \Rightarrow C_2$, where C_1 and C_2 are conjunctions of conditions and each condition is either $a_i = v_i$ or $a_i \in [l_i, u_i]$ with v_i being a value (for categoric or numeric) and u_i, l_i being upper and lower bound values (for numeric) from the domain of the attribute a_i . The support of the rule $C_1 \Rightarrow C_2$ is the same as the support of the conjunction $C_1 \wedge C_2$, while its confidence is the ratio of the supports of conditions $C_1 \wedge C_2$ and C_1 .

The different types of association rules can be categorized as follows.

1. Boolean: Here the association is between presence or absence of items.
2. Quantitative: Here the attribute values are partitioned into intervals, such as the age or income of an individual.
3. Categorical: Here an attribute is categorical, like the make of a car.
4. Single dimensional: A sample rule is of the form
 $\text{buys}(X, \text{"computer"}) \Rightarrow \text{buys}(X, \text{"financial_management_software"})$.
5. Multidimensional: A sample rule is of the form
 $\text{age}(X, \text{"30...39"}) \wedge \text{income}(X, \text{"42K...48K"})$
 $\Rightarrow \text{buys}(X, \text{"high_resolution_TV"})$.
6. Multilevel: This involves rules with attributes at different levels of abstraction in a concept hierarchy.
 - (a) Considering "*laptop_computer*" to be at a lower level in the concept hierarchy, as compared to "*computer*," we have rules of the form
 $\text{age}(X, \text{"30...39"}) \Rightarrow \text{buys}(X, \text{"laptop_computer"})$ and
 $\text{age}(X, \text{"30...39"}) \Rightarrow \text{buys}(X, \text{"computer"})$, or
 - (b) when the attributes span across multiple hierarchies, involving concepts "*computer*" and "*printer*", we can have a cross-level rule involving "*computer*" and "*b/w_printer*" (at a lower level in the

second hierarchy) of the form
 “computer” $\Rightarrow$ “b/w_printer”.

In the following sections we provide an overview on the different kinds of association rules and their mining, as reported in the literature. Section 7.2 deals with some efficient algorithms for mining association rules, using the candidate generation and test methods. Section 7.3 gives some insight into depth-first search methods for rule mining. The concept of interestingness of rules is provided in Section 7.4. In Sections 7.5–7.8 we describe multilevel rules, online generation of rules, generalized rules, and scalable mining of rules, respectively. Section 7.9 deals with some other variants of association rule mining. Fuzzy association rules are reviewed in Section 7.10. Finally, Section 7.11 concludes the chapter.

7.2 CANDIDATE GENERATION AND TEST METHODS

In this section we describe the *A priori* algorithm [1, 2] and some of its variants, including the partition algorithm [3].

7.2.1 A priori algorithm

This algorithm computes *frequent itemsets* from a transactions database over multiple iterations. Each iteration involves (i) candidate generation and (ii) candidate counting and selection. Utilizing the knowledge about infrequent itemsets, obtained from previous iterations, the algorithm prunes *a priori* those candidate itemsets that cannot become frequent. After discarding every candidate itemset that has an infrequent subset, the algorithm enters the candidate counting step.

Table 7.1 Sample transactions to demonstrate association rule mining

Transaction ID	Purchased items
1	{1, 2, 3}
2	{1, 3}
3	{1, 4}
4	{2, 5, 6}

7.2.1.1 Example 1: Let us consider an example in Table 7.1, depicting the items purchased by customers in four transactions. For a minimum support of 50% (here, two transactions) and a minimum confidence of 50%, we have the following rules

- (i) $1 \Rightarrow 3$ with 50% support and 66% confidence;
- (ii) $3 \Rightarrow 1$ with 50% support and 100% confidence.

The objective is to generate confident rules, having at least the minimum confidence. The problem decomposition proceeds as follows:

- Find all sets of items that have minimum support, typically using the *Apriori* algorithm [1, 2]. This is the most expensive phase of the search, and involves lots of research for reducing the complexity.
- Use the *frequent itemsets* to generate the desired rules. Given m items there can be potentially 2^m frequent itemsets.

Consider Table 7.2. For the rule $1 \Rightarrow 3$, we have

$$\text{Support} = \text{Support}(\{1, 3\}) = 50\%$$

and

$$\text{Confidence} = \frac{\text{Support}(\{1, 3\})}{\text{Support}(\{1\})} = 66\%.$$

Table 7.2 Computation of frequent itemsets

Frequent itemset	Support (%)
{1}	75
{2}	50
{3}	50
{1, 3}	50

The *A priori* algorithm is outlined as follows. Let F_k be the set of frequent itemsets of size k , let C_k be the set of candidate itemsets of size k , and let F_1 be the set of large items. We start from $k = 1$.

1. for all items in frequent itemset F_k repeat steps 2–4.
2. Generate new candidates C_{k+1} from F_k .
3. for each transaction T in the database, increment the count of all candidates in C_{k+1} that are contained in T .
4. Generate frequent itemsets F_{k+1} of size k from candidates in C_{k+1} with minimum support.

The final solution is $\bigcup_k F_k$.

A key observation is that every subset of a frequent itemset is also frequent. This implies that a candidate itemset in C_{k+1} can be pruned if even one of its subsets is not contained in F_k .

Table 7.3 Example transactions database for frequent itemset generation

Transaction ID	Purchased items
1	{1, 3, 4}
2	{2, 3, 5}
3	{1, 2, 3, 5}
4	{2, 5}

Table 7.4 Stages of Apriori algorithm demonstrating frequent itemset generation

C_1	Count	Support (%)	F_1	C_2	Count	Support (%)	F_2
{1}	2	50	—	{2, 3}	2	50	—
{2}	3	75	{2}	{2, 5}	3	75	{2, 5}
{3}	3	75	{3}	{3, 5}	2	50	—
{4}	1	25	—	—	—	—	—
{5}	3	75	{5}	—	—	—	—

7.2.1.2 Example 2: Let us explain the *A priori* algorithm with an example database of transactions provided in Table 7.3. Consider Table 7.4, with a minimum *Support* > 50%. After the first scan of the database, we have the candidate itemsets C_1 along with their corresponding *Supports*, as {1} : 50%, {2} : 75%, {3} : 75%, {4} : 25%, and {5} : 75%.

The frequent itemsets F_1 consist of {2}, {3}, and {5}, each with *Support* of 75%.

Now the candidate itemsets C_2 are {2, 3}, {2, 5}, {3, 5}, with *Supports* of 50%, 75%, 50%, respectively.

The corresponding frequent itemset F_2 becomes {2, 5} with a *Support* of 75%.

The rules generated are

$$2 \Rightarrow 5 \text{ with } \textit{Confidence} = \frac{\textit{Support}\{2,5\}}{\textit{Support}\{2\}} = 100\% \text{ and}$$

$$5 \Rightarrow 2 \text{ with } \textit{Confidence} = \frac{\textit{Support}\{2,5\}}{\textit{Support}\{5\}} = 100\%.$$

However, in this method, multiple passes have to be made over the database for each different value of minimum support and confidence. This number can be as large as the longest frequent itemset. For very large databases of transactions, this may involve considerable input-output (I/O) and lead to unacceptable response times for online queries. Moreover, the potential

number of frequent itemsets is exponential to the number of different items, although the actual number of frequent itemsets can be considerably smaller.

Constraints are sometimes specified to focus only on the interesting portions of a database. An example could be to find the association rules where the prices of items are at most 200 dollars ($max < 200$). Incorporating constraints can result in efficiency. The anti-monotonicity property ensures that when an itemset violates the constraint, so does any of its supersets (say, $min > X$ or $max < X$). The *A priori* algorithm uses this property for pruning the rulebase.

7.2.2 Partition algorithm

The main characteristics of the partition algorithm [3] are given below.

- Logically divide the horizontal database into a number of nonoverlapping partitions, which can be accommodated in main memory.
- Each partition is read, and locally frequent itemsets are generated.
- All locally frequent itemsets are merged, and a second pass is made through all the partitions.
- Global counts of all chosen itemsets are obtained.

A key observation, here, is that a globally frequent itemset must be locally frequent in at least one of the non-overlapping partitions. This implies that all frequent itemsets are guaranteed to be found. It minimizes I/O by scanning the database only twice. In the first pass, it generates the set of all potentially or locally frequent itemsets, while in the second pass it counts their global support. However, the Partition algorithm may enumerate too many false positives (itemsets locally frequent in some partition but not globally frequent) in the first pass. Moreover, if this local frequent set does not fit in main memory then additional database scans are required.

7.2.3 Some extensions

There exist some other variants of the *A priori* algorithm. These include (i) consideration that a high confidence may not essentially imply high correlation, (ii) incorporation of correlations, and (iii) pruning of the large number of mined rules.

The DHP algorithm [4] involves hashing in the framework of the *A priori* algorithm. It tries to reduce the number of candidates by collecting approximate counts in the previous level. Here a k -itemset is considered to be in C_k only if it is hashed into a bucket satisfying the minimum support criterion. However, like *A priori*, it requires as many database passes as the longest itemset.

The method in Refs. [5] and [6] finds all frequent itemsets using random sampling. A *negative border* is indicative of infrequent itemsets, whose subsets are all frequent. The major steps are as follows.

- Scan the database to count support for frequent itemsets, and for itemsets in a negative border.
- If no itemset in the negative border is frequent, then no more passes over the database is needed.
- Otherwise, scan the database to count support for candidate itemsets generated from the negative border.

The negative border is useful in increasing the efficiency of generation of large itemsets and in derivation of *negative* association rules.

Dynamic Itemset Counting [7] dynamically counts the support for all supersets of an itemset (with frequent subsets) if, during a pass, an itemset becomes frequent. It counts candidates of varying lengths as the database scan progresses and is thereby able to reduce the number of scans over the *A priori* algorithm.

7.3 DEPTH-FIRST SEARCH METHODS

The *A priori* algorithm is a breadth-first search technique that exhaustively evaluates all possible combinations of itemsets at a particular level. An efficient variation of this is a depth-first search algorithm *Eclat* [8], used for association rule mining. A search tree is generated in terms of the itemsets using depth-first traversal. This results in an efficient utilization of the memory and has applicability to the handling of large data.

Often the *A priori* algorithm needs to generate a huge number of candidate sets. For example, in order to discover a frequent pattern of size 100, such as $\{a_1, \dots, a_{100}\}$, it needs to generate more than $2^{100} \approx 10^{30}$ candidates. Moreover, it may need to repeatedly scan the database while checking a large set of candidates using pattern matching.

The Frequent-Pattern growth (FP-growth) algorithm avoids this candidate generation phase all together, by employing a divide-and-conquer strategy [9]. It *compresses* the database representing frequent (or representative) items into a Frequent-Pattern tree (FP-tree), while retaining the itemset association information. The compressed database is divided into a set of conditional (or projected) databases, each associated with one frequent item. Finally, each compressed database is mined separately.

Example 3: Let us consider Tables 7.3 and 7.4 for describing the FP-growth algorithm. In the first scan of the transactional database we obtain the list of items, sorted in descending order on their counts as

$$L = \{(2, 3), (3, 3), (5, 3), (1, 2), (4, 1)\},$$

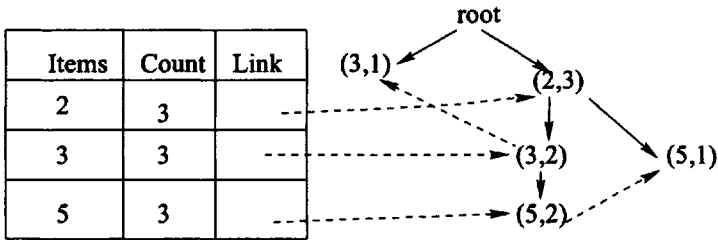


Fig. 7.1 An example FP-tree.

where the number following the comma indicates the count. Considering only those items having minimum support of 3, for simplified understanding, we have

$$L' = \{(2, 3), (3, 3), (5, 3)\}.$$

The resultant FP-tree of Fig. 7.1 is constructed as follows. First the *root* node is created. The database is scanned a second time, and the items in each transaction are processed in the order given in L' while creating a branch corresponding to each transaction. For example, the scan of the first transaction $\{1, 3, 4\}$ of Table 7.3 yields the item $\{3\}$, as only it has minimum support. This leads to the leftmost branch of the tree with node $(3, 1)$, with item $\{3\}$ being linked to the *root*. The second transaction $\{2, 3, 5\}$ is found to be in proper order as L' , and it generates a branch from the *root* with three nodes $(2, 1)$, $(3, 1)$, and $(5, 1)$. The third transaction $\{1, 2, 3, 5\}$ yields the itemset $\{2, 3, 5\}$ according to L' , and it shares the second branch of the tree. Hence the counts along these three nodes are incremented by one along the entire path, now resulting in nodes $(2, 2)$, $(3, 2)$, and $(5, 2)$. The fourth transaction $\{2, 5\}$, being again in order as L' , is used to generate the rightmost branch of the tree with a common prefix for item $\{2\}$. Therefore, the nodes along this path from the *root* now become $(2, 3)$ and $(5, 1)$.

Tree traversal is facilitated by constructing an *item header table*, as shown in Fig. 7.1, so that each item points to its node(s) of occurrence in the tree through a chain of dotted links. Now mining of frequent patterns in databases reduces to recursively mining the FP-tree. The algorithm starts from the last item in list L' , as an initial suffix pattern, and constructs its *conditional pattern base* or sub-database consisting of the set of *prefix paths* in the FP-tree co-occurring with it.

Example 4: Here we demonstrate the tree traversal method for generating frequent itemsets. Let us start from item $\{5\}$. Based on the node-link connections from the root of Fig. 7.1, the two paths selected are $\{(2, 3), (5, 1)\}$ and $\{(2, 3), (3, 2), (5, 2)\}$. We observe that the samples with a frequent item $\{5\}$ are $\{(2, 2), (3, 2), (5, 2)\}$ for transaction IDs 2, 3, and $\{(2, 1), (5, 1)\}$ for

transaction ID 4, from Table 7.3. The frequent patterns generated, observing minimum support from both these patterns, are therefore $\{(2,3), (5,3)\}$ or simply the itemset $\{2,5\}$.

Analyzing item $\{3\}$, we obtain paths $\{(3,1)\}$ and $\{(2,3), (3,2)\}$ from the root. However these lead to no frequent patterns. Hence the solution of the FP-growth method is the set of frequent itemsets $\{2,5\}$.

Instead of finding long frequent patterns, the FP-growth method resorts to searching recursively for shorter ones followed by a concatenation of the suffix. The use of the least frequent item as a suffix serves to significantly reduce the search costs. The FP-growth method is efficient and scalable for mining long frequent patterns. It is found to be faster [9] than the *Tree-projection* algorithm, which recursively projects a database into a tree of projected databases, as well as the *A priori* algorithm.

7.4 INTERESTING RULES

Unlike accuracy, which involves evaluating the performance of a large number of rules exhibiting typical behavior for a particular situation, *interestingness* measures the deviation from prior expectation of the general information inherent in these prevalent rules. Mining's payoff is in extracting such *surprising* phenomena that lie hidden in the data. The concept of surprisingness is explained from the information theoretic viewpoint in Section 3.2.1. Now let us outline what makes a rule surprising. It

- does not match prior expectation and
- cannot be trivially derived from simpler rules.

It is evident that rules having very low *interestingness* values need not be examined, as they are likely to contain well-known aspects about the domain. So we need some kind of measure to evaluate this interestingness, which can again be objective or subjective [10]. While objective measures categorize the interestingness of a pattern in terms of its structure and the underlying data used in the discovery process, the subjective measures additionally incorporate the views of the person inspecting the pattern. These lead to the development of parameters like unexpectedness and actionability. A rule is interesting to a user if it is unexpected (or surprising) and if (s)he can act on the knowledge provided by the rule to her/his advantage.

The different interestingness measures are provided below. Note that these are in addition to the quantitative measures of Section 2.4.2, used for evaluating the performance of rules.

1. *Simplicity*: This is desirable for human comprehension, as a more complex rule structure implies that it is more difficult to interpret and is

therefore less interesting. It is measured in terms of the rule length or the number of decision tree nodes or leaves.

2. *Certainty*: This is the reliability (or accuracy) in classification rules. It is measured as

$$\text{confidence}(X \Rightarrow Y) = \frac{\text{no.of_tuples_containing_both_X_and_Y}}{\text{no.of_tuples_containing_X}}. \quad (7.1)$$

3. *Utility*: This is measured as

$$\text{support}(X \Rightarrow Y) = \frac{\text{no.of_tuples_containing_both_X_and_Y}}{\text{total_no.of_tuples}}. \quad (7.2)$$

4. *Novelty*: This corresponds to patterns that contribute new information or increased performance. It implies some sort of data exception, which differs from what is expected based on a statistical model or user beliefs.
5. *Interest*: This also provides a measure of the new information provided, and it can be viewed as an estimation of the increase in probability of Y caused by the occurrence of X , that is, $P(Y|X) - P(Y)$. It is expressed as

$$\text{interest}(X \Rightarrow Y) = \text{confidence}(X \Rightarrow Y) - \frac{\text{no.of_tuples_containing_Y}}{\text{total_no.of_tuples}}. \quad (7.3)$$

7.5 MULTILEVEL RULES

Data can be generalized by replacing low-level concepts with their more general high-level concepts from a concept hierarchy. When a rule involves attributes from different levels in such a concept hierarchy, it is termed *Multilevel*. A rule is typically said to be *redundant* if it does not offer any additional information and is less general (namely, at a lower level).

Let us consider an example concept hierarchy of *computer* (support: 10%) at level 1, followed by descendants *laptop* (support: 6%) and *desktop* (support: 4%) at the next lower level 2. There exist several approaches to mine Multilevel rules. These are mentioned below.

- *Uniform minimum support* for all levels: The search avoids examining itemsets containing any item whose ancestors do not have the minimum support. For example, if the minimum support for the levels is 5%, then only the association involving *laptop* can be considered at level 2. However, if the minimum support threshold is set too high, it may miss several meaningful associations occurring at lower abstraction levels.

Moreover, if this threshold is too low, it may lead to the generation of many uninteresting associations at higher abstraction levels.

- *Reduced minimum support at lower levels:* Now let the minimum support at level 2 be reduced to (say) 3%. This overcomes the first problem mentioned above, so that the association involving *desktop* also becomes frequent at level 2.
- *Level-by-level independent:* Each node is examined regardless of whether or not its parent node is frequent. Here both concepts *laptop* and *desktop* are examined.
- *Level-cross filtering by single item:* An item at the i th level is examined if and only if its parent at the $(i - 1)$ th level is frequent. This may miss associations between low-level items that are frequent based on a reduced minimum support, but whose ancestors do not satisfy (higher) minimum support. For example, if the minimum support at levels 1 and 2 are set at 12% and 3% respectively, then the concepts at level 2 are not examined here since the concept at level 1 (i.e., *computer*) is not frequent.
- *Controlled level-cross filtering by single item:* Users may choose to lower the level passage threshold at higher concept levels to allow descendants of subfrequent items at lower levels to be examined. For example, if the level passage threshold at level 1 is now set at 8%, then the descendants of the subfrequent concept *computer* (support: 10%) can be involved for determining associations.
- *Level-cross filtering by k -itemset:* A k -itemset at the i th level is examined, if and only if its parent k -itemset at the $(i - 1)$ th level is frequent. This restriction can lead to filtering out of many valuable patterns. Let us now modify the above example to include the concept pair ($k = 2$) *computer & printer* (support: 7%) at level 1, followed by descendants *laptop & b/w printer* (support: 1%), *laptop & color printer* (support: 2%), *desktop & b/w printer* (support: 1%), and *desktop & color printer* (support: 3%) at level 2. If the minimum support at levels 1 and 2 are now set at 5% and 2%, respectively, then all 2-itemsets at level 2 can now be examined to determine associations.

7.6 ONLINE GENERATION OF RULES

It is often hard for a user to guess, *a priori*, how many rules might satisfy a given level of support and confidence. In most cases, one finds that the number of redundant rules is significantly larger than the number of essential rules. To circumvent these problems, an online algorithm is described in Ref. [11]. The data are preprocessed and stored such that a graph theoretic

search algorithm, with complexity proportional to the size of the output, may be applied for effectively handling repeated online queries. It is independent of the size of the transactional (preprocessed) data. The algorithm is capable of quickly discovering association rules, in compact form, from large itemsets with specific items in the antecedent or consequent.

In most cases, the number of redundant rules is significantly larger than the number of essential rules. This kind of redundancy arises when we consider rules having more than one item in the consequent. For example, if the rule $X \Rightarrow YZ$ is true for a given minimum support and confidence, then rules such as $XY \Rightarrow Z$, $XZ \Rightarrow Y$, $X \Rightarrow Y$, and $X \Rightarrow Z$ are said to be redundant. Mining of nonredundant association rules helps significantly in the reduction of irrelevant noise. The kinds of online queries that this system can support are outlined below.

1. Find all association rules above a certain level of *min_support* and *min_confidence*.
2. At a certain level of *min_support* and *min_confidence*, find all association rules concerned with the set of items X .
3. Find the number of association rules or itemsets in any of the above two cases.
4. Find the level of *min_support* at which exactly k -itemsets exist containing the set of items Z .
5. For a particular level of *min_confidence* c , find the level of *min_support* at which exactly k single-consequent rules exist involving the set of items Z .

An itemset X is said to be *adjacent* to an itemset Y , if one of them can be obtained from the other by adding a single item. An itemset X is called the parent of itemset Y (descendant or child) if the latter can be generated from the former by adding one item. The algorithm proceeds by forming an *adjacency lattice* L , which is a directed acyclic graph. For each vertex $v(J)$ in L which is a descendant of $v(I)$, support $S(J) \leq S(I)$. The number of edges in the adjacency lattice is equal to the sum of the number of items in the primary itemsets. For a given itemset I (including the null set $\{\}$), the algorithm finds all itemsets J such that $v(J)$ is reachable from $v(I)$ by a directed path in the lattice L and satisfies $S(J) \geq s$, where s indicates minimum support.

The adjacency lattice, so generated from the transactions, is then traversed to mine association rules.

7.7 GENERALIZED RULES

Items in a database are often stored as concept hierarchies. Often there exist associations across these hierarchies, resulting in generalized association rules.

Let us consider a hierarchy of *clothes*, categorized as *outwear* (consisting of *jackets* and *pants*) and *shirts*. Let there be another hierarchy of *footwear* grouped as *shoes* and *hiking boots*. In this case, a rule *clothes* $\Rightarrow$ *footwear* may hold even if the rule *clothes* $\Rightarrow$ *shoes* does not hold (depending upon the corresponding levels of confidence and support). Approaches to handle such rules are provided in the literature [12, 13].

A graph-based approach [14] is used to discover primitive (conventional), generalized, and multilevel association rules. In a concept hierarchy, the terminal nodes are the database items while the nonterminal nodes are the generalized items. Here the database is scanned only once to construct an *association graph*, which is then traversed to generate all large itemsets. The basic steps of the algorithm are as follows.

- Numbering phase: Here all items are assigned an integer value.
- Large item generation: This consists of determining the items with support not less than a user specified minimum support.
- Association graph construction: This consists of determining the association between large items.
- Association pattern generation: This is done by traversing the association graph.
- Association rule generation: Here one generates all generalized and multilevel rules.

Table 7.5 Transactions demonstrating Bit vector computation

Transaction ID	Itemset
1	CAD
2	ECB
3	ABCE
4	EB

The algorithm scans the database and computes a *Bit vector* for each item. The length of the vector is equal to the number of transactions in the database. If an item appears in the i th transaction, the i th bit of the corresponding bit vector BV_i is set to 1, else it is 0.

Example 5: Table 7.5 depicts a sample set of transactions of items to demonstrate Bit vector generation. Let the minimum support be 50%, that is, 2 transactions. The BV s for items A, B, C, E are (1010), (0111), (1110), (0111), respectively. The support for itemset $\{i_1, \dots, i_k\}$ is the number of

ones in $BV_{i_1} \wedge \dots \wedge BV_{i_k}$. For every two large items i, j ($i < j$), if the number of ones in $BV_i \wedge BV_j$ achieves the user-specified minimum support, then a directed edge from item i to item j is created in the association graph. Considering itemset $\{i, j\}$ to be a large 2-itemset, sample rules generated from Table 7.5 are $A \Rightarrow C$ (with support 2); $B \Rightarrow C, E$ (with supports 2,3 respectively); and $C \Rightarrow E$ (with support 2).

However, the algorithm assumes that the bit vectors fit in main memory, and thus scalability could be a problem for databases with millions of transactions.

7.8 SCALABLE MINING OF RULES

The idea of scalable mining in large databases is important because the search space is often exponential in the number of database attributes. Most current approaches are iterative in nature, requiring multiple database scans. This makes them very expensive. Use of sampling can be sensitive to data skew, and this can adversely affect the performance of an algorithm. Moreover, most existing approaches use very complicated data structures involving poor locality and thereby incur additional space and computational overheads.

A scalable algorithm, described in Ref. [15], utilizes the structural properties of frequent itemsets to facilitate fast discovery. Efficient lattice traversal techniques are presented to quickly identify all the long frequent itemsets and their subsets (if required). The approach is insensitive to data skew, and the simple intersection operations involved make it an attractive option for direct implementation in distributed systems using SQL. Its main characteristics are as follows.

- It uses a vertical transactions ID (TID)-list, associating each itemset with the list of transactions in which it occurs. All frequent itemsets are enumerated via TID-list intersections.
- The original search space (lattice) is decomposed into smaller sublattices, which can be processed independently in the main memory. One can use prefix-based and maximal-clique¹-based partition for this purpose.
- Bottom-up, top-down and efficient hybrid searches are used for enumerating the frequent items within each sublattice.
- The algorithm requires only a few database scans, thereby minimizing I/O costs. It also retains linear scalability in the number of transactions.

¹A graph consists of a set of vertices and edges. A graph is complete if there is an edge between all pairs of vertices. A complete subgraph of a graph is called a *clique*.

- Some of the assertions made in the algorithm include the following.
 - All subsets of a frequent itemset are frequent.
 - The maximal frequent itemsets uniquely determine all frequent itemsets.
 - Two itemsets are in the same class if they share a common k length *prefix*.

The algorithm is found to outperform several approaches like *A priori* and Partition (described in Section 7.2).

7.9 OTHER VARIANTS

In this section we dwell on some variants like quantitative association rules, temporal association rules, correlation rules, localized associations, and optimized association rules.

7.9.1 Quantitative association rules

As mentioned earlier, a boolean association rule is an implication written as $X \Rightarrow Y$ among items (or attributes) X and Y . The support of the rule is s , if s tuples contain both X and Y as attributes. The confidence of the rule is c ($c < 1$, expressed as a percentage) if c of all tuples that contain X also contain Y .

A more general form of such rules is the quantitative association rule [16]. A sample quantitative association rule is

10% of married people with age between 50 and 70 have at least 2 cars.

The algorithm involves discretizing the domains of quantitative attributes into intervals, in order to reduce the domain into a categorical one. Here A and B are vectors of intervals, each of which is in one of the attributes in X and Y , respectively. The support of the rule is s , if s tuples fall in intervals of A and B , when projected to attributes X and Y respectively. The confidence of the rule is c , if c of all the tuples that fall in intervals in A also fall in intervals in B .

7.9.2 Temporal association rules

Temporal rules can be used to describe the rich temporal character inherent in the data. Consider a sample rule *diaper* $\Rightarrow$ *baby food*, with a support of 5% and confidence of 87%. The support of this rule may jump to 25% between 6 to 9 PM on weekdays. The problem is to determine how to find rules that follow some interesting user-defined temporal patterns. The challenge is to

design efficient algorithms that do much better than finding every rule in every time unit. Approaches in this direction are provided in the literature [17, 18].

7.9.3 Correlation rules

Association rules, typically, do not capture correlations. For example, suppose 90% customers buy coffee, 25% buy tea, and 20% buy both tea and coffee. The rule *tea* $\Rightarrow$ *coffee* has a high support of 0.2 and confidence 0.8. But the items {tea, coffee} are not correlated. The expected support of customers buying both is $0.9 \times 0.25 = 0.225$.

In correlation rules, the correlation between the occurrences of X and Y is expressed as

$$corr_{X,Y} = \frac{P(X \cup Y)}{P(X)P(Y)}, \quad (7.4)$$

where a value < 1 indicates a negative correlation, $= 1$ signifies independence, and > 1 implies a positive correlation between the items, and $P(\cdot)$ refers to the corresponding probability.

Fast itemset counting has been used to find correlated events. This has applications in (i) medicine, to find redundant tests, (ii) cross selling in retail and (iii) banking. These rules help improve the predictive capability of classifiers that assume attribute independence.

7.9.4 Localized associations

Aggarwal et al. [19] have designed an algorithm effective in discovering localized associations, from individual segments, that expose a customer pattern which is more specific than the aggregate behavior. Such personalized associations are applicable to more useful target marketing. For example, those transactions which are drawn from extremely cold geographical regions may contain correlations corresponding to heavy winter apparel, whereas these may not be present in the aggregate data because they are often not present to a very high degree in the rest of the database. An attempt to find such correlations, using global analysis by lowering the support, will result in finding a large number of uninteresting and redundant "correlations" which are created simply by chance throughout the dataset.

The CLustering for ASsociation Discovery (CLASD) algorithm [19] can also be used to segment categorical data. Concepts from both agglomerative and partitional clustering are used in conjunction with random sampling, in order to make it robust, practical, and scalable for very large databases. The affinity $A(i, j)$ between two items i and j is expressed as

$$A(i, j) = \frac{sup(\{i, j\})}{sup(\{i\}) + sup(\{j\}) - sup(\{i, j\})}, \quad (7.5)$$

where $\text{sup}(X)$ denotes the aggregate support of a set of items X . The *similarity* between a pair of transactions $T = \{i_1, \dots, i_m\}$ and $T' = \{j_1, \dots, j_n\}$ is defined as the average affinity of their items. Here

$$\text{Sim}(T, T') = \frac{\sum_{p=1}^m \sum_{q=1}^n A(i_p, j_q)}{m * n}. \quad (7.6)$$

The overall approach uses a set of cluster representatives or seeds to create the partitions and is always larger than the final number of clusters c . In each iteration, the closest pair of representatives are merged to reduce their number by a factor α . Random sampling is performed on the database to assign the transactions to seeds. The value of the sample size is also increased by a factor α in each iteration. Thus later phases of the clustering benefit from larger sample sizes, and robust computations are possible in later iterations.

7.9.5 Optimized association rules

Association rules are built from atomic conditions. For the atomic condition $a_i = v_i$, if v_i is a value from the domain of attribute a_i , the condition is referred to as *instantiated*; else, if v_i is a variable, the condition is termed as *uninstantiated*. Optimized association rules are permitted to contain uninstantiated attributes. The problem is to determine instantiations, such that either the support or confidence of the rule is maximized. Optimized association rules are generalized [20] by allowing the rules to contain disjunctions over an arbitrary number of uninstantiated attributes, having either categorical or numeric values. Useful information is extracted about seasonal and local patterns involving multiple attributes. Pruning is incorporated using a combination of depth-first search, branch and bound techniques, and a graph search algorithm.

7.10 FUZZY ASSOCIATION RULES

Quantitative association rules, as mentioned in Section 7.9.1, require the specification of appropriate intervals along each attribute. However, these intervals may often not be concise and meaningful enough for human experts to discover nontrivial knowledge. Fuzzy sets can be used from this perspective to represent intervals with non-sharp boundaries, thereby generating fuzzy association rules (introduced in Section 2.3.3). Assignment of meaningful linguistic terms to the fuzzy sets makes these rules more understandable. A sample fuzzy association rule is

10% of married old people have at least several cars.

Algorithms for mining fuzzy association rules are provided in the literature [21]. Typically, an expert is required to provide the fuzzy sets for the

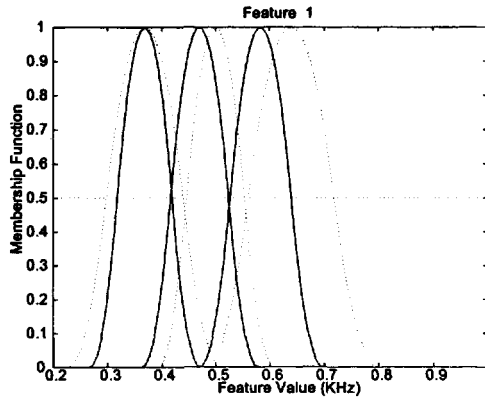


Fig. 7.2 Overlapping fuzzy sets.

quantitative attributes, along with their corresponding membership functions. Sometimes these functions are also determined by clustering and subsequent tuning. Pruning is another feature necessary to retain only the *interesting* rules.

Fuzzy set concepts are integrated with the *A priori* algorithm (Section 7.2.1) to mine interesting fuzzy association rules, using the Fuzzy Transaction Data mining Algorithm (FTDA) [21]. Quantitative values in transactions are transformed to natural and more understandable linguistic terms, which are then filtered to generate fuzzy associations.

Let there be N transactions data each with n attribute values, a set of membership functions, and predefined minimum support s and confidence c . The steps of the FTDA are outlined as follows.

1. Transform the quantitative value a_{ij} of each transaction t^i , $i = 1, \dots, N$, for each attribute a_j , $j = 1, \dots, n$, into a fuzzy set μ_{ij_k} represented as

$$\left(\frac{\mu_{ij_1}}{R_{j_1}} + \frac{\mu_{ij_2}}{R_{j_2}} + \dots + \frac{\mu_{ij_l}}{R_{j_l}} \right),$$

using the given membership functions. Here R_{j_k} is the k th linguistic partition (e.g., *low*, *medium*, *high*, etc.) of attribute a_j , μ_{ij_k} is the fuzzy membership value for a_{ij} in region R_{j_k} , and $l (= |a_j|)$ is the number of fuzzy partitions used for a_j .

(**Example:** Consider Fig. 7.2. Let there be a quantitative input feature (in KHz), denoted as a_j . The corresponding value of transaction t^i along this axis is given as a_{ij} . Using fuzzy sets, we split this axis into l overlapping partitions of linguistic terms, say, *low*, *medium*, and *high*. Let these partitions be represented as R_{j_1} for *low*, R_{j_2} for *medium*, and R_{j_3} for *high*, with centers r_{j_1} , r_{j_2} , r_{j_3} , respectively. The three

overlapping membership functions (drawn with solid lines) indicate low, medium, and high regions of the feature axis, from left to right. Here we use π membership functions, as explained in Section 2.2.2.1. The overlap between adjacent partitions is termed p , uniformly for each pair, along the axis. A quantitative feature value a_{ij} is hence mapped to a three-dimensional vector $[\mu_{ij_1}, \mu_{ij_2}, \mu_{ij_3}]$, whose components (lying in $[0,1]$) refer to the membership value to the corresponding linguistic term (low R_{j_1} , medium R_{j_2} , or high R_{j_3}). Note that the membership function at each of the centers $(r_{j_1}, r_{j_2}, r_{j_3})$ has a value of 1.)

2. Calculate the scalar cardinality of each attribute region R_{j_k} in the transaction data as

$$count_{j_k} = \sum_{i=1}^N \mu_{ij_k}. \quad (7.7)$$

3. Find $count_j^{\max} = \max_{k=1}^{|a_j|} count_{j_k}$, for $j = 1, \dots, n$. Let $R_j^{\max}$ be the linguistic partition with $count_j^{\max}$ for attribute a_j .
4. Check whether the $count_j^{\max}$ of each $R_j^{\max}$, $j = 1, \dots, n$, is larger than or equal to the support s , and then put it in the set of frequent 1-itemsets (F_1). Therefore

$$F_1 = \{R_j^{\max} | count_j^{\max} \geq s\} \quad (7.8)$$

for $1 \leq j \leq n$.

5. Set $r = 1$, where r represents the number of items kept in the current frequent itemsets.
6. Generate the candidate set C_{r+1} from F_r in a manner similar to that in the *A priori* algorithm.
7. **for** each newly formed $(r + 1)$ -itemset R [corresponding to the $R_j^{\max}$ along j th axis by Eq. (7.8)], with items $(R_1, R_2, \dots, R_{r+1})$ in C_{r+1} , **do** the following substeps.

- (a) Calculate the fuzzy value of each transaction data t^i in R as $\mu_{iR} = \mu_{iR_1} \wedge \mu_{iR_2} \wedge \dots \wedge \mu_{iR_{r+1}}$, where μ_{iR_j} is the membership value of t^i in region R_j . If the minimum operator is used for the intersection, we have

$$\mu_{iR} = \min_{j=1}^{r+1} \mu_{iR_j}. \quad (7.9)$$

- (b) Calculate the scalar cardinality of R in the transaction data as

$$count_R = \sum_{i=1}^N \mu_{iR}. \quad (7.10)$$

- (c) if count_R is larger than or equal to the support s then put R in F_{r+1} .
8. if F_{r+1} is null then go to the next step; else set $r = r + 1$ and repeat steps 6–8.
9. Construct the association rules for all frequent q -itemset R with items $(R_1, R_2, \dots, R_q)$, $q \geq 2$, using the following substeps.
- (a) Form all possible association rules as

$$R_1 \wedge \dots \wedge R_{k-1} \wedge R_{k+1} \wedge \dots \wedge R_q \rightarrow R_k, \quad (7.11)$$

for $k = 1, \dots, q$.

- (b) Calculate the confidence values of all association rules using

$$\frac{\sum_{i=1}^N \mu_{iS}}{\sum_{i=1}^N (\mu_{iR_1} \wedge \dots \wedge \mu_{iR_{k-1}} \wedge \mu_{iR_{k+1}} \wedge \dots \wedge \mu_{iR_q})}.$$

10. Output the rules with confidence values larger than or equal to the pre-defined confidence c .

The rules mined can serve as meta-knowledge concerning the given transactions. Since each attribute uses only the linguistic term with the maximum cardinality, the number of items remains the same as that of the original attributes. Instead, if in step 4, all the regions with support values larger than the threshold are considered, then the algorithm would generate more rules while also consuming more time. Thus trade-off exists between rule completeness and time complexity.

Determination of appropriate fuzzy sets R_{j_k} , $k = 1, \dots, l$, for quantitative attribute a_j , is made using clustering. Here optimization of a goodness index is made based on cluster compaction and separation [22]. The number l of such clusters and their corresponding updated centers r_{j_i} , $i = 1, \dots, l$, are obtained. Let the initial set of overlapping membership functions be indicated by the dotted line in Fig. 7.2. Clustering is used to tune the centers of these functions, thereby translating the membership functions along the feature axis. The final situation, after updating, is depicted by the solid line in the figure.

Initially a quantitative attribute interval is divided into l linguistic sub-intervals around the cluster centers, with a coverage (or overlapping) of $p\%$ between adjacent linguistic partitions. The non-fuzzy partitioning is obtained as a special case with $p = 0$. For the i th linguistic fuzzy set R_{j_i} , the effective upper and lower bounds $d_{j_i}^+$ and $d_{j_i}^-$ are expressed as

$$d_{j_i}^+ = r_{j_i} + \frac{1}{100} * \left(\frac{100 - p}{2} \right) * (r_{j_{i+1}} - r_{j_i}), \quad (7.12)$$

$$d_{ji}^- = r_{ji} - \frac{1}{100} * \left(\frac{100 - p}{2} \right) * (r_{ji} - r_{ji-1}), \quad (7.13)$$

respectively, with a uniform overlap p between each pair of adjacent partitions.

Interestingness measures, based on statistics and information theory, have been designed [22] for fuzzy association rules. Given a database $D = \{t^1, t^2, \dots, t^N\}$ with N transactions, let (Z, C) be an attribute-fuzzy set pair (itemset) for attribute $Z = X \cup Y$ having associated fuzzy set $C = A \cup B$. Here X is A is the antecedent, and Y is B the consequent of the fuzzy association rule, such that X and Y are disjoint.

If the rule $(X, A) \Rightarrow (Y, B)$ is interesting, it should have enough *fuzzy support* $FS_{(Z,C)}$ and a high *fuzzy confidence* $FC_{((X,A),(Y,B))}$. These are computed as

$$FS_{(Z,C)} = \frac{\sum_{i=1}^N \prod_{j=1}^m t^i(Z_j, C_j)}{N}, \quad (7.14)$$

where m is the number of items in itemset (Z, C) , and

$$FC_{((X,A),(Y,B))} = \frac{FS_{(Z,C)}}{FS_{(X,A)}}. \quad (7.15)$$

Fuzzy covariance is based on the co-occurrence of the antecedent (X, A) and consequent (Y, B) and is defined as

$$Cov_{((X,A),(Y,B))} = FS_{(Z,C)} - FS_{(X,A)} * FS_{(Y,B)}. \quad (7.16)$$

Fuzzy correlation, additionally, takes the pattern distribution into consideration. It is expressed as

$$Corr_{((X,A),(Y,B))} = \frac{Cov_{((X,A),(Y,B))}}{\sqrt{Var_{(X,A)} * Var_{(Y,B)}}}, \quad (7.17)$$

where $Var_{(X,A)} = FS_{(X,A)}^2 - (FS_{(X,A)})^2$ and $FS_{(X,A)}^2 = \frac{\sum_{i=1}^N (\prod_{j=1}^m t^i(X_j, A_j))^2}{N}$ for (X, A) . In both these cases, only a positive value implies that the antecedent and consequent are related: The higher the value, the greater their relation.

Let us now describe the information theoretic measures. The *independence entropy* $H_{((X,A),(Y,B))}$ represents the amount of information needed per transaction, using a (false) assumption of independence, when the true probability is $FS_{(Z,C)}$ [Eq. (7.14)] with true entropy $H_{(Z,C)}$ for (Z, C) . Here

$$H_{((X,A),(Y,B))} = -FS_{(Z,C)} \log_2 (FS_{(X,A)} FS_{(Y,B)}) - (1 - FS_{(Z,C)}) \log_2 (1 - FS_{(X,A)} FS_{(Y,B)})$$

and

$$H_{(Z,C)} = -FS_{(Z,C)} * \log_2 FS_{(Z,C)} - (1 - FS_{(Z,C)}) * \log_2 (1 - FS_{(Z,C)}).$$

[Note the analogy with the conventional entropy expression of Eq. (5.1)]. Using these, we obtain a good measure of correlation termed *unconditional entropy* UE that is defined as

$$UE_{((X,A),(Y,B))} = H_{((X,A),(Y,B))} - H_{(Z,C)}. \quad (7.18)$$

This difference is always larger when the dependence is higher.

The *unconditional entropy* $H_{(Y,B)}$ of the consequent (Y, B) is expressed as

$$H_{(Y,B)} = -FS_{(Y,B)} * \log_2 FS_{(Y,B)} - (1 - FS_{(Y,B)}) * \log_2 (1 - FS_{(Y,B)}).$$

The conditional probability $P_{((Y,B)|(X,A))}$ is the same as fuzzy confidence $FC_{((X,A),(Y,B))}$ of Eq. (7.15) and is different from $P_{(Y,B)}$. We use

$$H_{((Y,B)|(X,A))} = -FC_{((X,A),(Y,B))} \log_2 (FC_{((X,A),(Y,B))}) - (1 - FC_{((X,A),(Y,B))}) \log_2 (1 - FC_{((X,A),(Y,B))}).$$

The interestingness measure *conditional entropy* $CE_{((X,A),(Y,B))}$ is given as

$$CE_{((X,A),(Y,B))} = H_{(Y,B)} - H_{((Y,B)|(X,A))}. \quad (7.19)$$

Genetic algorithms are used [23] to tune the fuzzy support s and confidence c , while generating fuzzy rules based on the *A priori* algorithm. The fitness function is chosen to maximize the classification accuracy and minimize the number of fuzzy rules.

Mining of fuzzy generalized rules, expressing associations across higher-level concept hierarchies (fuzzy taxonomies), is described in Ref. [24]. The *A priori* algorithm is extended to handle fuzzy sets in the generalized framework. Linguistic hedges, like *very*, *more or less*, etc., are incorporated in these rules to extract more meaningful knowledge.

7.11 CONCLUSIONS AND DISCUSSION

Mining of associations among items in large groups of transactions constitutes an integral part of data mining. In this chapter we have described efficient mining of association rules. These include the candidate generation and test methods, and depth-first search methods. The concept of interesting rules has been highlighted. Techniques involving multilevel rules, online generation of rules, generalized rules, and scalable mining of rules have been outlined. Fuzzy association rules have also been described, along with several fuzzy interestingness measures.

Association rules have recently been applied to texture classification and image segmentation [25]. The statistical and structural information of an image are embedded in the local intensity variation patterns of the texture

regions. Association rules are used to identify these frequently-occurring patterns in the image, and discover relationships which have significant discriminative power. The frequency of occurrence of these local patterns, within a region, are used as the texture features.

In the following chapter we deal with rule mining in the soft computing framework. A modular strategy enables splitting of the problem into subtasks. This sort of *divide-and-conquer* approach is useful from the context of data mining.

REFERENCES

1. R. Agrawal, T. Imielinski, and A. Swami, "Mining association rules between sets of items in large databases," in *Proceedings of 1993 ACM SIGMOD International Conference on Management of Data* (Washington, D.C.), pp. 207–216, May 1993.
2. R. Agrawal and R. Srikant, "Fast algorithms for mining association rules in large databases," in *Proceedings of 20th International Conference on Very Large Databases*, pp. 478–499, September 1994.
3. A. Savasere, E. Omiecinski, and S. Navathe, "An efficient algorithm for mining association rules in large databases," in *Proceedings of 1995 International Conference on Very Large Data Bases (VLDB '95)* (Zurich, Switzerland), pp. 432–443, September 1995.
4. J. S. Park, M. S. Chen, and P. S. Yu, "An effective hash-based algorithm for mining association rules," in *Proceedings of 1995 ACM-SIGMOD International Conference on Management of Data (SIGMOD '95)* (San Jose, CA), pp. 175–186, May 1995.
5. H. Toivonen, "Sampling large databases for association rules," in *Proceedings of 1996 International Conference on Very Large Data Bases (VLDB '96)* (Bombay, India), pp. 134–145, September 1996.
6. H. Mannila and H. Toivonen, "Levelwise search and borders of theories in knowledge discovery," *Data Mining and Knowledge Discovery*, vol. 1, pp. 241–258, 1997.
7. S. Brin, R. Motwani, J. D. Ullman, and S. Tsur, "Dynamic itemset counting and implication rules for market basket analysis," in *Proceedings of 1997 ACM-SIGMOD International Conference on Management of Data (SIGMOD '97)* (Tuscon, AZ), pp. 255–264, May 1997.
8. M. Zaki, S. Parthasarathy, M. Ogihara, and W. Li, "New algorithms for fast discovery of association rules," in *Proceedings of 3rd International*

Conference on Knowledge Discovery and Data Mining (KDD '97) (California), pp. 283–296, 1997.

9. J. Han and M. Kamber, *Data Mining: Concepts and Techniques*. San Diego: Academic Press, 2001.
10. A. Silberschatz and A. Tuzhilin, "What makes patterns interesting in knowledge discovery systems," *IEEE Transactions on Knowledge and Data Engineering*, vol. 8, pp. 970–974, 1996.
11. C. C. Aggarwal and P. S. Yu, "A new approach to online generation of association rules," *IEEE Transactions on Knowledge and Data Engineering*, vol. 13, pp. 527–540, 2001.
12. R. Srikant and R. Agrawal, "Mining generalized association rules," in *Proceedings of 1995 International Conference on Very Large Data Bases (VLDB '95)* (Zurich, Switzerland), pp. 407–419, September 1995.
13. J. Han and Y. Fu, "Discovery of multiple-level association rules from large databases," in *Proceedings of 1995 International Conference on Very Large Data Bases (VLDB '95)* (Zurich, Switzerland), pp. 420–431, September 1995.
14. S. J. Yen and A. L. P. Chen, "A graph-based approach for discovering various types of association rules," *IEEE Transactions on Knowledge and Data Engineering*, vol. 13, pp. 839–845, 2001.
15. M. J. Zaki, "Scalable algorithms for association mining," *IEEE Transactions on Knowledge and Data Engineering*, vol. 12, pp. 372–390, 2000.
16. R. Srikant and R. Agrawal, "Mining quantitative association rules in large relational tables," in *Proceedings of 1996 ACM-SIGMOD International Conference on Management of Data (SIGMOD '96)* (Montreal, Canada), pp. 1–12, June 1996.
17. B. Oezden, S. Ramaswamy, and A. Silberschatz, "Cyclic association rules," in *Proceedings of 1998 International Conference on Data Engineering (ICDE '98)* (Orlando, FL), pp. 412–421, February 1998.
18. S. Ramaswamy, S. Mahajan, and A. Silberschatz, "On the discovery of interesting patterns in association rules," in *Proceedings of 1998 International Conference on Very Large Data Bases (VLDB '98)* (New York), pp. 368–379, August 1998.
19. C. C. Aggarwal, C. Procopiuc, and P. S. Yu, "Finding localized associations in market basket data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 14, pp. 51–62, 2002.

20. R. Rastogi and K. Shim, "Mining optimized association rules with categorical and numeric attributes," *IEEE Transactions on Knowledge and Data Engineering*, vol. 14, pp. 29–50, 2002.
21. T. P. Hong, C. S. Kuo, and S. C. Chi, "Mining association rules from quantitative data," *Intelligent Data Analysis*, vol. 3, pp. 363–376, 1999.
22. A. Gyenesei and J. Teuhola, "Interestingness measures for fuzzy association rules," vol. 2168 of *Lecture Notes in Artificial Intelligence*, pp. 152–164, Heidelberg: Springer-Verlag, 2001.
23. Y. C. Hu, R. S. Chen, and G. H. Tzeng, "Finding fuzzy classification rules using data mining techniques," *Pattern Recognition Letters*, vol. 24, pp. 509–519, 2003.
24. G. Chen and Q. Wei, "Fuzzy association rules and the extended mining algorithms," *Information Sciences*, vol. 147, pp. 201–228, 2002.
25. J. A. Rushing, H. S. Ranganath, T. H. Hinke, and S. J. Graves, "Using association rules as texture features," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 23, pp. 845–858, 2001.

This page intentionally left blank

8

Rule Mining with Soft Computing

8.1 INTRODUCTION

Rule generation refers to the extraction and/or refinement of rules. It can involve traditional rulebase generation by knowledge engineers, or incorporate some sort of automation. When applied from the perspective of large data, we term it as *rule mining*. As mentioned in Section 1.10, this includes association, classification and dependency rule mining. In this chapter we concentrate on the generation (or mining) of classification rules.

Rule generation from ANNs is gaining in popularity in recent times due to its capability of providing some insight to the user about the symbolic knowledge embedded within the network. Fuzzy sets are an aid in providing this information in a more human comprehensible or natural form, and can handle uncertainties at various levels. The neuro-fuzzy approach, symbiotically combining the merits of connectionist and fuzzy approaches, constitutes a key component of soft computing based rule generation [1].

Andrews et al. [2] have provided a classification scheme for connectionist rule extraction algorithms. They take into consideration the

- expressive power of the rules: (1) propositional or Boolean logic i.e., crisp or nonfuzzy, (2) nonconventional logic i.e., probabilistic or fuzzy;
- *translucency* of view taken in the algorithm about the underlying ANN units: (1) decompositional approach (more analytical), where each internal element of the *transparent* network is examined, (2) pedagogical or *blackbox* approach, where one observes only the input-output behavior of the *opaque* network;

- extent to which the underlying ANN incorporates specialized training regimes i.e., *portability*;
- quality of the rules: (1) accuracy i.e., generalization to test cases, (2) fidelity i.e., whether they can mimic the behavior of the ANN from which they were generated, (3) consistency i.e., whether they produce the same classification of test instances over different training instances, (4) comprehensibility, in terms of the size of the rule set and the number of antecedents per rule;
- algorithmic complexity of the technique.

Taha and Ghosh [3] have considered additional issues related to rule extraction. These include the granularity of explanation, modifiability, theory refinement capability (to handle incompleteness, inconsistency, and/or inaccuracy of initial domain knowledge), stability (robustness) to corruption in data or knowledge, and scalability for large datasets (or rulebases).

A recent trend in neural network design for large-scale problems is to split the original task into simpler subtasks and use a subnetwork module for each of the subtasks [4]. It has been shown that by combining the output of several subnetworks in an ensemble, one can improve the generalization ability over that of a single large network [5]. This type of *divide-and-conquer* strategy makes it possible to effectively mine large volumes of data while discovering information. Incorporation of high-level domain knowledge is also a useful feature of data mining. The use of knowledge-based networks is a step in this direction.

The present chapter provides a study on the modular approach to rule generation, with a focus on data mining. Section 8.2 provides a survey on existing connectionist rule generation methods, along with their hybridizations with other soft computing tools. Knowledge-based models are also discussed from this perspective. The concept of modular hybridization is elaborated upon in Section 8.3. Section 8.4 concludes the chapter.

8.2 CONNECTIONIST RULE GENERATION

The primary input to a connectionist rule generation algorithm is typically a representation of a trained feedforward ANN (of Section 2.2.3), in terms of its nodes and links, and sometimes the dataset. One interprets one or more hidden and output units into rules, which may later be combined and simplified to arrive at a more comprehensible rule set. These rules can also provide new insights into the application domain. The models are usually suitable in data-rich environments and seem to be capable of overcoming the problem of the *knowledge acquisition bottleneck* faced by knowledge engineers while designing the knowledge base of traditional expert systems. The trained link weights and node activation of the ANN are used to automatically generate

the rules, either for later use in a traditional expert system or for refining the initial domain knowledge or for providing justification (explanation) in the case of an inferred decision.

In this section we provide a review on rule generation using neural (connectionist) models, along with their hybridizations involving other soft computing tools like fuzzy sets, genetic algorithms, and rough sets [1]. The use of knowledge-based networks, for this purpose, is also discussed in the soft computing framework.

8.2.1 Neural models

Let us first consider the layered connectionist model by Gallant [6] used for rule generation in the medical domain. The inputs and outputs consist of *crisp* variables in all cases. Generally, the symptoms are represented by the input nodes while the diseases and possible treatments correspond to the intermediate and/or output nodes. The model deals with *sacrophagal* problems, using a linear discriminant network (with no hidden nodes) that is trained by the simple *pocket algorithm*. The absence of the hidden nodes and nonlinearity limits the utility of the system in modeling complex decision surfaces. Dependency information regarding the variables, in the form of an adjacency matrix, is provided by the expert. Every input variable x is approximated by three Boolean variables x'_1, x'_2, x'_3 . Cell activation is discrete, taking on values $+1, -1$, or 0 , corresponding to logical values of *true*, *false*, or *unknown*. Each cell computes its new activation y'_i as a linear discriminant function of the x'_j s.

Rules are generated by traversing the trained connection weights as follows:

1. List all inputs that are known and have contributed to the ultimate positivity of a discriminant.
2. Arrange the list by decreasing absolute value of the weights.
3. Generate clauses for an IF-THEN rule from this ordered list.

The user can also be queried to supplement incomplete input information. During question generation, the system selects that unknown output variable whose *confidence* is maximum. Then it backtracks along the connection weights to find an unknown input variable, whose value is queried from the user.

Ishikawa [7] demonstrates the training of a network using *structural learning with forgetting*. An examination of the resultant simplified and nonredundant network architecture leads to easy extraction of rules. The stationary (nonactive) positive weights get reduced and negative weights increased using a decay factor. A total of 8124 samples of mushrooms, with 22 attributes each, have been studied for the two-class (edible or poisonous) problem. The method selects two or four most relevant attributes. For the two-attribute

case, *odor* and *spore-print-color* were found to be important. A sample extracted rule is as follows:

IF (*odor* = *almond* OR *anise* OR *none*) AND (*spore-print-color* $\neq$ *green*) THEN *mushroom* = *edible*.

Duch et al. [8] modified this algorithm by constraining the weights to $\{+1, -1, 0\}$. This is supposed to result in the extraction of rules with more logical interpretation. They have also used a generalization of RBF networks for interpreting node functions as rules.

Setiono [9] has used a pruned network for extracting compact, meaningful rules, in terms of hidden unit activation. The activation are clustered into discrete values, and a process of splitting of hidden units and creation of new subnetwork is repeated until each hidden unit has only a small number of inputs connected to it. A penalty term augments a cross-entropy error function that is minimized to encourage weight decay and remove redundant weights. The accuracy and number of rules generated are claimed to be better than those obtained by C4.5 [10]. Setiono has also reported [11] the extraction of *M* of *N* type rules (described in Section 8.2.3) from a trained feedforward network whose weights and inputs are restricted to values in $\{-1, 1\}$. The rules are claimed to possess desirable qualities like accuracy, simplicity and fidelity.

Setiono and Liu [12] have also developed oblique decision trees that partition the attribute space by hyperplanes (not necessarily axis-parallel) and can readily be translated into a set of rules. Since an oblique decision tree classifies patterns based on linear combinations of input attributes, the rules are more compact than that generated by an univariate tree over the same domain. Comparison is provided with other decision tree-based approaches, like C4.5 and CART [13]. The compactness of these oblique rules is said to result in better rule comprehensibility and consistency.

8.2.2 Neuro-fuzzy models

Nauck et al. [14, 15] have developed NEFCON, NEFCLASS, and NEFPROX, using generic fuzzy perceptron, to model Mamdani-type [16] neuro-fuzzy systems. Since fuzzy systems are designed to exploit the tolerance for imprecision, here they are not used to generate an *exact* solution. The learning procedure uses a fuzzy error, and can operate on both fuzzy sets and rules. The incremental rule learning algorithm can create a rulebase from scratch by adding rule after rule, or can also operate on prior knowledge. The knowledge base of the fuzzy system is implicitly embedded in the network structure. The system is claimed to be simple and highly interpretable, as well as suitable in providing support to users during decision-making involving classification and function approximation.

Rhee and Krishnapuram [17] generate rules from minimal approximate fuzzy aggregation networks, involving hybrid operators with compensatory behavior at the neuronal level. The linguistic labels and the corresponding triangular membership functions for the input features are estimated from the training data. The compensatory parameters are learned during gradient descent, to estimate the type of aggregation employed. Pruning of redundant features and/or hidden nodes helps in generating appropriate rules in terms of AND-OR operators that are represented by these hybrid functions. Zhang and Kandel [18] have also used compensatory fuzzy operators to effectively learn fuzzy IF-THEN rules from both well- and ill-defined data.

Mitra and Pal [19] have used a fuzzy logical MLP for inferencing and rule generation. The model consists of logical neurons employing conjugate pairs of *t-norms* T and *t-conorms* S , like *min-max* and *product-probabilistic sum*, in place of the *weighted sum* and *sigmoidal* functions of the conventional MLP. Various fuzzy implication operators are used to introduce different amounts of interaction during error backpropagation. The built-in AND-OR structure of the model helps it to generate appropriate rules, expressed as disjunction of conjunctive clauses.

8.2.3 Using knowledge-based networks

One of the major problems in connectionist or neuro-fuzzy design is the choice of the optimal network structure. This has an important bearing on any performance evaluation. Moreover, the models are generally very data-dependent, and the appropriate network size also depends on the available training data. Various methodologies developed for selecting the optimal network structure include growing and pruning of nodes and links, employing genetic search, and embedding initial knowledge in the network topology. The last approach – embedding initial knowledge – is usually followed in the case of knowledge-based networks. It is formally shown [20] that such knowledge-based networks require relatively smaller training set sizes for correct generalization. When the initial knowledge fails to explain many instances, additional hidden units and connections need to be added. The initial encoded crude domain knowledge may be refined with experience, by performing learning in the data environment. The resulting networks generally involve less redundancy in their topology. Embedding of prior knowledge, involving user interaction, is also a desirable feature of data mining.

8.2.3.1 Connectionist models Towell and Shavlik [21] have designed a hybrid learning system KBANN and have applied it to problems of molecular biology. Disjunctive rules are rewritten as multiple conjunctive rules while mapping into the network structure. It is primarily a theory *refinement* system that is capable of pruning an inserted rule set, but not capable of adding new rules. It is largely topology-preserving and assumes that the initial domain theory is basically correct and nearly complete.

An expansion of the network guided by both the domain theory and training data has been reported in TopGen by Opitz and Shavlik [22]. Dynamic addition of hidden nodes are made at the best place by heuristically searching through the space of possible network topologies, in a manner analogous to the adding of rules and conjuncts to the symbolic rulebase. This approach uses a specialized ANN architecture with a specialized training algorithm. It generates sparser rule sets as compared to KBANN and overcomes the latter's limitation of not being able to extend a relatively weak initial domain theory. The additional computational expense is justified in terms of the human expert's willingness to wait for an extended period of time for better predictive accuracy.

A way of using the knowledge of the trained neural model to extract revised rules for the problem domain is described by Fu [23] (*Subset* algorithm) and Towell and Shavlik [24] (*M of N* algorithm). Knowledge, in the form of rules in disjunctive normal form, is encoded into the network. The other links represent low-weighted connections, allowing subsequent refinement. It is assumed that the neurons have binary inputs and hard-limiting activation functions. Even though the algorithms are exponential in complexity, their inherent simplicity makes them extremely useful.

The *Subset* algorithm [23] initially searches for any single weight (at the input of a neuron) exceeding its bias, and it rewrites all conditions so found as rules with single input variable. The breadth-first search for all the hidden and output nodes (over the input links) continues for increasing sizes of sets, until all such sets have been explored and possibly rewritten as rules in disjunctive normal form. The extracted rules are simple to understand and their size can be restricted by specifying the number of premises (antecedents) to be considered. Finally the algorithm removes subsumed and overly general rules. The main steps of the algorithm are outlined as follows:
For each hidden and output unit repeat steps 1 and 2.

1. Extract up to β_p subsets of the positively weighted incoming links, whose summed weight is greater than the bias of the unit.
2. **for** each subset $\mathcal{P}$ of β_p subsets found in step 1, **repeat** the following.
 - (a) Extract up to β_n minimal subsets of negatively weighted links, whose summed weight is greater than the bias of the unit *minus* the sum of $\mathcal{P}$.
 - (b) Let $\mathcal{Z}$ be a new predicate used nowhere else.
 - (c) With each subset $\mathcal{N}$ of β_n subsets found in step 2(a), form the rule "IF $\mathcal{N}$ THEN $\mathcal{Z}$."
 - (d) Form the rule "IF $\mathcal{P}$ and NOT $\mathcal{Z}$ THEN *name-of-unit*."

However, some of the problems associated with this algorithm are as follows [3]. It requires lengthy, exhaustive searches of size $O(2^k)$ for a hidden or output

node with fan-in of k . It extracts a large set of rules, up to $\beta_p * (1 + \beta_n)$, where β_p and β_n are the number of subsets of positively and negatively weighted links, respectively. Some of the generated rules may be repetitive, as permutations of rule antecedents are not taken care of automatically. Moreover, there is no guarantee that all useful knowledge embedded in the trained network will be extracted. To avoid the otherwise prohibitive combinatorics, all implementations of *Subset* algorithm typically use heuristics.

The *Subset* algorithm has been modified by Towell and Shavlik [24] to design the *M of N* algorithm for extracting meaningful rules. A general rule in this case is of the form: IF (*at least M* of the following *N* antecedents are true) THEN The rationale is to find a group of links that form an *equivalence class*, whose members have similar effect (weight values) and can be used interchangeably with one another.

The steps of this algorithm include

- *clustering* the weights of each (hidden and output) neuron into groups,
- *averaging* their values to create equivalence classes,
- *eliminating* low value weights if they have no significant effect on the sign of the total activation, and
- *optimizing*, by freezing the remaining weights and retraining the biases using the backpropagation algorithm.

This is followed by *rule extraction*. Arithmetic is performed such that one searches for all weighted antecedents, which, when summed up, exceed the threshold value of a given neuron. When possible, the rules are simplified to eliminate superfluous weights and thresholds.

This algorithm has good generalization (accuracy), but can have degraded comprehensibility [2]. Note that the algorithm considers groups of links as equivalence classes, thereby generating a bound on the number of rules rather than establishing a ceiling on the number of antecedents. The computational complexity is $O(k^3 + (k^2 \cdot j))$, where j is the number of examples. Additionally, the rule extraction procedure involves a backpropagation step requiring significant computational overhead.

8.2.3.2 Incorporating fuzzy sets A brief survey on the knowledge-based networks involving fuzziness at different stages is provided here. Knowledge extracted from experts in the form of membership functions and fuzzy rules (in AND-OR form) is used to build and preweight the neural net structure, which is then tuned using training data.

A major problem of using MLPs to refine rule-based knowledge [23, 24] is the preservation of symbolic knowledge (stability) under the weight tuning mechanism (plasticity) of the backpropagation algorithm. Another limitation is that unless the initial rulebase is roughly complete, the initial network architecture may not be sufficiently rich for handling the problem domain

by incorporating new knowledge. The Adaptive Resonance Theory (ART) architecture [25] is designed to circumvent this *stability-plasticity* dilemma. Learning in ARTMAP [26] is match-based (not error-based); it does not wash away existing knowledge and the meanings of units do not shift. Use of fuzzy set theoretic concepts lead to the formulation of the fuzzy ARTMAP [27].

Tan [28] has used a generalization of fuzzy ARTMAP, called 'cascade ARTMAP', as a fuzzy knowledge-based network. It represents intermediate attributes and rule cascades of rule-based knowledge explicitly, and performs multistep inferencing. A rule insertion algorithm translates IF-THEN symbolic rules into cascade ARTMAP architecture. This knowledge can be refined and enhanced by the learning algorithm. During learning, new recognition categories (rules) can be created dynamically to cover the deficiency of the domain theory. The extracted rules involve discrete inputs and are of good quality. The algorithmic complexity is linear in the number of recognition categories. Results indicate that the performance is superior to that of the KBANN [21], ID3 and MLP. The extracted rules are claimed [28] to be simpler and more accurate than the M of N rules [24]. Besides, each extracted rule is associated with a confidence factor that indicates its importance or usefulness. This allows ranking and evaluation of the extracted knowledge.

Machado and Rocha [29] have used an interval-based representation for membership grades (*MGI*) to allow reasoning with different types of uncertainty, namely, vagueness, ignorance, and relevance. The model incorporates the facilities of incremental learning, inference, inquiry, censorship of input information, and explanation, as in expert systems. The utility-based inquiry process permits significant reduction of consultation cost or risk and gives the system the common sense property possessed by experts when selecting tests to be performed. The ability to criticize input data when they disrupt a trend of acceptance or rejection observed for a hypothesis mimics the behavior of experts, who are often able to detect suspicious input data and either reject them or ask for their confirmation. The explanation algorithm provides responses to queries such as *how* a particular conclusion was reached or *why* a particular question was formulated. The network forms a set of pathways that compete to send the largest evidential flow to the output neuron representing the hypothesis. The structure of the winning pathway represents a chain of fuzzy pseudo-production rules, which can be presented to the user either in a graphical format or as English text.

Application of this algorithm has been made to the deforestation monitoring of the Amazon region, using Landsat-V satellite images. The classes considered are forest, savanna, water, deforested area, cloud, and shadow. Eighty-two numerical features of spectral, textural, and geometric nature were measured on each image segment (of spectrally homogeneous regions, generated by region growing). Fuzzy classification allows the modeling of complex situations such as transition phenomena (as in the regeneration of forest in a previously burned area) or multiple classification (as in the case of forest overcast by clouds).

A model by Mitra et al. [30] has been developed for classification, inferencing, querying, and rule generation. It is capable of generating both *positive* (indicating the belongingness of a pattern to a class) and *negative* (indicating its degree of *not* belonging to a class) rules in linguistic form to justify any decision reached. This is found to be useful for inferencing in ambiguous cases. The knowledge encoding procedure, unlike many other methods [21, 23], involves a nonbinary weighting mechanism. The *a priori* class information and the distribution of pattern points in the feature space are taken into account while encoding the crude domain knowledge from the dataset among the connection weights, using fuzzy intervals and linguistic sets in the process. Each pattern class is modeled in terms of positive and negative hidden nodes. The trained knowledge-based network is used for rule generation in IF-THEN form. These rules describe the extent to which a test pattern belongs or does not belong to one of the classes, in terms of antecedent and consequent clauses provided in natural form. Backtracking along maximal weighted paths of the trained net, utilizing its input and output activation (with confidence factor), enables generation of these clauses.

8.2.3.3 Incorporating genetic algorithms Opitz and Shavlik [31] use the domain theory of Towell and Shavlik [21, 24] to generate a knowledge-based network structure that is evolved using GAs. Random perturbation is applied to create an initial set of candidate networks or *population*. A node is perturbed by either deleting it or adding new nodes to it. Next, these networks are trained using backpropagation and placed back into the population. New networks are created by using crossover and mutation operators, specifically designed to function on these networks. The algorithm tries to minimize the destruction of the rule structure of the crossed-over networks, by keeping intact nodes belonging to the same syntactic rule (i.e., the nodes highly connected to each other). The mutation operator adds diversity to a population, while still maintaining a directed heuristic search technique for choosing where to add nodes. In this manner the algorithm searches the topology space, in order to find suitable networks which are then trained using backpropagation.

8.2.3.4 Incorporating rough sets Concept of rough sets is integrated with fuzzy-neural network for designing a knowledge-based rough-fuzzy MLP [32, 33], where the theory of rough sets is utilized for extracting crude domain knowledge in the form of IF-THEN rules. These rules are encoded among the connection weights, with their number and syntax automatically determining the network topology in terms of hidden nodes and links. Methods are derived to model convex decision regions with single-object representatives, as well as arbitrary decision regions with multiple-object representatives. From the perspective of pattern recognition, this implies using a single prototype to model a (convex) decision region in the first case. In the second case, this means using multiple prototypes to serve as representatives of any arbitrary decision region. A three-layered fuzzy MLP is considered, where the feature

space gives the condition attributes and where the output classes give the decision attributes, so as to result in a decision table. Rules are then generated from the table by computing relative reducts. The dependency factors of these rules are encoded as the initial weight values of the links.

8.3 MODULAR HYBRIDIZATION

It is believed that the use of Modular Neural Network enables a wider use of ANNs for large-scale systems. Embedding modularity (i.e., to perform local and encapsulated computation) into neural networks leads to many advantages as compared to the use of a single network. For instance, constraining the network connectivity increases its learning capacity and permits its application to large-scale problems [4] with relevance to data mining. It is easier to encode *a priori* knowledge in modular neural networks. In addition, the number of network parameters can be reduced by using modularity. This feature speeds computation and can improve the generalization capability of the system [34].

This section describes a way of integrating subnetworks of rough-fuzzy MLPs, using a modular evolutionary algorithm, for classification and rule generation [35, 36]. Rough set theory is applied for extracting dependency rules directly from real-valued attribute table consisting of fuzzy membership values. This helps in preserving all the class representative points in the dependency rules by adaptively applying a threshold that automatically takes care of the shape of membership functions.

An l -class classification problem is split into l two-class subproblems. Crude subnetwork modules are initially encoded, for each two-class sub-problem, from the dependency rules. These subnetworks are then combined and the final network is evolved using a GA with *restricted* mutation operator which utilizes the knowledge of the modular structure already generated, for faster convergence. The GA tunes the fuzzification parameters, and network weight and structure simultaneously, by optimizing a single fitness function. This methodology helps in imposing a structure on the weights, which results in a network more suitable for rule generation. Performance of the algorithm is compared with related techniques.

8.3.1 Rough fuzzy MLP

As pointed out in Section 5.6.3, an n -dimensional pattern $\mathbf{X}_i$ is represented in the form of Eq. (5.18), with $y_1^0, \dots, y_{3n}^0$ being the activations of the $3n$ neurons in the input layer of the network. An l -class problem domain corresponds to l nodes in the output layer, such that the membership of the i th pattern in class k is defined by Eq. (5.23). The basics of rough sets have already been introduced in Section 2.2.6, for the benefit of the readers. Before going to the

details of the modular hybrid system, we provide in this section an overview of the mathematical preliminaries using rough set theoretic concepts for the interested reader.

8.3.1.1 Rule generation Let $S = \langle U, A \rangle$ be a decision table, with C and $D = \{d_1, \dots, d_l\}$ its sets of condition and decision attributes, respectively. Divide the decision table $S = \langle U, A \rangle$ into l tables $S_i = \langle U_i, A_i \rangle$, $i = 1, \dots, l$, corresponding to the l decision attributes $d_1, \dots, d_l$, where

$$U = U_1 \cup \dots \cup U_l \text{ and } A_i = C \cup \{d_i\}.$$

Let $\{x_{i1}, \dots, x_{ip}\}$ be the set of those objects of U_i that occur in S_i , $i = 1, \dots, l$.

Now for each d_i -reduct $B = \{b_1, \dots, b_k\}$ (say), a discernibility matrix (denoted $M_{d_i}(B)$) from the d_i -discernibility matrix is defined as follows [32].

$$c_{ij} = \{a \in B : a(x_i) \neq a(x_j)\}, \quad (8.1)$$

for $i, j = 1, \dots, n$.

For each object $x_j \in x_{i1}, \dots, x_{ip}$, the discernibility function $f_{d_i}^{x_j}$ is defined as

$$f_{d_i}^{x_j} = \bigwedge \{ \bigvee (c_{ij}) : 1 \leq i, j \leq n, j < i, c_{ij} \neq \emptyset \}, \quad (8.2)$$

where $\bigvee (c_{ij})$ is the disjunction of all members of c_{ij} . Then $f_{d_i}^{x_j}$ is brought to its conjunctive normal form (*c.n.f.*). One thus obtains a dependency rule r_i , namely, $P_i \leftarrow d_i$, where P_i is the disjunctive normal form (*d.n.f.*) of $f_{d_i}^{x_j}$, $j \in i_1, \dots, i_p$.

The dependency factor df_i for r_i is given by

$$df_i = \frac{\text{card}(\text{POS}_i(d_i))}{\text{card}(U_i)}, \quad (8.3)$$

where $\text{POS}_i(d_i) = \bigcup_{X \in I_{d_i}} l_i(X)$, and $l_i(X)$ is the lower approximation of X with respect to I_i . In this case, $df_i = 1$ [32].

8.3.1.2 Knowledge encoding Consider the case of feature j for class C_k in the l -class problem domain. The inputs for the i -th representative sample X_i are mapped to the corresponding three-dimensional feature space of $\mu_{\text{low}(a_j)}(X_i)$, $\mu_{\text{medium}(a_j)}(X_i)$, and $\mu_{\text{high}(a_j)}(X_i)$ by Eq. (5.18). Let these be represented by L_j , M_j , and H_j , respectively. As the method considers multiple objects in a class, a separate $n_k \times 3n$ -dimensional attribute-value decision table is generated for each class C_k (where n_k indicates the number of objects in C_k).

The absolute distance between each pair of objects is computed along each attribute L_j , M_j , H_j for all j . We modify Eq. (8.1) to directly handle a real-valued attribute table consisting of fuzzy membership values. We define

$$c_{ij} = \{a \in B : |a(x_i) - a(x_j)| > Th\} \quad (8.4)$$

for $i, j = 1, \dots, n_k$, where Th is an adaptive threshold. Note that the adaptivity of this threshold is in-built, depending on the inherent shape of the membership function.

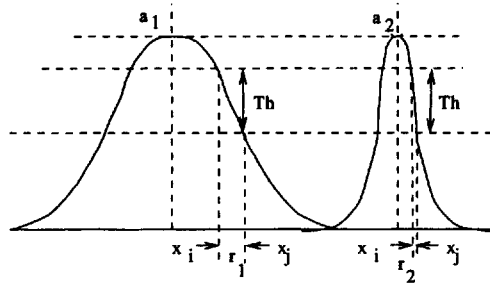


Fig. 8.1 Illustration of adaptive thresholding of membership functions.

Consider Fig. 8.1. Let a_1 , a_2 correspond to two membership functions (attributes), with a_2 being steeper than a_1 . It is observed that $r_1 > r_2$. This results in an implicit adaptivity of Th while computing c_{ij} in the discernibility matrix directly from the real-valued attributes. This is particularly useful in modeling multimodal class distributions.

The hidden layer nodes model the first-level (innermost) operator in the antecedent part of a rule, which can be either a conjunct or a disjunct. The output layer nodes model the outer-level operands, which can again be either a conjunct or a disjunct. For each inner-level operator, corresponding to one output class (one dependency rule), one hidden node is dedicated. Only those input attributes that appear in this conjunct or disjunct are connected to the appropriate hidden node, which in turn is connected to the corresponding output node. Each outer level operator is modeled at the output layer by joining the corresponding hidden nodes. Note that a single attribute (involving no inner level operators) is directly connected to the appropriate output node via a hidden node, to maintain uniformity in rule mapping.

Let the dependency factor for a particular dependency rule for class C_k be $df = \alpha = 1$ by Eq. (8.3). The weight w_{ki}^1 between a hidden node i and output node k is set at $\frac{\alpha}{fac} + \epsilon$, where fac refers to the number of outer-level operands in the antecedent of the rule and ϵ is a small random number taken to destroy any symmetry among the weights. Note that $fac \geq 1$ and each hidden node is connected to only one output node. Let the initial weight so clamped at a hidden node be denoted as β . The weight $w_{ia_j}^0$ between an attribute a_j (where a corresponds to *low* (L), *medium* (M), or *high* (H)) and hidden node i is set to $\frac{\beta}{facd} + \epsilon$, such that $facd$ is the number of attributes connected by the corresponding inner-level operator. Again $facd \geq 1$. Thus for an l -class problem domain there are at least l hidden nodes. All other possible connections in the resulting network are set as small random numbers. It is to be mentioned that the number of hidden nodes is automatically determined from the number of dependency rules, while their connectivity follows from the syntax of these rules.

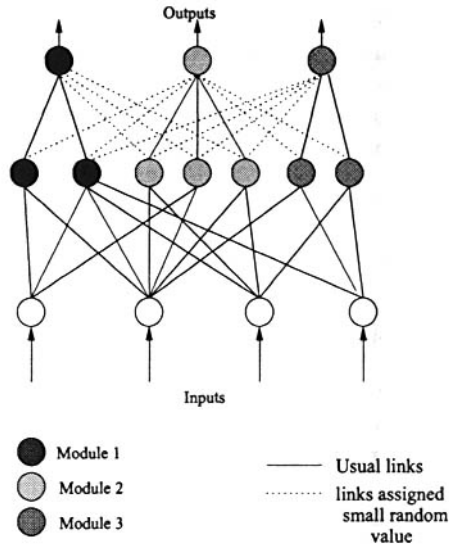


Fig. 8.2 Intra- and intermodule links.

8.3.2 Modular knowledge-based network

This involves two phases. First an l -class classification problem is split into l two-class problems. Rough set theoretic concepts are used to encode domain knowledge into each of the l subnetworks, using Eqs. (8.2)–(8.4). The number of hidden nodes and connectivity of the knowledge-based subnetworks is automatically determined. A two-class problem leads to the generation of one or more crude subnetworks, each encoding a particular decision rule. Let each of these constitute a pool. So we obtain $m \geq l$ pools of knowledge-based modules. Each pool k is perturbed to generate a total of n_k subnetworks, such that $n_1 = \dots = n_k = \dots = n_m$. These pools constitute the initial population of subnetworks, which are then evolved independently using genetic algorithms.

At the end of training, the modules (or subnetworks) corresponding to each two-class problem are concatenated to form an initial network for the second phase. The intermodule links are initialized to small random values as depicted in Fig. 8.2. A set of such concatenated networks forms the initial population of the GA. Note that the individual modules cooperate, rather than compete, with each other while evolving towards the final solution. The mutation probability for the intermodule links is now set to a high value, while that of intramodule links is set to a relatively lower value. This sort of *restricted* mutation helps preserve some of the localized rule structures, already extracted and evolved, as potential solutions. The initial population for the GA of the entire network is formed from all possible combinations

of these individual network modules and random perturbations about them. This ensures that for complex multi-modal pattern distributions all the different representative points remain in the population. The algorithm then searches through the reduced space of possible network topologies. The steps are summarized below, followed by an example.

1. **for each class, generate rough set dependency rules.**
2. Map each of the dependency rules to a separate subnetwork module (fuzzy MLP).
3. Partially evolve each of the subnetworks using conventional GA.
4. Concatenate the subnetwork modules to obtain the complete network. For concatenation the intramodule links are left unchanged while the intermodule links are initialized to low random values. Note that each of the subnetworks solves a two-class classification problem, while the concatenated network solves the actual l -class problem. Every possible combination of subnetwork modules is generated to form a pool of networks.
5. The pool of networks is evolved using a *modified* GA with an adaptive or variable mutation operator. The mutation probability is set to a low value for the intramodule links and to a high value for the intermodule links.

8.3.2.1 Example Consider a problem of classifying a two dimensional data into two classes. The input fuzzifier maps the features into a six dimensional feature space. Let a sample set of rules obtained from rough set theory be

$$C_1 \leftarrow (L_1 \wedge M_2) \vee (H_2 \wedge M_1), C_2 \leftarrow M_2 \vee H_1, C_2 \leftarrow L_2 \vee L_1,$$

where L_j, M_j, H_j correspond to $\mu_{low}(a_j), \mu_{medium}(a_j), \mu_{high}(a_j)$, respectively. For the first phase of the GA three different pools are formed, using one crude subnetwork for class 1 and two crude subnetworks for class 2, respectively. Three partially trained subnetworks result from each of these pools. They are then concatenated to form $(1 \times 2) = 2$ networks. The population for the final phase of the GA is formed with these networks and perturbations about them. The steps followed in obtaining the final network is illustrated in Fig. 8.3. The corresponding impact of the different stages of network evolution on the decision regions, generated in the feature space, is depicted in Fig. 2.9.

8.3.2.2 Characteristics Use of this scheme for generating modular knowledge-based networks has several advantages.

- Sufficient reduction in training time is obtained, as the above approach parallelizes the GA to an extent. Because the search string of the GA for subnetworks is smaller, more than linear decrease in searching time is

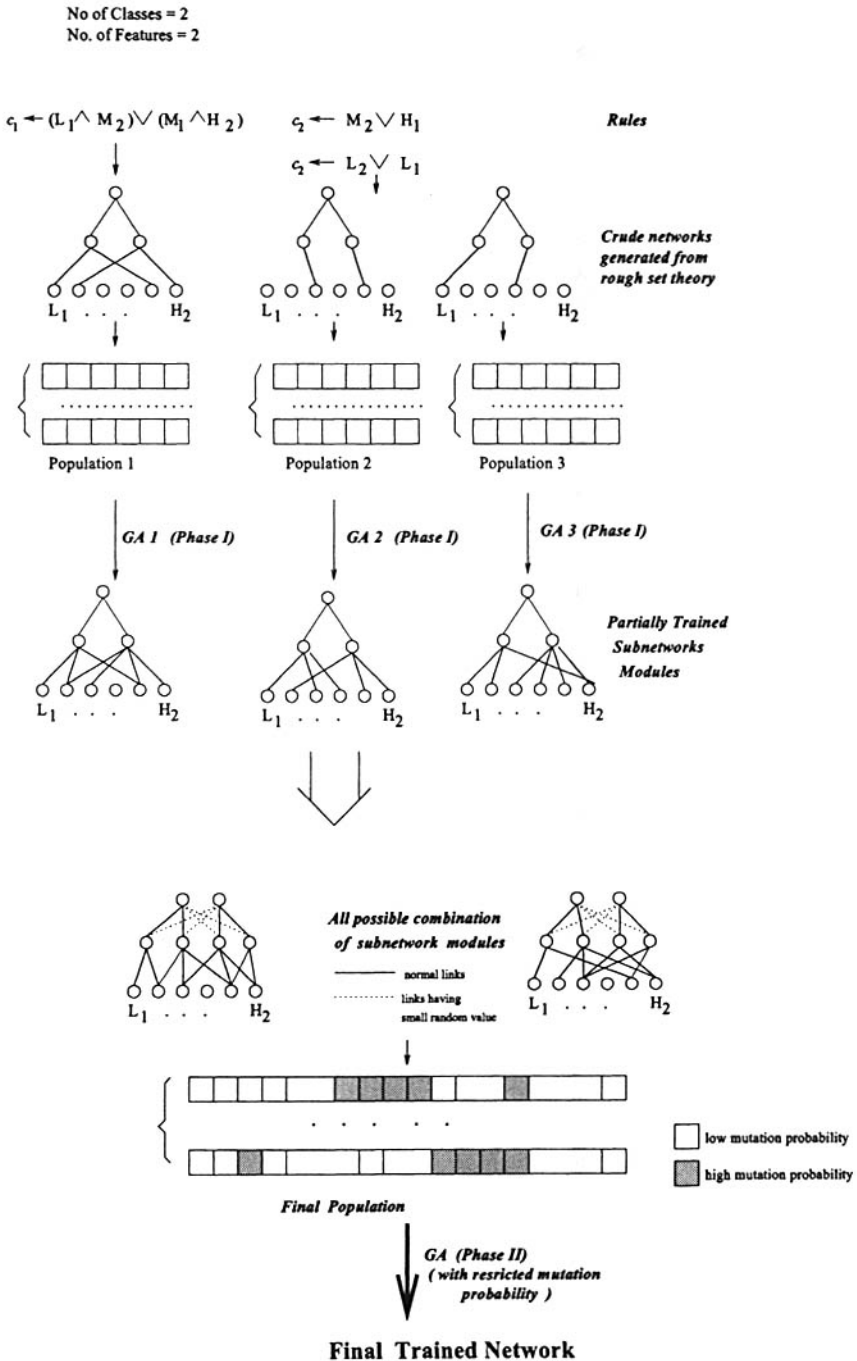


Fig. 8.3 Steps for designing a sample modular rough-fuzzy MLP.

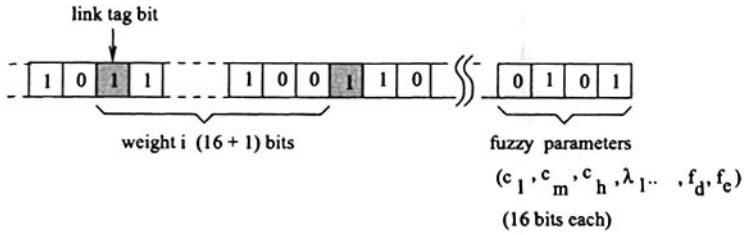
obtained. Also very small number of training cycles are required in the refinement phase, as the network is already very close to the solution.

- The use of rough sets for knowledge encoding provides an established mathematical framework for network decomposition. The search space is reduced, leading to shorter training time. The initial network topology is also automatically determined and provides good *building blocks* for the GA.
- The algorithm indirectly constrains the solution in such a manner that a structure is imposed on the connection weights. This is helpful for subsequent rule extraction from the weights, as the resultant network has sparse but strong interconnection among the nodes.

8.3.3 Evolutionary design

Here we describe the use of GAs for evolving the weight values as well as the structure of the modular subnetworks. The input and output fuzzification parameters are also tuned. The initial population consists of all possible networks generated from rough set theoretic rules. As explained in Section 2.2.5, GAs involve three basic procedures, namely, (i) encoding of the problem parameters in the form of binary strings, (ii) application of genetic operators like crossover and mutation, and (iii) selection of individuals based on some objective function to create a new population. Each of these aspects is discussed below with relevance to this algorithm [36].

8.3.3.1 Chromosomal representation The problem variables consist of the weight values and the input/output fuzzification parameters. Each of the weights is encoded into a binary word of 16-bit length, where [000...0] decodes to -128 and [111...1] decodes to 128. An additional bit is assigned to each weight to indicate the presence or absence of the corresponding link. If this bit is 0, then the remaining bits are unrepresented in the phenotype. The total number of bits in the string is therefore *dynamic*. Thus a total of 17 bits are assigned for each weight. The fuzzification parameters tuned are the centers (c) and radius (λ) for each of the linguistic attributes *low*, *medium*, and *high* of each feature, and output fuzzifier parameters f_d and f_e . These are also coded as 16-bit strings in the range [0, 2]. The chromosome is obtained by concatenating all the above strings. Sample values of the string length are around 2000 bits for reasonably sized networks.



Initial population is generated by coding the networks obtained by rough set-based knowledge encoding and by random perturbations about them. A population size of 64 was considered.

8.3.3.2 Genetic operators Here we provide details on the implementation aspects of the different genetic operators, namely, crossover, mutation, selection, and fitness function used. These notions were introduced earlier in Section 2.2.5.

Crossover: It is obvious that due to the large string length, single-point crossover would have little effectiveness. Multiple-point crossover is adopted, with the distance between two crossover points being a random variable between 8 and 24 bits. The crossover probability is fixed at 0.7.

Mutation: Because the search string is very large, the influence of mutation is more on the search. Each of the bits in the string is chosen to have some mutation probability ($pmut$), but with a spatiotemporal variation. The mutation probabilities vary along the encoded string, with the bits corresponding to intermodule links being assigned a higher probability as compared to intramodule links. This is done to ensure least alterations in the structure of the individual modules already evolved, by incorporating the domain knowledge extracted through rough set theory.

Choice of fitness function: In GAs the fitness function is the final arbiter for string creation, and the nature of the solution obtained depends on the objective function. An objective function of the form described below is chosen.

$$F = \alpha_1 f_1 + \alpha_2 f_2, \quad (8.5)$$

where

$$f_1 = \frac{\text{No. of correctly classified samples in training set}}{\text{Total no. of samples in training set}}$$

$$f_2 = 1 - \frac{\text{No. of links present}}{\text{Total no. of links possible}}.$$

Here α_1 and α_2 determine the relative weightage of each of the factors. α_1 is taken to be 0.9 and α_2 is taken as 0.1, to give more importance to the classification score as compared to the network size in terms of number of links. Note that we optimize the network connectivity, weights, and input-output fuzzification parameters simultaneously.

Selection: This is done by the *roulette wheel* method. The probabilities are calculated on the basis of ranking of the individuals in terms of the objective

function. *Elitism* is incorporated in the selection process by comparing the fitness of the best individual of a new generation to that of the current generation. If the latter has a higher value, then the corresponding individual replaces a randomly selected individual in the new population.

8.3.4 Rule extraction

A rule extraction algorithm [37], based on the hybrid model of Section 8.3.2, is presented here. The performance of the rules is evaluated quantitatively. A quantitative comparison of the rule extraction algorithm is made with some existing ones like *Subset* [23], *M of N* [24], and *X2R* [38]. The steps of the algorithm are provided below.

1. Compute the following quantities:

$P\text{Mean}$ = Mean of all positive weights, $P\text{Thres}_1$ = Mean of all positive weights less than $P\text{Mean}$, $P\text{Thres}_2$ = Mean of all weights greater than $P\text{Mean}$. Similarly, calculate $N\text{Mean}$, $N\text{Thres}_1$, and $N\text{Thres}_2$ for negative weights.

2. for each hidden and output unit

- (a) for all weights greater than $P\text{Thres}_2$, search for positive rules; and for all weights less than $N\text{Thres}_2$, search for negative rules, only, by *Subset* method.
- (b) Search for combinations of positive weights above $P\text{Thres}_1$ and negative weights greater than $N\text{Thres}_2$ that exceed the bias. Similarly, search for negative weights less than $N\text{Thres}_1$ and positive weights below $P\text{Thres}_2$.

3. Associate with each rule j a confidence factor cf_j , given by Eq. (2.36).

Since the learning algorithm imposes a structure on the network, resulting in a sparse network having few strong links, the $P\text{Thres}$ and $N\text{Thres}$ values are well separated. Hence the above rule extraction algorithm generates most of the embedded rules over a small number of computational steps.

The computational complexity of the algorithm is as follows. Let the network have i , h , o numbers of input, hidden, and output nodes, respectively. Let us make the assumption that $i = h = o = k$. Let the fraction of weights having value in $[0, P\text{Thres}_1)$, $[P\text{Thres}_1, P\text{Thres}_2)$, $[P\text{Thres}_2, \infty)$ be p_1 , p_2 , p_3 , respectively. Similarly, let the corresponding fractions for negative weights be n_1 , n_2 , n_3 . Then the computational complexity (C) becomes

$$C = k \cdot (2^{(p_2+p_3)k+1} + 2^{(n_2+n_3)k+1} + 2^{(p_3+n_1)k+1} + 2^{(p_1+n_3)k+1}).$$

If $n_1, n_2, p_1, p_2 \ll p_3, n_3$,

$$C \approx 4k(2^{p_3k} + 2^{n_3k}) = 4k(e^{ln_2.p_3k} + e^{ln_2.n_3k}).$$

Also if $p_3, n_3 \ll 1$,

$$C \approx 4k(1 + \log 2(p_3 + n_3))k + 0.5(\log 2(p_3 + n_3))^2 k^2, \text{ i.e., } C \approx O(k^3).$$

An important consideration is the order of application of rules in a rule base. Since most of the real-life patterns are noisy and overlapping, rulebases obtained are often not totally consistent. Hence multiple rules may fire for a single example. Several existing approaches apply the rules sequentially [3], often leading to degraded performance. The confidence factors, associated with the extracted rules, help in circumventing this problem. The quantitative measures used to evaluate these generated rules have been described in Section 2.4.2.

8.3.5 Results

This genetic-rough-neuro-fuzzy algorithm has been implemented on both real-life (speech, medical) and artificially generated data [35, 36]. In this section we provide sample results on the *Vowel* data. Figure 5.10 depicts the six Telugu (it is a language in India) vowel classes in the F_1 - F_2 plane. These overlapping classes will be denoted by $C_1, C_2, \dots, C_6$. The rough set theoretic technique is applied on the data to extract some knowledge, which is initially encoded among the connection weights of the subnetworks. The data are first transformed into a nine-dimensional linguistic space.

The methodology described here is termed Model S. Other models compared include:

- (i) Model O: An ordinary MLP trained using backpropagation (BP).
- (ii) Model F: A fuzzy MLP trained using BP.
- (iii) Model R: A rough-fuzzy MLP trained using BP.

8.3.5.1 Classification Recognition scores obtained by Model S are presented in Table 8.2. It also shows a comparison with other related MLP-based classification methods (Models O, F, R). In all cases, 10% of the samples are used as training set, and the remaining samples are used as test set. Ten such independent runs are performed, and the mean value and standard deviation of the classification accuracy, computed over them, are presented in Table 8.2. The dependency rules, as generated via rough set theory and used for encoding crude domain knowledge, are shown in Table 8.1. The values of input fuzzification parameters used are also presented.

The classification accuracies obtained by the models are analyzed for statistical significance. Tests of significance are performed for the inequality of means (of accuracies) obtained using the different algorithms. Since both mean pairs and the variance pairs are unknown and different, a generalized version of *t*-test is appropriate in this context [39]. The test confidence level considered was 95%. In Table 8.2, we present the mean and standard deviation (SD) of the accuracies. Based on these, the value of the test statistics is computed. If the value exceeds the corresponding tabled value, the means are unequal with statistical significance (algorithm having higher mean accuracy being significantly superior to the one having lower value).

Table 8.1 Rough set dependency rules for *Vowel* data and input fuzzification parameter values

C_1	$\leftarrow$	$M_1 \vee L_3$
C_1	$\leftarrow$	$M_1 \vee M_2$
C_2	$\leftarrow$	$M_2 \vee M_3 \vee (H_1 \wedge M_2)$
C_2	$\leftarrow$	$M_2 \vee H_3$
C_3	$\leftarrow$	$(L_1 \wedge H_2) \vee (M_1 \wedge H_2)$
C_3	$\leftarrow$	$(L_1 \wedge H_2) \vee (L_1 \wedge M_3)$
C_4	$\leftarrow$	$(L_1 \wedge L_2) \vee (L_1 \wedge L_3) \vee (L_2 \wedge M_3) \vee (L_1 \wedge M_3)$
C_5	$\leftarrow$	$(H_1 \wedge M_2) \vee (M_1 \wedge M_3) \vee (M_1 \wedge M_2) \vee (M_2 \wedge L_1)$
C_5	$\leftarrow$	$(H_1 \wedge M_2) \vee (M_1 \wedge M_2) \vee (H_1 \wedge H_3) \vee (H_2 \wedge L_1)$
C_5	$\leftarrow$	$(L_2 \wedge L_1) \vee (H_3 \wedge M_3) \vee M_1$
C_6	$\leftarrow$	$L_1 \vee M_3 \vee L_2$
C_6	$\leftarrow$	$M_1 \vee H_3$
C_6	$\leftarrow$	$L_1 \vee H_3$
C_6	$\leftarrow$	$M_1 \vee M_3 \vee L_2.$

Fuzzification parameters:

Feature 1: $c_L = 0.348$, $c_M = 0.463$, $c_H = 0.613$, $\lambda_L = 0.115$, $\lambda_M = 0.150$, $\lambda_H = 0.134$

Feature 2: $c_L = 0.219$, $c_M = 0.437$, $c_H = 0.725$, $\lambda_L = 0.218$, $\lambda_M = 0.253$, $\lambda_H = 0.288$

Feature 3: $c_L = 0.396$, $c_M = 0.542$, $c_H = 0.678$, $\lambda_L = 0.146$, $\lambda_M = 0.140$, $\lambda_H = 0.135$

It is observed from Table 8.2 that Model S performs the best with the least network size as well as least number of sweeps. For Model R, the classification performance on test set is marginally better than that of Model S, but with significantly higher number of links and training sweeps required. Comparing models F and R, we observe that the incorporation of domain knowledge in the latter through rough sets boosts its performance. The variation of the classification accuracy of the models with iteration is also studied. Model S converges after about 90 iterations of the GA, providing the highest accuracy compared to all the other models. The backpropagation-based models require about 2000–5000 iterations for convergence.

Table 8.2 Comparative performance of different models on *Vowel* data

Models	Model O		Model F		Model R		Model S	
	Train	Test	Train	Test	Train	Test	Train	Test
Accuracy(%)	65.4,	64.1,	84.1,	81.8,	86.7,	86.0,	87.1,	85.8,
(Mean, SD)	0.5	0.5	0.4	0.5	0.3	0.2	0.2	0.2
No. of links	131		210		152		84	
Sweeps	5600		5600		2000		90	

SD: Standard Deviation

It is observed that Model F results in a dense network with weak links, while the incorporation of rough sets, modular concepts, and GAs in Model S produces a sparse network with strong links. The latter is, therefore, more

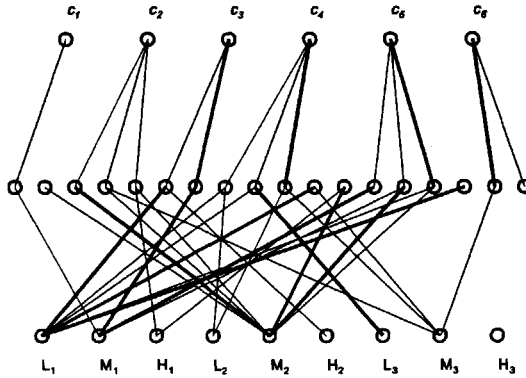


Fig. 8.4 Positive connectivity of the network obtained for *Vowel* data, using Model S. (Bold lines indicate weights greater than $PThres_2$, while others indicate values between $PThres_1$ and $PThres_2$).

suitable for rule extraction. The connectivity (positive weights) of the trained network is shown in Fig. 8.4.

8.3.5.2 Rule extraction We use the algorithm explained in Section 8.3.4 to extract refined rules from the trained network (Model S). These rules are compared to those obtained by the *Subset* method [23], *M of N* method [24], a pedagogical method X2R [38], and a decision tree-based method C4.5 [10] in terms of the quantitative performance measures (Section 2.4.2). The set of rules extracted from the network is presented in Table 8.3, along with their certainty (confidence) factors (*cf*). The values of the fuzzification parameters of the membership functions L, M, and H are also mentioned.

A comparison of the performance indices of the extracted rules is presented in Table 8.4. Since the network obtained using Model S contains fewer links, the generated rules are less in number and have high *certainty factor*. Accordingly, it possesses relatively higher percentage of uncovered region, though the accuracy does not suffer much. On the other hand, the *Subset* algorithm achieves the highest *accuracy* but requires the largest number of rules and computation time. In fact, while the accuracy (computation time) of *Subset* method is marginally better (or worse) than Model S, the size of the rule base of Model S is significantly lower.

The *accuracy*, *user's accuracy* and *kappa* achieved by Model S, are better than those of *M of N*, X2R, and C4.5. The X2R algorithm requires least computation time but achieves least accuracy with more rules. The *Conf* index is the minimum for rules extracted by Model S; it also has a high *fidelity* of 94.22%.

Table 8.3 Rules extracted from trained network for *Vowel* data and input fuzzification parameter values

$C_1 \leftarrow M_1 \vee L_3 \vee M_2$	$cf = 0.851$
$C_1 \leftarrow H_1 \vee M_2$	$cf = 0.755$
$C_2 \leftarrow M_2 \vee M_3$	$cf = 0.811$
$C_2 \leftarrow \overline{M}_1 \wedge \overline{H}_1 \wedge L_2 \wedge M_2$	$cf = 0.846$
$C_3 \leftarrow L_1 \vee H_2$	$cf = 0.778$
$C_4 \leftarrow L_1 \wedge L_2 \wedge \overline{L}_3$	$cf = 0.719$
$C_5 \leftarrow M_1 \wedge H_2$	$cf = 0.881$
$C_5 \leftarrow M_1 \wedge M_2$	$cf = 0.782$
$C_5 \leftarrow H_1 \wedge M_2$	$cf = 0.721$
$C_6 \leftarrow \overline{H}_2$	$cf = 0.717$

Fuzzification parameters :

Feature 1 :	$c_L = 0.34,$	$c_M = 0.502,$	$c_H = 0.681,$
	$\lambda_L = 0.122,$	$\lambda_M = 0.154,$	$\lambda_H = 0.177$
Feature 2 :	$c_L = 0.217,$	$c_M = 0.431,$	$c_H = 0.725,$
	$\lambda_L = 0.211,$	$\lambda_M = 0.250,$	$\lambda_H = 0.288$
Feature 3 :	$c_L = 0.380,$	$c_M = 0.540,$	$c_H = 0.675,$
	$\lambda_L = 0.244,$	$\lambda_M = 0.212,$	$\lambda_H = 0.224$

Table 8.4 Comparative performance of rules extracted by various methods for *Vowel* data

Algorithm	Accuracy (%)	User's accuracy (%)	Kappa (%)	Uncovered region (%)	No. of rules	CPU time (sec)	Conf
Model S	81.02	83.31	78.17	3.10	10	1.1	1.4
Subset	82.01	82.72	77.29	2.89	16	1.4	1.9
M of N	79.00	80.01	74.55	2.10	14	1.2	1.9
X2R	76.00	75.81	72.34	2.72	14	0.9	1.7
C4.5	79.00	79.17	77.21	3.10	16	1.0	1.5

8.4 CONCLUSIONS AND DISCUSSION

In this chapter we have provided an exhaustive survey of rule generation in the soft computing framework involving hybridization of its different tools like fuzzy sets, ANNs, rough sets, and GAs. Incorporation of domain knowledge involving user interaction generates knowledge-based networks. This is a useful feature in data mining. Rule generation from fuzzy and nonfuzzy knowledge-based networks are found to result in refined rules.

A divide-and-conquer strategy involving modular subnetworks is effective for mining large datasets. Such a methodology, incorporating the four soft computing tools (namely, ANNs, fuzzy sets, GAs, and rough sets), has been used for designing a knowledge-based network for pattern classification and rule generation. The algorithm involves synthesis of several fuzzy MLP modules, each encoding the rough set rules for a particular class. These knowledge-based modules are refined using a GA. The genetic operators are implemented in such a way that they help preserve the modular structure already evolved. It is found that this scheme results in superior performance in terms of classification score, training time, and network sparseness (thereby enabling easier extraction of rules).

The extracted rules are compared with some of the related rule extraction techniques on the basis of some quantitative performance indices. It is observed that these rules are less in number, yet accurate, and have high certainty factor and low confusion with less computation time. The investigation, besides having significance in soft computing research, has potential for application to large-scale problems involving knowledge discovery tasks [40].

REFERENCES

1. S. Mitra and Y. Hayashi, "Neuro-fuzzy rule generation: Survey in soft computing framework," *IEEE Transactions on Neural Networks*, vol. 11, pp. 748–768, 2000.
2. R. Andrews, J. Diederich, and A. B. Tickle, "A survey and critique of techniques for extracting rules from trained artificial neural networks," *Knowledge-Based Systems*, vol. 8, pp. 373–389, 1995.
3. I. A. Taha and J. Ghosh, "Symbolic interpretation of artificial neural networks," *IEEE Transactions on Knowledge and Data Engineering*, vol. 11, pp. 448–463, 1999.
4. B. M. Happel and J. J. Murre, "Design and evolution of modular neural network architectures," *Neural Networks*, vol. 7, pp. 985–1004, 1994.

5. L. Hansen and P. Salamon, "Neural network ensembles," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 12, pp. 993–1001, 1990.
6. S. I. Gallant, "Connectionist expert systems," *Communications of the Association for Computing Machinery*, vol. 31, pp. 152–169, 1988.
7. M. Ishikawa, "Structural learning with forgetting," *Neural Networks*, vol. 9, pp. 509–521, 1996.
8. W. Duch, R. Adamczak, and K. Grabczewski, "Extraction of logical rules from neural networks," *Neural Processing Letters*, vol. 7, pp. 211–219, 1998.
9. R. Setiono, "Extracting rules from neural networks by pruning and hidden-unit splitting," *Neural Computation*, vol. 9, pp. 205–225, 1997.
10. J. R. Quinlan, *C4.5: Programs for Machine Learning*. San Mateo, CA: Morgan Kaufmann, 1993.
11. R. Setiono, "Extracting M of N rules from trained neural networks," *IEEE Transactions on Neural Networks*, vol. 11, pp. 512–519, 2000.
12. R. Setiono and H. Liu, "A connectionist approach to generating oblique decision trees," *IEEE Transactions on Systems, Man, and Cybernetics, Part B: Cybernetics*, vol. 29, pp. 440–444, 1999.
13. L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Monterey, CA: Wadsworth and Brooks/Cole, 1984.
14. D. Nauck, F. Klawonn, and R. Kruse, *Foundations of Neuro-Fuzzy Systems*. Chichester, England: John Wiley & Sons, 1997.
15. D. Nauck and R. Kruse, "Neuro-fuzzy systems for function approximation," *Fuzzy Sets and Systems*, vol. 101, pp. 261–271, 1999.
16. E. H. Mamdani and S. Assilian, "An experiment in linguistic synthesis with a fuzzy logic controller," *International Journal of Man Machine Studies*, vol. 7, pp. 1–13, 1975.
17. F. C. H. Rhee and R. Krishnapuram, "Fuzzy rule generation methods for high-level computer vision," *Fuzzy Sets and Systems*, vol. 60, pp. 245–258, 1993.
18. Y. Q. Zhang and A. Kandel, "Compensatory neuro-fuzzy systems with fast learning algorithms," *IEEE Transactions on Neural Networks*, vol. 9, pp. 83–105, 1998.
19. S. Mitra and S. K. Pal, "Logical operation based fuzzy MLP for classification and rule generation," *Neural Networks*, vol. 7, pp. 353–373, 1994.

20. L. M. Fu, "Learning capacity and sample complexity on expert networks," *IEEE Transactions on Neural Networks*, vol. 7, pp. 1517-1520, 1996.
21. G. G. Towell and J. W. Shavlik, "Knowledge-based artificial neural networks," *Artificial Intelligence*, vol. 70, pp. 119-165, 1994.
22. D. W. Opitz and J. W. Shavlik, "Dynamically adding symbolically meaningful nodes to knowledge-based neural networks," *Knowledge-Based Systems*, vol. 8, pp. 310-311, 1995.
23. L. M. Fu, "Knowledge-based connectionism for revising domain theories," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 23, pp. 173-182, 1993.
24. G. G. Towell and J. W. Shavlik, "Extracting refined rules from knowledge-based neural networks," *Machine Learning*, vol. 13, pp. 71-101, 1993.
25. G. A. Carpenter and S. Grossberg, "A massively parallel architecture for a self-organising neural pattern recognition machine," *Computer Vision, Graphics and Image Processing*, vol. 37, pp. 54-115, 1987.
26. G. A. Carpenter, S. Grossberg, and J. H. Reynolds, "ARTMAP: Supervised real-time learning and classification of nonstationary data by a self-organizing neural network," *Neural Networks*, vol. 4, pp. 565-588, 1991.
27. G. A. Carpenter, S. Grossberg, N. Markuzon, J. H. Reynolds, and D. B. Rosen, "Fuzzy ARTMAP: a neural network architecture for incremental supervised learning of analog multidimensional maps," *IEEE Transactions on Neural Networks*, vol. 3, pp. 698-713, 1992.
28. A. H. Tan, "Cascade ARTMAP: Integrating neural computation and symbolic knowledge processing," *IEEE Transactions on Neural Networks*, vol. 8, pp. 237-250, 1997.
29. R. J. Machado and A. F. da Rocha, "Inference, inquiry, evidence censorship, and explanation in connectionist expert systems," *IEEE Transactions on Fuzzy Systems*, vol. 5, pp. 443-459, 1997.
30. S. Mitra, R. K. De, and S. K. Pal, "Knowledge-based fuzzy MLP for classification and rule generation," *IEEE Transactions on Neural Networks*, vol. 8, pp. 1338-1350, 1997.
31. D. W. Opitz and J. W. Shavlik, "Connectionist theory refinement: Genetically searching the space of network topologies," *Journal of Artificial Intelligence Research*, vol. 6, pp. 177-209, 1997.
32. M. Banerjee, S. Mitra, and S. K. Pal, "Rough fuzzy MLP: Knowledge encoding and classification," *IEEE Transactions on Neural Networks*, vol. 9, pp. 1203-1216, 1998.

33. S. Mitra, M. Banerjee, and S. K. Pal, "Rough knowledge-based network, fuzziness and classification," *Neural Computing and Applications*, vol. 7, pp. 17–25, 1998.
34. E. B. Baum and D. Haussler, "What size nets give valid generalizations?," *Neural Computations*, vol. 1, pp. 151–160, 1989.
35. P. Mitra, S. Mitra, and S. K. Pal, "Staging of cervical cancer with soft computing," *IEEE Transactions on Biomedical Engineering*, vol. 47, pp. 934–940, 2000.
36. S. Mitra, P. Mitra, and S. K. Pal, "Evolutionary modular design of rough knowledge-based network using fuzzy attributes," *Neurocomputing*, vol. 36, pp. 45–66, 2001.
37. S. K. Pal, S. Mitra, and P. Mitra, "Rough Fuzzy MLP: Modular evolution, rule generation and evaluation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 15, pp. 14–25, 2003.
38. H. Liu and S. T. Tan, "X2R: A fast rule generator," in *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics* (Vancouver, Canada), pp. 215–220, 1995.
39. E. L. Lehmann, *Testing of Statistical Hypotheses*. New York: John Wiley & Sons, 1976.
40. S. Mitra, S. K. Pal, and P. Mitra, "Data mining in soft computing framework: A survey," *IEEE Transactions on Neural Networks*, vol. 13, pp. 3–14, 2002.

9

Multimedia Data Mining

9.1 INTRODUCTION

There has been a significant growth in information technology over the last decade because of the ever-growing power of the computing systems, advances in data storage management, and tremendous progress in communication technologies. A conventional information processing system has been mainly based on alphanumeric data with a very structured data representation, and the type of computation involved is mainly number crunching in nature. However, representation of digital information is no longer restricted in the form of numeric and alphanumeric data only.

Classically, databases were formed by tuples of numeric and alphanumeric contents. Today, information processing revolves around different datatypes in higher-order abstraction of data representation, such as text, document, image, video, graphics, speech, audio, hypertext, markup languages, etc. The growth in Internet technologies have added a new dimension to the interactive usage of these different datatypes as well. Interactive processing and management of these different datatypes is another important aspect of multimedia processing.

There has been tremendous demand in storage and management of multimedia data over the last decade. Progress in data compression research has been one of the enabling factors in this direction. A multimedia database management system stores and manages these large collection of multimedia datatypes, namely, text, digital image, digital video, graphics, voice, audio, etc. Popularity of multimedia database systems is increasing, with the growing

use of interactive audio-visual systems, digital camera, scanner, CD-ROM, Internet, etc. Multimedia databases fall in the category of very large database management system. Today's *World Wide Web* can be considered to be a distributed multimedia database of the largest size ever used.

Multimedia data are usually semistructured and often very much unstructured, as compared to numeric and alphanumeric data structures that are usually dealt with in traditional database management systems. Most of the classical data mining algorithms and techniques were mainly developed for mining information from structured data, such as relational, transactional, and data warehouse data. They had been used typically in financial and business market analysis, forecast, etc. However, in reality not all datatypes are very structured, and a substantial amount of information is available as semistructured or unstructured multimedia data. As a result, retrieval of multimedia data is challenging [1].

Most of the information in government, business and other institutions are stored in the form of text databases. As a matter of fact, recent studies show that more than 80% of company information is stored in the form of textual documents. Visual information, available in the form of still imagery, video, and graphics, is a natural medium of communication by humans.

A tremendous amount of visual information is being processed and communicated everyday in this digital multimedia age. The medical community today relies upon the visual information conveyed by the digital medical images such as X-ray, MRI, CAT/SCAN, etc. Security and surveillance applications, based on online video and imaging applications, is becoming popular. Similarly, voice, audio, and speech data contain a huge amount of information in digital form as well. As a result, retrieval and mining of information from multimedia data will have significant impact in the current and future information age.

It is obvious that development of data mining techniques particularly useful for multimedia datatypes will continue to be challenging, and there needs to be focused efforts to influence this area of study. Development of sophisticated data mining technologies for multimedia data could very well be the catalyst for the next-generation information technology revolution.

In this chapter we provide a study on the developments in multimedia data mining. Section 9.2 deals with different aspects of text mining. Section 9.3 presents issues in image mining, including color images and content-based image retrieval. Video mining, encompassing content-based video retrieval and MPEG standard, is described in Section 9.4. Section 9.5 provides an introduction to Web mining. Finally, Section 9.6 concludes the chapter.

9.2 TEXT MINING

Text data stored in most of the text databases are usually semistructured and possibly unstructured. There is a vast amount of information available in text

or document databases, in the form of electronic publication of books, digital libraries, electronic mails, electronic media, technical and business documents, reports, research articles, Web pages in the Internet, hypertext, markup languages, etc. In order to aid mining of information from large collections of such text databases, special types of data mining methodologies have been developed recently. This domain of study is popularly known as '*text mining*' [2]–[4].

Text mining is an emerging area of study, currently under rapid development in scientific research. In addition to traditional data mining methodologies, text mining uses techniques from many multidisciplinary scientific fields in order to gain insight, understand and interpret, and automatically extract information from large quantities of text data available in text databases distributed everywhere. The functionalities of text mining methodologies have been mainly built on the results of *text analysis* techniques. Some of the other areas that have recently influenced text mining are string matching, text searching, artificial intelligence, machine learning, information retrieval, natural language processing, statistics, information theory, soft computing, etc. The *Internet search* engines, combined with various text analysis techniques, have paved the way for online text mining as well.

9.2.1 Keyword-based search and mining

The search of text databases is different from the search techniques applied in traditional relational database management systems. A crude way of mining text databases is to apply keyword based searching. In this simplistic approach the documents are considered to be strings, with a set of *keywords* being the signature of the text data and indexed accordingly. A keyword can be searched inside a text file using string matching techniques, which may involve exact match or approximate match as explained in Chapter 4. String-matched keywords or patterns, found inside the text, are then used to index the documents. After the documents have been identified by the keywords, traditional data mining techniques (such as classification, clustering, rule mining, etc.) can be applied with probably some degree of success depending upon the characteristics of the collection of the documents in the text database.

There are two major problems with such a simplistic approach that does not take the semantic meaning of the keywords into consideration. These two challenging problems are *synonymy* and *polysemy*, which have been a long standing problem in the area of *natural language processing*. A keyword provided by the user may not appear at all in the document, whereas the document may be very much related to the keyword because the same thing can often be described in different ways in a natural language.

For example, the keyword could be '*woman*', whereas the document may not exactly contain any instance of the word '*woman*' but contain the word '*lady*' frequently. This is known as the *synonymy* problem. This problem

can be addressed to some extent by just filtering the document such that the words of similar meaning are replaced by a chosen canonical token word. For example, the words 'automobile', 'vehicle', and 'vehicular' can simply be replaced by the word 'car'. Similarly, the words 'is', 'are', 'am', 'were', 'was', 'been', 'being' can be replaced by the word 'be' when they appear in a document. However, this is not a very practical proposition, because it is not possible to maintain a universal list from the dictionary of English language to form the tokens of such types of words.

It is also possible that the same word may have different meanings in different contexts. For example, the word '*mining*' has different meaning in the context of '*data mining*' as compared to the aspect of '*coal mining*'. This is called the *polysemy* problem. Hence the success of the *natural language processing*, coupled with other artificial intelligence areas to solve these problems, will have great influence on text mining in the long run.

9.2.2 Text analysis and retrieval

Text analysis has been a field of study in natural language processing and information retrieval for quite a while. Since most of the Internet search techniques are text-based, text analysis also received prominence with the growth of the Internet.

Usually, text data are semistructured, and easy to read and interpret by humans. Text analysis techniques can be applied to extract relevant key features from a text, categorize the text documents based on its semantic contents, index the documents, extract the overview of large collection of text documents, organize large collections of documents in efficient ways, improve the effectiveness of automatic search process, detect duplicate documents in large text databases, etc.

In full-text retrieval systems, automatic indexing of the documents are often done based on statistical analysis of the common words and phrases that appear in the document. One such simple method for automated text document indexing can be defined by the following steps.

1. Find the unique words in each document in the document database.
2. Calculate the frequency of occurrence of each of these unique words for each document in the document database.
3. Compute the total frequency of occurrence of each word across all the documents in the database.
4. Sort the words in ascending order of their frequency of occurrence in the database.
5. Remove the words with very high frequency of occurrences from this sorted list.

6. Remove the words with low frequency of occurrences from this sorted list.
7. Use the remaining words as index for the text database.

9.2.3 Mathematical modeling of documents

The *text* data can be loosely considered as a composition of two basic units, namely, *document* and *term* [2, 5]. In the general sense, a *document* is a structured or semistructured segment of a text. For example, this book is a *text document* and it is structured in the form of a number of chapters, where each chapter is composed of sections, and each section may be composed of a number of subsections and paragraphs, etc. Similarly, an electronic mail can be considered a document because it contains a message header, title, and content of the message, in a defined structured fashion. There are many such documents that exist in practice. Some other examples are source codes, Web pages, spreadsheets, telephone directory, etc. A *term* is a word or group of words or a phrase that exists in a document. Terms are extracted from the documents using one of the string matching algorithms described in Chapter 4.

We can model a *text* document using this definition of *document* and *term* [2, 5]. Let us consider a set of N documents $D = (d_1, d_2, d_3, \dots, d_N)$ and a set of M terms $T = (t_1, t_2, t_3, \dots, t_M)$. We can model each document d_i as a vector $V_i = (v_{i,1}, v_{i,2}, \dots, v_{i,M})$ in the M -dimensional space R^M . The entry $v_{i,j}$ represents a measure of association of the term t_j with the document d_i . The value of $v_{i,j}$ is 0 if the document d_i does not contain the term t_j and is nonzero otherwise. In simple boolean representation, $v_{i,j} = 1$ if the term t_j appears in document d_i . However, this measure is not found to be very robust in text retrieval. The more popular and practical measure of association ($v_{i,j}$) is the *term frequency*, which is simply defined as the number of occurrences of the term t_j in document d_i . Using this approach, the text is simply modeled as a *document-term frequency matrix* as depicted in Fig. 9.1.

In Fig. 9.1, we have shown a 5×4 array to represent the *document-term frequency matrix* for a set of five *documents* and four *terms*. Let us assume that the selected terms are

- $t_1 = \text{monkey}$,
- $t_2 = \text{bird}$,
- $t_3 = \text{flower}$,
- $t_4 = \text{sky}$.

The second row in the matrix is the vector (5, 9, 4, 3) representing the document d_2 , in which the term monkey appears 5 times, bird appears 9 times, flower appears 4 times, and sky appears 3 times, respectively.

	t_1	t_2	t_3	t_4
d_1	10	8	1	0
d_2	5	9	4	3
d_3	0	15	10	1
d_4	23	0	0	7
d_5	52	19	2	8

Fig. 9.1 Document-term frequency matrix for five documents and four terms.

It is possible that some of the terms may appear more frequently in the documents set of many documents than the others. This may represent the fact that these terms are more important than others in determining the content of a document. For example, the term 'information' is definitely more important than the words 'is', 'the', 'are', 'am', 'of', etc. in any English text. The problem with document-term frequency matrix model is that it does not capture this phenomena. In order to increase the discrimination power for these terms, the corresponding term frequencies can be weighted by *inverse-document frequency* (*IDF*). The *inverse-document frequency* of term t_j is defined by

$$IDF_j = 1 + \log \frac{n_j}{N}, \quad (9.1)$$

where N is the number of documents and n_j is the number of documents that contains the term t_j . The *IDF* favors the terms that appear in more documents than the others. The discriminating power can further be improved by updating each entry $v_{i,j}$ in the document-term frequency matrix as

$$\bar{v}_{i,j} = \frac{v_{i,j}}{\sum_{k=1}^M v_{i,k}} * IDF_j. \quad (9.2)$$

9.2.4 Similarity-based matching for documents and queries

When a document is modeled using the document-term frequency matrix representation or its variants, as explained above, the relative ordering of words in the text gets lost. Thereby the syntactic information of formation of the text, such as the grammar for the sentence structure in English text, also disappears. In spite of this, the term frequency modeling has been found to be very effective in a variety of text or document retrieval applications such as query processing, comparison of documents, document analysis, etc.

Once the document is represented in the matrix model, we can apply a distance measure to find the similarity of two documents. The simplest approach is to find the Euclidean distance between the two vectors, corresponding to the two documents. For example, if we want to search a query document d_q in the document database $D = (d_1, d_2, d_3, \dots, d_N)$, we first form the frequency vector $v_q = (v_{q,1}, v_{q,2}, \dots, v_{q,M})$ for the M terms of the term set $T = (t_1, t_2, t_3, \dots, t_M)$. The Euclidean distance between the query document d_q and the document d_i in the document database D is

$$\delta(d_q, d_i) = \sqrt{\sum_{j=1}^M (v_{q,j} - v_{i,j})^2}. \quad (9.3)$$

We can also apply other well-defined statistical distance measures [6] (such as Mahalanobis distance, Manhattan distance, etc.) to find the similarity between two documents. However, the *cosine measure* of two vectors has been found to be very effective in the comparison of two documents. The *cosine measure* of two vectors v_i and v_j can be computed as

$$\cos(v_i, v_j) = \frac{\sum_{k=1}^M [v_{i,k} * v_{j,k}]}{\sqrt{\left[\sum_{k=1}^M v_{i,k}^2 \sum_{k=1}^M v_{j,k}^2 \right]}}. \quad (9.4)$$

This is essentially the cosine of the angle between the two vectors. The cosine measure is nothing but the inner product of the two vectors, after both v_i and v_j have been normalized to have unity length. As a result, the cosine measure reflects the relative distribution of the terms in the vectors and this measure has been found to be very effective in document matching.

Using the numeric values of the above distance measures, we can find the similarity amongst the documents in a document collection. Similarity-based indices can be developed for these documents, followed by the application of traditional data mining techniques for clustering, classification and other operations on the documents based on these indices.

Queries can be expressed by the same term-based representation, considering the query itself as a document formed with a set of terms. As a result, we can also apply the above principles for query matching in a document. The query is expressed as a vector of weights corresponding to the terms that appear in the query, and the weight becomes implicitly zero for those terms not existent in the query. In its simplest form, the vector can contain weight one for the terms existing in the query and zero for others. The distance of the vector is then measured from the vectors corresponding to the documents in the document database.

The main discrepancy of the above document-term frequency matrix approach is that it loses the information regarding syntactic relationship amongst the words in the documents. The other problem with the document-term frequency matrix approach is that a query may contain terms with semantically same, but physically different terminology, as compared to the terms used to index a document. For example, the query may contain the term '*lad*' whereas the document may have been indexed by '*boy*'. Although these two words are semantically the same, from the similarity perspective they are quite different.

One way to solve this problem is to use a predefined dictionary or knowledge base (a thesaurus or ontology) linking semantically related terms together. However, such an approach is inherently very subjective regarding how the terms are related and how similar they are semantically with respect to the content of a particular database. Moreover, the thesaurus could be prohibitively large to contain all possible cases from English or any other human language.

In spite of reasonably good similarity measures, the computational requirements of the above approach is very high. In most of the practical text document databases, the number of terms in the term set could be more than 50,000 and the number of documents could also be very large in the document database. This makes the dimension of the document-term frequency matrix very high and prohibitively large for computational requirements. This high dimensionality also leads the matrix to be very sparse and can further enhance the difficulty in identifying the terms in the document.

9.2.5 Latent semantic analysis

In order to reduce the dimensionality of a matrix, an efficient technique has been developed to analyze text based on the popular *Singular Value Decomposition* used in *principal component analysis* [7, 8]. This technique is called *Latent Semantic Indexing*, and the text analysis using this method is called *Latent Semantic Analysis* (LSA) [2, 3]. As the name implies, this technique helps in extracting the hidden semantic structure from the text rather than just the usage of the term occurrences. LSA also provides a good metric of semantic similarities among documents, based on a term's context. The theory behind singular value decomposition of a matrix and generation of principal components has been discussed in detail in Section 3.7.

The dimensionality of the original document-term frequency matrix F is often prohibitively large. The *Latent Semantic Analysis* (LSA) approximates the original $M \times N$ document-term frequency matrix F to a much smaller matrix of size $N \times K$, using only K principal components generated by the singular value decomposition (SVD) method described in Section 3.7. In reality, the value of K can be much smaller than the original dimension N of the *term* set. Typical values of N could be 10,000 to 50,000 or more while K can be in the order of 100 or less, literally without significant loss of information. The SVD approach exploits the possible redundancy in the terms of the document.

Let us consider an example to better understand the significance of LSA. We can assume that the terms such as '*image*', '*picture*', '*pixel array*' can be expected to contain redundancy in the semantic sense in a document, because they are quite related and essentially mean the same thing. A text document discussing about multimedia or image processing aspects may contain all these terms, and they may appear in the document-term frequency matrix all together. The intuitive logic behind the principal component representation of the document-term frequency matrix is that the reduced matrix may capture semantic relationships among the original terms by creating new terms that compactly reflect the semantic content of the text document. As a result, the terms '*image*', '*picture*', '*pixel array*' are effectively mapped into a single principal component term, and this new term can be considered to indicate that the document is related to a multimedia or image processing topic. Hence this term can be used to create an index for the document. If a query contains the term '*picture*', the retrieval system based on this LSA approach will successfully identify a document related to '*image*' or '*pixel array*'. On the other hand, the simple document-term frequency-based approach will fail to properly identify this situation.

The LSA approach for text indexing employs the transformed document-term frequency matrix to compare the similarity between two documents by distance measures (of Section 9.2.4) or to extract a prioritized list of (say, N) matches for a query. The indices generated through text analysis can be used to classify the documents. Then association rule mining can be applied to the terms to discover sets of associated terms, which can be used to distinguish one class of documents from others.

The text mining process can be broadly separated in two phases, namely, *text refinement* and *knowledge extraction*. In the text refinement phase, the original unstructured or semistructured text document is transformed into a chosen intermediate form or model. In the second phase, the knowledge is then discovered from this intermediate model by extracting patterns and mining rules.

9.2.6 Soft computing approaches

Since the free-form text is usually unstructured or semistructured, the application of soft computing approaches can be promising to analyze the imprecise nature of the data and extraction of patterns for knowledge discovery. Recently, there have been some developments in this direction.

Inductive learning, using fuzzy decision tree (Section 5.6), has been developed for imprecise text mining [9]. A concept relation dictionary and a classification tree are generated from a random set of daily business reports database of text classes concerning retailing. In their experiments the authors use 10,000 evaluation examples, with a decision tree of maximum 90 nodes (37 intermediate and 53 terminal).

An approach in Ref. [10] uses fuzzy association thesaurus and query expansion for text retrieval. Fuzzy composition operations like *max-min*, *max-product* and *sum-product* are used for constructing the thesaurus. Interactive query expansion shows the user, upon initial query, a ranked list of documents suggested by the system based on the fuzzy relation composition. A measure of similarity helps select the correlated terms using queries with/without weight. The experiments use a collection of daily news in Chinese, with 981 homogeneous and 700 heterogeneous text documents.

An HTML document can be viewed as a structured entity, in which document subparts are identified by *tags* and each such subpart consists of text delimited by a distinct tag. A fuzzy representation of HTML documents is described in Ref. [11]. The HTML document is represented by a sum of fuzzy set terms

$$R(d) = \sum_{t \in T} \frac{\mu_d(t)}{t}, \quad (9.5)$$

where the importance of each term t in document d is given by the membership value

$$\mu_d(t) = \sum_{i=1}^n (\mu_{tag_i}(d, t) * w_i) * g(IDF_t),$$

w_i is the normalized importance weight associated with tag_i , n corresponds to the number of tags, $g(\cdot)$ is a normalization function, and IDF_t is the inverse-document frequency. The significance of an index term is computed by weighting the occurrence of the term with the importance of the tag associated with it.

Recommending alternate queries during textual information retrieval is an important feature in a Web-based search engine, because users often do not know the exact terms to locate the information relevant to their interests. Fuzzy ontology is used for query refinement in a domain search engine named Personalizing Abstract Search Service (PASS) [12]. Fuzzy *narrower-* and *broader-than* term relations are defined using a fuzzy conjunction operator. Pruning of redundant relations is made by analyzing the sets of relations in-

volving more than two terms. PASS is used to provide the abstracts of papers from the *IEEE Transactions on Neural Networks* journal.

A key issue in text mining is keyword extraction. This allows the automatic categorization and classification of text documents. Keyword extraction can be done using clustering methods. Relational Alternating Cluster Estimation (RACE), based on Levenshtein distance, was used to automatically extract the 20 most relevant keywords from Internet documents in Ref. [13]. Using these keywords, corresponding to the cluster centers, a classification rate of more than 80% could be achieved.

Self-organization of the Web (WEBSOM) [14, 15], based on Kohonen's SOM (Section 2.2.3.3), has been used for exploring document collections. A textual document collection is organized onto a graphical map display that provides an overview of the collection and facilitates interactive browsing. The browsing can be focused by locating some interesting documents on the map using content addressing. The excellent visualization capabilities of SOM are utilized for this purpose.

9.3 IMAGE MINING

Traditional data mining techniques have been developed mainly for structured datatypes. The image datatype does not belong to this structured category, suitable for interpretation by a machine, and hence the mining of image data is a challenging problem. Content of an image is visual in nature and the interpretation of the information conveyed by an image is mainly subjective, based on the human visual system.

Image data have been used for machine vision, based on extraction of desired features from an image and interpretation of these features for particular applications. This is a challenging area of study, and it has been extensively explored in pattern recognition and machine vision for quite some time. Although interpretation of the image content by the human visual system is a natural and apparently effortless procedure, it remains a mystery how the human brain processes this information. Hence modeling the process of human interpretation of the semantic content of images is still a research challenge. As a result, it is difficult to define a single set of algorithms or functionalities that can claim to comprise a complete set of image mining tools.

The research and development of mining image data is relatively new, and has become an emerging field of study today. Most of the activities in mining image data have been in the search and retrieval of images based on the analysis of similarity of a query image or its feature(s) with the entries in the image database. The image retrieval systems can be broadly categorized into two categories based on the type of searches, using either description of an image or its visual content.

In the first category, the images are described based on user-defined texts [16, 17]. The images are indexed and retrieved based on these rudimentary

descriptions, such as their size, type, date and time of capture, identity of owner, keywords, or some text description of the image. As a result, this is often called *description based* or *text-based image retrieval* process. The image indices are predefined based on these descriptions, and they are searched on these indices when a query is posed as

“find the images from a image database which matches with the given set of descriptions, e.g. images captured on January 8 to January 12, 1999 and size bigger than 100 KByte.”

The text-based descriptions of the images are usually typed manually for each image by human operators, because the automatic generation of keywords for the images is difficult without incorporation of visual information and feature extraction. As a result, this is a very labor-intensive process and is impractical in today's multimedia information age. Moreover, since the description of images are very much subjective, the automated process to generate a text-based description for indexing of the images could be very inaccurate and incomplete. In the second category, the query can be posed as

“find the images similar to a given query image.”

This second category of similarity based image retrieval process is called *Content Based Image Retrieval* (CBIR) [18]–[22]. In CBIR systems, the images are searched and retrieved based on the visual content of the images. Based on these visual contents, desirable images features can be extracted and used as index or basis of search. *Content-based image retrieval* is highly desirable and has increasingly become a growing area of study towards the successful development of image mining techniques.

9.3.1 Content-Based Image Retrieval

There are, in general, three fundamental modules in a content-based image retrieval system. These are

1. Visual content or feature extraction,
2. Multidimensional indexing, and
3. Retrieval.

The images in an image database are indexed-based on extracted inherent visual contents (or features) such as color, texture, pattern, image topology, shape of objects and their layouts and locations within the image, etc. An image can be represented by a multidimensional vector of the extracted features from the image. The feature vector actually acts as the *signature* of the image. This feature vector can be assumed to be associated to a point in the

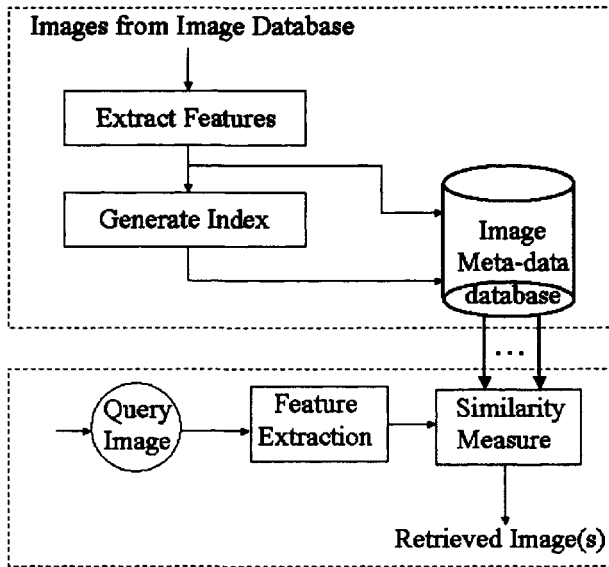


Fig. 9.2 Architecture of a Content Based Image Retrieval System.

multidimensional space. As an example, an image can be represented by an N -dimensional feature vector whose first n_1 components may represent *color*, the next n_2 components may represent *shape*, the following n_3 components may represent some image *topology*, and finally n_4 components may represent *texture* of the image, so that there are $N = n_1 + n_2 + n_3 + n_4$ components. As a result, an *example image* can simply be used as a query using visual content based indexing.

The query image can be analyzed to extract the visual features and can be compared to find matches with the indices of the images stored in the database. The extracted image features are stored as meta-data, and images are indexed based on these meta-data information. This meta-data information comprises some measures of the extracted image features. The feature vectors of similar images will then be clustered in the N -dimensional space. Retrieving *similar* images to a *query* image then boils down to finding the indices of those images in the N -dimensional search space whose feature vectors in the N -dimensional space are within some threshold of proximity to the point of the query image. This indexing structure is popularly known as *Multidimensional Access Structure* (MAS) [23].

The architecture for a possible content based image retrieval system is shown in Fig. 9.2. The CBIR systems architecture is essentially divided into two parts. In the first part, the images from the image database are processed off-line. The features from each image in the image database are extracted to

form the meta-data information of the image, in order to describe the image using its visual content features. Next these features are used to index the image, and they are stored into the meta-data database along with the images. In the second part, the retrieval process is depicted. The query image is analyzed to extract the visual features, and these features are used to retrieve the similar images from the image database. Rather than directly comparing two images, similarity of the visual features of the query image is measured with the features of each image stored in the meta-data database as their signatures. Often the similarity of two images are measured by computing the distance between the feature vectors of the two images. The retrieval systems returns the first k images, whose distance from the query image is below some defined threshold.

Several image features have been used to index images for content-based image retrieval systems. Most popular amongst them are color, texture, shape, image topology, color layout, region of interest, etc. We discuss some of these features in greater detail in the following sections.

9.3.2 Color features

Color is one of the most widely used visual features in content based image retrieval [24]–[27]. While we can perceive only a limited number of gray levels, our eyes are able to distinguish thousands of colors and a computer can represent even millions of distinguishable colors in practice. Color has been successfully applied to retrieve images, because it has very strong correlations with the underlying objects in an image. Moreover, color feature is robust to background complications, scaling, orientation, perspective, and size of an image. A color pixel in a digital image is represented by three color channels (usually Red, Green and Blue). It is well known that any color can be produced by mixing these three primary colors.

Although we can use any color space for computation of a color histogram HSV (Hue, Saturation, and Value), HLS (Hue, Lightness, and Saturation), CIE color spaces (such as CIELAB, CIELUV) [28] were found to deliver better results as compared to the RGB space. Since these color spaces are visually (or perceptually) uniform compared to the RGB, they are found to be more effective to measure color similarities between images. We describe here the HSV color space, because that is widely used in the CBIR community. In the HSV color space,

- *Hue* of a color represents the relative color appearance; that is, ‘redness’, ‘greenness,’ and so on,
- *Value* indicates the darkness of the color (or the perceived illuminance), and
- *Saturation* represents the strength of the color.

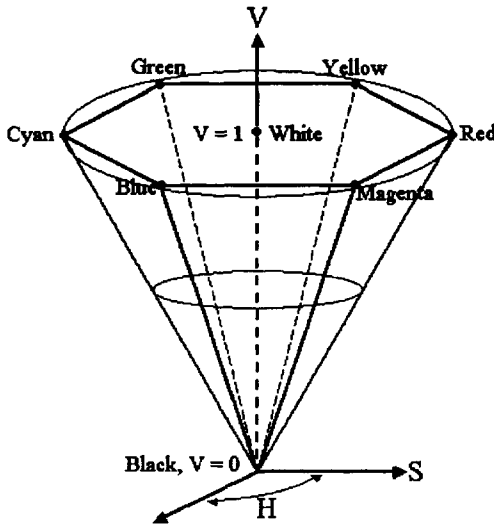


Fig. 9.3 Perceptual representation of HSV color space.

The perceptual representation of the HSV color space has a conical shape, as shown in Fig. 9.3. The 'Value' (V) varies along the vertical axis of the cone, the 'Hue' (H) varies along the periphery of the circle of the cone and is represented as an angle about the vertical axis, and the 'Saturation' (S) varies along the radial distance as shown in Fig. 9.3. We have shown the vertices a hexagon on the periphery of the circle in Fig. 9.3 to show six colors separated by 60° angles. Red is at 0° (coincides with 300°), Yellow at 60° , Green at 120° , Cyan at 180° , Blue at 240° , and Magenta at 300° . The complementary colors are 180° apart. For example, Blue and Yellow are the complementary colors. Apex of the cone ($V = 0$) represents 'Black'. Again, $V = 1$ and $S = 0$ represents 'White'. The colors have their maximum luminosity ($V = 1$) at the periphery of the circle. $V = 1$ and $S = 1$ represent the 'pure' hues for any color.

If the red, green, and blue component values of a color pixel in the RGB space are R , G , and B respectively, they can be linearly normalized to fractions r , g , and b in the range $[0, 1]$ and these normalized values r , g , and b are used to transform into the HSV space. Transformation of RGB to HSV color space can be accomplished by the following set of equations [28]:

$$V = \max(r, g, b), \quad (9.6)$$

$$S = \begin{cases} 0 & \text{if } V = 0 \\ V - \frac{\min(r, g, b)}{V} & \text{if } V > 0, \end{cases} \quad (9.7)$$

$$H = \begin{cases} 0 & \text{if } S = 0 \\ \frac{60 * (g - b)}{S * V} & \text{if } V = r \\ 60 * \left[2 + \frac{(b - r)}{S * V} \right] & \text{if } V = g \\ 60 * \left[4 + \frac{(r - g)}{S * V} \right] & \text{if } V = b, \end{cases} \quad (9.8)$$

$$H = H + 360 \quad \text{if } H < 0. \quad (9.9)$$

9.3.2.1 Color histogram: This is the most commonly used color feature in CBIR [24, 25]. Color histogram has been found to be very effective in characterizing the global distribution of colors in an image, and it can be used as an important feature for image characterization. To define color histograms, the color space is quantized into a finite number of discrete levels. Each of these levels becomes a bin in the histogram. The color histogram is then computed by counting the number of pixels in each of these discrete levels. There are many different approaches to quantize a color space to determine the number of such discrete levels [24]–[26].

Using the color histogram, we can find the images that have similar color distribution. One can think of the simplest measure of similarity by computing the distance between two histograms. Let us consider that $H^{(1)} = \{h_1^{(1)}, h_2^{(1)}, \dots, h_K^{(1)}\}$ and $H^{(2)} = \{h_1^{(2)}, h_2^{(2)}, \dots, h_K^{(2)}\}$ are two feature vectors generated from the color histograms of two images, where $h_j^{(1)}$ and $h_j^{(2)}$ are the count of pixels in the j th bin of the two histograms respectively, and K is the number of bins in each histogram. We can define a simple distance between two histograms as

$$d_1 = \sum_{j=1}^K |h_j^{(1)} - h_j^{(2)}|. \quad (9.10)$$

There is another popular distance measure between two histograms, popularly known as *histogram intersection*. The histogram intersection is the total number of pixels common to both the histograms. This can be computed as

$$I(H^{(1)}, H^{(2)}) = \sum_{j=1}^K \min(h_j^{(1)} - h_j^{(2)}). \quad (9.11)$$

The above equations can be normalized to maintain the value of the distance measure in the range $[0, 1]$. To normalize $I(H^{(1)}, H^{(2)})$, it is divided either

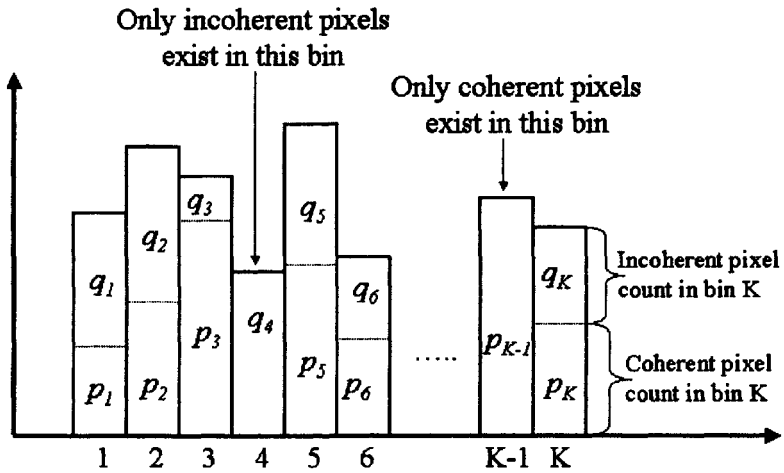


Fig. 9.4 Partitioning the color histogram with coherence and noncoherence pixel counts.

by the total number of pixels in one of the histograms or by the size of the image.

9.3.2.2 Color coherence vector: One problem with the above color histogram-based similarity measure approach is that the global color distribution doesn't reflect the spatial distribution of the color pixels locally in the image. This cannot distinguish whether a particular color is sparsely scattered all over the image or it appears in a single large region in the image.

The color coherence vector [27]-based approach was designed to accommodate the information of spatial color into the color histogram. Here we can classify each pixel in an image, based on whether it belongs to a large uniform region or not. For example, we can consider a region to be uniformly colored if it consists of the same color and the area of the region is above a certain threshold (say, 2%) of the whole image area. We refer to the pixels in these regions as *coherent* pixels.

In this approach, each histogram bin is divided into two parts. One contains the count of pixels belonging to a large uniformly colored region and the other contains the same colored pixels belonging to a sparse region. Let us consider that the i th bin of the histogram contains p_i coherent pixels and q_i incoherent

pixels. Using this partition, the *color coherence vector* of an image can be expressed as $\{(p_1, q_1), (p_2, q_2), \dots, (p_K, q_K)\}$. We have shown this in Fig. 9.4. It should be noted that $\{p_1 + q_1, p_2 + q_2, \dots, p_K + q_K\}$ is the original color histogram without this distinguishing power. The color coherence vectors provide superior retrieval results with this additional distinguishing capability, as compared to the global color histogram method [27].

9.3.2.3 Color moment: This is a compact representation of the color feature to characterize a color image [25]. It has been shown that most of the color distribution information is captured by the three low-order moments. The first-order moment (μ) captures the mean color, the second-order moment (σ) captures the standard deviation, and the third-order moment captures the skewness (θ) of color. These three low-order moments ($\mu_c, \sigma_c, \theta_c$) are extracted for each of the three color planes, using the following mathematical formulation.

$$\mu_c = \frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N p_{ij}^c, \quad (9.12)$$

$$\sigma_c = \left[\frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N (p_{ij}^c - \mu_c)^2 \right]^{\frac{1}{2}}, \quad (9.13)$$

$$\theta_c = \left[\frac{1}{MN} \sum_{i=1}^M \sum_{j=1}^N (p_{ij}^c - \mu_c)^3 \right]^{\frac{1}{3}}, \quad (9.14)$$

where p_{ij}^c is value of the c th color component of the color pixel in the i th row and j th column of the image. As a result, we need to extract only nine parameters (three moments for each of the three color planes) to characterize the color image. Weighted Euclidean distance between the color moments of two images has been found to be effective to calculate color similarity [25].

9.3.2.4 Linguistic color tag: The global color distribution using color histogram does not take advantage of the fact that the adjacent histogram bins might actually represent roughly the same color, because of the limited ability of the human perceptual system. Moreover, only a limited number of color shades are sufficient for visual discrimination between two images. To take advantage of this, a color matching technique based on *linguistic tags*, to identify a color with a name, has been proposed recently [29]. The concept behind this technique is to construct *equivalence classes* of colors, which are identified by *linguistic tags* (or color name such as pink, maroon, etc.) that perceptually appear the same to the human eye but are distinctly different from that of neighboring subspaces. Using this approach, the dimensionality of color features are significantly reduced. This also helps to reduce computations for color similarity measures. A color palette of 15 colors corresponding to

15 equivalent classes were developed by Iqbal and Aggarwal, using this technique, and was applied effectively in color similarity measure to distinguish color images [29].

9.3.3 Texture features

Texture is a very interesting image feature that has been used for characterization of images, with application in content-based image retrieval. There is no single formal definition of *texture* in the literature. However, a major characteristic of texture is the *repetition of a pattern or patterns over a region* in an image. The elements of patterns are sometimes called *textons*. The size, shape, color, and orientation of the textons can vary over the region. The difference between two textures can be in the degree of variation of the textons. It can also be due to spatial statistical distribution of the textons in the image. Texture is an innate property of virtually all surfaces, such as bricks, fabrics, woods, papers, carpets, clouds, trees, lands, skin, etc. It contains important information regarding underlying structural arrangement of the surfaces in an image. When a small area in an image has wide variation of discrete tonal features, the dominant property of that area is *texture*. On the other hand, the *gray tone* is a dominant property when a small area in the image has very small variation of discrete tonal features. Texture analysis has been an active area of research in pattern recognition since the 1970s [30, 31].

A variety of techniques have been used for measuring textural similarity. In 1973, Haralick et al. proposed co-occurrence matrix representation of texture feature to mathematically represent gray level spatial dependence of texture in an image [30]. In this method the co-occurrence matrix is constructed based on the orientation and distance between image pixels. Meaningful statistics are extracted from this co-occurrence matrix, as the representation of texture. Since basic texture patterns are governed by periodic occurrence of certain gray levels, co-occurrence of gray levels at predefined relative positions can be a reasonable measure of the presence of texture and periodicity of the patterns.

Several texture features such as *entropy*, *energy*, *contrast*, and *homogeneity*, can be extracted from the co-occurrence matrix of gray levels of an image [32]. The gray level co-occurrence matrix $C(i, j)$ is defined by first specifying a displacement vector $d_{x,y} = (\delta x, \delta y)$ and then counting all pairs of pixels separated by displacement $d_{x,y}$ and having gray levels i and j . The matrix $C(i, j)$ is normalized by dividing each element in the matrix by the total number of pixel pairs. Using this co-occurrence matrix, the texture features metrics are computed as follows [32].

$$Entropy = - \sum_i \sum_j C(i, j) \log C(i, j), \quad (9.15)$$

$$Energy = \sum_i \sum_j C^2(i, j), \quad (9.16)$$

$$Contrast = \sum_i \sum_j (i - j)^2 C(i, j), \quad (9.17)$$

$$Homogeneity = \sum_i \sum_j \frac{C(i, j)}{1 + |i - j|}. \quad (9.18)$$

Practically, the co-occurrence matrix $C(i, j)$ is computed for several values of displacement $d_{x,y}$, and the one which maximizes a statistical measure is used.

Tamura et al. proposed computational approximations to the texture features, based on the psychological studies in visual perception of textures [31]. The texture properties they found visually meaningful for texture analysis are *coarseness*, *contrast*, *directionality*, *linelikeness*, *regularity*, and *roughness*. These texture features have been used in many content based image retrieval systems [18, 19]. Popular signal processing techniques have also been used in texture analysis and extraction of visual texture features. Wavelet transforms (Section 2.2.7) have been applied in texture analysis and classification of images, based on multiresolution decomposition of the images and representing textures in different scales [33]–[36]. Amongst the different wavelet filters, Gabor filters were found to be very effective in texture analysis.

9.3.4 Shape features

Shape is another image feature applied in CBIR. Shape can roughly be defined as the description of an object minus its position, orientation and size. Therefore, shape features should be invariant to *translation*, *rotation*, and *scale*, for an effective CBIR, when the arrangement of the objects in the image are not known in advance. To use *shape* as an image feature, it is essential to segment the image to detect object or region boundaries; and this is a challenge. Techniques for shape characterization can be divided into two categories.

The first category is *boundary-based*, using the outer contour of the shape of an object. The second category is *region-based*, using the whole shape region of the object. The most prominent representatives of these two categories are *Fourier Descriptors* [37] and *Moment Invariants* [38]. The main idea behind the *Fourier Descriptors* is to use the Fourier-transformed boundaries of the objects as the shape features, whereas the idea behind *Moment Invariants* is to use region-based geometric moments that are invariant to translation and rotation. Hu identified seven normalized central moments as shape features, which are also scale invariant. We provide expressions for these seven invariants below.

9.3.4.1 Moment invariants Let $F(x, y)$ denote an image in the two-dimensional spatial domain. *Geometric Moment* [39]–[41] of order $p + q$ is denoted as

$$m_{p,q} = \sum_x \sum_y x^p y^q F(x, y), \quad (9.19)$$

for $p, q = 0, 1, 2, \dots$. The central moments are expressed as

$$\mu_{pq} = \sum_x \sum_y (x - x_c)^p (y - y_c)^q F(x, y), \quad (9.20)$$

where $x_c = \frac{m_{1,0}}{m_{0,0}}$, $y_c = \frac{m_{0,1}}{m_{0,0}}$, and (x_c, y_c) is called the center of the region or object. Hence the *Central Moments*, of order up to 3, can be computed as

$$\left. \begin{aligned} \mu_{0,0} &= m_{0,0} \\ \mu_{1,0} &= 0 \\ \mu_{0,1} &= 0 \\ \mu_{2,0} &= m_{2,0} - x_c m_{1,0} \\ \mu_{0,2} &= m_{0,2} - y_c m_{0,1} \\ \mu_{1,1} &= m_{1,1} - y_c m_{1,0} \\ \mu_{3,0} &= m_{3,0} - 3x_c m_{2,0} + 2m_{1,0} x_c^2 \\ \mu_{1,2} &= m_{1,2} - 2y_c m_{1,1} - x_c m_{0,2} + 2y_c^2 m_{1,0} \\ \mu_{2,1} &= m_{2,1} - 2x_c m_{1,1} - y_c m_{2,0} + 2x_c^2 m_{0,1} \\ \mu_{0,3} &= m_{0,3} - 3y_c m_{0,2} + 2y_c^2 m_{0,1} \end{aligned} \right\} \quad (9.21)$$

The *normalized central moments*, denoted $\eta_{p,q}$, are defined as

$$\eta_{p,q} = \frac{\mu_{p,q}}{\mu_{0,0}^\gamma}, \quad (9.22)$$

where

$$\gamma = \frac{p + q + 2}{2} \quad (9.23)$$

for $p + q = 2, 3, \dots$. A set of seven *transformation invariant moments* can be derived from the second- and third-order moments as follows.

$$\left. \begin{aligned} \phi_1 &= (\eta_{2,0} + \eta_{0,2}) \\ \phi_2 &= (\eta_{2,0} - \eta_{0,2})^2 + 4\eta_{1,1}^2 \\ \phi_3 &= (\eta_{3,0} - 3\eta_{1,2})^2 + (3\eta_{2,1} - \eta_{0,3})^2 \\ \phi_4 &= (\eta_{3,0} + \eta_{1,2})^2 + (\eta_{2,1} + \eta_{0,3})^2 \\ \phi_5 &= (\eta_{3,0} - 3\eta_{1,2})(\eta_{3,0} + \eta_{1,2})[(\eta_{3,0} + \eta_{1,2})^2 - 3(\eta_{2,1} + \eta_{0,3})^2] \\ &\quad + (3\eta_{2,1} - \eta_{0,3})(\eta_{2,1} + \eta_{0,3})[3(\eta_{3,0} + \eta_{1,2})^2 - (\eta_{2,1} + \eta_{0,3})^2] \\ \phi_6 &= (\eta_{2,0} - \eta_{0,2})[(\eta_{3,0} + \eta_{1,2})^2 - (\eta_{2,1} + \eta_{0,3})^2] \\ &\quad + 4\eta_{1,1}(\eta_{3,0} + \eta_{1,2})(\eta_{2,1} + \eta_{0,3}) \\ \phi_7 &= (3\eta_{2,1} - \eta_{0,3})(\eta_{3,0} + \eta_{1,2})[(\eta_{3,0} + \eta_{1,2})^2 - 3(\eta_{2,1} + \eta_{0,3})^2] \\ &\quad + (3\eta_{1,2} - \eta_{3,0})(\eta_{2,1} + \eta_{0,3})[3(\eta_{3,0} + \eta_{1,2})^2 - (\eta_{2,1} + \eta_{0,3})^2] \end{aligned} \right\} \quad (9.24)$$

This set of normalized central moments is invariant to translation, rotation, and scale changes in an image.

In addition to the geometric moments, the circularity, aspect ratio, symmetry, and concavity are also used for segmentation and shape detection in images [42]. We exclude detailed discussions on these features here.

The problem with shape-based CBIR system development is that the shape features need very accurate segmentation of images to detect the object or region boundaries. Image segmentation is an active area of research and most of the segmentation algorithms are still computationally very expensive for on-line image segmentation. Robust and accurate segmentation of images still remains a challenge in computer vision. As a result, shape feature based image retrieval has been mainly limited to image databases where objects or regions are readily available.

9.3.5 Topology

A digital image can be represented by one or more topological properties [40], which typically represent the geometric shape of an image. The interesting characteristic of topological properties is that when changes are made to the image itself, such as stretching, deformation, rotation, scaling, translation, or other rubber-sheet transformations, these properties of the image do not change. As a result, topological properties can be quite useful in characterization of images and can be used as a signature of an image content to use in content based image retrieval.

One topological property of a digital image is known as *Euler number* [40]. The *Euler number* is usually computed in a binary image. However, it can be extended to characterize gray-tone images as well by defining a vector of Euler numbers of the binary planes of the gray-tone image. This has been called the *Euler Vector* [43]. The *Euler number* is defined as the difference between number of *connected components* and number of *holes* in a binary image. Hence if an image has C connected components and H number of holes, the *Euler number* E of the image can be defined as

$$E = C - H. \quad (9.25)$$

The *Euler number* of the binary image in Fig. 9.5(a) is 0 because the image has one connected component and one hole, whereas the Euler number of Fig. 9.5(b) is 2 because it has seven connected components and five holes. The binary image of letter B will have Euler number -1 because it has one connected component and two holes.

The Euler number remains invariant despite the transformation of the image due to translation, rotation, stretching, scaling, etc. For some classes of digital images, Euler numbers have strong discriminatory power. In other words, once the Euler number for a particular digital image is known, the digital image may readily be distinguished from other digital images in its class.

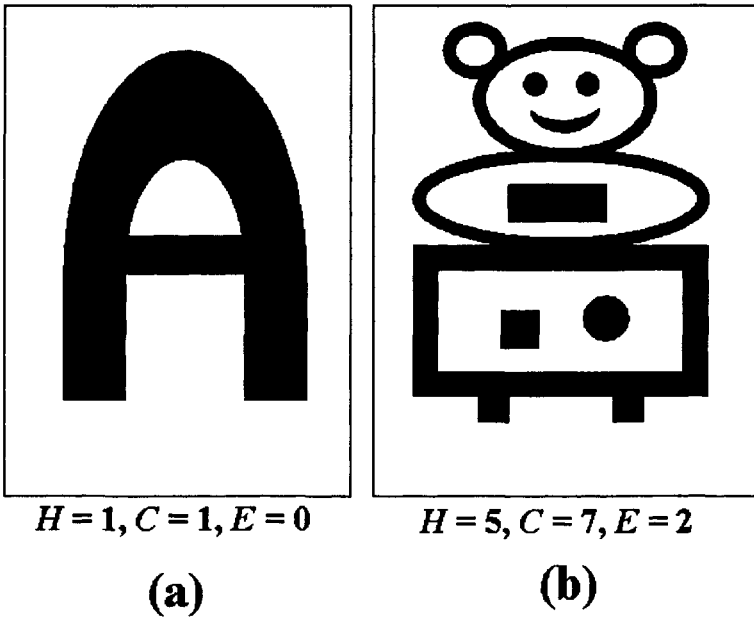


Fig. 9.5 Sample binary images with (a) Euler number 0 and (b) Euler number 2.

This implies that the Euler number may be used for more efficient searching or matching of digital images. For example, the Euler number may be used in medical diagnosis such as the detection of malaria infected cells. As the Euler number of an infected cell is often different from that of a normal cell, the malaria infected cells may be identified by calculating the Euler number of each cell image. The *Euler number* may also be used for image searching, such as in a database of logo images.

9.3.5.1 Euler Vector Bishnu et al. [43] extended the above usage of Euler number to gray-tone images, by defining a vector of Euler numbers. This vector is called the *Euler Vector*. Intensity value of each pixel in an 8-bit gray-tone image can be represented by an 8-bit binary vector b_i , $i = 0, 1, \dots, 7$, that is, $(b_7, b_6, b_5, b_4, b_3, b_2, b_1, b_0)$, where $b_i \in \{0, 1\}$. The i th bit plane is formed with b_i 's from all the pixels in the gray-tone image. To define Euler vector, we retain the first four most significant bit planes corresponding to (b_7, b_6, b_5, b_4) , because they contain most of the information of the image. This 4-bit binary vector is converted to its corresponding reflected gray code (g_7, g_6, g_5, g_4) , where $g_7 = b_7$, $g_6 = b_7 \otimes b_6$, $g_5 = b_6 \otimes b_5$, $g_4 = b_5 \otimes b_4$, and $\otimes$ denotes the binary XOR (modulo-2) operation.

9.3.5.2 Definition *Euler vector* of a gray-tone image is a 4-tuple (E_7, E_6, E_5, E_4) , where E_i is the *Euler number* of the bit-plane formed with reflected gray codes g_i of all the pixels in the image.

Gray code representation of intensity values offers a distinct advantage over standard binary representation in this particular context. *Euler vector* is found to be more insensitive to noise and other changes, when the gray code is used. This happens because two consecutive numbers have unit Hamming distance in gray-code representation and, for most of the cases, a small change in intensity values cannot affect all the 4 bit planes simultaneously in gray representation. The *Euler vector* can be used as a quick combinatorial signature of an image and has been used for image matching in content-based image retrieval [43, 44].

9.3.6 Multidimensional indexing

Multidimensional indexing is an important component of content-based image retrieval. Development of indexing techniques has been an active area in database management, computational geometry, and pattern recognition. However, the notion of indexing has subtle difference in different communities. The notion of indexing in multimedia data mining and content-based image retrieval is different than its notion in the traditional database management systems. In traditional database management system (particularly for relational database), the indexing refers to the access structure of the database files in terms organization of the records. Indexes are specified based on one or more attributes of the records in order to process queries based on those attributes. These record and file structures are well organized and supported by an access structure such as hashing, B-tree, etc. In the information retrieval community, the indexing mechanism is concerned with the process to assign terms (or phrases or keywords or descriptors) to a document so that the document can be retrieved based on these terms. The indexing in content-based image retrieval or mining multimedia data is similar to the notion adopted in the information retrieval. The primary concern of indexing is to assign a suitable description to the data in order to detect the information content of the data. As we explained in the previous sections, the descriptors of the multimedia data are extracted based on certain features or feature vector of the data. These content descriptors are then organized into a suitable access structure for retrieval.

The key issues in indexing for content-based image retrieval are (i) reduction of high dimensionality of the feature vectors, (ii) finding an efficient data structure for indexing, and (iii) finding suitable similarity measures.

In CBIR, the dimensionality of features vectors is normally very high. Today the dimensionality is typically of the order of 10^2 . With the exploration of multimedia content, this order may grow in future. Before indexing, it is very important to reduce the dimensionality of the feature vectors. The most

popular approach to reduce high dimensionality is application of the principal component analysis, based on singular decomposition of the feature matrices. The theory behind singular value decomposition of a matrix and generation of principal components for reduction of high dimensionality has been discussed in detail in Section 3.7. The technique has also been elaborated in Section 9.2.5 with regard to text mining. This can be applied to both text and image datatypes in order to reduce the high dimensionality of the feature vectors and hence simply the access structure for indexing the multimedia data.

After dimensionality reduction, it is very essential to select an appropriate multidimensional indexing data structure and algorithm to index the feature vectors. There are a number of approaches proposed in the literature. The popular amongst them are multidimensional binary search trees [45], R-Tree [46], variants of R-Tree such as R^* -Tree [47], SR-tree [48], SS-tree [49], Kd-tree [50], etc. All these indexing methods provide reasonable performance for dimensions up to around 20, and the performance deteriorates after that. Moreover, most of these tree-based indexing techniques have been designed for traditional database queries such as point queries and range queries, but not for similarity queries for multimedia data retrieval. There have been some limited efforts in this direction. Multimedia database indexing particularly suitable for data mining applications remain a challenge. So exploration of new efficient indexing schemes and their data structure will continue to be a challenge for the future.

After indexing of images in the image database, it is important to use a proper similarity measure for their retrieval from the database. Similarity measures based on statistical analysis have been dominant in CBIR. Distance measures such as Euclidean distance, Mahalanobis distance, Manhattan distance [6], and similar techniques have been used for similarity measures. Distance of histograms and histogram intersection methods have also been used for this purpose, particularly with color features.

Another aspect of indexing and searching is to have minimum disk latency while retrieving similar objects. Chang et al. proposed a clustering technique to cluster similar data on disk to achieve this goal, and they applied a hashing technique to index the clusters [51]. In spite of lots of development in this area, finding new and improved similarity measures still remains a topic of interest in computer science, statistics, and applied mathematics.

9.3.7 Results of a simple CBIR system

Saha et al. proposed a simple image matching scheme for an experimental content-based image retrieval system that uses moment, shape, and texture features extracted from the images [42]. The shape features have been extracted using moment computation (feature set A) of the regions, combined with circularity, aspect ratio, concavity, and symmetricity metrics (feature set B). The texture of the images have been generated using the energy,

Table 9.1 CBIR match performance (%)

Image group	A	B	A+B	A+B+C
AEROPLANE	71.3	69.68	73.39	73.23
CAR	78.90	74.11	78.30	82.56
FLOWER	51.22	38.64	43.64	43.48
ANIMAL	25.44	48.45	48.89	49.56
Overall	62.70	38.90	64.80	66.55

A: Moment invariants only.

B: Circularity, symmetry, aspect ratio and concavity.

C: Energy, entropy, contrast and homogeneity.

entropy, contrast, and homogeneity measure as image features (set C). We present here results of this system using around 290 samples taken from an image database containing several classes of images, including animals, cars, flowers, etc. Distance of the query image from the database images have been computed simply as $\sum_i |f_i - f'_i|$, where f_i and f'_i are the values of the i th feature of the database image and query image respectively. The top ten closest images have been taken as the query result, excluding the query image itself if it is present in the database.

Table 9.2 CBIR weighted match performance (%)

Image group	All features (No weight)	All features (With weight)
AEROPLANE	73.23	69.84
CAR	82.56	86.75
FLOWER	43.48	77.03
ANIMAL	49.56	72.44
Overall	66.55	77.79

Tables 9.1 and 9.2 provide the results, where each database image is used as a query image to find the top ten from the database. From Table 9.1, we observe that the overall performance can be improved by mixing the different feature sets for image query.

Further improvement can be achieved by assigning weights to each feature as follows:

$$\sum_i W_{gi} |f_i - f'_i|,$$

where W_{gi} is the weight of the i th feature for image group g and the database image belongs to image group g . Here W_{gi} is computed as $\frac{1}{1 + \sigma_{gi}}$, where σ_{gi} is the standard deviation of the i th feature value of the images in group g . Table 9.2 indicates that this improves the matching performance.

The summary of the results is presented in Fig. 9.6 for eight query images. In each row the first column indicates the query image. For all rows the five best matches are shown, with the query image (as all of them are present in the database) naturally coming first as the best match.

9.4 VIDEO MINING

Currently text-based search engines are commercially available, and they are predominant in the *World Wide Web* for search and retrieval of information. However, demand for search and mining multimedia data based on its content description is growing. Search and retrieval of contents is no longer restricted to traditional database retrieval applications. As an example, it is often required to find a video clip of a certain event in a television studio. In the future the content customers will demand to search and retrieve video clips based on content description in different forms. It is not difficult to imagine that one may want to mine and download the images or video clips containing the presence of Mother Teresa from the Internet or search and retrieve them from a video archival system. It is even possible to demand for retrieval of a video which contains a tune of a particular song.

In order to meet the demands for retrieval of audio-visual contents, there is a need of efficient solution to search, identify and filter various types of audio-visual content of interest to the user using non-text based technologies. Recognizing this demand, the MPEG (Moving Picture Expert Group) standard committee, under the auspices of the International Standard Organization, is engaged in a work item to define a standard for multimedia audio-visual content description interface [52]. JPEG2000 is the new standard for still picture compression and has been developed in such a way that metadata information can be stored in the file header for access and retrieval by users as well [53, 54]. There is a mode in JPEG2000 standard which particularly focuses on compressing moving pictures or video and its content description.

All these developments will influence effective mining of video data in the near future. Video mining is yet to take off as a mainstream active area of research and development by the data mining community. The development has so far been restricted to retrieval of video content only. However, there are ample opportunities that data mining principles can offer in conjunction with the video retrieval techniques, towards the successful development of video data mining. In order to influence the readers towards this direction, we present here a brief description of the MPEG-7 standard for multimedia con-

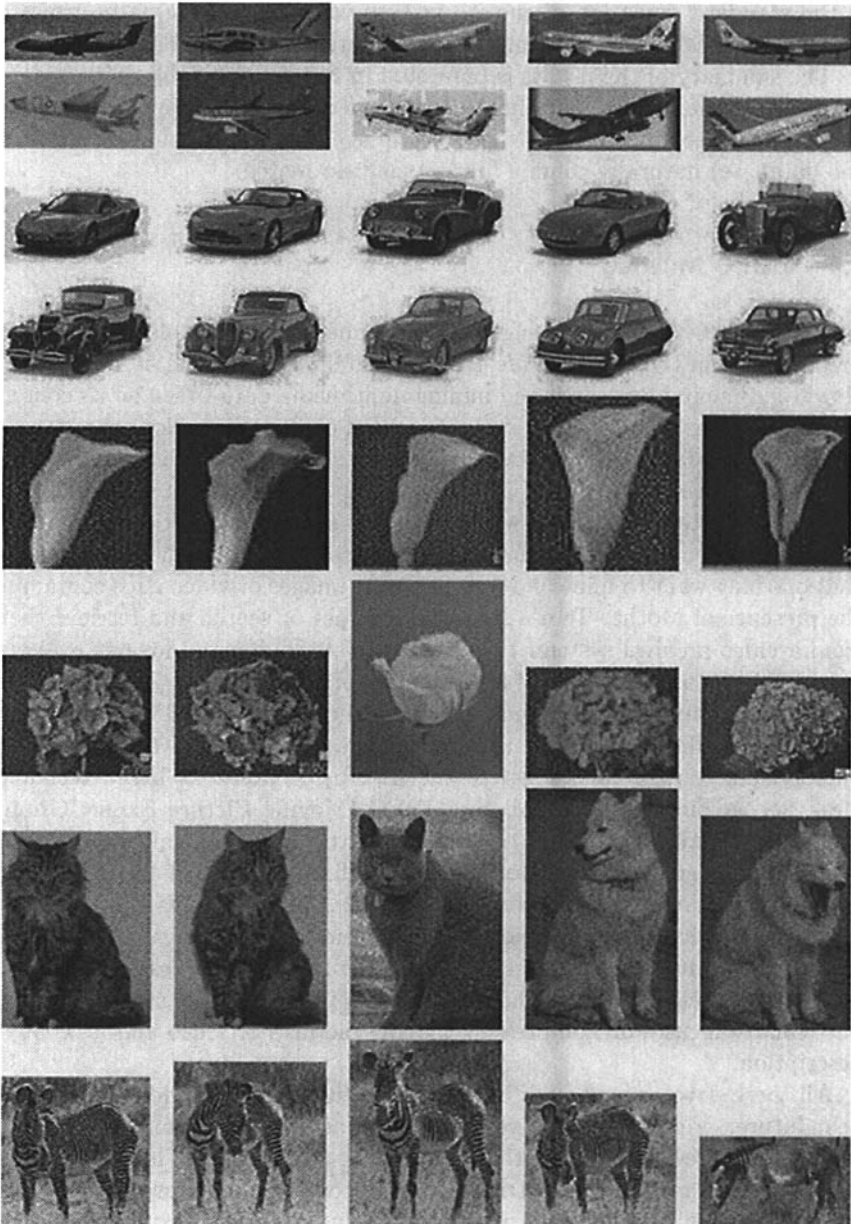


Fig. 9.6 Results of image matching in the CBIR system.

tent description interface and a general discussion on a possible video retrieval system. Application of data mining techniques on top of this development is left to the readers for their imagination.

9.4.1 MPEG-7: Multimedia content description interface

There is a wrong notion amongst many of us that MPEG-7 is another video compression standard. However, adoption of data compression principles are essential to define this standard for compact description of the multimedia contents. The goal of the MPEG-7 activity is not to define another video compression standard. The purpose of this standardization activity is to specify a standard set of descriptors that can be used to identify content and permit search for a particular multimedia content in a multimedia information system.

The goal of MPEG-7 is to define a standard set of descriptors that can be used to describe various types of multimedia information, as well as the relationship between the various descriptors and their structures. In principle these descriptors will not depend on the way the content is available, either on the form of storage or on their format. For example, video information can be encoded with any compression scheme (MPEG-1, MPEG-2, MPEG-4, JPEG, JPEG2000, or any other proprietary algorithm) or it can be uncompressed in its raw format without any encoding. It is even possible to generate a description of an analog video, or a picture drawn on paper. The audio-visual data description in MPEG-7 may include still pictures, video, graphics, audio, speech, three-dimensional models, and information about how these data elements are combined in the multimedia presentation.

The top level scope of the MPEG-7 standard is shown in Fig. 9.7. The block diagram emphasizes that only the audio-visual description of multimedia data is meant to be standardized. The standard neither defines nor deals with the mechanism for extraction of features from the multimedia data, nor is it connected with its encoding or search and retrieval mechanism. Accordingly, the MPEG committee was chartered to standardize the following elements as described in the requirement document for MPEG-7 work item [55]:

1. A set of *descriptors* - A *descriptor* is a representation of a feature, such as color, shape, texture, image topology, motion, or title, to name a few. The descriptor defines the syntax and semantics of representation of the feature.
2. A set of *description schemes* - A *description scheme* specifies the structure and semantics of the relationships between its components, which may be both *descriptors* and *description schemes* as well.
3. A set of *coding schemes* for the descriptors.
4. A *Description Definition Language* (DDL) to specify the description schemes and descriptors.

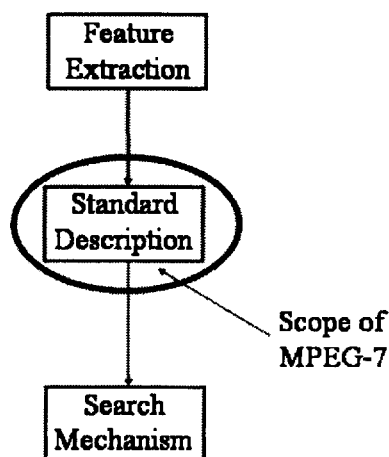


Fig. 9.7 Scope of the MPEG-7 standard.

The goal of MPEG-7 work item is a tall order. The final output of the standard is yet to be seen.

9.4.2 Content-based video retrieval system

Content-based image retrieval techniques can be extended, in principle, to video retrieval systems. However, this is not very straightforward because of the temporal relationship of video frames and their inherent structure. A video is not only a sequence of pictures, it represents the actions and events in a chronological order to convey a story and represent a moving visual information. In other words, one may argue that each video clip can be considered as a sequence of individual still pictures and each individual frame can then be indexed and stored using the traditional content-based image retrieval techniques. Again, this is not very practical given the number of frames in a good quality video clip of even a few minutes. This also does not capture the story structure, which is a collection of actions and events in time.

A generic video retrieval system, which can fit in MPEG-7 model, is shown in Fig. 9.8. As shown in this figure, a video clip is first temporarily segmented into video *shots*. A *shot* is a piece of a video clip (i.e., a group of frames or pictures), where the video content from one frame to the adjacent frames

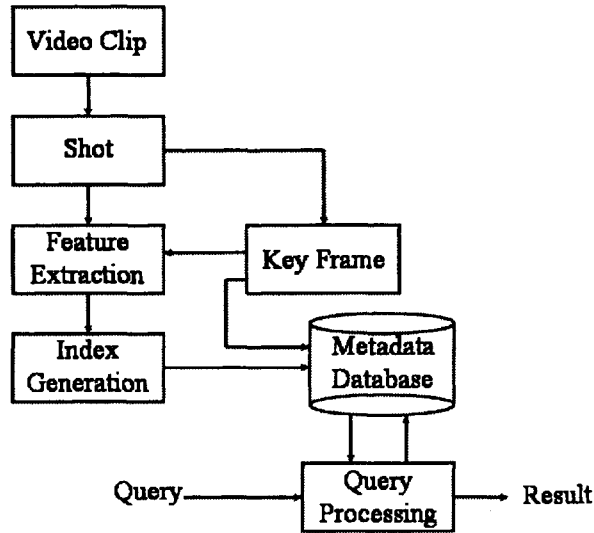


Fig. 9.8 Video retrieval systems architecture.

does not change abruptly. One of these frames in a shot is considered to be a *key frame*. This key frame is considered to be a representative for the picture content in that shot. Sequence of key frames can define the sequence of events happening in the video clip. This is very useful to identify the type and content of the video. Detection of shots and extraction of the key frames from video clips is a research challenge [56, 57].

As an individual still picture, each key frame can be segmented into a number of objects with desired meaningful image features such as shape, texture, color, topology, and many others as defined in Section 9.3 for content-based image retrieval. The semantic relationship between these individual features or feature vectors defines an object of interest to the user. We can apply similar feature extraction techniques and generate index structures for the key frames using the feature vectors, as described for content-based image retrieval. These indexed feature data and the corresponding key frames are stored in the *metadata* database. Collection of these metadata information describes the content of the video clip.

In a video retrieval system, the *query processing* depends upon the applications. It can be similar to content-based image retrieval, as described in Section 9.3, or it can be more complex depending upon the type of query processing. In its simplest form, a picture can be supplied to the video retrieval system as a *query* image. This query image is then matched with each and every key frame stored in the metadata database. The matching technique is

repeated, based on the extraction of different features from the query image, followed by the matching of these features with the stored features of the key frames in the database. Once a match of the query picture with a key frame is found, the piece of video can be identified by the index of the key frame along with the actual shot and the video clip.

Success of the MPEG-7 standardization process will have a significant impact in multimedia and video content retrieval and will influence the future development of multimedia data mining on a standardized platform based on its definition.

9.5 WEB MINING

World Wide Web is the largest database that ever existed. It serves as a widely distributed data or information repository system to warehouse different datatypes. Extraction of information and knowledge discovery from World Wide Web is an important area of study. The problem for knowledge extraction from World Wide Web is even more challenging because the data stored in the Web is very dynamic in nature, and constantly changing due to continuous update of the existing data or Web pages and addition of new information every moment. The complexity of Web pages is greater than that of traditional text document databases. Web data is often composed of different multimedia datatypes, mixed and sometimes interspersed together. As a result, Web mining will remain to be a challenge in the coming years.

Web mining [58] uses data mining techniques to automatically retrieve, extract, and evaluate information for knowledge discovery from the Web. Almost 99% of the data in the Web is useless for a particular user, and often it does not represent any relevant information that the user is looking for. Taking into account the huge amount of data storage and manipulation needed for (say) a simple query, the processing essentially requires adequate tools suitable for extracting only the relevant, sometimes hidden, knowledge as the final result of the problem under consideration.

The subtle difference between data mining and Web mining suggests the use of new or modified tools and algorithms for appropriate handling of the Internet. Web mining typically addresses semistructured or unstructured data, like Web and log files with mixed knowledge involving multimedia, flow data, etc., often represented by imprecise or incomplete information.

Web mining can be broadly categorized as

1. **Web Content Mining** of multimedia documents, involving text, hyper-text, image, audio, and video. This deals with the extraction of concept hierarchies or relations from the Web, along with their automatic categorization.

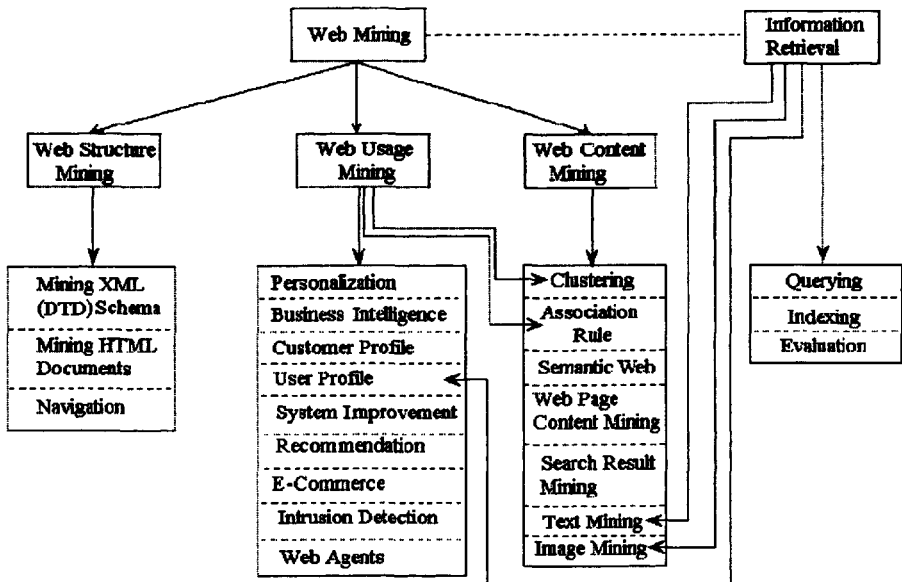


Fig. 9.9 A Web mining taxonomy.

2. Web Structure Mining of interdocument links, provided as a graph of links in a site or between sites. For example, in Google a page is important if important pages point to it.
3. Web Usage Mining of the data generated by the users' interactions with the Web, typically represented as Web server access logs, user profiles, user queries, and mouse-clicks. This includes trend analysis (of the Web dynamics information) and Web access association or sequential pattern analysis.

Fig. 9.9 provides a taxonomy for Web mining [59]. The different functions falling under the three major categories are highlighted. Web mining typically encompasses ways of improving search or customization by (i) learning about the users interests based on access patterns, (ii) providing users with pages, sites, and advertisements of interest, and (iii) using XML to improve search and information discovery on the Web.

9.5.1 Search engines

Web search engines are programs written to query and retrieve information stored in databases (fully structured), HTML pages (semistructured) and free text (unstructured) on the Web. The most popular indices have been created by Web robots such as AltaVista and WebCrawler, which scan millions of

Web documents and store an index of the words in the documents. There are over a dozen different indices currently in active use, each with a unique interface and a database covering a different fraction of the Web. Some of these indices have been discussed, with respect to text mining, in Section 9.2.

MetaCrawler presents the next level of the information food chain, by providing a single unified interface for Web document searching [60]. It submits the query to nine indices in parallel, and then it collates the results and prunes them. Thus instead of tackling the Web directly, MetaCrawler mines robot-created searchable indices.

Future resource discovery systems will make use of automatic text categorization technology to classify Web documents into categories. This technology could facilitate the automatic construction of Web directories, such as Yahoo, by discovering documents that fit Yahoo categories. Alternatively, the technology could be used to filter the results of queries to searchable indices. For example, in response to a query such as “find me product reviews of Encarta,” a discovery system could take documents containing the word “Encarta” found by querying searchable indices and then identify the subset that corresponds to product reviews.

It often happens that, upon matching the user’s query with locally cached documents or those in remote Web sites, the search engine comes up with too many poorly matched documents. Some Web search algorithms, such as Hyperlink Induced Topic Search (HITS) and PageRank, rank these matched documents according to some content criteria like structural information of hyperlinks. Thereby, the best fits of the ranked matches can be presented to the user.

There are two important types of Web pages, namely,

- *authorities*, providing the best source of information about a given topic, and
- *hubs*, providing a collection of links to authorities.

The hyperlink structure connects the different pages of the Web. A good hub is a page that points to several good authorities, while a corresponding good authority is a page that is pointed to by many good hubs. HITS works on this mutually reinforcing relationship by searching for good hubs and good authorities.

The HITS algorithm proceeds by

1. *sampling*, where it generates a focused collection of Web pages that are likely to be rich in relevant information, and
2. *weight propagation*, where an iterative procedure is pursued to determine an estimate of the hubs and authorities in order to obtain the most relevant hubs and authorities.

The Web is considered as a directed graph of pages. HITS initially constructs a subgraph based on the query terms, using a root set of pages generated by

some index-based search engine. The goal is to generate a subgraph rich in relevant and authoritative pages. The root set is expanded to a base set by including all pages that are linked to from here, but limited by a prespecified cutoff size.

Let each page p belonging to the base set V be associated with positive weights a_p and h_p , corresponding to its authority and hub weights. The objective of weight propagation (updating) is to extract good hubs and authorities from this base set V . The weights are normalized, so that their total sum remains bounded. Initially, all a and h values are set to uniform constants. If a page p is pointed to by many good hubs, its authority weight is increased by updating

$$a_p = \sum h_q, \quad (9.26)$$

$\forall q$, such that $q \rightarrow p$, that is, for all pages q that link to p . Similarly, if a page p points to many good authorities, its hub weight is increased as

$$h_p = \sum a_q, \quad (9.27)$$

$\forall q$, where $p \rightarrow q$. Note an interesting analogy with the weight updating in STIRR, as explained in Section 6.6. Typically, pages with large weights represent a dense pattern of linkages. HITS finally outputs a short list of pages with the largest hub weights and largest authority weights, corresponding to the given search topic.

While HITS basically works on the static information concerning a Web site structure, LOGSOM uses dynamic information about user behavior using the SOM (described in Section 2.2.3.3). LOGSOM can be used by, say, a company to analyze the navigation patterns of its Web pages by its potential customers, in order to make improved decisions.

Here the users' navigation patterns are mapped onto a two-dimensional map. It utilizes the excellent clustering and visualization abilities of the SOM. The system starts with a Web log file, consisting of the date, time, and address of the requested Web pages, along with the IP address of the user's machine. Clustering using c -means (Section 6.3.1.1) may initially be applied for reduction of the huge volume of Web data. LOGSOM causes the mapping of each Web page (URL) onto a SOM unit, based on its similarity with other URLs in terms of user navigation patterns. The similarity is measured in terms of the distance on the map.

9.5.2 Soft computing approaches

The huge amount of multivariate information offered by the Web has opened up new possibilities for many areas of research. Due to the involvement of human interaction in Web information (like text, image, sound, and linkages between them), new tools and methodologies need to be extended in order to deal with the incomplete or imprecise information. Soft computing promises to open up new avenues in this direction.

Considering the Web as a large distributed multimedia database, an extension of methodologies to deal with them and their mining algorithms can be considered under image mining. The huge volumes of compressed and uncompressed information stored in images, and their subjective evaluation by humans in interaction with the Web, is another focus of current data mining research requiring soft computing-based treatment [59].

There is a growing awareness that, in practice, it is easy to discover a huge amount of information from the Web, where most of these patterns are actually obvious, redundant, and useless or uninteresting to the user. To prevent the user from being overwhelmed by a large number of uninteresting patterns, soft computing techniques are needed to identify only the useful or interesting patterns and present them to the user. Different interestingness measures (of Sections 7.4 and 7.10) can be used for this purpose.

According to Zadeh [61], fuzzy logic may serve as the backbone of the semantic Web, an extension of the current Web in which information is given well-defined meaning. This enables computers and people to work in greater cooperation. The role of fuzzy sets in Web mining [62] holds promise in

- handling of fuzzy queries involving natural language and/or linguistic quantifiers like *almost*, *about*,
- evaluation of the matching of documents as a fuzzy similarity relation,
- deduction and summarization,
- document and user clustering, and
- information fusion in multimedia data.

User feedback is a process where the user expresses an opinion about the documents that the system has retrieved as an answer to a certain query. This user evaluation or opinion can be (a) used for training the classification system and (b) reflected in the corresponding user profile. Fuzzy linguistic modeling is useful in handling such preference relations. This information can also be utilized by (say) marketing experts to analyze user interests.

Ordinary end-users often face difficulties in formulating a precise representation of their information needs in a Boolean query. This affects the efficiency of the information retrieval process. Hence Web search engines require the use of fuzzy aggregation operators. These are especially suitable in flexible query answering and information retrieval. Fuzzy statistics may be used to emphasize the significance of the occurrence of a term in certain portions of a document, like URL, title, abstract, etc.

9.5.2.1 Clustering The importance of clustering to Web mining, specifically in the domains of Web Content and Web Usage mining, make Web clustering an interesting topic of research. This includes clustering of Web documents, snippets, and access logs. Usually the Web involves overlapping clusters. So

a crisp usage of metrics is better replaced by fuzzy sets that can reflect, in a more natural manner, the degree of belongingness (or membership) to a cluster.

The fuzzy *c*-medoids (of Section 6.5.1) is used to cluster relational data from Web documents and snippets in Ref. [63]. The algorithms are applied to a collection of 1042 abstracts from the Cambridge Scientific Abstract Web site, corresponding to ten topics. A preprocessing stage is used to filter and remove irrelevant words, in order to generate the input feature vector that is computed using the *inverse-document frequency* method. This 500-dimensional feature vector (keywords) is reduced using principal component analysis, resulting in a selection of 10 eigenvector values. The algorithms are also tested on a collection of snippets, corresponding to 200 Web documents collected by a search engine in response to a query. The results are extended in Ref. [64] using robust estimators, providing a computational complexity $O(n \log n)$ instead of $O(n^2)$. This is suitable for clustering noisy data that are characteristic of Web documents.

Relational alternating cluster estimation (RACE) [65] is used to handle non-numerical patterns in relational datasets. The authors consider two types of patterns related to the Web, namely, (i) document contents such as text parts of Web pages (Web content mining) and (ii) sequences of Web pages visited by users, such as Web logs (Web usage mining).

9.5.2.2 Association rule mining Association rules, in the context of Web mining, refer to the determination of (say) those URLs that tend to be requested together. This can be categorized under Web Usage and Web Content mining.

Web mining of inference amplification is made in Ref. [66], using inference logic from fuzzy cognitive maps in three phases. The first stage mines association rules with the *A priori* algorithm (Section 7.2.1), from a Web-log database. A corresponding fuzzy knowledge map involving causality (the cognitive map uses values between -1 and $+1$) is built in the second stage, incorporating both positive and negative rules. In the final stage, the system applies inference amplification in order to enrich the resulting Web mining association rules. Experiments are made using a Web-log file from a real online shop, specializing in computers and peripherals. The causal knowledge is represented as an adjacency matrix, including the connectivity $W^T = \{w_{ij} | w_{ij} \in \{-1, +1\}\}$. Here w_{ij} indicates the causal value (weight) of the arc from vertex/node C_i to C_j .

9.5.2.3 Web navigation Navigation is categorized as Web Structure mining. An optimization of the path for surfing the Web, given a target, is described in Ref. [67]. The connected Web sites are represented by a directed graph with source and destination nodes, and in addition to a links set along the path. The frequency of accessing various links (access rate) and the time taken to retrieve target pages (retrieval rate) are considered as the decisive factors. These are affected by criteria like the availability of channels, server capability,

and access time. The expected access rate and the required retrieval rate are expressed as fuzzy sets. Link weights are associated with appropriate linguistic values. A fuzzy opinion matrix expresses subjective estimation of users during their surfing of specified links on the path. The general evaluation of the link, with respect to interest rate of users, is provided as the intersection among the estimation made by the different users. The optimal path is computed as the minimum fuzzy distance estimated between a fuzzy *Hurwicz opinion set* (derived in terms of optimism–pessimism index) and the actual requirement set.

9.5.2.4 Web personalization The increasing popularity of the Internet and the exponential increase in the number of its users has led to the creation of new paradigms of knowledge discovery, like Web personalization, mining bookmarks, mining e-mail correspondences, recommendation systems, and so on. These are grouped as Web Usage mining.

Mining typical user profiles and URL associations from the vast amount of access logs is an important component of Web personalization, which deals with tailoring a user's interaction with the Web information space based on information about her/him. Nasraoui et al. [68] have defined a *user session* as a temporally compact sequence of Web accesses by a user and used a dissimilarity measure between two Web sessions to capture the organization of a Web site. Automatic discovery of user session profile is made using fuzzy Competitive Agglomeration for Relational Data (CARD) algorithm. Complex, non-Euclidean distance/similarity measures can be handled in this framework. Log files are collected from a real site in order to mine the user profiles. The experimental results "capture" the pattern for users in profiles like outside visitor, prospective student, those that attend lessons by the same professor, and so on. To discover user profiles from real log files, various fuzzy clustering methods have been applied.

9.5.2.5 Information retrieval Typically, four main constituents can be identified in the process of information retrieval from the Internet.

- Indexing: generation of document representation
- Querying: expression of user preferences through natural language or terms connected by logical operators
- Evaluation: performance of matching between user query and document representation
- User profile construction: storage of terms representing user preferences, especially to enhance the system retrieval in future accesses by the user

Due to the presence of *multimedia* information repositories consisting of mixed media data, the information retrieved can be text as well as image document or a mixture of both.

Fuzzy genes are used as intelligent agents for information retrieval from the Web [69]. User profiles are built from the user preferences, represented by chromosomes made up of a vector of fuzzy genes. Each chromosome is associated with a fitness corresponding to the system's belief in the hypothesis that the chromosome, as a query, represents the user's information needs. Every gene represents, by a fuzzy set, the number of occurrences that characterizes the documents considered relevant by the user. The fitness of the chromosome is adjusted based on the comparison between the user's evaluation of the retrieved documents and the score computed by the system. GAs are used to track the user's preferences and adapt the profile by incorporating her/his relevance feedback, while fuzzy sets handle the imprecision in the user's preferences and evaluation of the retrieved documents.

9.6 CONCLUSIONS AND DISCUSSION

In this chapter, we discussed how the data mining techniques can be applied in multimedia data. Multimedia data consist of a mixture of text, image, video, graphics, audio, speech, hypertext, markup languages, etc. Multimedia databases are large databases, and mining multimedia data is challenging because of the inherently unstructured characteristics of these data. Keeping this in view, we have presented here different aspects of multimedia data mining principles and applications.

Textual documents were considered in the framework of text mining. We have discussed how the semistructured text can be modeled and feature vectors can be generated from this model, so that they can be indexed and used for query and mining operations. We have presented issues and principles behind image mining, particularly in the light of content-based image retrieval. The underlying principles of content-based image retrieval, based on several image features such as color, shape, texture, etc., have been provided. We have introduced the purpose and target of MPEG-7 standard for Multimedia Content Description Interface. Video retrieval system was described, in the light of MPEG-7 as well as content-based image retrieval. Finally we discussed Web mining, and the role of soft computing in mining Web data.

REFERENCES

1. M. T. Maybury, ed., *Intelligent Multimedia Information Retrieval*. Menlo Park, CA: AAAI Press, 1997.
2. D. Hand, H. Mannila and P. Smyth, *Principles of Data Mining*. Cambridge, MA: The MIT Press, 2001.

3. M. Kantardzic, *Data Mining: Concepts, Models, Methods, and Algorithms*. Hoboken, NJ: John Wiley & Sons, 2002.
4. J. Han and M. Kamber, *Data Mining: Concepts and Techniques*. San Francisco, CA: Morgan Kaufmann Publishers, 2001.
5. G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing and Management*, vol. 24, pp. 513-523, 1988.
6. P. A. Devijver and J. Kittler, eds., *Pattern Recognition Theory and Applications*. Berlin: Springer-Verlag, 1987.
7. I. T. Jolliffe, *Principal Component Analysis*. New York: Springer-Verlag, 1986.
8. A. N. Netravali and B. Haskell, *Digital Pictures*. New York: Plenum Press, 1988.
9. S. Sakurai, Y. Ichimura, A. Suyama, and R. Orihara, "Inductive learning of a knowledge dictionary for a text mining system," in *Proceedings of 14th International Conference on Industrial and Engineering Applications of Artificial Intelligence and Expert Systems (IEA/AIE 2001)* (L. Monostori, J. Vancza, and M. Ali, eds.), vol. LNAI 2070, Berlin: Springer-Verlag, 2001, pp. 247-252.
10. H. M. Lee, S. K. Lin, and C. W. Huang, "Interactive query expansion based on fuzzy association thesaurus for web information retrieval," in *Proceedings of the 10th IEEE International Conference on Fuzzy Systems*, pp. 2:724-2:727, 2001.
11. A. Molinari and G. Pasi, "A fuzzy representation of HTML documents for information retrieval systems," in *Proceedings of the Fifth IEEE International Conference on Fuzzy Systems*, pp. 1:107-1:112, 1996.
12. D. H. Widyantoro and J. Yen, "Using fuzzy ontology for query refinement in a personalized abstract search engine," in *Proceedings of Joint 9th IFSA World Congress and 20th NAFIPS International Conference* (Vancouver, Canada), pp. 1:610-1:615, July 2001.
13. T. A. Runkler and J. C. Bezdek, "Relational clustering for the analysis of Internet newsgroups," in *Exploratory Data Analysis in Empirical Research, Proceedings of the 25th Annual Conference of the German Classification Society* (O. Opitz and M. Schwaiger, eds.), Studies in Classification, Data Analysis, and Knowledge Organization, Berlin: Springer-Verlag, 2002, pp. 291-299.
14. T. Kohonen, S. Kaski, K. Lagus, J. Salojarvi, J. Honkela, V. Paatero, and A. Saarela, "Self organization of a massive document collection," *IEEE Transactions on Neural Networks*, vol. 11, pp. 574-585, 2000.

15. S. Kaski, T. Honkela, K. Lagus, and T. Kohonen, "WEBSOM – Self-organizing maps of document collections," *Neurocomputing*, vol. 21, pp. 101–117, 1998.
16. H. Tamura and N. Yokoya, "Image database systems: A survey," *Pattern Recognition*, vol. 17, pp. 29–43, 1984.
17. S.-K. Chang and A. Hsu, "Image information systems: Where do we go from here?," *IEEE Transactions on Knowledge and Data Engineering*, vol. 4, pp. 431–442, 1992.
18. M. Flickner, H. Sawhney, W. Niblack, J. Ashley, Q. Huang, B. Dom, M. Gorkani, J. Hafine, D. Lee, D. Petkovic, D. Steele, and P. Yanker, "Query by image and video content: The QBIC system," *IEEE Computer Magazine*, vol. 28, pp. 23–32, 1995.
19. A. Pentland, R. Picard, and S. Sclaroff, "Photobook: Content-based manipulation of image databases," *International Journal of Computer Vision*, vol. 18, pp. 233–254, 1996.
20. A. Gupta and R. Jain, "Visual information retrieval," *Communications of the ACM*, vol. 40, pp. 71–79, 1997.
21. J. R. Smith and S.-F. Chang, "Visually searching the web for content," *IEEE Multimedia Magazine*, vol. 4, pp. 12–20, 1997.
22. W.-Y. Ma and B. S. Manjunath, "Netra: A toolbox for navigating large image databases," *Multimedia Systems*, vol. 7, pp. 184–198, 1999.
23. H. Samet, *Application of Spatial Data Structures: Computer Graphics, Image Processing and GIS*. Reading, MA: Addison-Wesley, 1990.
24. M. Swain and D. Ballard, "Color indexing," *International Journal of Computer Vision*, vol. 7, pp. 11–32, 1991.
25. M. Stricker and M. Orengo, "Similarity of color images," in *Proceedings of SPIE Storage and Retrieval for Image and Video Databases III*, vol. 2185 (San Jose, CA), pp. 381–392, February 1995.
26. H. J. Zhang and D. Shong, "A scheme for visual feature-based image indexing," in *Proceedings of SPIE Conference on Storage and Retrieval for Image and Video Databases III*, vol. 2185 (San Jose, CA), pp. 36–46, February 1995.
27. G. Pass and R. Zabith, "Histogram refinement for content-based image retrieval," in *Proceedings of of IEEE Workshop and Applications of Computer Vision*, pp. 96–102, 1996.
28. H. R. Kang, *Color technology for electronic imaging devices*. Bellingham, Washington, USA: SPIE Optical Engineering Press, 1997.

29. Q. Iqbal and J. K. Aggarwal, "Combining structure, color and texture for image retrieval: A performance evaluation," in *Proceedings of International Conference on Pattern Recognition* (Quebec City, Canada), 2002.
30. R. M. Haralick, J. Shanmugam, and I. Dinstein, "Texture feature for image classification," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 3, pp. 610-621, 1973.
31. H. Tamura, S. Mori, and T. Yamawaki, "Texture features corresponding to visual perception," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 8, pp. 460-473, 1978.
32. R. Jain, R. Kastuni, and B. G. Schunck, *Machine Vision*. New York: McGraw-Hill, 1995.
33. T. Chang and C. J. Kuo, "Texture analysis and classification with tree-structured wavelet transform," *IEEE Transactions on Image Processing*, vol. 2, pp. 429-441, 1993.
34. A. Laine and J. Fan, "Texture classification by wavelet packet signatures," *IEEE Transactions on Pattern Recognitions and Machine Intelligence*, vol. 15, pp. 1186-1191, 1993.
35. W.-Y. Ma and B. S. Manjunath, "A comparison of wavelet features for texture annotation," in *Proceedings of IEEE Int. Conf. on Image Processing II* (Washington, D.C.), pp. 256-259, 1995.
36. B. S. Manjunath and W.-Y. Ma, "Texture features for browsing and retrieval of image data," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 18, pp. 837-842, 1996.
37. E. Persoon and K. Fu, "Shape discrimination using Fourier descriptors," *IEEE Transactions on Systems, Man and Cybernetics*, vol. 7, pp. 170-179, 1977.
38. M. K. Hu, "Visual pattern recognition by moment invariants," in *Computer Methods in Image Analysis* (J. K. Aggarwal, R. O. Duda, and A. Rosenfeld, eds.), Los Angeles: IEEE Computer Society, 1977.
39. W. K. Pratt, *Digital Image Processing*. New York: Wiley-Interscience, 1991.
40. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*. Reading, MA: Addison-Wesley, 1993.
41. D. Luo, *Pattern Recognition and Image Processing*. New York: Horwood Publishing, 1998.
42. S. K. Saha, A. K. Das, and B. Chanda, "Content-based image retrieval: An experiment with shape and texture features," *Technical Report No.*

- BEC09-04-2002*, Department of Computer Science and Technology, Bengal Engineering College (Deemed University), India, May 2003.
43. A. Bishnu, B. B. Bhattacharya, M. K. Kundu, C. A. Murthy, and T. Acharya, "Euler vector: A combinatorial signature of gray-tone images," in *Proceedings of the IEEE Int. Conf. on Information Technology: Coding & Computing* (Las Vegas), pp. 121–126, April, 2002.
 44. A. Bishnu, P. Bhunre, B. B. Bhattacharya, M. K. Kundu, C. A. Murthy, and T. Acharya, "Content based image retrieval: Related issues with Euler Vector," in *Proceedings of the IEEE Int. Conf. on Image Processing* (Rochester, NY), pp. II:585–II:588, September, 2002.
 45. J. L. Bentley, "Multidimensional binary search trees used for associative searching," *Communications of the ACM*, vol. 18, pp. 509–517, 1975.
 46. A. Guttman, "R-trees: A dynamic index structure for spatial searching," in *Proceedings of ACM SIGMOD* (Boston), pp. 47–57, June, 1984.
 47. N. Beckmann, H.-P. Kriegel, R. Schneider, and B. Seeger, "The R^* -tree: An efficient and robust access method for points and rectangles," in *Proc. of ACM SIGMOD*, May, 1990.
 48. N. Katamaya and S. Satoh, "The SR-tree: An index structure for higher-dimensional nearest neighbor queries," in *Proc. of the Int. Conf. on Management of Data, ACM SIGMOD*, pp. 13–15, May, 1997.
 49. A. W. David and J. Ramesh, "Similarity indexing with SS-tree," in *Proc. of the 12th Int. Conf. on Data Engineering* (New Orleans), pp. 516–523, February, 2002.
 50. M. Overmars, M. D. Berg, M. V. Kreveld, and O. Schwarzkopf, *Computational Geometry: Algorithms and Applications*. New York: Springer-Verlag, 1997.
 51. E. Chang, C. Li, J. Z. Wang, P. Mork, and G. Wiederhold, "Searching near-replicas of images via clustering," in *Proc. of the SPIE Symposium of Voice, Video, and Data Communications* (Boston), pp. 281–292, September, 1999.
 52. MPEG-7: ISO/IEC JTC1/SC29/WG211, N2207, Context and objectives, March, 1998.
 53. "Information technology – JPEG2000 Image Coding System," Final Committee Draft Version 1.0 ISO/IEC JTC 1/SC 29/WG 1 N1646R, March 2000.
 54. D. S. Taubman and M. W. Marcellin, *JPEG2000: Image Compression Fundamentals, Standards and Practice*. Boston: Kluwer Academic Publishers, 2002.

55. MPEG-7: ISO/IEC JTC1/SC29/WG211, N2727, MPEG-7 requirements, MPEG Seoul Meeting, 1999.
56. B. C. O'Connor, "Selecting key frames of moving image documents: A digital environment for analysis and navigation," *Microcomputers for Information Management*, vol. 8, pp. 119–133, 1991.
57. A. Nagasaka and Y. Tanaka, "Automatic video indexing and full-search for video appearances," in *Visual Database Systems* (E. Knuth and I. M. Wegener, eds.), Elsevier Science Publishers, Vol. II, Amsterdam, pp. 113–127, 1992.
58. R. Kohavi, B. Masand, M. Spilipoulou, and J. Srivastava, "Web mining," *Data Mining and Knowledge Discovery*, vol. 6, pp. 5–8, 2002.
59. D. Arotaritei and S. Mitra, "Web mining: A survey in the fuzzy framework," *Fuzzy Sets and Systems*, 2003 (accepted).
60. O. Etzioni and M. Perkowitz, "Adaptive Web sites: An AI challenge," in *Proceedings of 15th National Conference on Artificial Intelligence* (Madison, WI), July 1998.
61. L. A. Zadeh, "A new direction in AI: Towards a computational theory of perceptions," *AI Magazine*, vol. 22, pp. 73–84, 2001.
62. F. Crestani and G. Pasi, eds., *Soft Computing in Information Retrieval: Techniques and Application*, vol. 50. Heidelberg: Physica-Verlag, 2000.
63. R. Krishnapuram, A. Joshi, and L. Yi, "A fuzzy relative of the k -medoids algorithm with application to document and snippet clustering," in *Proceedings of IEEE International Conference on Fuzzy Systems* (Korea), pp. 3:1281–3:1286, August, 1999.
64. R. Krishnapuram, A. Joshi, O. Nasraoui, and L. Yi, "Low complexity fuzzy relational clustering algorithms for Web mining," *IEEE Transactions on Fuzzy Systems*, vol. 9, pp. 595–607, 2001.
65. T. A. Runkler and J. Bezdek, "Web mining with relational clustering," *International Journal of Approximate Reasoning*, vol. 32, pp. 217–236, 2003.
66. K. C. Lee, J. S. Kim, N. H. Chung, and S. J. Kwon, "Fuzzy cognitive map approach to Web mining inference amplification," *Expert Systems with Applications*, vol. 22, pp. 197–211, 2002.
67. C. W. Chong, V. Ramachandran, and C. Eswaran, "Path optimization using fuzzy distance approach," in *Proceedings of 1999 IEEE International Fuzzy Systems Conference (FUZZ-IEEE 99)* (Seoul, Korea), pp. III:1771–III:1774, August 1999.

68. O. Nasraoui, H. Frigui, R. Krishnapuram, and A. Joshi, "Extracting web user profiles using relational competitive fuzzy clustering," *International Journal on Artificial Intelligence Tools*, vol. 9, 2000.
69. M. J. Martin-Bautista, H. L. Larsen, and M. A. Vila, "A fuzzy genetic algorithm approach to an adaptive information retrieval agent," *Journal of the American Society for Information Science*, vol. 50, pp. 760-771, 1999.

This page intentionally left blank

10

Bioinformatics: An Application

10.1 INTRODUCTION

Computational Molecular Biology [1] is an interdisciplinary subject involving fields as diverse as biology, computer science, information technology, mathematics, physics, statistics, and chemistry. Bioinformatics [2]–[4] concentrates on the information science related aspect of this. Since the inception of the Human Genome Project, Bioinformatics has drawn a lot of attention. Upon completion of this project, one needs to analyze and interpret the vast amount of data that are now available. This involves the decoding of 50,000–100,000 human genes. Data mining holds promise in this direction. High-dimensional clustering is a potential for grouping genes and proteins with many attributes. In addition to the combinatorial approach for solutions, there also exists scope for soft computing, especially for generating low-cost, low-precision good solutions.

Bioinformatics [2]–[4] can be defined as the application of computer technology to the management of biological information. It encompasses a study of the inherent genetic information, underlying molecular structure, resulting biochemical function, and the exhibited phenotypic symptoms. Here computers are used to gather, store, analyze, and integrate biological and genetic information, which can then be applied to gene-based drug discovery and development. Current research involves the design and implementation of programs and systems for the storage, management, and analysis of the vast amounts of DNA and protein sequence data. *Biological data mining* is an

emerging field of research and development for further progress in this direction [4].

The high-volume, data-driven nature of modern experimental biology has led to the creation of many databases that contain genomes, protein sequences, gene expression data, and other datatypes. Researchers often retrieve information from these databases based on one characteristic, such as amino acid sequence, gene annotation, or protein name. Answering queries involves some form of data analysis, such as statistical significance, clustering, or sequence homology search. The Basic Alignment Search Tool (BLAST) [5] is typically the first Bioinformatics tool a biologist uses when examining a new DNA or protein sequence. BLAST compares the new sequence to all sequences in the database to find the most similar known sequences. Advances related to recognizing protein interactions, improving homology search, and identifying cellular location, in the context of data mining, are discussed in Ref. [6]. Techniques from string matching with examples demonstrating how they can be applied in DNA search were presented earlier in Section 4.3 in Chapter 4.

Unlike a genome, which provides only static sequence information, microarray experiments produce gene expression patterns that provide dynamic information about cell function. This information is useful while investigating complex interactions within the cell. For example, data mining methods can ascertain and summarize the set of genes responding to a certain level of stress in an organism [3]. Microarray technologies have been utilized to evaluate the level of expression of thousands of genes in colon, breast and blood cancer treatment [4]. The sheer volume of the data makes it impossible to view a large microarray clustering result on a 2D/3D display. Efficient interactive visualization tools are needed to facilitate pattern extraction from microarray datasets. Gene expression data being typically high-dimensional, it requires appropriate data mining strategies like clustering and string matching for further analysis.

Recent years have seen (a) an explosion in the availability of structural information pertaining to drug targets and (b) the growth of computational chemistry and Bioinformatics methods to exploit these. Simultaneously, combinatorial chemistry and screening technologies have greatly advanced, analyzing large amounts of structure-based design inputs in order to discover small molecule leads and then optimizing their potency and *pharmacokinetic* properties to produce useful drugs.

Proteins constitute an important ingredient of living beings and are made up of a sequence of amino acids. There can be a large number of 3D states for a protein. The determination of an optimal conformation constitutes protein folding. This promises to provide enormous information on the presence of active sites and possible drug interaction. Incomplete folds can cause disabling diseases like Alzheimer's and "mad cow" syndrome. Misfolding also causes respiration and locomotion failures, as attempted in *biowarfare*, because protein functions cannot be fully carried out in that environment.

The folding of a protein is a highly complex process. There are nearly 100,000 proteins encoded in the human genome, and there are thought to be more than a thousand fundamentally distinct structural architectures into which folded proteins can be classified. To establish how a newly formed *polypeptide* sequence of amino acids finds its way to its correct fold, rather than the countless alternatives, is one of the greatest challenges in modern structural biology. To respond to this challenge requires interdisciplinary research involving the most advanced techniques available to the chemists and structural biologists. A good survey on molecular modeling of proteins and prediction of their structures is provided in Ref. [7].

Soft computing tools like neural networks and genetic algorithms have been used for analyzing the different protein structures and folds [8]–[11]. Since the work entails processing huge amounts of incomplete or ambiguous data, the learning ability of neural networks, uncertainty handling capacity of fuzzy sets, and the searching potential of genetic algorithms are utilized in this direction.

In this chapter we focus on the Information Science-related aspects of Bioinformatics, particularly microarray data, gene expression profiles, protein structure prediction and folding, including soft computing-based methods. Section 10.2 provides an introduction to the basics of protein structure and microarray data. The Information Science-related aspects of Bioinformatics are described in Section 10.3, with appropriate emphasis on data mining. High-dimensional clustering of microarray data to analyze gene expressions is discussed in Section 10.4. Mining of association rules in Bioinformatics is described in Section 10.5. The role of soft computing is highlighted in Section 10.6. Finally, Section 10.7 concludes the chapter.

10.2 PRELIMINARIES FROM BIOLOGY

Proteins are built up by polypeptide chains of amino acids, which consist of the deoxyribonucleic acid (DNA) as the building block. In this section we provide a basic understanding of the protein structure and DNA microarray data.

10.2.1 Deoxyribonucleic acid

The nucleus of a cell contains chromosomes that are made up of the double helical DNA molecules. The DNA consists of two strands of phosphate and deoxyribose sugar molecules, joined by covalent bonds. To each sugar molecule is attached one of the four nitrogenous bases, namely, adenine (*A*), cytosine (*C*), guanine (*G*), thymine (*T*). Note that uracil (*U*) exists in place of *T* in ribose sugar. Bases between the strand pairs are attached by hydrogen bonds, such that either *AT* or *GC* comes together. The *monomeric* units of

nucleic acids, within the DNA, are called nucleotides. A nucleotide is a combination of a phosphate, a sugar, and a purine or a pyrimidine base, where a purine (or pyrimidine) consists of *A, G* (or *C, T*).

There are approximately 3 billion base pairs in our body. The DNA is a string of *A, C, G, T*, and the whole stretch of DNA is called the genome of the organism. Genes are coded in fragments of DNA (either strand) that are dispersed in the genome. Each gene contains information to produce a single protein. The DNA codes for protein (unbranched organic polymers), and the enzymes and hormones are also proteins. Understanding what parts of the genome encodes which genes is the topic of the Human Genome project.

DNA in the human genome is arranged into 24 distinct chromosomes, that are physically separate molecules ranging in length from about 50 million to 250 million base pairs. Each chromosome contains many genes, the basic physical and functional units of heredity. However, genes comprise only about 2% of the human genome; the remainder consists of noncoding regions, whose functions may include providing chromosomal structural integrity and regulating where, when, and in what quantity proteins are made.

The DNA is *transcribed* to produce messenger (*m*)-RNA, which is then *translated* to produce protein. The *m*-RNA is single-stranded and has a ribose sugar molecule. There exist '*Promoter*' and '*Termination*' sites in a gene, responsible for the initiation and termination of transcription. Translation consists of mapping from triplets (codons) of four bases to the 20 amino acids building block of proteins.

10.2.2 Amino acids

An amino acid is an organic molecule consisting of an amine (NH) and a carboxylic (CO) acid group (backbone), together with a side chain (hydrogen atom and residue R) that differentiates between them. More than one triplet of DNA can map to the same amino acid, but the same triplet cannot map to two different amino acids. A sequence of amino acids, held together by peptide bonds forming a polypeptide chain, endow a protein with its 3D structure. In humans a gene consists of exons that get translated into an amino acid sequence, separated by introns (that are not translated). The carboxyl and amino groups of a pair of amino acids react through hydrolysis (removal of water molecule) to link and form a peptide bond. Similar reactions occur along the chain to form a protein molecule.

Proteins are polypeptides, formed within cells as a linear chain of amino acids. The length of a protein can vary from 10s to 1000s of amino acid monomers. Each monomer or amino acid, itself, consists of 10 or more atoms, making the macromolecule extremely large and complex. It is therefore not surprising that the many interactions cause the chain to fold and twist. Proteins in the living world reach a stable, unique, 3D structure, and this is responsible for a biological function.

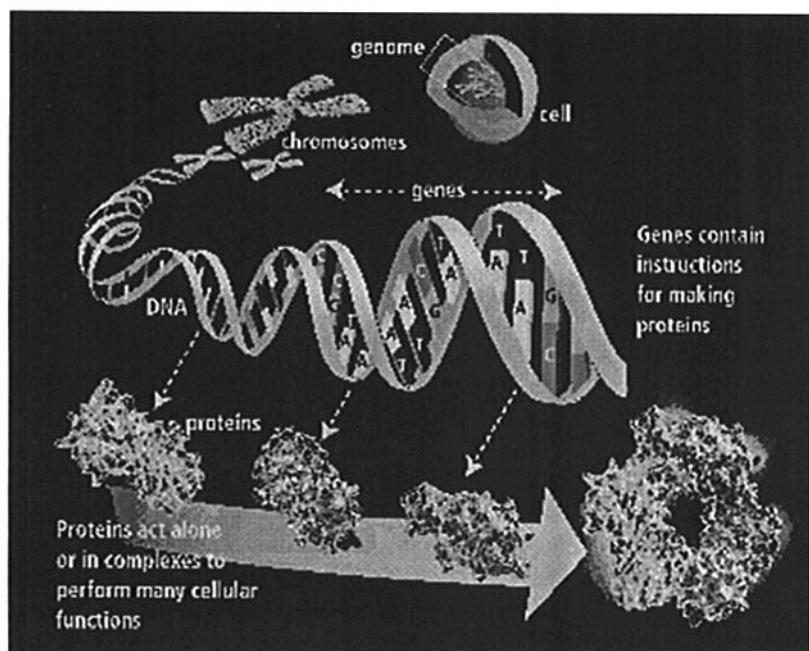


Fig. 10.1 Molecular biology

10.2.3 Proteins

Although genes get a lot of attention, the proteins are the ones that perform most life functions and even make up the majority of cellular structures in our body. Proteins are large, complex molecules made up of smaller subunits called amino acids. Chemical properties that distinguish the 20 different amino acids cause the protein chains to fold up into specific three-dimensional structures that define their particular functions in the cell. Figure 10.1 depicts the whole system.

The constellation of all proteins in a cell is called its *proteome*. Unlike the relatively unchanging genome, the dynamic proteome changes from minute to minute in response to tens of thousands of intra- and extracellular environmental signals. A protein's chemistry and behavior are specified by the gene sequence and by the number and identities of other proteins made in the same cell at the same time and with which it associates and reacts. Studies to explore protein structure and activities, known as proteomics, will be the focus of much research for decades to come and will help elucidate the molecular basis of health and diseases. Proteins are the primary carriers of signals in an organism. Figure 10.2 summarizes how information gets transferred to proteins.

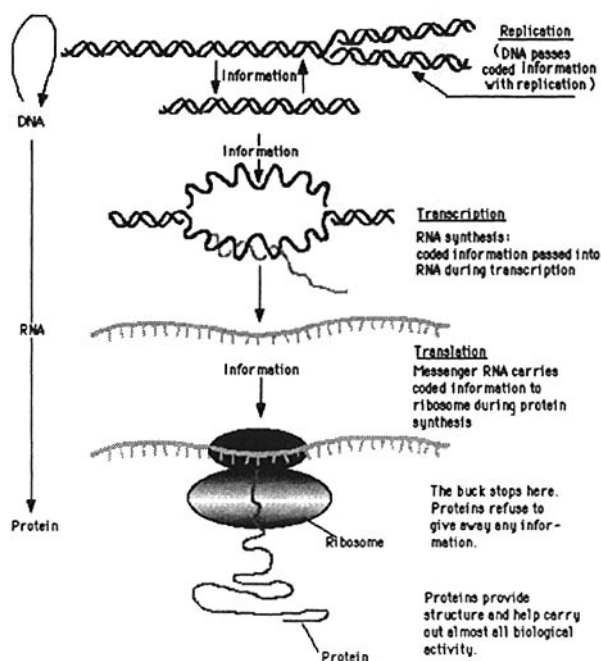


Fig. 10.2 Central dogma of molecular biology.

Proteins are involved in virtually every biological process in a living system. They are synthesized on ribosomes as linear chains of, typically, several hundred amino acids in a specific order from information encoded within the cellular DNA. In order to function, these chains must fold into the unique native three-dimensional structures that are characteristic of the individual proteins. In a cell, this takes place in a complex highly crowded molecular environment. There are several families of cellular proteins whose job is to catalyze the folding process of the other proteins that are required by the living organism.

In a cellular environment, molecular *chaperones* help to protect the incompletely folded polypeptide chains from aggregating. Even after the folding process is complete, however, a protein can subsequently experience conditions under which it unfolds, at least partially, and then again becomes prone to aggregation. It is becoming clear that the failure of proteins to fold correctly or to remain folded under all appropriate physiological conditions can give rise to a wide range of pathological conditions. Diseases associated with misfolding now include genetic, sporadic, and even infectious ailments.

10.2.4 Microarray and gene expression

DNA microarrays (gene arrays or gene chips) [3] usually consist of thin glass or nylon substrates containing specific DNA gene samples spotted in an array by a robotic printing device. Researchers spread fluorescently labeled *m*-RNA from an experimental condition onto the DNA gene samples in the array. This *m*-RNA binds (hybridizes) strongly with some DNA gene samples and weakly with others, depending on the inherent double helical characteristics. A laser scans the array and sensors to detect the fluorescence levels (using red and green dyes), indicating the strength with which the sample expresses each gene. The logarithmic ratio between the two intensities of each dye is used as the gene expression data. The relative abundance of the spotted DNA sequences in a pair of DNA or RNA samples is assessed by evaluating the differential hybridization of the two samples to the sequences on the array.

Gene expression levels can be determined for samples taken (i) at multiple time instants of a biological process (different phases of cell division) or (ii) under various conditions (tumor samples with different histopathological diagnosis). Each gene corresponds to a high-dimensional row vector of its expression profile.

10.3 INFORMATION SCIENCE ASPECTS

Bioinformatics requires handling large volumes of data, involving natural interaction with information science. One needs to consider the problems of data storage, analysis, and retrieval, along with the computational modeling and simulation. Data mining, image processing, and visualization are the other important constituents required to help the user with a visual environment that facilitates high-dimensional data dependent on many parameters. Information theory based lossless data compression techniques can play a vital role in management of this high volume of data.

Genomes cannot be sequenced all at once. This involves around 40K–100K base pairs, broken into 400–2000 small fragments of about 1000 base pairs. It is an NP-hard combinatorial problem, involving factors such as (i) unknown relative position and orientation of fragments within original DNA sequence and (ii) noisy and partial information about sequence of each fragment. The different aspects currently handled by information scientists include

- Protein folding problem
- Genomic sequence analysis
- Homology search and multiple alignment
- Searching biological databases.

In this section we concentrate on some of these problems in detail. The data mining aspects are appropriately highlighted.

10.3.1 Protein folding

Given the *primary* structure of a protein, in terms of a linear sequence of amino acids, the aim is to predict its 3D structure. The 3D structure is related to the functionality, say the diversity or specificity involved. It is useful in designing efficient drugs to cure diseases. However, considering all interactions governed by the laws of physics and chemistry to predict 3D positions of different atoms in the protein molecule, a reasonably fast computer would need one day to simulate 1 ns of folding. This gives an idea about the complexity of the problem under consideration.

Protein folding is a thermodynamically determined problem. It is also a reaction involving other interacting amino acids and water molecules. The *secondary* structure can involve an α helix (with the CO group of the i th residue hydrogen (H)-bonded to the NH group of the $(i + 4)$ th one) or a β sheet (corrugated or hairpin structure) formed by the H-bonds between the amino acids. The parts of the protein that are not characterized by any regular H-bonding patterns are called random coils. Neural nets (MLP) have been used to predict the secondary structure of proteins [8][10]. These are described in further detail in Section 10.6.

The *tertiary* structure refers to the 3D conformation of the protein. Since the search space is large, there can exist many possible 3D structural conformations for a short primary sequence. The objective is to determine the minimum energy state for a polypeptide chain folding. GAs have been used for the prediction of tertiary structures [11]. This is discussed in Section 10.6. There also exists a *quaternary* structure pertaining to certain proteins that contain more than one polypeptide chain.

The process of protein folding involves minimization of an energy function, that is expressed in terms of several variables like bond lengths, bond angles and torsional angles. The major factors affecting folding include (i) hydrogen bonding, (ii) hydrophobic effect, (iii) electrostatic interactions, (iv) Van der Waals forces, and (v) conformational entropy.

10.3.1.1 Hydrogen bonding Polypeptides contain numerous proton donors and acceptors, both in their backbone and in the R-groups of the amino acids. The environment in which proteins are found also contains ample H-bond donors and acceptors of the water molecule. H-bonding, therefore, occurs not only within and between polypeptide chains but also with the surrounding aqueous medium.

10.3.1.2 Hydrophobic forces Proteins are composed of amino acids that contain either hydrophilic or hydrophobic R-groups. It is the nature of the interaction of the different R-groups with the aqueous environment that plays a major role in shaping protein structure. The spontaneous folded state of globular proteins is a reflection of a balance between (a) the opposing energetics of H-bonding between hydrophilic R-groups and the aqueous environment

and (b) the repulsion from the aqueous environment by the hydrophobic R-groups. The hydrophobicity of certain amino acid R-groups tends to drive them away from the exterior of proteins into the interior. This driving force restricts the available conformations into which a protein may fold.

10.3.1.3 Electrostatic forces Electrostatic forces are mainly of three types, namely, charge-charge, charge-dipole and dipole-dipole. Typical charge-charge interactions, which favor protein folding, are those between oppositely charged R-groups. A substantial component of the energy involved in protein folding is from charge-dipole interactions. This refers to the interaction of ionized R-groups of amino acids with the dipole of the water molecule. The slight dipole moment that exists in the polar R-groups of amino acid also influences their interaction with water. It is, therefore, understandable that the majority of the amino acids found on the exterior surfaces of globular proteins contain charged or polar R-groups.

10.3.1.4 Van der Waals forces There are both attractive and repulsive Van der Waals forces that control protein folding. Attractive Van der Waals forces involve the interactions among induced dipoles, which arise from fluctuations in the charge densities occurring between adjacent uncharged nonbonded atoms. Repulsive Van der Waals forces involve the interactions that occur when uncharged nonbonded atoms come very close together but do not induce dipoles. The repulsion is the result of the electron-electron interaction that occurs as two clouds of electrons begin to overlap. Although Van der Waals forces are extremely weak, relative to other forces governing a stable conformation, it is the huge number of such interactions that occur in large protein molecules that makes them significant to the folding of proteins.

10.3.2 Protein structure modeling

There are three broad approaches to predicting the structure of an unknown protein, based on its similarity to known protein structures. These are

- **Comparative or homology modeling:** Predicts protein structure based on the strength of a protein sequence similarity to known structures. This is also termed template detection, and it is generally undertaken in case of greater than 30% similarity with an available template structure.
- **Protein threading:** Given a sequence of proteins with unknown structure and a database of known folds, it finds the most plausible fold after evaluating the quality or stability of the arrangement. This approach is typically undertaken in case of around 20–30% similarity.
- **Ab initio:** Calculates coordinates of protein sequences without reference to existing protein structures. This technique is followed when the sim-

ilarity is even lower, and hence the former two approaches cannot be applied.

Detecting repeatedly occurring 3D structures in molecules can help biologists to understand their functions. This is particularly useful in modeling the tertiary structures of proteins. Wang et al. [12] have approached the problem by finding *patterns* in 3D graphs. Each node in the graph is an undecomposable or atomic unit and has a label, with edges linking them. Patterns are defined as *rigid substructures* that may occur in a graph after allowing for an arbitrary number of whole-structure rotations and translations, as well as a small number of user-specified edit operations (namely, relabeling, deleting, inserting) either in the patterns or in the graph. A geometric hashing technique is used to hash node-triplets of the graphs into a 3D table and compress their label-triplets. The algorithm proceeds by finding candidate patterns from the graphs and calculating their occurrence numbers to determine which of them satisfy the user-specified requirements. Applications are made in scientific data mining by (i) locating frequently occurring *motifs* in two families of proteins and applying the discovered patterns to classify large 3D protein structures and (ii) clustering chemical compounds based on the 3D patterns occurring in them. The performance of the algorithm is good in terms of high recall and precision rates, both for classification and clustering.

10.3.3 Genomic sequence analysis

Gene hunting is a complex task. This is because coding regions account for only 3% of the human genome; and of the 694,000 sequences available in public databases, only around 2000 unique protein structures have been found. Experimental structure elucidation by X-ray crystallography and/or NMR is very slow. Data mining can be usefully applied to deal with such large volumes of data. Correctly predicting the position of genes in the DNA and knowing their encoded 3D protein structure will be helpful in drug and vaccine design and individual characteristics determination. GENSCAN [13] has been designed for this purpose using the Hidden Markov Model.

10.3.4 Homology search

Proteins in different organisms are related to one another by evolution from a common ancestor. Those proteins related with respect to the common ancestor are called *homologues*. This relationship can be recognized by multiple sequence comparisons. A similar primary structure leads to a similar 3D structure, resulting in a similar functionality of the proteins. It is even more challenging to find similarity between two distant homologues. The problem of finding closely related homologues has been solved to some extent by adopting the techniques in string matching for string alignment and similarity matching algorithms. As a matter of fact, string matching techniques have been widely

used DNA sequencing and homology search in Bioinformatics. We have described the foundation of string matching algorithms and some of the classical algorithms for string matching including k -approximate matching of strings in Chapter 4. The principles behind the k approximate string matching, edit distance between two strings, and pattern search with k -mismatches can be adopted in the development of homology search as well as biological database search.

The dynamic programming approach has been adopted to find DNA sequence alignment. The principle is similar to the one described in Section 4.4. However, the traditional dynamic programming method for local alignment is too slow. Fast Alignment (FASTA) [14] and Basic Local Alignment Search Tool (BLAST) [5] are often found to be more efficient.

10.3.4.1 Dynamic programming for sequence alignment Dynamic programming is a very powerful mathematical and algorithmic tool and has been used in many real-life problems. We have shown how this tool has been used in approximate string matching in Chapter 4. Here we use the dynamic programming approach for solving sequence alignment of two DNA strings.

As we know, a string is defined as a sequence of characters or symbols from an alphabet. Let us consider two strings $S_1 = S_1[1 \cdots m]$ and $S_2 = S_2[1 \cdots n]$ of length m and n respectively. We further assume that these strings do not have any blank character to begin with. Let us denote the blank character as ‘_’. A string S can be transformed to a string $\bar{S}$ by inserting one or more blank characters in any position in the string S . For example, string $S = \text{‘two books’}$ of length 8 can be transformed to $\bar{S} = \text{‘two_book_s’}$ of length 10.

An alignment of S_1 and S_2 is the choice of inserting blank characters in S_1 or S_2 or both, so that there is no k for which both $\bar{S}_1[k]$ and $\bar{S}_2[k]$ are blanks. For example, alignment of $S_1 = \text{‘mood’}$ and $S_2 = \text{‘iglou’}$ may be $\bar{S}_1 = \text{‘_mood’}$ and $\bar{S}_2 = \text{‘iglou_’}$.

We can assign a scoring factor for the above alignment [15]. Although there are a number of ways to assign scoring factor, the most common scoring factor used by many algorithms is the linear additive scoring scheme. The score $\omega(a, b)$ maps two characters or symbols a and b from an alphabet (including a blank) to real numbers, and $\omega(a, b) = \omega(b, a)$. The score Ω of the alignment of two strings is calculated by accumulating some matching points in each character location. Let us assume that for each match one gets 2 points, 0 points for each mismatch, and -1 for each blank. Then the score of alignment of the above example will be

$$\begin{aligned}\Omega(\bar{S}_1, \bar{S}_2) &= \omega(_, i) + \omega(m, g) + \omega(_, l) + \omega(o, o) + \omega(o, o) + \omega(d, _) \\ &= (-1) + 0 + (-1) + 2 + 2 + (-1) = 1.\end{aligned}$$

A high score obviously reflects a good alignment between two strings, and hence intuitively may detect the organisms with close evolutionary relationships. Based on this assumption, we can now formulate the dynamic programming problem as follows.

Given two strings S_1 and S_2 and a scoring scheme, an alignment of S_1 and S_2 is optimal if there is no other alignment with a higher score. The highest possible score is called the similarity of S_1 and S_2 , denoted by $\text{Sim}(S_1, S_2)$. Let

$$f(i, j) = \text{sim}(S_1[1 \cdots i], S_2[1 \cdots j]),$$

where $i = 1$ to $|S_1|$ and $j = 1$ to $|S_2|$, and then

$$\text{sim}(S_1, S_2) = f(|S_1|, |S_2|).$$

The key idea behind the solution is that for $i, j > 1$, if we know the values of $f(i-1, j)$, $f(i, j-1)$, and $f(i-1, j-1)$, then we can easily compute, by a recursive step [16],

$$f(i, j) = \max \left\{ \begin{array}{l} f(i-1, j-1) + \omega(S_1[i], S_2[j]), \\ f(i-1, j) + \omega(S_1[i], -), \\ f(i, j-1) + \omega(-, S_2[j]) \end{array} \right\} \quad (10.1)$$

The first element of the maximum occurs when $S_1[i]$ and $S_2[j]$ are both non-blanks, the second element occurs when $S_1[i]$ is non-blank but the character following $S_2[j]$ is blank, and the third element in the maximum occurs when the character following $S_1[i]$ is a blank and $S_2[j]$ is a non-blank. This two-dimensional recursion for $f(i, j)$ can be computed in a tabular fashion, similar to that shown in Section 4.4 for k -approximate string matching, with the two strings to be aligned written along the row and column of the table [15]. We avoid the details here.

When the alignments are computed over the whole of strings S_1 and the whole string S_2 , the problem is called a global alignment problem. The local alignments of S_1 and S_2 deals with alignments of a substring of S_1 and a substring of S_2 . Using linear scoring schemes, $\text{Sim}(S_1, S_2)$ can be computed in time $O(|S_1| * |S_2|)$ by the method of dynamic programming. However, it is still slow for local alignment. 'Fast Alignment' (FASTA) [14] and 'Basic Local Alignment Search Tool' (BLAST) [5] are often more efficient. We describe below the BLAST algorithm.

10.3.4.2 The BLAST algorithm BLAST is a heuristic method to find the highest locally optimal alignments between a query sequence and a database [5]. It does not allow the presence of gaps in between. BLAST improves the overall speed of search while retaining good sensitivity (important as the databases continue to grow), by breaking the query and database sequences into fragments (words) and initially seeking matches between these fragments. The idea behind the BLAST algorithm can be best illustrated by an example. If the query is 'MOONLIGHT', the search is performed using a small stretch of say 4 letters. For example 'MOON' closely matches with other words of same length like 'SOON', 'MOOD', 'GOON', etc. These words are then extended to locate 'MOONLIGHT' in the database. The three main steps of

the algorithm are as follows.

Step (1)(a): For a query of length L , find all overlapping words of length W , where $W = 3$ for proteins and $W = 3$ for DNA. For example, for the query *ABCDEFGH* with $W = 3$ and $L = 8$, there will be $L - W + 1 = 6$ overlapping words of length 3, namely, *ABC*, *BCD*, *CDE*, *DEF*, *EFG*, *FGH*.

Step (1)(b): For each of these 6 words from the query, we find a list of (close relatives) words that will score at least a threshold T when using a suitable substitution matrix (e.g., PAM or BLOSUM matrices that determine the similarity between amino acids). For example, *ADC* has close words *ADD*, *AEC*, *EDC*, which score above T . The threshold T is chosen by the user. A similar decomposition is done for the database once and for all. In the database all the sequences can be thought of as being combined into one big sequence, which is then divided into words of length W .

Step (2): The word list generated from the query is compared with the database list to identify the exact matches. This comparison can be accelerated by using a hash table.

Step (3): Word hits are extended in either direction to generate an alignment with a score exceeding the threshold S (High-score Segment Pair, HSP). In the example above, *MOONLIGHT* is an HSP.

After the alignment is done, the match must be statistically checked to see if it occurred purely by chance or if there is indeed some homology between the sequences. Here T dictates the speed and sensitivity of the search. A low value of T reduces the possibility of missing HSPs with the required S score. However, lower T values also increase the size of the hit list generated, and hence the execution time and memory required. BLAST is unlikely to be as sensitive for all protein searches as a full dynamic programming algorithm. Nevertheless, the underlying statistics provide a direct estimate of the significance of any match found.

Some of the efficient extensions to BLAST include

- Gapped BLAST [17]: Allows insertions and deletions to be introduced into alignments.
- Position-specific iterative BLAST (Psi-BLAST) [17]: Includes gaps, as well as searches for distant homologies by building a profile (general characteristics).
- Pattern Hit Initiated BLAST (Phi-BLAST) [18]: Amino acid pattern present in a query is defined and searched.

With the availability of nucleic acid sequence information from viruses and bacteria, efforts have been directed towards generation of highly specific

pathogen signatures [19], including foot-and-mouth and human immunodeficiency viruses. BLAST was used to compare the entire genome of the target pathogen with all the microbial genomes in Genbank. Several gigabytes of raw BLAST output were parsed, and all exact match regions of the pathogen genome were *masked out* after comparison. The remaining part of the genome was considered to be *potentially unique* to the target pathogen.

10.4 CLUSTERING OF MICROARRAY DATA

Microarrays provide a powerful basis to monitor the expression of thousands of genes, in order to identify mechanisms that govern the activation of genes in an organism. Short DNA patterns (or binding sites) near the genes serve as switches that control gene expression. Therefore, similar patterns of expression correspond to similar binding site patterns. A major cause of coexpression of genes is their sharing of the regulation mechanism (coregulation) at the sequence level. Control (or promoter) regions in the neighborhood of the genes contain specific short sequence patterns, called *binding sites*. The detection of such “statistically overrepresented patterns in DNA or amino acid sequences” is called “*motif*” *finding*. Clustering of coexpressed genes, into biologically meaningful groups, have been integrated with the discovery of binding motifs in order to discover regulatory motifs from microarray data [20]. Clustering also helps in inferring the biological role of an unknown gene that is coexpressed with a known gene(s).

Motifs are found in two broad ways. Word counting methods from string matching are based on counting the number of occurrences of each DNA word (or oligonucleotide) and comparing this number with the expected number of occurrences based on some statistical model. Probabilistic sequence models build a likelihood function for the sequences based on the motif occurrences and a model of the background sequence. Optimization methods like expectation maximization (EM) and Gibbs sampling are used to search for good motif configurations.

Clustering of gene expression profiles often require preprocessing along the following lines.

1. Normalizing the hybridization intensities within a single array experiment.
2. Transforming the data using a nonlinear function, like the logarithm in case of expression ratios.
3. Estimating and replacing missing values in expressions, or adapting existing algorithms to handle missing values.
4. Filtering gene expression profiles, to eliminate those that do not satisfy some simple criteria.

5. Standardizing or rescaling the profiles, say, to generate vectors of length one.

Cluster validation is essential, from both the biological and statistical perspectives, in order to biologically validate and objectively compare the results generated by different clustering algorithms. Statistical cluster validation is done by evaluating cluster coherence, predictive accuracy, and robustness to noise. Based on biological intuition, a cluster result can be termed reliable if the within-cluster distance is small (i.e., all genes within a cluster are tightly coexpressed) and the maximal inter-cluster distance is large (or, the cluster has an average profile well delineated from the remaining dataset). Quantitative statistics like Dunn's validity index [21] and Figure of Merit [22] have been used to compare different clustering algorithms.

A categorization of clustering algorithms (dedicated to microarray data) is provided by Moreau et al. [20], grouping some of the later methods designed specifically to handle gene expression profiles to be in the second generation.

10.4.1 First-generation algorithms

First-generation clustering algorithms, used on gene expression profiles, include *c*-means [23], self-organizing maps (SOMs) [24], and hierarchical clustering [25]–[27]. The SOM neural net has been applied to the exploratory data analysis of gene expression data from a yeast DNA microarray [28]. Hierarchical clustering is the most widely used method, given its good visualization properties. Clusters are generated by cutting the dendrogram at a certain level. However, it is arbitrary to predict the level that would correspond to the best *biological results*. Moreover, the quadratic complexity of the method is a source of inconvenience for mining the large volumes of data. In case of *c*-means algorithm, the center corresponds to the average expression vector. But the prediction of the number of clusters in advance is often arbitrary from the context of biology. SOMs require a selected node in the gene expression space (along with its neighbors) to be rotated in the direction of a selected gene expression profile (pattern). Again, the predefinition of a two-dimensional topology of nodes can be a problem considering *biological relevance*.

An inherent problem with all these algorithms is that they force every data point into a cluster. In case of microarray data, a considerable number of genes do not contribute to the biological process being studied and hence lack coexpression with other genes. Inclusion of such *noisy* genes in any cluster causes contamination, making it less suitable for further analysis. Besides, the computational and memory complexities of these methods often limit the number of expression profiles that can be handled.

10.4.2 Second-generation algorithms

These include the self-organizing tree algorithm (SOTA) [29], model-based clustering [30, 31], and quality-based clustering [32, 33].

The SOTA [29] is a dynamic binary tree that combines the characteristics of SOMs and divisive hierarchical clustering. As in SOMs, the gene expression profiles are sequentially and iteratively presented at the terminal nodes, and the mapping of the node that is closest (along with its neighboring nodes) is appropriately updated. Upon convergence, the node containing the most variable (measured in terms of distance) population of expression profiles is split into sister nodes, causing a growth of the binary tree. Unlike conventional hierarchical clustering, SOTA is linear in complexity to the number of profiles. The number of clusters need not be known in advance, as in *c*-means algorithm. A statistical procedure is followed for terminating the growing of the tree, thereby eliminating the need for an arbitrary choice of cutting level as in hierarchical models. However, no validation is provided to establish its biological relevance.

Model-based clustering [30, 31] typically assumes multivariate probability (normal) distributions to represent clusters. The covariance matrix for each cluster is expressed by its eigenvalue decomposition, which controls the orientation, shape, and volume parameters. Initially the model parameters are estimated by the EM algorithm using fixed cluster number and covariance structure. Then the best from this group of models is selected, based on an information theoretic criterion.

Quality-based clustering (QT-Clust) [32] generates clusters with a quality guarantee (user-defined threshold), ensuring that each member of a cluster is coexpressed with all other members. It is a greedy procedure, generating clusters satisfying the quality guarantee with a maximum number of expression profiles. The algorithm terminates when the number of points in the largest remaining cluster falls below a specified threshold. However, the user-defined quality guarantee is often difficult for biologists to handle and often leads to extensive parameter fine-tuning. The computational complexity is quadratic in the number of expression profiles.

Adaptive quality-based clustering [33] uses an estimate of the quality of the cluster, so that the initial center is located in a region of locally dense gene expression profiles. In an adaptive step, the quality of the cluster is re-estimated to determine significant coexpression of genes in terms of a *significance level*. The process is repeated until the relative difference between the initial and re-estimated quality is sufficiently small. The computational complexity of the algorithm is linear in the number of expression profiles and has been biologically validated. However, the approach is heuristic and not proven to converge in every situation.

10.5 ASSOCIATION RULES

Rule mining in Bioinformatics analyzes large collections of molecules to discover some regularity among molecules of a specific class. For example, in drug discovery a biologist tries to find new drug candidates based on experimental evidence of activity against a certain disease by screening the data. In chemical synthesis success prediction, on the other hand, the aim is to detect molecular features that inhibit the desired reaction.

Linear fragments or chains of atoms have been mined [34] using a technique similar to the *A priori* algorithm of Section 7.2.1. However, this restriction to linear fragments eliminates consideration of ring-like substructures that are common to most real-world applications involving molecular representations.

Association rules have been mined for molecular fragments, which help discriminate between different classes of activity on the National Cancer Institute's HIV-screening dataset [35]. Here a restricted depth-first search algorithm, similar to the *Eclat* association rule mining strategy [36], is used to determine connected substructures. It maintains parallel embeddings of a fragment in all molecules throughout the growth process of the search tree, while exploiting the local order of the atoms and bonds (single, double, triple, or aromatic) in the fragment to prune the tree.

Association rules can reveal biologically relevant associations between different genes, which are highly expressed or repressed in diseased and/or healthy cells. They can also be used to express associations between cellular environmental effects and gene expression, to diagnose (say) a profiled tumor sample or a drug treatment given to cells in the sample before profiling. Efficient mining of association rules from 300 gene expression profiles of yeast, demonstrating diverse mutations and chemical treatments, has been reported in Ref. [37] using the *A priori* algorithm. A gene expression profile is considered to be a single transaction, with each transcript or protein being an item in it. The measured value of the gene expression data is put in bins and is marked as being up (highly expressed), down (highly repressed), or neither up nor down.

10.6 ROLE OF SOFT COMPUTING

In this section we attempt to highlight the role of soft computing in Bioinformatics. The learning ability of neural networks has been utilized to predict the secondary structure of proteins [8]–[10]. The searching potential of GAs has been applied for prediction of the tertiary structure of proteins [11]. The uncertainty handling capacity of fuzzy sets is used in Ref. [38].

10.6.1 Predicting protein secondary structure

Prediction of protein structure from the primary amino acid sequence is a challenging task. The problem has been approached from several angles. A step on the way to a prediction of the full 3D structure of protein is predicting the local conformation of the polypeptide chain, called the secondary structure.

The whole framework was pioneered by Chou and Fasman [39]. They used a statistical method, with the likelihood of each amino acid being one of the three (α , β , coil) secondary structures being estimated from known proteins. Around 1988 the first attempts were made by Qian and Sejnowski [8], to use multilayer perceptrons to predict protein secondary structure. The performance of this method was improved by Rost and Sander [9], by using multiple-sequence alignment.

This has been further developed by Riis and Krogh [10], with ensembles of combining networks, for greater accuracy in prediction. The *Softmax* method is used to provide simultaneous classification of an input pattern into multiple classes. A normalizing function at the output layer ensures that the three outputs always sum to one. A logarithmic likelihood cost function is minimized, instead of the usual squared error. A window is selected from all the single structure networks in the ensemble. The output is determined for the central residue, with the prediction being chosen as the largest of the three outputs normalized by Softmax. Figure 10.3 provides the overall network structure.

10.6.2 Predicting protein tertiary structure

Protein binding sites exhibit highly selective recognition of small organic molecules, utilizing features like complex three-dimensional *lock* (active sites) into which only specific *keys* (drug molecules) will fit. This has been exploited by medicinal chemists in the design of molecules to selectively augment or retard biochemical pathways and thereby exhibit desired clinical effects. X-ray crystallography has been used to reveal the structure of many of these binding sites. Given the active site geometry of a protein molecule, it is now possible to obtain computer-aided design of therapeutic molecules in order to predict and explain the binding mode of novel chemical entities. This, in essence, is the *docking problem* that is of paramount importance in drug design.

Any solution to the docking problem requires a powerful search technique to explore the conformation space available to the protein and *ligand*, along with a good understanding of the process of molecular recognition to devise scoring functions for reliably predicting binding modes. A ligand can be a drug molecule, or even an enzyme. Genetic Optimization for Ligand Docking (GOLD) [40] is an automated ligand docking program that uses a GA to explore the full range of ligand conformational flexibility with partial flexibility of the protein, while satisfying the requirement that the ligand must displace loosely bound water molecules upon binding. A simple scoring function is

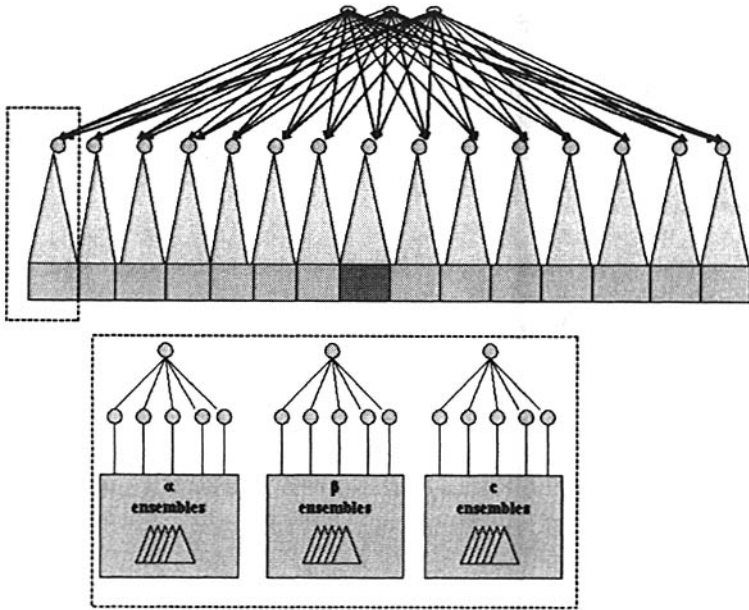


Fig. 10.3 The ensemble of combining and filtering network.

used to rank the generated binding modes. This comprises terms attributed to hydrogen bonding, pairwise dispersion potential to account for hydrophobic energy of binding, and molecular mechanics-based internal energy of the ligand.

Inspection of the X-ray crystallographic structures of proteins with associated high-affinity ligands reveals that the ligands appear to conform closely to the shape of the binding cavity, by maximizing their hydrophobic contribution and interacting at a number of hydrogen bonding sites. Additionally, dispersive interactions between protein and ligand atoms are involved along with an entropic contribution from the displacement of water molecules from the active site into the solvent. Sufficiently accurate simulation of many of these interactions may be enough to predict the binding mode of the majority of high-affinity ligands.

Tertiary protein structure prediction, using GAs, has also been reported in Ref. [11]. The objective is to generate a set of *native-like* conformations of a protein based on a force field. Proteins can be represented in terms of (a) three-dimensional Cartesian coordinates of its atoms and (b) the torsional angle Rotamers.

The Cartesian coordinates representation has the advantage of being easily converted to and from the 3D conformation of a protein. Bond lengths, b , are specified in these terms. However, it has the disadvantage that a mutation operator would in most instances create invalid protein conformations, where

some atoms lie too far apart or collide. Therefore a filter is needed to eliminate invalid individuals.

In the torsional angles representation, the protein is described by a set of angles under the assumption of constant standard binding geometries. The different angles involved are the

1. Bond angle θ ,
2. Torsional angle ϕ , between N (amine group) and C_α ,
3. angle ψ , between C_α and C' (carboxyl group),
4. Peptide bond angle ω , between C' and N , and
5. Dihedral angle.

The potential energy $U(r_1, \dots, r_N)$ between N atoms is minimized. It is expressed in terms of the different factors mentioned in Section 10.3.1, and is computed as

$$\begin{aligned}
 U(r_1, \dots, r_N) = & \sum_i K_b (b_i - b_0^i)^2 + \sum_i K_\theta (\theta_i - \theta_0^i)^2 \\
 & + \sum_i K_\phi [1 - \cos(n\phi_i - \delta)] + \sum_{i,j} \frac{q_i q_j}{4\pi\epsilon_0\epsilon_r r_{ij}} \\
 & + \sum_{i,j} \epsilon \left[\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - 2 \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right],
 \end{aligned} \tag{10.2}$$

where the first three harmonic terms on the right-hand side involve the bond length, bond angle, and torsional angle of covalent connectivity, with b_0^i and θ_0^i indicating the down-state (low energy) bond length and bond angle, respectively for the i th atom. The effects of hydrogen bonding and that of solvents (for nonbonded atom pairs i, j , separated by at least four atoms) is taken care of by the electrostatic Coulomb interaction and Van der Waals' interaction, modeled by the last two terms of the expression. Here K_b , K_θ , K_ϕ , and δ are constants, q_i and q_j are the charges of atoms i and j , separated by distance r_{ij} , and ϵ indicates the dielectric constant.

A protein acquires a folded conformation favorable to the solvent present. The calculation of the entropy difference between a folded and unfolded state is based on the interactions between a protein and solvent pair. The term E_{pe} , for a conformation, is a function of its actual diameter. Here the diameter is defined to be the largest distance between a pair of carbon atoms in a conformation. We have

$$E_{pe} = 4^{(\text{actual_diameter} - \text{expected_diameter})} \text{ kcal/mol.} \tag{10.3}$$

This is the conformational entropy constituent of potential energy, in addition to the factors involved in Eq. (10.2). It implies that extended conformations

have larger energy values and are therefore less fit for reproduction than *global* conformations.

An active site structure determines the functionality of a protein. A ligand (enzyme or drug) docks into an active site of a protein. GOLD [40] evaluates nonmatching bonds while minimizing the potential energy (fitness function), defined in terms of the Van der Waals' internal and external (or ligand-site) energy, torsional (or dihedral) energy, and hydrogen bonds. The output is the ligand and protein conformations, associated with the fittest chromosome in the population, when the GA terminates. The files handled are the Cambridge Crystallographic Database (by Cambridge University-Glaxo collaboration), Brookhaven Protein Database (PDB), and the Rotamer library.

10.6.3 Determining binding sites

The DEZYMER algorithm [41] is used to identify potential new ligand binding sites in proteins of known 3D structure. It alters only the sequence and side chain structure, leaving the protein backbone fold intact. The algorithm searches for a constellation of backbone positions arranged such that if appropriate side chains were placed there, they would bind the ligand according to a predefined geometry of interaction. It avoids bad *steric* contact between atom pairs. Atomic close packing is preserved to fill the available space to the fullest extent, while satisfying potential hydrogen-bond-forming groups.

The protein database (PDB) contains known 3D atomic coordinates of the target protein. The Rotamer libraries for the different side chains provide a collection of allowed conformations of side chains obtained from the crystal structures of proteins. A predefined ligand binding geometry is provided considering bond lengths, bond angles, dihedral angles and nonbonded (Van der Waals) contacts. The algorithm measures the quality of fit between the desired geometry and the one found in a particular test configuration, and it arrives at or near a global minimum in the dihedral space for a given sequence in a predicted design. A test configuration is generated by evaluating an objective function. Fuzzy set theoretic concepts have been applied in the DEZYMER algorithm [38], to handle existing uncertainties in the modeling.

10.6.4 Classifying gene expression data

Classification of acute leukemia, having highly similar appearance in gene expression data, has been made by combining a pair of classifiers trained with mutually exclusive features [42]. Gene expression profiles were constructed from 72 patients having acute lymphoblastic leukemia (ALL) or acute myeloid leukemia (AML), each constituting one sample of the DNA microarray. Each pattern consists of 7129 gene expressions. Feature selection was employed to generate 25 top-ranked genes for the experiment.

A neural network combines the outputs of the multiple classifiers. Feature selection with nonoverlapping correlation (such as Pearson and Spearman correlation coefficients) encourages the classifier ensemble to learn different aspects of the training data in a wide solution space. The recognition accuracy and generalization capacity are higher than those involving support vector machines [43, 44], SOM, decision tree, and k -NN classifier.

10.7 CONCLUSIONS AND DISCUSSION

Bioinformatics is a new area of science where the combination of statistics, molecular biology, and computational methods is used for analyzing and processing biological information like gene, DNA, RNA, and proteins. Proteins play a very important role in Bioinformatics. Improper folding of protein structure is responsible for causing many diseases. Therefore, accurate structure prediction of proteins is one of the main goals for proteomics.

The principles and results of string matching have been used to solve many problems in Bioinformatics, especially for DNA sequencing, alignment, homologue search, etc. We have discussed these issues in this chapter. We have also shown how string matching techniques can be used to search patterns in DNA sequence in Chapter 4.

Soft computing tools like neural networks and genetic algorithms have been used for analyzing the different protein structures and folds. Since the work entails processing huge amounts of incomplete or ambiguous data, the learning ability of neural networks, uncertainty handling capacity of fuzzy sets, and the searching potential of GAs are utilized in this direction. Artificial neural networks have been used for the determination of secondary structures of proteins. GAs are used for the prediction tertiary structure, by minimizing a potential energy function.

Microarray Bioinformatics has aided in a massive parallelization of experimental biology [3]. Hierarchical clustering has been shown to be effective in microarray data analysis for identifying genes with similar profiles and, possibly, similar functions. Recent decision tree packages let users manipulate incoming data and the rules generated, then examine the results with color- and size-coded visualizations. This capacity to interact and explore lets domain experts apply their knowledge by quickly testing hypotheses and performing exploratory data analysis. Microarrays, sequenced genomes, and the explosion of Bioinformatics research have led to astonishing progress in our understanding of molecular biology. Hybrid approaches, combining powerful algorithms and interactive visualization tools with the strengths of fast processors, hold promise for enhanced performance in the near future.

Interactive clustering with good visualization is being applied in the Bioinformatics domain [45]. Users are often willing to accept models with non-optimal generalization performance, provided that they can explore the underlying decision process and possibly influence the construction of the model

interactively using domain knowledge. The neighborhood of each cluster, expressed in terms of a representative sample, is analyzed to select the optimal cluster representative. Visual clustering results are provided for AIDS Antiviral Screen dataset [45].

Bioinformatics, with its huge volume of high-dimensional data, holds ample promise for the emergent field of biological data mining. Development of lossless data compression holds promise to compress and manage such huge volume of data. Some of the techniques used in very large database management systems and data mining area to reduce high-dimensionality of data need to be utilized in biological data also, for efficient storage, retrieval, and data management.

Some of the implications of the just completed Human Genome project have thrown up the following possibilities.

- Cure of genetic diseases like sickle cell anemia, etc., caused by a certain mutation, could become a reality.
- Cure for cancer could be contemplated.
- Sequencing genomes of bacteria could be useful in energy production, environmental remediation, waste management, and biotechnology industry.
- Tackling the docking problem for drug design.
- Interference with the laws of Nature by humans.

However, the field being still in a nascent state, we tried to compile in this chapter a concise treatise on the varied aspects of Bioinformatics currently available in literature. We attempted to bring in flavors from applications involving string matching, classification, clustering, and rule mining, which pertain to the scope of this book. We look forward to a successful marriage between biological data and mining tools, in the near future, for the betterment of the human race.

REFERENCES

1. S. L. Salzberg, D. B. Searls, and S. Kasif, eds., *Computational Methods in Molecular Biology*. Amsterdam: Elsevier Sciences B. V., 1998.
2. P. Baldi and S. Brunak, *Bioinformatics: The Machine Learning Approach*. Adaptive Computation and Machine Learning, Cambridge, MA: The MIT Press, 2001.
3. "Special Issue on Bioinformatics," *IEEE Computer*, vol. 35, July 2002.

4. "Special Issue on Bioinformatics, Part I: Advances and Challenges," *Proceedings of the IEEE*, vol. 90, November 2002.
5. S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman, "Basic Local Alignment Search Tool," *Journal of Molecular Biology*, vol. 215, pp. 403–410, 1990.
6. L. Hirschman, J. C. Park, J. Tsujii, L. Wong, and C. H. Wu, "Accomplishments and challenges in literature data mining for biology," *Bioinformatics*, vol. 18, pp. 1553–1561, 2002.
7. A. Neumaier, "Molecular modeling of proteins and mathematical prediction of protein structure," *SIAM Reviews*, vol. 39, pp. 407–460, 1997.
8. N. Qian and T. Sejnowski, "Predicting the secondary structure of globular proteins using neural network models," *Journal of Molecular Biology*, vol. 202, pp. 865–884, 1988.
9. B. Rost and C. Sander, "Prediction of protein secondary structure at better than 70 accuracy," *Journal of Molecular Biology*, vol. 232, pp. 584–599, 1993.
10. S. K. Riis and A. Krogh, "Improving prediction of protein secondary structure using structured neural networks and multiple sequence alignments," *Journal of Computational Biology*, vol. 3, pp. 163–183, 1996.
11. S. Schulze-Kremer, "Genetic algorithms for protein tertiary structure prediction," in *Parallel Problem Solving from Nature II* (R. Männer and B. Manderick, eds.). Amsterdam: Elsevier Sciences B. V., 1992.
12. X. Wang, J. T. L. Wang, D. Shasha, B. A. Shapiro, I. Rigoutsos, and K. Zhang, "Finding patterns in three-dimensional graphs: Algorithms and applications to scientific data mining," *IEEE Transactions on Knowledge and Data Engineering*, vol. 14, pp. 731–749, 2002.
13. C. Burge and S. Karlin, "Prediction of complete gene structures in human genomic DNA," *Journal of Molecular Biology*, vol. 268, pp. 78–94, 1997.
14. W. R. Pearson, and D. J. Lipman, "Improved tools for biological sequence comparison", *Proceedings of the National Academy of Sciences*, vol. 85, pp. 2444–2448, 1988.
15. J. Setubal and J. Meidanis. *Introduction to Computational Molecular Biology*. PWS Publishing Company, 1997.
16. R. M. Karp, "Mapping in genome: Some combinatorial problems arising in molecular biology," in *Proceedings of the Twenty-Fifth Annual ACM Symposium on the Theory of Computing* (San Diego, CA), pp. 278–285, 1993.

17. S. F. Altschul, T. L. Madden, A. A. Schaffer, J. Zhang, Z. Zhang, W. Miller, and D. J. Lipman, "Gapped BLAST and PSI-BLAST: A new generation of protein database search programs," *Nucleic Acids Research*, vol. 25, pp. 3389–3402, 1997.
18. Z. Zhang, A. A. Schaffer, W. Miller, T. L. Madden, D. J. Lipman, E. V. Koonin, and S. F. Altschul, "Protein sequence similarity searches using patterns as seeds," *Nucleic Acids Research*, vol. 26, pp. 3986–3990, 1998.
19. J. P. Fitch, S. N. Gardner, T. A. Kuczmarski, S. Kurtz, R. Myers, L. L. Ott, T. R. Slezak, E. A. Vitalis, A. T. Zemla, and P. M. McCready, "Rapid development of nucleic acid diagnostics," *Proceedings of the IEEE*, vol. 90, pp. 1708–1721, 2002.
20. Y. Moreau, "Functional Bioinformatics of microarray data: From expression to regulation," *Proceedings of the IEEE*, vol. 90, pp. 1722–1743, 2002.
21. F. Azuaje, "A cluster validity framework for genome expression data," *Bioinformatics*, vol. 18, pp. 319–320, 2002.
22. K. Y. Yeung, D. R. Haynor, and W. L. Ruzzo, "Validating clustering for gene expression data," *Bioinformatics*, vol. 17, pp. 309–318, 2001.
23. S. Tavazoie, J. D. Hughes, M. J. Campbell, R. J. Cho, and G. M. Church, "Systematic determination of genetic network architecture," *Nature Genetics*, vol. 22, pp. 281–285, 1999.
24. P. Tamayo, D. Slonim, J. Mesirov, Q. Zhu, S. Kitareewan, E. Smitrovsky, E. S. Lander, and T. R. Golub, "Interpreting patterns of gene expression with self-organizing maps: Methods and applications to hematopoietic differentiation," *Proceedings of National Academy of Sciences USA*, vol. 96, pp. 2907–2912, 1999.
25. M. B. Eisen, P. T. Spellman, P. O. Brown, and D. Botstein, "Cluster analysis and display of genome-wide expression patterns," *Proceedings of National Academy of Sciences USA*, vol. 95, pp. 14863–14868, 1998.
26. J. Quackenbush, "Computational analysis of microarray data," *Nat. Rev. Genetics*, vol. 2, pp. 418–427, 2001.
27. G. Sherlock, "Analysis of large-scale gene-expression data," *Current Opinion in Immunology*, vol. 12, pp. 201–205, 2000.
28. K. Torkkola, R. M. Gardner, T. Kaysser-Kranich, and C. Ma, "Self-organizing maps in mining gene expression data," *Information Sciences*, vol. 139, pp. 79–96, 2001.
29. J. Herrero, A. Valencia, and J. Dopazo, "A hierarchical unsupervised growing neural network for clustering gene expression patterns," *Bioinformatics*, vol. 17, pp. 126–136, 2001.

30. D. Ghosh and A. M. Chinnaiyan, "Mixture modeling of gene expression data from microarray experiments," *Bioinformatics*, vol. 18, pp. 275–286, 2002.
31. K. Y. Yeung, C. Fraley, A. Murua, A. E. Raftery, and W. L. Ruzzo, "Model-based clustering and data transformations for gene expression data," *Bioinformatics*, vol. 17, pp. 977–987, 2001.
32. L. J. Heyer, S. Kruglyak, and S. Yooseph, "Exploring expression data: Identification and analysis of coexpressed genes," *Genome Research*, vol. 9, pp. 1106–1115, 1999.
33. F. De Smet, J. Mathys, K. Marchal, G. Thijs, B. De Moor, and Y. Moreau, "Adaptive quality-based clustering of gene expression profiles," *Bioinformatics*, vol. 18, pp. 735–746, 2002.
34. S. Kramer, L. de Raedt, and C. Helma, "Molecular feature mining in HIV data," in *Proceedings of 7th International Conference on Knowledge Discovery and Data Mining (KDD-2001)* (San Francisco, CA), pp. 136–143, 2001.
35. C. Borgelt and M. R. Berthold, "Mining molecular fragments: Finding relevant substructures of molecules," in *Proceedings of IEEE International Conference on Data Mining* (Maebashi, Japan), December 2002.
36. M. Zaki, S. Parthasarathy, M. Ogihara, and W. Li, "New algorithms for fast discovery of association rules," in *Proceedings of 3rd International Conference on Knowledge Discovery and Data Mining (KDD 97)* (California), pp. 283–296, 1997.
37. C. Creighton and S. Hanash, "Mining gene expression databases for association rules," *Bioinformatics*, vol. 19, pp. 79–86, 2003.
38. J. S. Fetrow and J. Skolnick, "Method for prediction of protein function from sequence using the sequence-to-structure-to-function paradigm with application to Glutaredoxins/Thioredoxins and t_1 Ribonucleases," *Journal of Molecular Biology*, vol. 281, pp. 949–968, 1998.
39. P. Chou and G. Fasman, "Prediction of the secondary structure of proteins from their amino acid sequence," *Advances in Enzymology*, vol. 47, pp. 45–148, 1978.
40. G. Jones, P. Willett, R. C. Glen, A. R. Leach, and R. Taylor, "Development and validation of a genetic algorithm for flexible docking," *Journal of Molecular Biology*, vol. 267, pp. 727–748, 1997.
41. H. W. Hellinga and F. M. Richards, "Construction of new ligand binding sites in proteins of known structure," *Journal of Molecular Biology*, vol. 222, pp. 763–785, 1991.

42. S. B. Cho and J. Ryu, "Classifying gene expression data of cancer using classifier ensemble with mutually exclusive features," *Proceedings of the IEEE*, vol. 90, pp. 1744–1753, 2002.
43. T. S. Furey, N. Cristianini, N. Duffy, D. W. Bednarski, M. Schummer, and D. Haussler, "Support vector machine classification and validation of cancer tissue samples using microarray expression data," *Bioinformatics*, vol. 16, pp. 906–914, 2000.
44. A. Ben-Dor, L. Bruhn, N. Friedman, I. Nachman, M. Schummer, and N. Yakhini, "Tissue classification with gene expression profiles," *Journal of Computational Biology*, vol. 7, pp. 559–584, 2000.
45. M. R. Berthold, B. Wiswedel, and D. E. Patterson, "Neighborgram clustering: Interactive exploration of cluster neighborhoods," in *Proceedings of IEEE International Conference on Data Mining* (Maebashi, Japan), December 2002.

Index

- A priori algorithm, 269
- Active learning, 205
- Amino acid, 368
- ANN, 44
 - Kohonen net, 49
 - LMS error, 48
 - LVQ, 51
 - MLP, 47
 - perceptron, 46
 - RBF, 52
 - SOM, 50, 329
- Approximate reasoning, 41
- Artificial neural networks, 44
- Association rules, 9, 267
 - Bioinformatics, 381
 - candidate generation and test, 269
 - candidate itemsets, 270
 - confidence, 267, 270
 - depth-first search, 273
 - frequent itemsets, 270
 - fuzzy, 283
 - generalized, 278
 - localized, 282
 - multilevel, 276
 - online generation, 277
 - quantitative, 281
 - scalable mining, 280
 - support, 267, 270
 - temporal, 281
 - types, 268
- Backpropagation, 49
- Bayes theorem, 196
- Bayesian classifier, 196
 - naive, 196
- Bayesian network, 198
- Belief network, 198
- Bioinformatics, 2, 23, 143, 147, 365
 - classification, 385
 - clustering, 378
 - Hidden Markov model, 374
 - information science, 371
 - microarray, 23
 - protein folding, 24
 - rule mining, 381
 - soft computing, 367, 381
 - string matching, 169

- Biological data mining, 365
- Biometrics, 3, 26
- BIRCH, 241
- Bits per sample, 97
- BLAST, 366, 375, 376
- Boyer-Moore, 158
- Boyer-Moore-Horspool, 161
- BWT, 132

- c-means, 228, 233
- c-medoids, 228, 234
- c-modes, 254
- c-prototypes, 254
- C4.5, 182, 185
- CART, 182, 185
- Case-based reasoning, 203
- CBIR, 66, 330
 - multidimensional indexing, 342
- CBR, 203
- CF-tree, 241
- CLARA, 228, 238
- CLARANS, 228, 238
- Classification, 8, 9, 18, 181
 - Bioinformatics, 385
 - examples, 18
 - methods, 19
- Classifier
 - ANNs, 182
 - decision tree, 181
- CLIQUE, 243
- Clustering, 8, 19, 227
 - agglomerative, 236
 - Bioinformatics, 378
 - categorical, 251
 - conceptual, 228, 255
 - density-based, 239
 - divisive, 236
 - examples, 19
 - grid-based, 243
 - hierarchical, 228, 235, 241
 - methods, 20
 - partitional, 232, 238
 - partitive, 228
 - scalable, 237
 - symbolic, 228, 255
 - symbolic agglomerative, 256
 - symbolic validity indices, 257
 - Web mining, 261
- CODEC, 94
- Color, 332
- Color coherence vector, 335
- Color histogram, 334
- Color moment, 336
- Color space, 332
 - CIE, 332
 - HLS, 332
 - HSV, 332
 - RGB, 332
- Color space conversion, 116
- Color tag, 336
- Competitive learning, 51
- Compressed pattern matching, 177
 - Huffman, 177
 - Lempel-Ziv, 178
- Compression ratio, 97
- Computational molecular biology,
 - 23, 365
- Content-based image retrieval, 330
- Content-based video retrieval, 348
- Correlation rules, 282
- CURE, 243

- Data compression, 90
 - coding complexity, 99
 - discrete memoryless model, 91
 - entropy, 96
 - examples, 10
 - lossless, 94, 96
 - lossy, 94, 96
 - model, 95
 - multimedia data, 12
 - performance measure, 96
 - quantization, 96
 - source coding algorithms, 99
- Data condensation, 12
- Data mining, 4, 5, 7-10
 - applications, 25
 - Bioinformatics, 365
 - challenges, 26
 - compression, 6, 11, 27, 89

- distributed, 28, 29
- examples, 2
- functions, 8
- fuzzy sets, 62
- GA, 70
- hybridization, 74
- interestingness, 7
- modularization, 75
- neural networks, 67
- preprocessing, 6
- rough sets, 72
- scalability, 9
- soft computing, 36
- spatial, 73
- visualization, 6, 27
- wavelets, 73
- Data warehousing, 4, 24
- Davies–Bouldin index, 258
- DBSCAN, 239
- DCT, 96
- Decision tree, 184
 - MDL, 193
 - oblique, 221
 - overfitting, 192
 - performance measure, 210
 - pruning, 186, 193
 - rule extraction, 194
- Decompression, 94
- Dendrogram, 228, 235
- Dictionary coding, 131
- Digital library, 2
- Distance
 - binary objects, 229
 - categorical objects, 231
 - edit, 172
 - Hamming, 172
 - Levenshtein, 172
 - numeric objects, 229
 - symbolic objects, 231
- DNA, 23, 367
- DNA microarray, 366, 371
- Dunn's index, 258
- DWT, 96
- Eager learners, 200
- Eclat, 273
- Entropy, 92, 185, 287
 - fuzzy, 206, 209
- Euler number, 340
- Evolutionary clustering, 250
- Expectation maximization, 235
- FASTA, 375
- FCM, 244
- FP-growth, 273
- FP-tree, 273
- Frequent-pattern growth, 273
- Fuzzy association rules, 283
- Fuzzy c-means, 244
 - conditional, 245
- Fuzzy c-medoids, 245
- Fuzzy clustering, 244
- Fuzzy decision tree, 183, 205
- Fuzzy entropy, 209
- Fuzzy ID3
 - classification, 207
 - rule evaluation, 213
 - rule generation, 212
 - rule mapping, 214
- Fuzzy knowledge-based network, 214
- Fuzzy logic, 41
- Fuzzy sets, 38, 39
 - association rules, 64, 283
 - basic operations, 43
 - classification, 205
 - clustering, 63, 244
 - functional dependencies, 65
 - granulation, 40
 - membership, 39, 40
 - membership function, 42
 - summarization, 65
- GA, 39, 55
 - association rules, 71
 - clustering, 250
 - multiobjective, 71
 - regression, 71
- Gene, 23
- Gene expression, 366, 371, 385
- Generalization, 192

- Genetic algorithms, 39, 55
- Genome, 23, 371
- Gini index, 185
- Granular clustering, 246
- Granular computing, 64, 203
- HITS, 352
- Homology search, 374
 - BLAST, 376
 - dynamic programming, 375
- HSV, 332
- Hubert's statistic, 257
- Huffman coding, 100
- Huffman tree, 100
- Human perceptual system, 28
- IBM IntelligentMiner, 189
- ID3, 182, 185, 187
- Image compression, 105
 - DPCM, 106
 - prediction error, 106
 - standard, 112
- Image mining, 17, 29, 66, 329
 - compression, 17
 - content-based retrieval, 17
 - multimedia data, 16
- Information, 92
- Information retrieval, 12
 - multimedia data, 13
- Information Theory, 91
- Instance-based learners, 199
- Instance-based learning, 183
- ISODATA, 234
- JPEG, 112
 - baseline, 113, 116
 - baseline compression, 119
 - baseline decompression, 126
 - hierarchical, 112
 - lossless coding, 113
 - MCU, 118
 - progressive DCT, 112
 - quantization matrix, 120
 - sequential DCT, 112
 - sequential lossless, 112
- JPEG2000, 129
- k-differences, 176
- k-NN classifier, 201
- Karp-Rabin, 165
- KDD, 4-7
- KL transform, 103, 108
- Knowledge discovery in databases, 4-7
- Knowledge-based networks, 46
- Knuth-Morris-Pratt, 152
- Kohonen network, 49
- Lazy learners, 200
- Leader clustering, 237
- Learning vector quantization, 51
- Lempel-Ziv coding, 131
- LOGSOM, 353
- LVQ, 51
- LZ family, 131, 139
- LZ77, 132
- LZ78, 133
- LZW, 136
- Market basket analysis, 20
- Mathematical Theory of Communication, 91
- McCulloch-Pitts neuron, 45
- Microarray, 378
- Minimum distance classifier, 199
- MLP, 47
- Modular hybridization, 302, 305, 308
 - evaluation, 311
- Modularization, 294
- Moment, 338
- MPEG, 345, 347
- Multilayer perceptron, 47
- Multimedia applications, 90
- Multimedia data, 320
- Multimedia processing, 319
- Multimedia retrieval, 26
- Neural networks, 38, 44
 - clustering, 69, 246
 - feedback, 45

- feedforward, 45
 - information retrieval, 69
 - regression, 69
 - rule evaluation, 67
 - rule extraction, 67
- Neuro-fuzzy computing, 53
- Noiseless Source Coding Theorem, 92
- OLAP, 24
- OLTP, 24
- PAM, 228, 234
- Partition algorithm, 272
- PDBSCAN, 240
- PPM, 131
- Predictive coding, 105
- Principal component analysis, 103, 326
- Probabilistic learning, 183
- Probability density function, 41
- Protein, 23, 366, 369
- Protein folding, 367, 372
- Protein structure, 372
 - binding site, 385
 - homology, 373
 - secondary, 382
 - tertiary, 382
 - threading, 373
- PUBLIC, 194
- Quality metric
 - MOS, 97
 - objective, 98
 - PSNR, 98
 - subjective, 97
- Quantitative association rules, 281
- Radial basis function network, 52
- RainForest, 182, 185, 192
- RBF, 52, 202
- Regression, 8, 18
- Regression tree, 186
- Reinforcement learning, 45
- ROCK, 252
- Rough c-means, 249
- Rough fuzzy MLP, 302
- Rough sets, 59
 - clustering, 249
 - document clustering, 73
 - rule mining, 72
- Rule extraction
 - connectionist, 293
 - modular hybridization, 310
- Rule generation, 8, 293
 - connectionist, 295
 - knowledge-based network, 297
 - M-of-N, 299
 - modular hybridization, 302
 - neuro-fuzzy, 296
 - Subset, 298
- Rule interestingness, 275
 - fuzzy, 287
- Rule mining, 20, 267, 293
 - association, 21
 - classification, 21
 - dependency, 21
- Run-length coding, 99
- Self-organizing map, 50
- Sequence analysis, 8
- Shape, 338
- Single-layer perceptron, 46
- Singular value decomposition, 103, 327
- Soft computing, 36, 37
 - Bioinformatics, 381
 - relevance, 37
- SOM, 247
 - clustering, 248
 - dynamic, 248
- SPRINT, 182, 185, 189
- STING, 243
- STIRR, 252
- String, 144
- String matching, 21, 143, 146
 - approximate, 22, 147, 171
 - Bioinformatics, 23, 169, 175
 - brute force, 148
 - finite automata, 150
 - fully specified, 22

- latent semantic analysis, 326
- linear-order, 150
- partially specified, 22
- Summarization, 8
- Supervised learning, 45
- Support vector machine, 204
- SVM, 183, 204
- Text analysis, 322
- Text compression, 15, 131
- Text mining, 14, 103, 143, 147, 321
 - keyword-based, 321
 - multimedia data, 14
 - similarity-based, 325
 - soft computing, 328
 - string matching, 15
- Text retrieval, 322
- Texture, 337
- Topology, 340
- Transform coding, 107
- Unsupervised learning, 45, 51
- Video mining, 345
- Video retrieval, 345
- Wagner-Fischer, 173
- WaveCluster, 248
- Wavelet coding, 110
- Wavelet transform, 73
- Wavelets, 61
 - clustering, 248
- Web mining, 15, 350
 - information retrieval, 16
 - soft computing, 353
- Web search engines, 351
- WWW, 3
- Ziv-Lempel coding, 131

This page intentionally left blank

About the Authors

Sushmita Mitra is an *Associate Professor* at the Machine Intelligence Unit, Indian Statistical Institute, Kolkata. She obtained her B.Sc. (Honors) in Physics and B.Tech and M.Tech in Computer Science from the University of Calcutta, India in 1984, 1987, and 1989, respectively. She was awarded a Ph.D. in Computer Science by the Indian Statistical Institute, Calcutta in 1995. Since 1989, she has been employed at the Indian Statistical Institute.

From 1992 to 1994 she was in the European Laboratory for Intelligent Techniques Engineering (RWTH), Aachen, as a *German Academic Exchange Service (DAAD)* Fellow. She was a *Visiting Professor* in the Computer Science Departments of the Meiji University, Japan in 1999, and Aalborg University Esbjerg, Denmark in 2002.

Dr. Mitra received the *National Talent Search Scholarship* (1978-1983) from the National Council for Educational Research and Training, India, the *IEEE TNN Outstanding Paper Award* in 1994 for her pioneering work in neuro-fuzzy computing, and the *CIMPA-INRIA-UNESCO Fellowship* in 1996. She is an author of the book *Neuro-Fuzzy Pattern Recognition: Methods in Soft Computing* (New York: Wiley, 1999) and has more than 50 research publications in referred international journals. According to the Science Citation Index (SCI), two of her papers have been ranked 3rd and 15th in the list of *Top-cited papers in Engineering Science* from India during 1992-2001. Currently, she is guest editing a special issue of the *Fuzzy Sets and Systems* on "Web Mining Using Soft Computing."

She served in the programme committees of many International conferences. Dr. Mitra is a Senior Member of IEEE. Her current research interests include data mining, pattern recognition, soft computing, and Bioinformatics.

Tinku Acharya is a Senior Executive Vice President and a Chief Science Officer of Avisere Inc., Tucson, Arizona. He is also an Adjunct Professor in the Department of Electrical Engineering, Arizona State University, Tempe, Arizona. He received his B.Sc. (Honors) in Physics and his B.Tech and M.Tech in Computer Science from University of Calcutta, India in 1984, 1987, and 1989, respectively. He received his Ph.D. in Computer Science from the University of Central Florida, Orlando, Florida in 1994.

Dr. Acharya served in Intel Corporation from June 1996 to June 2002, where he led several R&D teams in numerous projects towards development of algorithms and architectures in image and video processing, multimedia computing, PC-based digital camera, high-performance reprographics architecture for color photo-copiers, biometrics, Multimedia architecture for 3G Cellular Mobile Telephony, analysis of next generation microprocessor architecture, etc. During his tenure in Intel Corporation, he served as a "Principal Engineer" for a number of years. Before joining Intel Corporation in 1996, he was a consulting Engineer in AT&T Bell Laboratories (1995–1996) in New Jersey, a research faculty member at the Institute of Systems Research, University of Maryland at College Park (1994–1995), and held visiting faculty positions at Indian Institute of Technology (IIT), Kharagpur (in several occasions during 1998–2001). He also served as a Systems Analyst in National Informatics Center, Planning Commission, Government of India (1988–1990) prior to his doctoral studies in the United States. He collaborated in research and development with Palo Alto Research Center (PARC) of Xerox Corporation, Kodak Corporation, and many other academic institutions and research laboratories worldwide.

Dr. Acharya is the inventor of 44 awarded patents, and more than 85 patents are currently pending in the US Patent Office. His pioneering works got him international acclamation. He has been awarded the *Most Prolific Inventor* in Intel Corporation Worldwide in 1999 and *Most Prolific Inventor* in Intel Corporation Arizona site for five consecutive years (1997–2001). His contribution in intellectual properties has been specially mentioned in a leading business journal in Phoenix, Arizona in its January 2002 issue. Dr. Acharya received many awards including a "Divisional Recognition Award" by the Connected Product Division from Intel Corporation in 2000 for development of key intellectual properties in digital camera and image processing architectures.

He contributed in over 60 referred technical papers published in International Journals, conferences, and book chapters. He is also co-author of the book *Information Technology: Principles and Applications*, scheduled to be published by Prentice-Hall, New Delhi, India.

Dr. Acharya is a Senior Member of IEEE, SPIE Optical Society. He served in the US National Body of JPEG2000 Standard committee (1998–2002) and represented Intel Corporation in this committee as its primary member. He served in programme committees of several international conferences and many other professional bodies in academia and industry.

His current research interests are in computer vision for enterprise applications, multimedia computing, biometrics, multimedia data mining, VLSI architectures, and algorithms.